

Sổ tay ôn thi Java OCP 21

Giải thích tiếng Việt · 10 chương · 124 ví dụ · 200 câu hỏi có lời giải

Oracle Certified Professional: Java SE 21 Developer (1Z0-830)

Tất cả ví dụ và câu hỏi được kiểm chứng bằng OpenJDK 21.0.10 · Bản build 2026-09-28

Không tài liệu nào bảo đảm điểm tuyệt đối — hãy dùng sách để hiểu Java, không để học thuộc.

Kiểm tra font tiếng Việt: á â ã ä å ơ ư đ ư Đ

Giới thiệu — Cách dùng cuốn sổ tay này

Cuốn sách này dành cho ai?

Nói thật trước khi bắt đầu

Thông tin kỳ thi (tóm tắt — xem [DECISION.md](#))

Cấu trúc mỗi chương

Mọi đáp án đều được máy kiểm chứng

Học thế nào cho hiệu quả?

Kiến thức nền cần có

Nguồn tham khảo (Sources)

DECISION — Chọn kỳ thi nào? 1Z0-830 (Java SE 21) hay 1Z0-829 (Java SE 17)

1. Kết luận ngắn

2. Bảng so sánh

3. Mục tiêu thi 1Z0-830 (10 nhóm)

4. Lý do chọn 1Z0-830

5. Còn 1Z0-831 (Java SE 25) thì sao?

6. Hệ quả cho phần còn lại của task

Nguồn tham khảo (Sources)

Chương 1 — Dữ liệu: số, boolean, chuỗi và ngày giờ

Mục tiêu

Giải thích đơn giản

Ví dụ

Đi sâu

Lỗi và bẫy thường gặp (Exam traps)

Góc nhìn từ TypeScript

Tóm tắt

Bài tập (có lời giải)

Đọc thêm (link chính thức — bị chặn trong sandbox nên mình chưa mở được)

Nguồn tham khảo (Sources)

Chương 2 — Luồng điều khiển (Controlling Program Flow)

Mục tiêu

Giải thích đơn giản

Ví dụ

Đi sâu

Lỗi và bẫy thường gặp (Exam traps)

Góc nhìn từ TypeScript

Tóm tắt

Bài tập (có lời giải)

Đọc thêm (link chính thức — bị chặn trong sandbox nên mình chưa mở được)

Nguồn tham khảo (Sources)

Chương 3 — Hướng đối tượng (Using Object-Oriented Concepts in Java)

Mục tiêu

Giải thích đơn giản

Ví dụ

Đi sâu

Lỗi và bẫy thường gặp (Exam traps)

Góc nhìn từ TypeScript

Tóm tắt

Bài tập (có lời giải)

Đọc thêm (link chính thức — bị chặn trong sandbox nên mình chưa mở được)

Nguồn tham khảo (Sources)

Chương 4 — Ngoại lệ (Handling Exceptions)

Mục tiêu

Giải thích đơn giản

Ví dụ

Đi sâu

Lỗi và bẫy thường gặp (Exam traps)

Góc nhìn từ TypeScript

Tóm tắt

Bài tập (có lời giải)

Đọc thêm (link chính thức — bị chặn trong sandbox nên mình chưa mở được)

Nguồn tham khảo (Sources)

Chương 5 — Mảng và Collections (Working with Arrays and Collections)

Mục tiêu

Giải thích đơn giản

Ví dụ

Đi sâu

Lỗi và bẫy thường gặp (Exam traps)

Góc nhìn từ TypeScript

Tóm tắt

Bài tập (có lời giải)
Đọc thêm (link chính thức — bị chặn trong sandbox nên mình chưa mở được)
Nguồn tham khảo (Sources)

Chương 6 — Lambda và Stream (Working with Streams and Lambda expressions)
Mục tiêu
Giải thích đơn giản
Ví dụ
Đi sâu
Lỗi và bẫy thường gặp (Exam traps)
Góc nhìn từ TypeScript
Tóm tắt
Bài tập (có lời giải)
Đọc thêm (link chính thức — bị chặn trong sandbox nên mình chưa mở được)
Nguồn tham khảo (Sources)

Chương 7 — Module, đóng gói và triển khai (Packaging and Deploying Java Code)
Mục tiêu
Giải thích đơn giản
Ví dụ
Đi sâu
Lỗi và bẫy thường gặp (Exam traps)
Góc nhìn từ TypeScript
Tóm tắt
Bài tập (có lời giải)
Đọc thêm (link chính thức — bị chặn trong sandbox nên mình chưa mở được)
Nguồn tham khảo (Sources)

Chương 8 — Đồng thời (Managing Concurrent Code Execution)
Mục tiêu
Giải thích đơn giản
Ví dụ
Đi sâu
Lỗi và bẫy thường gặp (Exam traps)
Góc nhìn từ TypeScript
Tóm tắt
Bài tập (có lời giải)
Đọc thêm (link chính thức — bị chặn trong sandbox nên mình chưa mở được)
Nguồn tham khảo (Sources)

Chương 9 — I/O và NIO.2 (Using Java I/O API)
Mục tiêu
Giải thích đơn giản
Ví dụ
Đi sâu
Lỗi và bẫy thường gặp (Exam traps)
Góc nhìn từ TypeScript
Tóm tắt
Bài tập (có lời giải)
Đọc thêm (link chính thức — bị chặn trong sandbox nên mình chưa mở được)
Nguồn tham khảo (Sources)

Chương 10 — Bản địa hoá (Implementing Localization)
Mục tiêu
Giải thích đơn giản
Ví dụ
Đi sâu
Lỗi và bẫy thường gặp (Exam traps)
Góc nhìn từ TypeScript
Tóm tắt
Bài tập (có lời giải)
Đọc thêm (link chính thức — bị chặn trong sandbox nên mình chưa mở được)
Nguồn tham khảo (Sources)

Lịch học 8 tuần (có phương án rút gọn 6 tuần)
Nguyên tắc
8 tuần
Phương án rút gọn 6 tuần (khi đã vững Java cơ bản)
Theo dõi tiến độ (tự điền)
Mẹo làm bài thi

GLOSSARY — Thuật ngữ Anh-Việt
COVERAGE — Mục tiêu thi 1Z0-830 → chương → ví dụ → câu hỏi
SOURCES — Nguồn học và nguồn kiểm chứng

1. Nguồn chính thức (ưu tiên cao nhất)
2. So sánh repo GitHub dùng để ôn 1Z0-830
3. Sách và khóa học nổi tiếng (chỉ giới thiệu link — có phí)

4. Chính sách bản quyền của sách này
5. Host bị chặn trong sandbox (không phải link hỏng)

Giới thiệu — Cách dùng cuốn sổ tay này

Cuốn sách này dành cho ai?

Cho **Nobin** và những người giống Nobin: dev frontend (TypeScript/Angular) đã biết Java cơ bản, đi làm full-time, muốn thi **Oracle Certified Professional: Java SE 21 Developer** — kỳ thi **1Z0-830** với hai mục tiêu: **điểm cao** và **hiểu Java thật sự** để làm backend.

Lý do chọn 1Z0-830 (thay vì 1Z0-829 / Java 17) nằm trong [DECISION.md](#).

Nói thật trước khi bắt đầu

- **Không tài liệu nào — kể cả cuốn này — bảo đảm bạn đạt điểm tuyệt đối, hay thậm chí bảo đảm bạn đậu.** Đề thi thật có câu hỏi mình không thể biết trước, Oracle có thể thay đổi đề, và kết quả phụ thuộc vào việc bạn luyện tập, ngủ đủ, và cả may mắn hôm thi.
- Mọi câu hỏi trong sách đều **tự viết** theo phong cách đề thi (không phải đề thật, không phải “dump”). Mục đích là luyện **cách suy nghĩ**, không phải học thuộc đáp án.
- Một số thông tin về kỳ thi (số câu, thời gian, điểm đậu, giá) được lấy từ kết quả tìm kiếm vì trang Oracle bị chặn trong môi trường viết sách; chúng được đánh dấu **UNVERIFIED**. Hãy kiểm tra lại trên trang Oracle trước khi đăng ký thi.

Thông tin kỳ thi (tóm tắt — xem DECISION.md)

	1Z0-830
Số câu	50 (UNVERIFIED)
Thời gian	120 phút (UNVERIFIED)
Điểm đậu	68% ≈ 34 câu (UNVERIFIED)
Dạng câu	Chọn 1 / chọn nhiều, phần lớn có code
Nhóm mục tiêu	10 nhóm, 26 mục tiêu con (theo cách chia của sách này) — xem COVERAGE.md

Cấu trúc mỗi chương

Mỗi chương ứng với **một nhóm mục tiêu** của đề thi và đi theo thứ tự “ý đơn giản → ví dụ thật → chi tiết”:

1. **Mục tiêu** — học xong làm được gì.
2. **Giải thích đơn giản** — ý chính bằng lời thường.
3. **Ví dụ** — 10+ chương trình nhỏ, kèm **output thật** (chạy bằng OpenJDK 21.0.10).
4. **Đi sâu** — quy tắc chi tiết mà đề thi hay hỏi.
5. **Lỗi và bẫy thường gặp (Exam traps)**.
6. **Góc nhìn từ TypeScript** — so sánh với thứ bạn đã biết.
7. **Tóm tắt**.
8. **Bài tập (có lời giải)** — 20 câu tự viết; mỗi câu có đáp án, lý do đúng, và lý do **từng** phương án sai.
9. **Nguồn tham khảo (Sources)**.

Mọi đáp án đều được máy kiểm chứng

Mỗi câu hỏi được lưu trong `examples/questions/<chương>/questions.yaml`. Script `tools/book.py` sinh file Java, **biên dịch và chạy thật** rồi so sánh với đáp án:

- Câu “in ra gì”: output thật phải khớp đúng phương án đúng.
- Câu “dòng nào lỗi biên dịch”: kiểm tra **từng dòng riêng lẻ** (sửa các dòng khác thì dòng đó vẫn lỗi; sửa tất cả thì biên dịch được).
- Câu “chèn đoạn nào...”: biên dịch/chạy **từng phương án**.
- Câu “phát biểu nào đúng”: **mỗi** phát biểu có một chương trình chứng minh đúng/sai.
- Câu về module/bundle: script tạo đúng cấu trúc thư mục rồi chạy `javac`, `java`, `jar`, `jlink` thật.

Chạy lại tất cả: `bash tools/run_all.sh` (xem [README.md](#)).

Học thế nào cho hiệu quả?

- Theo **lịch 8 tuần**: ~1 giờ ngày thường, ~3 giờ cuối tuần.
- Luôn **đoán output trước** khi nhìn đáp án. Sai thì ghi vào số lỗi.
- Làm câu hỏi **không dùng IDE** (đề thi không có IDE, không có gợi ý lỗi).
- Cuối lộ trình: 3 đề thi thử 50 câu / 120 phút, và cheat sheet 1 trang cho mỗi chương.

Kiến thức nền cần có

Biết viết class, method, `if`, vòng lặp, và chạy `javac Hello.java && java Hello`. Nếu chưa, hãy học phần nhập môn Java trước (khoảng 1–2 tuần) rồi quay lại.

Nguồn tham khảo (Sources)

- Thông tin kỳ thi và nguồn học: [DECISION.md](#), [SOURCES.md](#) (kiểm tra 2026-09-28).

DECISION — Chọn kỳ thi nào? 1Z0-830 (Java SE 21) hay 1Z0-829 (Java SE 17)

Ngày kiểm tra: 2026-09-28.

Cảnh báo trung thực: các trang chính thức của Oracle (`education.oracle.com`, `docs.oracle.com`, `mylearn.oracle.com`) và `dev.java` bị chặn trong sandbox nơi mình làm việc (lỗi 403 / `connect_rejected`). Vì vậy mọi con số về kỳ thi dưới đây lấy từ **đoạn trích kết quả tìm kiếm (WebSearch snippet)** có trỏ tới trang Oracle, và từ các nguồn phụ mình mở được. Tất cả đều được đánh dấu **UNVERIFIED** cho tới khi Nobin (hoặc một phiên có mạng) mở trang Oracle và xác nhận. Việc này nằm trong mục Blockers của `PROGRESS.md`.

1. Kết luận ngắn

Chọn 1Z0-830 — Java SE 21 Developer Professional. Cả cuốn sách, ví dụ và câu hỏi đều dùng **JDK 21** (máy kiểm tra: `OpenJDK 21.0.10`, biên dịch với `--release 21`).

2. Bảng so sánh

Tiêu chí	1Z0-830 — Java SE 21	1Z0-829 — Java SE 17	Nguồn / trạng thái
Tên kỳ thi	Java SE 21 Developer Professional	Java SE 17 Developer	Tiêu đề trang Oracle trong kết quả tìm kiếm [O1], [O2] — UNVERIFIED
Số câu hỏi	50	50	[S1], [S2], [G1] — UNVERIFIED
Thời gian	120 phút (~2,4 phút/câu)	90 phút (~1,8 phút/câu)	[S1], [S2], [G1] — UNVERIFIED
Điểm đậu	68% (≈ 34/50 câu)	68%	[S1], [S2], [G1] — UNVERIFIED
Giá	245 USD (thay đổi theo quốc gia)	245 USD (thay đổi theo quốc gia)	[S1], [S3] — UNVERIFIED ; giá ở Việt Nam có thể khác
Hình thức	Trắc nghiệm chọn 1 và chọn nhiều (multiple choice / multiple select), nhiều câu có code	Như bên trái	[S2], [S3] — UNVERIFIED
Nơi thi	Online qua Oracle University (có giám sát)	Online giám sát hoặc tại trung tâm	[G1], [S3] — UNVERIFIED
Tình trạng	Đang mở. Đã có bản mới hơn 1Z0-831 (Java SE 25) từ khoảng 2026-05	Theo snippet: retire ngày 2027-02-28	[S4], [S5], [G2] — UNVERIFIED
Nhóm mục tiêu thi	10 nhóm (xem mục 3) — không còn JDBC	11 nhóm, có “Accessing databases using JDBC”	[G1], [S2] — UNVERIFIED
Tính năng Java mới trong đề	record, sealed, pattern matching cho <code>instanceof</code> và <code>switch</code> , text block, virtual threads , sequenced collections (Java 21)	record, sealed, text block, switch expression (Java 17); không có virtual threads	[G1]

3. Mục tiêu thi 1Z0-830 (10 nhóm)

Danh sách này lấy từ bảng mục tiêu trong `intro.md` của repo `eh3rrera/ocpj21-book` [G1] (tác giả ghi là mục tiêu chính thức của Oracle) và khớp với snippet tìm kiếm của trang Oracle [S6]. **UNVERIFIED** so với trang Oracle gốc. Bản đầy đủ từng mục con nằm trong `COVERAGE.md`.

1. Handling Date, Time, Text, Numeric and Boolean Values
2. Controlling Program Flow
3. Using Object-Oriented Concepts in Java
4. Handling Exceptions
5. Working with Arrays and Collections
6. Working with Streams and Lambda expressions
7. Packaging and Deploying Java Code
8. Managing Concurrent Code Execution
9. Using Java I/O API
10. Implementing Localization

4. Lý do chọn 1Z0-830

1. **Nhiều thời gian hơn mỗi câu.** 120 phút cho 50 câu (1Z0-829 chỉ 90 phút). Đề OCP nổi tiếng là code dài và bẫy nhiều; thêm 30 phút giúp người đi làm full-time đọc kỹ hơn.
2. **Ít chủ đề “cũ” hơn.** 1Z0-830 bỏ nhóm JDBC. Thời gian ôn dồn vào ngôn ngữ và API lỗi.
3. **Kiến thức dùng được ngay khi làm backend.** Java 21 là bản LTS; virtual threads, record, sealed type, pattern matching cho `switch` là những thứ Nobin sẽ gặp trong Spring Boot 3 hiện đại.
4. **1Z0-829 sắp retire** (snippet nói 2027-02-28, **UNVERIFIED**). Nếu học 6–8 tuần rồi thi, vẫn kịp, nhưng không có lợi thế gì so với 1Z0-830.
5. **Kiểm chứng được 100% code trong môi trường này.** Sandbox có sẵn OpenJDK 21.0.10, nên mọi ví dụ và câu hỏi đều được biên dịch và chạy thật bằng đúng phiên bản của kỳ thi.
6. **Tài liệu ôn nhiều và đã “chín”.** Sách Sybex cho 1Z0-830, sách mở `ocpj21-book` (CC BY-NC-SA), Enthuware, các khóa Pearson/Pluralsight đều đã có (xem SOURCES.md).

5. Còn 1Z0-831 (Java SE 25) thì sao?

Task chỉ yêu cầu so sánh 21 và 17, nên mình giữ đúng phạm vi. Nhưng khi tìm kiếm, mình thấy Oracle đã ra **1Z0-831 — Java SE 25 Developer Professional** (khoảng tháng 5/2026; 50 câu, 120 phút, 68% — theo snippet [S4] và issue GitHub của dev.java [G2]; **UNVERIFIED**).

Mình vẫn chọn 1Z0-830 vì: JDK 25 không có trong sandbox nên không kiểm chứng được code; tài liệu ôn cho Java 25 còn rất ít; và ~90% kiến thức Java 21 vẫn đúng cho Java 25. Đây là **câu hỏi cho Nobin** (xem PR): nếu muốn thi Java 25, phần lớn sách vẫn dùng được, chỉ cần thêm một chương “Java 22–25” (flexible constructor bodies, unnamed variables `_`, module import, scoped values, stream gatherers, compact source files...).

6. Hệ quả cho phần còn lại của task

- Mọi chương dùng tên nhóm mục tiêu của 1Z0-830.
- Mọi ví dụ và câu hỏi: biên dịch bằng `javac --release 21`, chạy bằng `java 21.0.10`.
- Không dùng tính năng preview của Java 21 (ví dụ: unnamed variables `_`, string templates), vì đề thi chỉ dùng tính năng chính thức.
- Đề thi thử: 50 câu, 120 phút, đậu khi đúng ≥ 34 câu (68%).

Nguồn tham khảo (Sources)

Trang Oracle (bị chặn trong sandbox — chỉ thấy qua kết quả tìm kiếm, **chưa mở được**):

- [O1] https://education.oracle.com/java-se-21-developer-professional/pexam_1Z0-830
- [O2] https://education.oracle.com/java-se-17-developer/pexam_1Z0-829
- [O3] https://education.oracle.com/java-se-25-developer-professional/pexam_1Z0-831

Đoạn trích tìm kiếm (WebSearch, 2026-09-28) — nguồn phụ:

- [S1] Snippet cho truy vấn “1Z0-830 ... number of questions duration passing score price”: 50 câu, 120 phút, 68%, 245 USD (kết quả gồm trang Oracle [O1] và các trang luyện thi).
- [S2] Snippet cho truy vấn “1Z0-829 ... 90 minutes 68%”: 50 câu, 90 phút, 68%, có nhóm JDBC.
- [S3] Snippet cho truy vấn “1Z0-829 exam price \$245 format”: 245 USD (tùy khu vực), online hoặc Pearson VUE.
- [S4] Snippet cho truy vấn “1Z0-831 ... release date”: Java 25 exam ra ngày 2026-05-01, 50 câu, 120 phút, 68%.
- [S5] Snippet cho truy vấn “1Z0-829 retired ... 2027”: “will retire on February 28, 2027”.
- [S6] Snippet cho truy vấn các nhóm mục tiêu 1Z0-830 (Handling Date, Time, Text... / Implementing Localization).

Nguồn đã mở được:

- [G1] `eh3rrera/ocpj21-book`, `intro.md` — <https://github.com/eh3rrera/ocpj21-book/blob/main/intro.md> (mở 2026-09-28; license CC BY-NC-SA 4.0; chỉ tham khảo thông tin, không sao chép nội dung)
- [G2] `java/devjava-content` issue #242 — <https://github.com/java/devjava-content/issues/242> (mở 2026-09-28)

Chương 1 — Dữ liệu: số, boolean, chuỗi và ngày giờ

Mục tiêu

Sau chương này, bạn làm được:

- Dùng đúng 8 kiểu nguyên thủy (primitive type) và các lớp bao (wrapper class) tương ứng (objective 1.1).
- Tính đúng giá trị biểu thức số học và logic: thứ tự ưu tiên (precedence), nâng kiểu (numeric promotion), ép kiểu (casting), và các hàm của Math (objective 1.2).
- Xử lý chuỗi với String, StringBuilder và text block (objective 1.3).
- Làm việc với ngày giờ: LocalDate, LocalTime, LocalDateTime, ZonedDateTime, Instant, Period, Duration, múi giờ và giờ mùa hè — daylight saving time (DST) (objective 1.4).

Giải thích đơn giản

Primitive và object. Java có 8 kiểu nguyên thủy: byte, short, int, long, float, double, char, boolean. Biến primitive chứa **giá trị** trực tiếp. Mọi thứ khác là **object**; biến chỉ chứa **tham chiếu (reference)** tới object. Mỗi primitive có một wrapper class: Integer, Long, Character, Boolean... Java tự chuyển qua lại giữa hai loại: **autoboxing** (int → Integer) và **unboxing** (Integer → int).

Biểu thức. Khi tính toán, Java “nâng” kiểu nhỏ lên kiểu lớn. Quy tắc quan trọng nhất cho đề thi: byte, short, char khi tham gia phép toán **luôn thành int**. Muốn gán về kiểu nhỏ hơn thì phải ép kiểu (cast) — trừ vài trường hợp đặc biệt (hằng số compile-time, toán tử +=).

Chuỗi. String là **bất biến (immutable)**: mọi method như toUpperCase() trả về chuỗi **mới**, chuỗi cũ không đổi. StringBuilder thì **thay đổi được (mutable)**: append, insert, reverse... sửa trực tiếp object và trả về chính nó.

Ngày giờ. Gói java.time cũng bất biến. LocalDate = ngày không có múi giờ, LocalTime = giờ, LocalDateTime = cả hai, ZonedDateTime = có múi giờ, Instant = một điểm trên trục thời gian (UTC). Period đo theo năm/tháng/ngày (lịch), Duration đo theo giây/nano (đồng hồ).

Ví dụ

Tất cả ví dụ nằm trong examples/ch01/. Chạy lại toàn bộ: python3 tools/book.py examples ch01. Output bên dưới là output thật, chạy bằng OpenJDK 21.0.10 (locale en_US, múi giờ UTC).

1. Primitive, literal và giá trị mặc định

Field (biến của lớp) có giá trị mặc định (0, false, '\u0000', null). Biến cục bộ (local variable) **không** có — dùng trước khi gán sẽ lỗi biên dịch.

examples/ch01/Ex01_Primitives.java

```
// objective: 1.1
// 8 kiểu nguyên thủy (primitive), literal và giá trị mặc định của field.
public class Ex01_Primitives {
    static int defaultInt;           // field: có giá trị mặc định
    static boolean defaultBool;
    static char defaultChar;
    static double defaultDouble;

    public static void main(String[] args) {
        System.out.println("byte : " + Byte.MIN_VALUE + " .. " + Byte.MAX_VALUE);
        System.out.println("short : " + Short.MIN_VALUE + " .. " + Short.MAX_VALUE);
        System.out.println("int : " + Integer.MIN_VALUE + " .. " + Integer.MAX_VALUE);
        System.out.println("long : " + Long.MIN_VALUE + " .. " + Long.MAX_VALUE);
        System.out.println("char : " + (int) Character.MIN_VALUE + " .. " + (int) Character.MAX_VALUE);
        System.out.println("float max = " + Float.MAX_VALUE);
        System.out.println("double max = " + Double.MAX_VALUE);

        int million = 1_000_000;      // dấu gạch dưới giúp dễ đọc
        int hex = 0xFF, oct = 017, bin = 0b1010;
        long big = 3_000_000_000L;    // cần hậu tố L
        float f = 1.5f;              // cần hậu tố f
        char c = 'A' + 1;             // hằng số int vừa kiểu char → OK
        System.out.println(million + " " + hex + " " + oct + " " + bin + " " + big + " " + f + " " + c);

        System.out.println("defaults: " + defaultInt + " " + defaultBool + " ["
            + (int) defaultChar + "]" + defaultDouble);
    }
}
```

Output thật (JDK 21.0.10):

```
byte : -128 .. 127
short : -32768 .. 32767
int : -2147483648 .. 2147483647
long : -9223372036854775808 .. 9223372036854775807
char : 0 .. 65535
float max = 3.4028235E38
double max = 1.7976931348623157E308
1000000 255 15 10 3000000000 1.5 B
defaults: 0 false [0] 0.0
```

2. Wrapper class và cache của Integer

examples/ch01/Ex02_Wrappers.java

```
// objective: 1.1
// Wrapper class, autoboxing/unboxing, cache của Integer và bẫy null.
public class Ex02_Wrappers {
    public static void main(String[] args) {
        Integer a = 127, b = 127; // autoboxing dùng Integer.valueOf → có cache -128..127
        Integer c = 128, d = 128;
        System.out.println("127 == 127 ? " + (a == b));
        System.out.println("128 == 128 ? " + (c == d));
        System.out.println("128 equals 128 ? " + c.equals(d));

        int p = Integer.parseInt("42"); // trả về int (primitive)
        Integer w = Integer.valueOf("42"); // trả về Integer (object)
        System.out.println(p + w); // unboxing rồi cộng: 84

        Long l = 42L;
        System.out.println("Long(42).equals(42) ? " + l.equals(42)); // 42 boxing thành Integer!

        System.out.println(Character.isDigit('7') + " " + Character.toUpperCase('x')
            + " " + Boolean.parseBoolean("TRUE") + " " + Boolean.parseBoolean("yes"));

        Integer nothing = null;
        try {
            int boom = nothing; // unboxing null → NullPointerException
            System.out.println(boom);
        } catch (NullPointerException e) {
            System.out.println("NPE khi unboxing null");
        }
    }
}
```

Output thật (JDK 21.0.10):

```
127 == 127 ? true
128 == 128 ? false
128 equals 128 ? true
84
Long(42).equals(42) ? false
true X true false
NPE khi unboxing null
```

3. Numeric promotion và tràn số

examples/ch01/Ex03_Promotion.java

```
// objective: 1.2
// Numeric promotion: byte/short/char + ... luôn thành int; compound assignment tự ép kiểu.
public class Ex03_Promotion {
    public static void main(String[] args) {
        byte x = 10, y = 20;
        // byte z = x + y; // KHÔNG biên dịch: x + y là int
        byte z = (byte) (x + y); // phải ép kiểu
        x += 5; // OK: += có ép kiểu ngầm (implicit cast)
        System.out.println(z + " " + x);

        short s = 1;
        char ch = 'a';
        var r1 = s + ch; // int
        var r2 = 1 + 2L; // long
        var r3 = 1L + 2.0f; // float
        var r4 = 'a' + 1.0; // double
        System.out.println(((Object) r1).getClass().getSimpleName() + " "
            + ((Object) r2).getClass().getSimpleName() + " "
            + ((Object) r3).getClass().getSimpleName() + " "
            + ((Object) r4).getClass().getSimpleName());

        int max = Integer.MAX_VALUE;
        System.out.println("overflow: " + (max + 1)); // tràn số, không có exception
        long ok = max + 1L; // tính bằng long ngay từ đầu
        System.out.println("long : " + ok);
    }
}
```

```

        long wrong = max * 2; // nhân bằng int rồi mới mở rộng
        System.out.println("int*int : " + wrong);
    }
}

```

Output thật (JDK 21.0.10):

```

30 15
Integer Long Float Double
overflow: -2147483648
long : 2147483648
int*int : -2

```

4. Ép kiểu (casting)

examples/ch01/Ex04_Casting.java

```

// objective: 1.2
// Ép kiểu thu hẹp (narrowing) cắt bit; ép double → int bỏ phần thập phân.
public class Ex04_Casting {
    public static void main(String[] args) {
        System.out.println((int) 3.99); // 3 (cắt, không làm tròn)
        System.out.println((int) -3.99); // -3
        System.out.println((byte) 200); // 200 - 256 = -56
        System.out.println((short) 70000); // 70000 - 65536 = 4464
        System.out.println((char) 66); // B
        System.out.println((int) 'z'); // 122
        System.out.println((int) 1e20); // bão hòa ở Integer.MAX_VALUE
        System.out.println((long) Double.NaN); // NaN → 0

        final int small = 100;
        byte fits = small; // hằng số compile-time vừa byte → OK không cần cast
        System.out.println(fits);

        double d = 10 / 4; // chia int trước → 2, rồi mới thành 2.0
        double e = 10 / 4.0;
        System.out.println(d + " " + e);
    }
}

```

Output thật (JDK 21.0.10):

```

3
-3
-56
4464
B
122
2147483647
0
100
2.0 2.5

```

5. Toán tử và thứ tự ưu tiên

examples/ch01/Ex05_Operators.java

```

// objective: 1.2
// Thứ tự ưu tiên (precedence), ++/--, short-circuit, % với số âm.
public class Ex05_Operators {
    static boolean check(String name, boolean v) {
        System.out.print(name + " ");
        return v;
    }

    public static void main(String[] args) {
        int i = 5;
        int j = i++ + ++i; // 5 + 7
        System.out.println("i=" + i + " j=" + j);

        System.out.println(2 + 3 * 4 - 6 / 4); // 2 + 12 - 1 = 13
        System.out.println(-7 / 2 + " " + -7 % 2 + " " + 7 % -2);
        System.out.println(1 + 2 + "3" + 4 + 5); // "3345"

        boolean r = check("A", false) && check("B", true); // B không chạy
        System.out.println("-> " + r);
        r = check("A", false) & check("B", true); // & luôn chạy cả hai
        System.out.println("-> " + r);
        r = check("A", true) || check("B", true);
        System.out.println("-> " + r);

        int k = 10;
    }
}

```

```

    k += k++ + ++k;          // k = 10 + (10 + 12)
    System.out.println("k=" + k);

    int t = 3;
    String size = t > 5 ? "big" : t > 2 ? "medium" : "small";
    System.out.println(size + " " + (5 & 3) + " " + (5 | 3) + " " + (5 ^ 3) + " " + (~5));
}
}

```

Output thật (JDK 21.0.10):

```

i=7 j=12
13
-3 -1 1
3345
A -> false
A B -> false
A -> true
k=32
medium 1 7 6 -6

```

6. Math API

examples/ch01/Ex06_MathApi.java

```

// objective: 1.2
// Các hàm hay gặp của Math.
public class Ex06_MathApi {
    public static void main(String[] args) {
        System.out.println(Math.round(2.5) + " " + Math.round(-2.5) + " " + Math.round(2.4f));
        System.out.println(Math.floor(-1.1) + " " + Math.ceil(-1.1) + " " + Math rint(2.5));
        System.out.println(Math.max(3, 7L) + " " + Math.min(-0.0, 0.0) + " " + Math.abs(-4.5));
        System.out.println(Math.pow(2, 10) + " " + Math.sqrt(-1) + " " + Math.cbrt(27));
        System.out.println(Math.abs(Integer.MIN_VALUE));          // vẫn âm! (tròn số)
        System.out.println(Math.floorDiv(-7, 2) + " " + Math.floorMod(-7, 2));
        System.out.println(1.0 / 0 + " " + -1.0 / 0 + " " + 0.0 / 0);
        System.out.println(0.1 + 0.2);
        try {
            System.out.println(Math.addExact(Integer.MAX_VALUE, 1));
        } catch (ArithmeticException e) {
            System.out.println("ArithmeticException: " + e.getMessage());
        }
        try {
            System.out.println(10 / 0);
        } catch (ArithmeticException e) {
            System.out.println("ArithmeticException: " + e.getMessage());
        }
    }
}

```

Output thật (JDK 21.0.10):

```

3 -2 2
-2.0 -1.0 2.0
7 -0.0 4.5
1024.0 NaN 3.0
-2147483648
-4 1
Infinity -Infinity NaN
0.30000000000000004
ArithmeticException: integer overflow
ArithmeticException: / by zero

```

7. String

examples/ch01/Ex07_Strings.java

```

// objective: 1.3
// String bất biến (immutable), string pool, các method hay gặp.
public class Ex07_Strings {
    public static void main(String[] args) {
        String s1 = "java";
        String s2 = "ja" + "va";          // hằng số compile-time → cùng object trong pool
        String part = "ja";
        String s3 = part + "va";          // tạo lúc chạy → object mới
        System.out.println((s1 == s2) + " " + (s1 == s3) + " " + s1.equals(s3) + " " + (s1 == s3.intern()));

        String s = " Hello World ";
        s.toUpperCase();                  // kết quả bị bỏ đi: String không đổi
        System.out.println "[" + s + "]");
        System.out.println "[" + s.strip() + "] [" + s.stripLeading() + "] [" + s.trim().toLowerCase() + "]");
    }
}

```

```

String t = "banana";
System.out.println(t.indexOf('a') + " " + t.indexOf("an", 2) + " " + t.lastIndexOf('a')
    + " " + t.charAt(2) + " " + t.substring(1, 3) + " " + t.substring(3));
System.out.println(t.replace('a', 'o') + " " + t.replace("an", "") + " " + t.contains("nan")
    + " " + t.startsWith("ban") + " " + t.startsWith("na", 2));
System.out.println("ab".repeat(3) + " " + " ".isBlank() + " " + "".isEmpty() + " " + "A".compareTo("C")
    + " " + "abc".equalsIgnoreCase("ABC"));
System.out.println(String.join("-", "a", "b", "c") + " " + "a,b,c".split(",").length
    + " " + "x".concat("y") + " " + String.valueOf(3.0));
System.out.println("%s has %d chars".formatted(t, t.length()));
try {
    System.out.println(t.substring(4, 2));
} catch (StringIndexOutOfBoundsException e) {
    System.out.println("StringIndexOutOfBoundsException");
}
}
}

```

Output thật (JDK 21.0.10):

```

true false true true
[ Hello World ]
[Hello World] [Hello World ] [hello world]
1 3 5 n an ana
bonono ba true true true
ababab true true -2 true
a-b-c 4 xy 3.0
banana has 6 chars
StringIndexOutOfBoundsException

```

8. StringBuilder

examples/ch01/Ex08_StringBuilder.java

```

// objective: 1.3
// StringBuilder có thể thay đổi (mutable); các method trả về chính nó (method chaining).
public class Ex08_StringBuilder {
    public static void main(String[] args) {
        StringBuilder sb = new StringBuilder("abc");
        sb.append(1).append('x').append(true);
        System.out.println(sb + " len=" + sb.length());
        sb.insert(0, "->").delete(2, 4).deleteCharAt(sb.length() - 1);
        System.out.println(sb);
        sb.reverse();
        System.out.println(sb);
        sb.setLength(3);
        System.out.println(sb + " " + sb.indexOf("u") + " " + sb.charAt(0));
        sb.replace(0, 1, "ZZZ").setCharAt(1, 'y');
        System.out.println(sb);

        StringBuilder a = new StringBuilder("hi");
        StringBuilder b = new StringBuilder("hi");
        System.out.println(a.equals(b) + " " + a.toString().equals(b.toString()) + " " + (a.compareTo(b) == 0));

        StringBuilder same = a.append("!"); // cùng một object
        System.out.println((same == a) + " " + a);

        String sub = a.substring(1); // substring trả về String, không đổi a
        System.out.println(sub + " " + a);
    }
}

```

Output thật (JDK 21.0.10):

```

abc1xtrue len=9
->c1xtru
urtx1c>-
urt 0 u
ZyZrt
false true true
true hi!
i! hi!

```

9. Text block

examples/ch01/Ex09_TextBlocks.java

```

// objective: 1.3
// Text block: thụt lề (incidental whitespace), \s, \n nổi dòng, dấu " bên trong.
public class Ex09_TextBlocks {
    public static void main(String[] args) {
        String json = ""

```

```

        {
            "name": "Nobin",
            "lang": "Java"
        }
        """;
System.out.print(json);
System.out.println("ends with newline? " + json.endsWith("\n"));

String noNewline = """
    one line \
    continued""";
System.out.println("[ " + noNewline + " ]");

String keepSpace = """
    a  \s
    b""";
System.out.println(keepSpace.replace(' ', '.'));

String closingLeft = """
    indented
    """; // dấu "" đóng lệch trái → giữ 4 dấu cách
System.out.print(closingLeft.replace(' ', '.'));

String quotes = """
    She said "hi" and \"\" is fine""";
System.out.println(quotes);
System.out.println(json.lines().count() + " lines");
    }
}

```

Output thật (JDK 21.0.10):

```

{
  "name": "Nobin",
  "lang": "Java"
}
ends with newline? true
[one line continued]
a...
b
...indented
She said "hi" and "" is fine
4 lines

```

10. LocalDate, LocalTime, LocalDateTime

examples/ch01/Ex10_LocalDate.java

```

// objective: 1.4
// LocalDate/LocalTime/LocalDateTime là bất biến; phép cộng trả về object mới.
import java.time.*;
import java.time.format.DateTimeFormatter;
import java.time.temporal.ChronoUnit;

public class Ex10_LocalDate {
    public static void main(String[] args) {
        LocalDate d = LocalDate.of(2024, Month.JANUARY, 31);
        d.plusDays(1); // kết quả bị bỏ → d không đổi
        System.out.println(d + " " + d.plusMonths(1) + " " + d.plusMonths(1).plusMonths(1));
        System.out.println(d.getDayOfWeek() + " " + d.isLeapYear() + " " + d.lengthOfMonth());

        LocalTime t = LocalTime.of(23, 30);
        System.out.println(t.plusHours(2) + " " + t.minusMinutes(90) + " " + LocalTime.MIDNIGHT);

        LocalDateTime dt = LocalDateTime.of(d, t);
        System.out.println(dt + " " + dt.plusMinutes(45));
        System.out.println(ChronoUnit.DAYS.between(LocalDate.of(2024, 1, 1), d));
        System.out.println(d.withDayOfMonth(1) + " " + d.with(DayOfWeek.MONDAY));

        LocalDate parsed = LocalDate.parse("2024-02-29");
        System.out.println(parsed.plusYears(1) + " " + parsed.format(DateTimeFormatter.ofPattern("dd/MM/yyyy")));
        try {
            LocalDate.of(2023, 2, 29);
        } catch (DateTimeException e) {
            System.out.println("DateTimeException: " + e.getMessage());
        }
    }
}

```

Output thật (JDK 21.0.10):

```

2024-01-31 2024-02-29 2024-03-29
WEDNESDAY true 31
01:30 22:00 00:00

```

```
2024-01-31T23:30 2024-02-01T00:15
30
2024-01-01 2024-01-29
2025-02-28 29/02/2024
DateTimeException: Invalid date 'February 29' as '2023' is not a leap year
```

11. Period, Duration, Instant

examples/ch01/Ex11_PeriodDuration.java

```
// objective: 1.4
// Period (năm/tháng/ngày) vs Duration (giây/nano) vs Instant (điểm trên trục thời gian UTC).
import java.time.*;

public class Ex11_PeriodDuration {
    public static void main(String[] args) {
        Period p = Period.of(1, 14, 40);
        System.out.println(p + " normalized=" + p.normalized());
        System.out.println(Period.ofWeeks(2) + " " + Period.ofYears(1).ofMonths(3)); // bấ: static method!
        System.out.println(Period.between(LocalDate.of(2024, 1, 31), LocalDate.of(2024, 3, 1)));

        Duration du = Duration.ofMinutes(135);
        System.out.println(du + " " + du.toHours() + "h " + du.toMinutesPart() + "m");
        System.out.println(Duration.ofDays(1) + " " + Duration.ofSeconds(90).plusMillis(500));
        System.out.println(Duration.between(LocalTime.of(10, 0), LocalTime.of(8, 30)));

        Instant i = Instant.parse("2024-03-10T06:59:00Z");
        System.out.println(i.plus(Duration.ofMinutes(2)) + " " + i.getEpochSecond());
        try {
            LocalDate.of(2024, 1, 1).plus(Duration.ofDays(1)); // LocalDate không hỗ trợ Duration
        } catch (Exception e) {
            System.out.println(e.getClass().getSimpleName() + ": " + e.getMessage());
        }
        System.out.println(LocalDate.of(2024, 1, 1).plus(Period.ofDays(1)));
    }
}
```

Output thật (JDK 21.0.10):

```
P1Y14M40D normalized=P2Y2M40D
P14D P3M
P1M1D
PT2H15M 2h 15m
PT24H PT1M30.5S
PT-1H-30M
2024-03-10T07:01:00Z 1710053940
UnsupportedTemporalTypeException: Unsupported unit: Seconds
2024-01-02
```

12. Múi giờ và giờ mùa hè (DST)

examples/ch01/Ex12_ZonesDst.java

```
// objective: 1.4
// ZonedDateTime và giờ mùa hè (daylight saving time, DST) ở America/New_York.
import java.time.*;

public class Ex12_ZonesDst {
    public static void main(String[] args) {
        ZoneId ny = ZoneId.of("America/New_York");
        // 2024-03-10: đồng hồ nhảy từ 02:00 lên 03:00 (gap) - 02:30 không tồn tại
        ZonedDateTime gap = ZonedDateTime.of(LocalDateTime.of(2024, 3, 10, 2, 30), ny);
        System.out.println("gap : " + gap);
        ZonedDateTime before = ZonedDateTime.of(LocalDateTime.of(2024, 3, 10, 1, 30), ny);
        System.out.println("+1h : " + before.plusHours(1));
        System.out.println("+1 day : " + before.plusDays(1)); // giữ giờ địa phương
        System.out.println("+24h : " + before.plusHours(24)); // cộng đúng 24 giờ thật

        // 2024-11-03: đồng hồ lùi từ 02:00 về 01:00 (overlap) - 01:30 xảy ra 2 lần
        ZonedDateTime overlap = ZonedDateTime.of(LocalDateTime.of(2024, 11, 3, 1, 30), ny);
        System.out.println("overlap : " + overlap + " / later: " + overlap.withLaterOffsetAtOverlap());
        System.out.println("hours in 2024-11-03: " + Duration.between(
            ZonedDateTime.of(LocalDate.of(2024, 11, 3), LocalTime.MIDNIGHT, ny),
            ZonedDateTime.of(LocalDate.of(2024, 11, 4), LocalTime.MIDNIGHT, ny)).toHours());

        ZonedDateTime hanoi = before.withZoneSameInstant(ZoneId.of("Asia/Ho_Chi_Minh"));
        System.out.println("same instant in VN: " + hanoi);
        System.out.println("same local in VN : " + before.withZoneSameLocal(ZoneId.of("Asia/Ho_Chi_Minh")));
        System.out.println(OffsetDateTime.of(2024, 1, 1, 12, 0, 0, 0, ZoneOffset.ofHours(7)).toInstant());
    }
}
```

Output thật (JDK 21.0.10):

```
gap      : 2024-03-10T03:30-04:00[America/New_York]
+1h      : 2024-03-10T03:30-04:00[America/New_York]
+1 day   : 2024-03-11T01:30-04:00[America/New_York]
+24h     : 2024-03-11T02:30-04:00[America/New_York]
overlap  : 2024-11-03T01:30-04:00[America/New_York] / later: 2024-11-03T01:30-05:00[America/New_York]
hours in 2024-11-03: 25
same instant in VN: 2024-03-10T13:30+07:00[Asia/Ho_Chi_Minh]
same local in VN  : 2024-03-10T01:30+07:00[Asia/Ho_Chi_Minh]
2024-01-01T05:00:00Z
```

13. Lỗi biên dịch khi thu hẹp kiểu

examples/ch01/Ex13_NarrowingError.java

```
// objective: 1.1, 1.2
// expect: compile-error
// Gán long cho int và dùng kết quả byte + byte mà không ép kiểu: javac báo lỗi.
public class Ex13_NarrowingError {
    public static void main(String[] args) {
        long big = 10L;
        int small = big;
        byte a = 1, b = 2;
        byte c = a + b;
        float f = 3.14;
    }
}
```

Output thật của javac (JDK 21.0.10) — cố ý không biên dịch được:

```
Ex13_NarrowingError.java:7: error: incompatible types: possible lossy conversion from long to int
    int small = big;
                   ^
Ex13_NarrowingError.java:9: error: incompatible types: possible lossy conversion from int to byte
    byte c = a + b;
           ^
Ex13_NarrowingError.java:10: error: incompatible types: possible lossy conversion from double to float
    float f = 3.14;
             ^
3 errors
```

Đi sâu

Bảng primitive

Kiểu	Kích thước	Khoảng giá trị / ghi chú	Literal
byte	8 bit	-128 .. 127	(byte) 1 hoặc hằng số vừa khoảng
short	16 bit	-32768 .. 32767	
int	32 bit	~ ±2,1 tỷ	10, 0xFF, 017 (bát phân), 0b1010, 1_000
long	64 bit		10L
float	32 bit	số thực, ~7 chữ số	1.5f (bắt buộc f)
double	64 bit	số thực, ~15 chữ số	1.5, 1e3, 1.5d
char	16 bit	0 .. 65535, không dấu	'A', 'A', 65 (nếu là hằng số)
boolean	—	true / false (không phải 0/1)	

Quy tắc dấu gạch dưới _ trong literal: chỉ được đặt **giữa hai chữ số**. 1_000 hợp lệ; _1000, 1000_, 1_.5, 0xFF đều lỗi.

Quy tắc nâng kiểu (numeric promotion) — JLS §5.6

- Nếu một toán hạng là double → cả hai thành double.
- Nếu không, nếu một là float → float.
- Nếu không, nếu một là long → long.
- Nếu không → int (kể cả byte + byte, char + char, short + byte).

Hệ quả: short s = 1; s = s + 1; lỗi biên dịch, nhưng s += 1; và s++ thì được, vì toán tử gán kết hợp (compound assignment) tự ép kiểu về kiểu của biến trái.

Ngoại lệ “hằng số compile-time”: final int k = 10; byte b = k; biên dịch được, vì k là hằng số và giá trị 10 vừa với byte. Nếu bỏ final thì lỗi. Điều này chỉ đúng cho byte, short, char, int — **không** áp dụng cho long → int hay double → float (float f = 3.14; luôn lỗi).

Wrapper class

- Integer.valueOf(...) và autoboxing dùng **cache** cho giá trị -128..127 (Javadoc của Integer.valueOf trong mã nguồn JDK 21 ghi rõ điều này). Vì vậy == giữa hai Integer 127 cho true, 128 cho false. **Luôn dùng equals** để so sánh wrapper.
- parseXXX trả về primitive, valueOf trả về wrapper.
- equals của wrapper kiểm tra **cả kiểu**: Long.valueOf(42).equals(42) là false vì 42 được boxing thành Integer.
- Unboxing null → NullPointerException.
- Wrapper không “nới rộng rồi boxing”: Long x = 5; **lỗi** (5 là int, chỉ boxing được thành Integer). long y = 5; thì OK.

Toán tử: thứ tự ưu tiên (từ cao xuống thấp, rút gọn)

Nhóm	Toán tử
Hậu tố	x++ x--
Tiền tố, một ngôi	++x --x + - ! ~ (cast)
Nhân chia	* / %
Cộng trừ	+ -
Dịch bit	<< >> >>>
So sánh	< > <= >= instanceof
Bằng	== !=
Bit / logic	& rồi ^ rồi \
Short-circuit	&& rồi \ \
Ba ngôi	? :
Gán	= += -= ... (kết hợp từ phải sang trái)

Biểu thức luôn được **đánh giá từ trái sang phải** (toán hạng trái tính trước), sau đó mới áp dụng thứ tự ưu tiên để ghép. Vì vậy i++ + ++i với i = 5 là 5 + 7.

+ với String: tính từ trái sang phải; khi gặp String thì từ đó trở đi là nối chuỗi. 1 + 2 + "3" + 4 + 5 → "3" + ... → "3345".

Math API cần nhớ

Method	Kết quả đáng nhớ
Math.round(double)	trả về long; làm tròn nửa lên (floor(x + 0.5)): round(-2.5) = -2
Math.round(float)	trả về int
Math.floor/ceil/rint	trả về double; rint(2.5) = 2.0 (làm tròn về số chẵn)
Math.max/min	theo kiểu đã nâng: max(3, 7L) là long
Math.abs(Integer.MIN_VALUE)	vẫn âm (trần số)
Math.pow, Math.sqrt	luôn double; sqrt(-1) = NaN
Math.addExact, multiplyExact	ném ArithmeticException khi tràn
Chia số nguyên cho 0	ArithmeticException; chia số thực cho 0 → Infinity / NaN

String: những method hay ra đề

- substring(begin, end): lấy từ begin tới **trước** end. end < begin → StringIndexOutOfBoundsException.
- indexOf trả về -1 nếu không thấy; charAt ngoài khoảng → exception.
- strip() hiểu khoảng trắng Unicode; trim() chỉ bỏ ký tự ' '. isBlank() vs isEmpty().
- split(",") bỏ các chuỗi rỗng **ở cuối**, nhưng giữ chuỗi rỗng ở giữa: "a,b,,c" → 4 phần tử.
- String pool: literal và biểu thức hằng số ("ja" + "va") dùng chung object; chuỗi tạo lúc chạy thì không. intern() trả về bản trong pool.

StringBuilder

- Không override equals → so sánh bằng == (tham chiếu). So nội dung: toString().equals(...) hoặc compareTo (có từ Java 11).
- delete(start, end), insert(offset, x), replace(start, end, str), reverse(), setLength(n).
- substring trả về String, **không** thay đổi builder.

Text block (Java 15+)

- Mở bằng """ rồi **xuống dòng ngay** (không được viết nội dung trên cùng dòng mở).
- Khoảng trắng thụt lề chung (incidental whitespace) bị bỏ; vị trí dấu """ đóng cũng tính vào.
- Khoảng trắng cuối dòng bị xóa; dùng \s để giữ một dấu cách. \ ở cuối dòng = nối dòng (không xuống dòng).

- Nếu `""` đóng nằm trên dòng riêng → chuỗi kết thúc bằng `\n`.

Ngày giờ

- Tất cả đều bất biến; quên gán kết quả (d. `plusDays(1);`) là bẫy kinh điển.
- `plusMonths` giữ ngày nếu được, nếu không thì lấy **ngày cuối tháng**: `31/1 + 1 tháng = 29/2/2024`.
- `Period.of(1, 14, 40)` **không** tự chuẩn hoá; `normalized()` chỉ gộp tháng thành năm, không đưng ngày.
- `Period.ofYears(1).ofMonths(3) = P3M`: các `ofXxx` là **static**, gọi nối chuỗi sẽ bỏ kết quả trước.
- `Duration` dùng được với `LocalTime`, `LocalDateTime`, `Instant`, `ZonedDateTime`; **không** dùng với `LocalDate` (`UnsupportedTemporalTypeException`). `Period` không dùng được với `LocalTime`.
- DST: khi giờ địa phương **không tồn tại** (gap), `ZonedDateTime` dời tới sau khoảng trống (02:30 → 03:30). Khi giờ địa phương **xảy ra hai lần** (overlap), mặc định chọn offset **sớm hơn**. `plusDays(1)` giữ giờ địa phương; `plusHours(24)` cộng đúng 24 giờ thật.

Lỗi và bẫy thường gặp (Exam traps)

1. `int x = 10L;`, `float f = 1.0;`, `long l = 3_000_000_000;` (literal int quá lớn) → **không biên dịch**.
2. `byte b = a + c;` với `a, c` là `byte` → lỗi (kết quả là `int`). Nhưng `b += c;` thì được.
3. `Integer` so sánh bằng `==`: đúng với `-128..127`, sai ngoài khoảng đó.
4. `Long l = 5;` lỗi; `long l = 5;` OK; `Long l = 5L;` OK.
5. Quên gán lại kết quả của `String/LocalDate` (bất biến) — nhưng `StringBuilder` thì không cần gán.
6. `StringBuilder.equals` so sánh tham chiếu.
7. `Math.round(-2.5)` là `-2`, không phải `-3`. `Math.round` trả về long với tham số double.
8. `i = i++;` → `i` **không đổi**.
9. `&& / ||` có thể **bỏ qua** vế phải (short-circuit) → biến trong vế phải không được tăng.
10. `Period.ofYears(1).ofMonths(3)` chỉ còn 3 tháng.
11. `substring(3, 1)` ném exception, không trả chuỗi rỗng.
12. Chuỗi rỗng ở cuối kết quả `split` bị bỏ.

Góc nhìn từ TypeScript

TypeScript	Java	Ghi chú
<code>number</code> (một kiểu cho tất cả)	<code>byte... double</code> (6 kiểu số)	Java có chia số nguyên thật: <code>7 / 2 == 3</code>
<code>bigint</code>	<code>long / BigInteger</code>	long tràn số âm thầm, không báo lỗi
<code>===</code> so sánh giá trị với primitive	<code>==</code> với primitive; <code>equals</code> với object	<code>Integer == Integer</code> so sánh tham chiếu!
<code>string</code> (bất biến)	<code>String</code> (bất biến)	Giống nhau. Nhưng Java có thêm <code>StringBuilder</code> mutable
Template literal <code>`Hi \${name}`</code>	<code>"Hi %s".formatted(name)</code> hoặc nối <code>+</code>	String templates của Java 21 là preview, không thi
Chuỗi nhiều dòng bằng backtick	Text block <code>"""</code>	Text block bỏ thụt lề chung, backtick thì không
<code>Date</code> (mutable, lộn xộn)	<code>java.time</code> (bất biến, rõ ràng)	Tương tự thư viện <code>Temporal</code> mới của JS
<code>undefined / null</code>	chỉ có <code>null</code> (cho object)	Primitive không bao giờ <code>null</code>

Tóm tắt

- `byte/short/char` + bất cứ gì → ít nhất `int`. Thu hẹp kiểu phải cast, trừ hằng số compile-time và `+=`.
- Wrapper: dùng `equals`; cache `-128..127`; unboxing `null` → NPE.
- Đánh giá trái sang phải, rồi mới ưu tiên. `&&`, `||` short-circuit.
- `String` bất biến, `StringBuilder` mutable và không override `equals`.
- Text block: bỏ thụt lề chung, `\s` giữ khoảng trắng, `\n` nối dòng.
- `java.time` bất biến; `Period` cho ngày, `Duration` cho giờ; DST gap dời giờ tới trước, overlap chọn offset sớm.

Bài tập (có lời giải)

20 câu hỏi tự viết theo phong cách đề thi. Mọi đáp án đã được script `tools/book.py` biên dịch và chạy để xác nhận (code trong `examples/questions/ch01/`). Làm hết rồi mới xem lời giải.

Câu hỏi

Câu 01-01 · Dễ · objective 1.1

Chương trình sau in ra gì?

```
public class Cache {
    public static void main(String[] args) {
        Integer a = 100, b = 100;
        Integer c = 1000, d = 1000;
        System.out.println((a == b) + " " + (c == d) + " " + c.equals(d));
    }
}
```

- A. true true true
- B. true false true
- C. false false true
- D. true false false

Câu 01-02 · Vừa · objective 1.2

Dòng nào gây lỗi biên dịch?

```
public class Promo {
    public static void main(String[] args) {
        short s = 10;
        s += 5;           // L1
        s = s * 2;         // L2
        char c = 'a';
        c++;              // L3
        final byte k = 3;
        byte b = k + 1;   // L4
    }
}
```

- A. Chỉ L1
- B. Chỉ L4
- C. Chỉ L2
- D. L2 và L4
- E. Không có lỗi, chương trình biên dịch được

Câu 01-03 · Vừa · objective 1.2

Chương trình sau in ra gì?

```
public class Incr {
    public static void main(String[] args) {
        int i = 3;
        int j = i++ * 2 + --i;
        System.out.println(i + " " + j);
    }
}
```

- A. 4 9
- B. 3 8
- C. 3 9
- D. 4 10

Câu 01-04 · Dễ · objective 1.3

Chương trình sau in ra gì?

```
public class Immutable {
    public static void main(String[] args) {
        String s = "java";
        s.concat(" rocks");
        s.toUpperCase();
        StringBuilder sb = new StringBuilder("java");
        sb.append(" rocks");
        System.out.println(s + " | " + sb);
    }
}
```

- A. JAVA ROCKS | java rocks
- B. java rocks | java rocks
- C. java | java
- D. java | java rocks

Câu 01-05 · Vừa · objective 1.3

Chương trình sau in ra gì?

```
public class Builder {
    public static void main(String[] args) {
        StringBuilder sb = new StringBuilder("0123456789");
        sb.delete(2, 5).insert(3, "-").reverse();
        System.out.println(sb);
    }
}
```

- A. 9876-510
- B. 98765-10
- C. 015-6789
- D. 9876-5210

Câu 01-06 · Khó · objective 1.3

Chương trình sau in ra gì?

```
public class Block {
    public static void main(String[] args) {
        String tb = ""
            A\s
            B \
            C
            ""
        System.out.print(tb.replace(' ', '.').replace("\n", "|"));
    }
}
```

- A. A|..B.C|
- B. A.|..B.C|
- C. A.|..B.|C|
- D. A.|B.C|

Câu 01-07 · Vừa · objective 1.4

Chương trình sau in ra gì?

```
import java.time.LocalDate;

public class Months {
    public static void main(String[] args) {
        LocalDate d = LocalDate.of(2023, 1, 31);
        d.plusDays(1);
        LocalDate e = d.plusMonths(1);
        System.out.println(d + " " + e);
    }
}
```

- A. 2023-02-01 2023-03-01
- B. 2023-01-31 2023-03-03
- C. Ném DateTimeException lúc chạy
- D. 2023-01-31 2023-02-28

Câu 01-08 · Khó · objective 1.4

Ở múi giờ America/New_York, ngày 2024-03-10 đồng hồ nhảy từ 02:00 lên 03:00. Chương trình sau in ra gì?

```
import java.time.*;

public class Dst {
    public static void main(String[] args) {
        ZoneId z = ZoneId.of("America/New_York");
        ZonedDateTime t = ZonedDateTime.of(2024, 3, 10, 1, 45, 0, 0, z);
        ZonedDateTime a = t.plusMinutes(30);
        ZonedDateTime b = t.plusDays(1).minusHours(24);
        System.out.println(a.toLocalTime() + " " + b.toLocalTime());
    }
}
```

- A. 03:15 00:45
- B. 03:15 01:45
- C. 02:15 01:45
- D. 02:15 00:45

Câu 01-09 · Vừa · objective 1.4

Chương trình sau in ra gì?

```
import java.time.*;

public class Amounts {
    public static void main(String[] args) {
        Period p = Period.ofMonths(1).ofDays(10);
        Duration d = Duration.ofHours(25);
        System.out.println(p + " " + d);
    }
}
```

- A. P1M10D PT25H
- B. P10D PT25H
- C. P1M10D P1DT1H
- D. P10D P1DT1H

Câu 01-10 · Vừa · objective 1.1

Chèn dòng nào vào chỗ `// INSERT CODE HERE` thì chương trình biên dịch được? (Chọn 2 đáp án.)

```
public class Decl {
    public static void main(String[] args) {
        // INSERT CODE HERE
    }
}
```

- A. Long a = 5;
- B. float b = 2.5;
- C. char c = 65;
- D. int d = 1_000_;
- E. double e = 0x1F;
- F. byte f = 128;

Câu 01-11 · Khó · objective 1.2

Chương trình sau in ra gì?

```
public class Rounding {
    public static void main(String[] args) {
        System.out.println(Math.round(-3.5) + " " + Math.round(3.49f) + " "
            + Math.floor(-3.5) + " " + Math.ceil(-3.5));
    }
}
```

- A. -4 3 -4.0 -3.0
- B. -3 3 -4.0 -3.0
- C. -3 3.0 -4.0 -3.0
- D. -4 3 -3.0 -4.0

Câu 01-12 · Vừa · objective 1.2

Chương trình sau in ra gì?

```
public class ShortCircuit {
    public static void main(String[] args) {
        int a = 0, b = 0;
        boolean r = (a++ > 0) && (b++ > 0);
        boolean s = (a++ > 0) | (b++ > 0);
        System.out.println(a + " " + b + " " + r + " " + s);
    }
}
```

- A. 2 2 false true
- B. 1 1 false true
- C. 2 1 false false
- D. 2 1 false true

Câu 01-13 · Khó · objective 1.1

Hai phát biểu nào đúng với Java 21? (Chọn 2 đáp án.)

- A. `Integer.valueOf(127) == Integer.valueOf(127)` luôn là `true`.
- B. `new Integer(5)` không còn biên dịch được trong Java 21.
- C. `Long.valueOf(10).equals(10)` trả về `true`.
- D. Unboxing một biến `Integer` có giá trị `null` ném `NullPointerException`.
- E. `Character.isLetter('9')` trả về `true`.

Câu 01-14 · Dễ · objective 1.3

Chương trình sau in ra gì?

```
public class Sub {
    public static void main(String[] args) {
        String s = "Programming";
        System.out.println(s.substring(3, 6) + s.indexOf('m') + s.charAt(s.length() - 1));
    }
}
```

- A. `gra6g`
- B. `gram6g`
- C. `gra7g`
- D. `ogr6g`

Câu 01-15 · Khó · objective 1.3

Chèn dòng nào vào chỗ `// INSERT CODE HERE` thì chương trình in ra `true`? (Chọn 2 đáp án.)

```
public class Pool {
    public static void main(String[] args) {
        String a = "hello";
        String b = "hel";
        // INSERT CODE HERE
    }
}
```

- A. `System.out.println(a == "hel" + "lo");`
- B. `System.out.println(a == b + "lo");`
- C. `System.out.println(a == (b + "lo").intern());`
- D. `System.out.println(a == new String("hello"));`
- E. `System.out.println(new StringBuilder(a).equals(new StringBuilder(a)));`

Câu 01-16 · Vừa · objective 1.4

Kết quả của chương trình là gì?

```
import java.time.*;

public class AddDuration {
    public static void main(String[] args) {
        LocalDate d = LocalDate.of(2024, 5, 20);
        LocalDate e = d.plus(Duration.ofDays(2));
        System.out.println(e);
    }
}
```

- A. In ra `2024-05-22`
- B. Không biên dịch được
- C. Ném `UnsupportedTemporalTypeException` lúc chạy
- D. In ra `2024-05-20`

Câu 01-17 · Khó · objective 1.2

Chương trình sau in ra gì?

```
public class Chars {
    public static void main(String[] args) {
        char c = 'A';
        c += 2;
        int i = c + 1;
        c++;
        System.out.println(c + " " + i + " " + (char) i + (c + 1));
    }
}
```

- A. `D 68 D69`

- B. D 68 DE
- C. E 68 D69
- D. D 68 68E

Câu 01-18 · Vừa · objective 1.4

Hai phát biểu nào đúng? (Chọn 2 đáp án.)

- A. `LocalTime.of(10, 0).plus(Period.ofDays(1))` ném exception lúc chạy.
- B. `Instant` có method `getYear()`.
- C. `Duration.between(t1, t2)` có thể trả về giá trị âm.
- D. `Period.between(...)` nhận hai tham số `LocalDateTime`.
- E. `LocalDate.of(2024, 13, 1)` trả về 2025-01-01.

Câu 01-19 · Dễ · objective 1.1

Chương trình sau in ra gì?

```
public class Defaults {
    static boolean flag;
    static double value;
    static String name;

    public static void main(String[] args) {
        System.out.println(flag + " " + value + " " + name);
    }
}
```

- A. false 0 null
- B. null null null
- C. Không biên dịch được vì field chưa được khởi tạo
- D. false 0.0 null

Câu 01-20 · Khó · objective 1.1

Những dòng nào gây lỗi biên dịch?

```
public class Lits {
    public static void main(String[] args) {
        long a = 2_147_483_648L; // L1
        int b = 2_147_483_648; // L2
        float c = 1e3f; // L3
        double d = 1_0_0_1; // L4
        int e = 0b1_0; // L5
        int f = 0x_1F; // L6
    }
}
```

- A. Chỉ L2
- B. L2 và L4
- C. L2 và L6
- D. L4 và L6
- E. L2, L4 và L6

Lời giải

Câu 01-01 — Đáp án: B

- **Vì sao đúng:** Autoboxing gọi `Integer.valueOf`, method này luôn cache các giá trị từ -128 đến 127. Hai biến `a`, `b` trở cùng một object nên `a == b` là `true`. 1000 nằm ngoài cache nên `c` và `d` là hai object khác nhau: `==` là `false`, còn `equals` so sánh giá trị nên `true`.
- **A sai:** `c == d` so sánh tham chiếu; 1000 nằm ngoài cache nên là hai object khác nhau.
- **C sai:** 100 nằm trong khoảng cache -128..127, nên `a == b` là `true`.
- **D sai:** `Integer.equals` so sánh giá trị số, 1000 bằng 1000 nên `true`.
- **Kiểm chứng:** `examples/questions/ch01/Q01_01/` — output confirmed (python3 tools/book.py questions ch01).

Câu 01-02 — Đáp án: C

- **Vì sao đúng:** `s * 2` được nâng kiểu (numeric promotion) thành `int`; gán `int` cho `short` mà không cast là lỗi. Các dòng khác hợp lệ: `+=` và `++` tự ép kiểu về kiểu của biến; `k + 1` là hằng số compile-time (vì `k` là `final` và được gán hằng số) với giá trị 4 vừa với `byte`.
- **A sai:** Toán tử gán kết hợp `+=` có ép kiểu ngầm: `s += 5` tương đương `s = (short)(s + 5)`.
- **B sai:** `k` là `final byte` gán bằng hằng số, nên `k + 1` là hằng số compile-time 4, vừa `byte` → hợp lệ.
- **D sai:** L4 hợp lệ (xem giải thích B).

- **E sai:** L2 gán kết quả `int` cho `short` nên không biên dịch.
- **Kiểm chứng:** `examples/questions/ch01/Q01_02/` — compile error confirmed at `['L2']` (python3 tools/book.py questions ch01).

Câu 01-03 — Đáp án: C

- **Vì sao đúng:** Biểu thức được đánh giá từ trái sang phải. `i++` cho giá trị 3 rồi `i` thành 4. `3 * 2 = 6`. Sau đó `--i` giảm `i` về 3 và cho giá trị 3. `j = 6 + 3 = 9`, `i = 3`.
- **A sai:** `--i` chạy sau `i++`, nên `i` cuối cùng là 3, không phải 4.
- **B sai:** `--i` cho giá trị 3 (không phải 2), vì `i` đã là 4 sau `i++`.
- **D sai:** `i` cuối cùng là 3 và `j` là `6 + 3 = 9`.
- **Kiểm chứng:** `examples/questions/ch01/Q01_03/` — output confirmed (python3 tools/book.py questions ch01).

Câu 01-04 — Đáp án: D

- **Vì sao đúng:** String bất biến: `concat` và `toUpperCase` trả về chuỗi mới nhưng kết quả không được gán lại, nên `s` vẫn là `"java"`. `StringBuilder` thay đổi được: `append` sửa trực tiếp `sb`.
- **A sai:** Kết quả của `concat/toUpperCase` bị bỏ đi; `s` không đổi.
- **B sai:** Như trên — `s.concat(...)` không sửa `s`.
- **C sai:** `sb.append` sửa chính object `sb`, không cần gán lại.
- **Kiểm chứng:** `examples/questions/ch01/Q01_04/` — output confirmed (python3 tools/book.py questions ch01).

Câu 01-05 — Đáp án: A

- **Vì sao đúng:** `delete(2, 5)` xoá chỉ số 2, 3, 4 → `"0156789"`. `insert(3, "-")` chèn trước chỉ số 3 → `"015-6789"`. `reverse()` đảo ngược → `"9876-510"`. Các method trả về cùng object nên nối chuỗi (chaining) được.
- **B sai:** Dấu `-` được chèn ở chỉ số 3 của `"0156789"`, tức là giữa 5 và 6, nên sau khi đảo là 9876-510.
- **C sai:** Đây là chuỗi trước khi `reverse()`; `reverse()` sửa chính builder.
- **D sai:** `delete(2, 5)` xoá cả ký tự 2 (chỉ số 2), nên không còn 2 trong kết quả.
- **Kiểm chứng:** `examples/questions/ch01/Q01_05/` — output confirmed (python3 tools/book.py questions ch01).

Câu 01-06 — Đáp án: B

- **Vì sao đúng:** Thụt lề chung (4 dấu cách, tính cả dòng `"""` đóng) bị bỏ. `\s` được xử lý sau khi xoá khoảng trắng cuối dòng nên giữ lại một dấu cách sau A. Dòng B giữ 2 dấu cách thụt lề thêm; dấu cách trước `\` không phải khoảng trắng cuối dòng nên được giữ, và `\` nối dòng với C. Dòng `"""` đóng riêng → có `\n` cuối. Kết quả `"A \n B C\n"`.
- **A sai:** `\s` giữ lại một dấu cách sau A, nên phải có `.` sau A.
- **C sai:** `\` ở cuối dòng là nối dòng (line continuation): không có xuống dòng giữa B và C.
- **D sai:** Chỉ bỏ phần thụt lề chung (4 dấu cách); dòng B thụt thêm 2 dấu cách nên còn `..B`.
- **Kiểm chứng:** `examples/questions/ch01/Q01_06/` — output confirmed (python3 tools/book.py questions ch01).

Câu 01-07 — Đáp án: D

- **Vì sao đúng:** `LocalDate` bất biến nên `d.plusDays(1)` không đổi `d`. `plusMonths(1)` từ 31/1/2023: tháng 2/2023 chỉ có 28 ngày, nên kết quả được điều chỉnh về ngày hợp lệ cuối cùng: 28/2/2023.
- **A sai:** Kết quả của `plusDays(1)` không được gán, `d` vẫn là 2023-01-31.
- **B sai:** `plusMonths` không “tràn” sang tháng sau; nó lấy ngày cuối tháng hợp lệ.
- **C sai:** `plusMonths` tự điều chỉnh ngày, không ném exception (khác với `LocalDate.of(2023, 2, 31)`).
- **Kiểm chứng:** `examples/questions/ch01/Q01_07/` — output confirmed (python3 tools/book.py questions ch01).

Câu 01-08 — Đáp án: A

- **Vì sao đúng:** `plusMinutes(30)` cộng thời gian thật: 01:45 EST + 30 phút = thời điểm mà đồng hồ hiển thị 03:15 EDT (02:15 không tồn tại). `plusDays(1)` giữ giờ địa phương: 2024-03-11 01:45 EDT. Trừ 24 giờ thật quay về 2024-03-10 lúc 00:45 EST, vì ngày 10/3 chỉ có 23 giờ.
- **B sai:** `plusDays(1)` rồi `minusHours(24)` không triệt tiêu nhau: ngày 10/3 chỉ dài 23 giờ, nên ra 00:45.
- **C sai:** 02:15 không tồn tại ngày hôm đó (nằm trong khoảng nhảy giờ - gap).
- **D sai:** 02:15 không tồn tại; `ZonedDateTime` luôn cho giờ hợp lệ là 03:15.
- **Kiểm chứng:** `examples/questions/ch01/Q01_08/` — output confirmed (python3 tools/book.py questions ch01).

Câu 01-09 — Đáp án: B

- **Vì sao đúng:** `ofDays` là method **static**; gọi nó qua kết quả của `ofMonths(1)` vẫn chỉ tạo `Period` 10 ngày, kết quả `ofMonths(1)` bị bỏ. `Duration.toString()` chỉ dùng giờ/phút/giây: 25 giờ in là `PT25H`.
- **A sai:** `Period.ofMonths(1).ofDays(10)` không cộng dồn: `ofDays` là static nên chỉ còn `P10D`.
- **C sai:** Cả hai phần đều sai: `P10D` (xem A) và `Duration` không bao giờ in phần ngày D trước T.
- **D sai:** `Duration.toString()` không đổi giờ ra ngày; 25 giờ là `PT25H`.
- **Kiểm chứng:** `examples/questions/ch01/Q01_09/` — output confirmed (python3 tools/book.py questions ch01).

Câu 01-10 — Đáp án: C, E

- **Vì sao đúng:** `char c = 65;` 65 là hằng số `int` vừa khoảng của `char` nên được gán không cần cast. `double e = 0x1F;` literal hex kiểu `int` (31) được mở rộng (widening) thành `double`.
- **A sai:** 5 là `int`; Java không nới rộng rồi mới boxing, nên không thể thành `Long`. Cần 5L.
- **B sai:** 2.5 là `double`; gán cho `float` cần 2.5f hoặc cast.
- **D sai:** Dấu `_` không được đứng cuối literal.
- **F sai:** 128 vượt quá `byte` (-128..127), cần cast.
- **Kiểm chứng:** `examples/questions/ch01/Q01_10/` — variants: CE satisfy compile (python3 tools/book.py questions ch01).

Câu 01-11 — Đáp án: B

- **Vì sao đúng:** `Math.round` làm tròn “nửa lên phía dương vô cực”: `round(-3.5) = floor(-3.5 + 0.5) = -3`, kiểu `long`. `round(3.49f) = 3` kiểu `int`. `floor` và `ceil` trả về `double`: `floor(-3.5) = -4.0`, `ceil(-3.5) = -3.0`.
- **A sai:** `Math.round(-3.5)` là -3 (làm tròn về phía dương), không phải -4.
- **C sai:** `Math.round(float)` trả về `int`, in ra 3, không có .0.
- **D sai:** `floor` luôn đi xuống (-4.0), `ceil` luôn đi lên (-3.0); và `round(-3.5)` là -3.
- **Kiểm chứng:** `examples/questions/ch01/Q01_11/` — output confirmed (python3 tools/book.py questions ch01).

Câu 01-12 — Đáp án: D

- **Vì sao đúng:** Dòng 1: `a++ > 0` là `0 > 0 = false` (a thành 1); `&&` short-circuit nên `b++` **không** chạy; `r = false`. Dòng 2: `|` luôn đánh giá cả hai vế: `1 > 0 = true` (a thành 2), `0 > 0 = false` (b thành 1); `s = true`.
- **A sai:** `b++` ở dòng 1 bị bỏ qua do `&&`, nên b chỉ tăng một lần.
- **B sai:** `a++` chạy ở cả hai dòng, nên `a = 2`.
- **C sai:** `true | false` là `true`.
- **Kiểm chứng:** `examples/questions/ch01/Q01_12/` — output confirmed (python3 tools/book.py questions ch01).

Câu 01-13 — Đáp án: A, D

- **Vì sao đúng:** Javadoc của `Integer.valueOf` bảo đảm luôn cache -128..127, nên cùng một object. Unboxing `null` gọi `intValue()` trên `null` → `NullPointerException`.
- **B sai:** Constructor `Integer(int)` bị đánh dấu deprecated (for removal) nhưng vẫn biên dịch và chạy trong Java 21 (chỉ có cảnh báo).
- **C sai:** 10 được boxing thành `Integer`; `Long.equals` trả về `false` khi kiểu khác nhau.
- **E sai:** '9' là chữ số (digit), không phải chữ cái.
- **Kiểm chứng:** `examples/questions/ch01/Q01_13/` — each option proven true/false by a program (python3 tools/book.py questions ch01).

Câu 01-14 — Đáp án: A

- **Vì sao đúng:** `substring(3, 6)` lấy chỉ số 3, 4, 5 → "gra" (chỉ số bắt đầu từ 0, end không lấy). `indexOf('m')` trả về vị trí **đầu tiên** là 6. Ký tự cuối (`length() - 1 = 10`) là 'g'.
- **B sai:** `substring(3, 6)` không lấy chỉ số 6, nên không có m.
- **C sai:** `indexOf` trả về lần xuất hiện đầu tiên (6), không phải lần cuối (7).
- **D sai:** Chỉ số bắt đầu từ 0: chỉ số 3 là g, không phải o.
- **Kiểm chứng:** `examples/questions/ch01/Q01_14/` — output confirmed (python3 tools/book.py questions ch01).

Câu 01-15 — Đáp án: A, C

- **Vì sao đúng:** A: + ưu tiên cao hơn `==`, nên so sánh `a == "hello"; "hel" + "lo"` là hằng số compile-time nên dùng chung object trong string pool → `true`. C: `intern()` trả về object trong pool, chính là a → `true`.
- **B sai:** b không phải `final`, nên `b + "lo"` được tạo lúc chạy thành object mới → `false`.
- **D sai:** `new String(...)` luôn tạo object mới → `false`.
- **E sai:** `StringBuilder` không override `equals`, nên so sánh tham chiếu của hai object khác nhau → `false`.
- **Kiểm chứng:** `examples/questions/ch01/Q01_15/` — variants: AC satisfy output (python3 tools/book.py questions ch01).

Câu 01-16 — Đáp án: C

- **Vì sao đúng:** `plus(TemporalAmount)` nhận cả `Period` lẫn `Duration` nên biên dịch được. Nhưng `Duration` cộng theo đơn vị giây, mà `LocalDate` không có phần giờ → ném `UnsupportedTemporalTypeException: Unsupported unit: Seconds`.
- **A sai:** Muốn cộng 2 ngày cho `LocalDate` phải dùng `plusDays(2)` hoặc `Period.ofDays(2)`.
- **B sai:** `Duration` là một `TemporalAmount`, nên lời gọi `plus` hợp lệ về kiểu.
- **D sai:** Chương trình không in gì vì exception xảy ra trước `println`.
- **Kiểm chứng:** `examples/questions/ch01/Q01_16/` — output confirmed (python3 tools/book.py questions ch01).

Câu 01-17 — Đáp án: A

- **Vì sao đúng:** `c += 2` → `'C'.i = 'C' + 1 = 68`. `c++` → `'D'`. Khi in: `c + " "` là nối chuỗi `"D "`, rồi `68`, rồi `(char) 68 = 'D'`, rồi `(c + 1)` trong ngoặc được tính trước theo kiểu `int = 69`.
- **B sai:** `(c + 1)` là phép cộng `char + int` → `int 69`, không phải ký tự `E`.
- **C sai:** `c` chỉ tăng một lần bằng `c++` sau khi đã là `'C'`, nên là `'D'`.
- **D sai:** `(char) i` in ký tự `D`, còn `(c + 1)` in số `69`.
- **Kiểm chứng:** `examples/questions/ch01/Q01_17/` — output confirmed (python3 tools/book.py questions ch01).

Câu 01-18 — Đáp án: A, C

- **Vì sao đúng:** `A: LocalDateTime` không có đơn vị ngày nên cộng `Period` ngày ném `UnsupportedTemporalTypeException`. `C`: nếu mốc thứ hai sớm hơn mốc thứ nhất, `Duration.between` trả về giá trị âm (ví dụ `PT-1H`).
- **B sai:** `Instant` chỉ là số giây từ epoch (UTC), không có method `getYear()` → lỗi biên dịch.
- **D sai:** `Period.between` chỉ nhận hai `LocalDate` → truyền `LocalDateTime` là lỗi biên dịch.
- **E sai:** Tháng 13 không hợp lệ → `DateTimeException; of` không tự “tràn” sang năm sau.
- **Kiểm chứng:** `examples/questions/ch01/Q01_18/` — each option proven true/false by a program (python3 tools/book.py questions ch01).

Câu 01-19 — Đáp án: D

- **Vì sao đúng:** `Field` (static hoặc instance) luôn có giá trị mặc định: `boolean` → `false`, `double` → `0.0`, `object` → `null`. Chỉ biến **cục bộ** mới phải gán trước khi dùng.
- **A sai:** `double` mặc định là `0.0`, và khi in ra có phần `.0`.
- **B sai:** Primitive không bao giờ là `null`.
- **C sai:** Quy tắc “phải gán trước khi dùng” chỉ áp dụng cho biến cục bộ, không cho field.
- **Kiểm chứng:** `examples/questions/ch01/Q01_19/` — output confirmed (python3 tools/book.py questions ch01).

Câu 01-20 — Đáp án: C

- **Vì sao đúng:** `L2`: literal `int` lớn nhất là `2 147 483 647`; `2 147 483 648` không có hậu tố `L` → “integer number too large”. `L6`: `_` không được đứng ngay sau tiền tố `0x`. Các dòng khác hợp lệ: `_` nằm giữa hai chữ số (`L4` có `1_0` và `0_1`), `1e3f` là `float` hợp lệ.
- **A sai:** `L6` cũng lỗi: `_` phải nằm giữa hai chữ số, không được sát `0x`.
- **B sai:** `L4` hợp lệ: mỗi `_` đều nằm giữa hai chữ số.
- **D sai:** `L4` hợp lệ, còn `L2` thì lỗi (literal int quá lớn).
- **E sai:** `L4` hợp lệ (xem B).
- **Kiểm chứng:** `examples/questions/ch01/Q01_20/` — compile error confirmed at [`'L2'`, `'L6'`] (python3 tools/book.py questions ch01).

Đọc thêm (link chính thức — bị chặn trong sandbox nên mình chưa mở được)

- JLS §4.2 Primitive Types, §5.1 Conversions, §5.6 Numeric Promotion, §15 Expressions: <https://docs.oracle.com/javase/specs/jls/se21/html/jls-5.html>
- Javadoc `java.lang.String`: <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/String.html>
- Javadoc gói `java.time`: <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/time/package-summary.html>

Nguồn tham khảo (Sources)

Các link dưới đây mình đã mở (mã nguồn JDK 21 chứa Javadoc gốc), ngày 2026-09-28:

- `Integer.java` (Javadoc của `valueOf` và `IntegerCache`): <https://raw.githubusercontent.com/openjdk/jdk21u/master/src/java.base/share/classes/java/lang/Integer.java>
- `String.java`: <https://raw.githubusercontent.com/openjdk/jdk21u/master/src/java.base/share/classes/java/lang/String.java>
- `Math.java` (`round`, `floorMod`, `addExact`): <https://raw.githubusercontent.com/openjdk/jdk21u/master/src/java.base/share/classes/java/lang/Math.java>
- `ZonedDateTime.java` (Javadoc về `gap/overlap`): <https://raw.githubusercontent.com/openjdk/jdk21u/master/src/java.base/share/classes/java/time/ZonedDateTime.java>
- `Period.java`: <https://raw.githubusercontent.com/openjdk/jdk21u/master/src/java.base/share/classes/java/time/Period.java>

Chương 2 — Luồng điều khiển (Controlling Program Flow)

Mục tiêu

Objective 2.1: viết đúng và **đọc đúng** các cấu trúc điều khiển:

- `if / else`, toán tử ba ngôi.
- `switch statement` (câu lệnh) và `switch expression` (biểu thức), dạng `:` và dạng `->`.
- Pattern matching trong `switch` (Java 21): `type pattern`, `record pattern`, `guard when`, `case null`.
- Vòng lặp `while`, `do-while`, `for`, `for-each`; `break`, `continue`, nhãn (label).
- Phát hiện code không thể tới (unreachable) và `switch` không đầy đủ (not exhaustive).

Giải thích đơn giản

Chương trình chạy từ trên xuống. **Luồng điều khiển (control flow)** là cách ta đổi thứ tự đó: chọn nhánh (`if`, `switch`) hoặc lặp lại (`for`, `while`).

Hai ý quan trọng nhất cho đề thi:

1. **Switch có hai “thế giới”**. Dạng cũ `case X`: có **fall-through** — chạy xong một case sẽ *rơi xuống* case sau nếu không có `break`. Dạng mới `case X ->` không bao giờ rơi xuống. Switch có thể là **statement** (chỉ làm việc) hoặc **expression** (trả về giá trị, phải đầy đủ mọi trường hợp).
2. **Pattern matching**. o `instanceof String s` vừa kiểm tra kiểu vừa tạo biến `s`. Trong Java 21, `switch` cũng làm được như vậy: `case String s ->`, kèm điều kiện `when`, và “mở” record: `case Point(int x, int y) ->`.

Ví dụ

Code trong `examples/ch02/`. Chạy lại: `python3 tools/book.py examples ch02`. Output thật, JDK 21.0.10.

1. `if / else` và “dangling else”

`examples/ch02/Ex01_IfElse.java`

```
// objective: 2.1
// if / else if / else, "dangling else", và điều kiện phải là boolean.
public class Ex01_IfElse {
    static String grade(int score) {
        if (score >= 90) return "A";
        else if (score >= 80) return "B";
        else if (score >= 50) return "C";
        else return "F";
    }

    public static void main(String[] args) {
        System.out.println(grade(95) + grade(85) + grade(50) + grade(10));

        int x = 5;
        if (x > 10)
            if (x > 20) System.out.println("> 20");
        else System.out.println("else thuộc về if GẦN NHẤT (x > 20)"); // không in gì!
        System.out.println("sau khối if lồng nhau");

        boolean done = false;
        if (done = true) { // gán, không phải so sánh — vẫn hợp lệ vì kiểu boolean
            System.out.println("done = " + done);
        }
        // if (x = 3) {} // lỗi biên dịch: int không phải boolean
    }
}
```

Output thật (JDK 21.0.10):

```
ABCF
sau khối if lồng nhau
done = true
```

2. Switch statement cổ điển và fall-through

`examples/ch02/Ex02_SwitchStatement.java`

```
// objective: 2.1
// switch statement kiểu cũ: rơi xuống (fall-through), default ở giữa, case phải là hằng số.
public class Ex02_SwitchStatement {
    static void test(int day) {
        final int WEEKEND = 6;
        System.out.print(day + ": ");
        switch (day) {
            case 1:
                System.out.print("Mon ");
            case 2:
                System.out.print("Tue ");
                break;
            default:
                System.out.print("Other ");
            case WEEKEND, 7: // nhiều nhãn trên một case (Java 14+)
                System.out.print("Weekend ");
        }
        System.out.println();
    }

    public static void main(String[] args) {
        test(1);
        test(2);
        test(4);
        test(7);

        String cmd = "stop";
        switch (cmd) { // switch trên String dùng equals()
            case "start" -> System.out.println("starting");
            case "stop" -> System.out.println("stopping");
            default -> System.out.println("unknown");
        }
        char c = 'b';
        switch (c) { case 'a': case 'b': System.out.println("a or b"); }
    }
}
```

Output thật (JDK 21.0.10):

```
1: Mon Tue
2: Tue
4: Other Weekend
7: Weekend
stopping
a or b
```

3. Switch expression, yield

examples/ch02/Ex03_SwitchExpression.java

```
// objective: 2.1
// switch expression: trả về giá trị, dùng -> hoặc yield, phải đầy đủ (exhaustive).
public class Ex03_SwitchExpression {
    enum Size { S, M, L, XL }

    static int price(Size s) {
        return switch (s) { // enum: liệt kê đủ hằng số thì không cần default
            case S, M -> 10;
            case L -> 12;
            case XL -> {
                int base = 12;
                yield base + 3; // khối { } phải dùng yield
            }
        };
    }

    public static void main(String[] args) {
        for (Size s : Size.values()) System.out.print(s + "=" + price(s) + " ");
        System.out.println();

        int n = 3;
        String word = switch (n) {
            case 1: yield "one"; // dạng ':' cũng dùng được trong switch expression
            case 2: yield "two";
            default: yield "many";
        };
        System.out.println(word);

        var kind = switch ("b") {
            case "a" -> 1;
            case "b" -> 2.5; // kiểu chung của các nhánh: double
            default -> 0;
        };
        System.out.println(kind);
    }
}
```

```
}  
}
```

Output thật (JDK 21.0.10):

```
S=10 M=10 L=12 XL=15  
many  
2.5
```

4. Pattern matching cho switch

examples/ch02/Ex04_PatternSwitch.java

```
// objective: 2.1, 3.5  
// Pattern matching cho switch (Java 21): type pattern, guard "when", case null.  
public class Ex04_PatternSwitch {  
    static String describe(Object o) {  
        return switch (o) {  
            case null -> "null!";  
            case Integer i when i > 100 -> "big int " + i;  
            case Integer i -> "int " + i;  
            case String s when s.isEmpty() -> "empty string";  
            case String s -> "string of " + s.length();  
            case int[] arr -> "int array of " + arr.length;  
            default -> "other " + o.getClass().getSimpleName();  
        };  
    }  
  
    public static void main(String[] args) {  
        Object[] data = {7, 500, "", "hello", new int[3], 2.5, null};  
        for (Object o : data) System.out.println(describe(o));  
  
        Object o = "abc";  
        try {  
            switch (o) {  
                case Integer i -> System.out.println("int");  
                default -> System.out.println("default, không null");  
            }  
            o = null;  
            switch (o) {  
                // không có case null -> NPE  
                case Integer i -> System.out.println("int");  
                default -> System.out.println("default");  
            }  
        } catch (NullPointerException e) {  
            System.out.println("NullPointerException: switch trên null không có case null");  
        }  
    }  
}
```

Output thật (JDK 21.0.10):

```
int 7  
big int 500  
empty string  
string of 5  
int array of 3  
other Double  
null!  
default, không null  
NullPointerException: switch trên null không có case null
```

5. Các loại vòng lặp

examples/ch02/Ex05_Loops.java

```
// objective: 2.1  
// while, do-while (luôn chạy ít nhất 1 lần), for nhiều biến, for-each.  
public class Ex05_Loops {  
    public static void main(String[] args) {  
        int i = 10;  
        while (i < 10) { System.out.println("while"); i++; }  
        do { System.out.println("do-while chạy 1 lần, i=" + i); i++; } while (i < 10);  
  
        for (int a = 0, b = 10; a < b; a += 3, b -= 3) {  
            System.out.print("(" + a + ", " + b + ") ");  
        }  
        System.out.println();  
  
        int[] nums = {1, 2, 3};  
        for (int n : nums) { n *= 10; } // n là bản sao: mảng không đổi  
        System.out.println(nums[0] + " " + nums[1] + " " + nums[2]);  
    }  
}
```

```

int count = 0;
for (;;) {                                     // vòng lặp vô hạn, thoát bằng break
    if (++count == 4) break;
}
System.out.println("count=" + count);

int k = 0;
while (k++ < 3) System.out.print(k + " ");
System.out.println("-> k=" + k);
}
}

```

Output thật (JDK 21.0.10):

```

do-while chạy 1 lần, i=10
(0,10) (3,7)
1 2 3
count=4
1 2 3 -> k=4

```

6. Nhãn (label) với break / continue

examples/ch02/Ex06_Labels.java

```

// objective: 2.1
// break / continue có nhãn (label) trong vòng lặp lồng nhau.
public class Ex06_Labels {
    public static void main(String[] args) {
        int[][] grid = {{1, 2, 3}, {4, -1, 6}, {7, 8, 9}};

        OUTER:
        for (int[] row : grid) {
            for (int v : row) {
                if (v < 0) break OUTER;           // thoát cả hai vòng
                System.out.print(v + " ");
            }
            System.out.println("| sau break OUTER");

            ROW:
            for (int r = 0; r < 3; r++) {
                for (int c = 0; c < 3; c++) {
                    if (c > r) continue ROW;       // sang hàng tiếp theo
                    System.out.print(grid[r][c] + " ");
                }
                System.out.println("| tam giác dưới");

                BLOCK: {
                    System.out.print("trong block ");
                    if (grid.length == 3) break BLOCK; // break có nhãn dùng được với block thường
                    System.out.print("không in ");
                }
                System.out.println("| hết block");
            }
        }
    }
}

```

Output thật (JDK 21.0.10):

```

1 2 3 4 | sau break OUTER
1 4 -1 7 8 9 | tam giác dưới
trong block | hết block

```

7. instanceof pattern và flow scoping

examples/ch02/Ex07_FlowScoping.java

```

// objective: 2.1, 3.5
// Pattern matching với instanceof và "flow scoping": biến pattern có mặt ở đâu?
public class Ex07_FlowScoping {
    static int len(Object o) {
        if (!(o instanceof String s)) {
            return -1;
        }
        return s.length();                     // s dùng được: nhánh trên luôn return
    }

    public static void main(String[] args) {
        System.out.println(len("hello") + " " + len(42));

        Object o = "Java";
        if (o instanceof String s && s.length() > 3) { // && : s đã được gán khi về phải chạy

```

```

        System.out.println("dài: " + s.toUpperCase());
    }
    // if (o instanceof String s || s.isEmpty()) {} // lỗi: s có thể chưa được gán

    Number n = 3.5;
    if (n instanceof Integer i) System.out.println("Integer " + i);
    else if (n instanceof Double d) System.out.println("Double " + (d * 2));

    String str = "x";
    if (str instanceof String t) System.out.println("Java 21 cho phép pattern không điều kiện: " + t);
    System.out.println(str instanceof CharSequence cs ? "CharSequence " + cs.length() : "no");
}
}

```

Output thật (JDK 21.0.10):

```

5 -1
dài: JAVA
Double 7.0
Java 21 cho phép pattern không điều kiện: x
CharSequence 1

```

8. Record pattern

examples/ch02/Ex08_RecordPatterns.java

```

// objective: 2.1, 3.5
// Record pattern (Java 21): "mở" record ngay trong instanceof và switch, kể cả lồng nhau.
public class Ex08_RecordPatterns {
    record Point(int x, int y) {}
    record Line(Point from, Point to) {}

    sealed interface Shape permits Circle, Square {}
    record Circle(double r) implements Shape {}
    record Square(double side) implements Shape {}

    static double area(Shape s) {
        return switch (s) {
            // sealed + record: đủ các trường hợp, không cần default
            case Circle(double r) -> Math.PI * r * r;
            case Square(var side) -> side * side;
        };
    }

    public static void main(String[] args) {
        Object o = new Line(new Point(0, 0), new Point(3, 4));
        if (o instanceof Line(Point(var x1, var y1), Point(int x2, int y2))) {
            System.out.println("length = " + Math.hypot(x2 - x1, y2 - y1));
        }
        System.out.printf("%.2f %.2f\n", area(new Circle(1)), area(new Square(3)));

        Object p = new Point(5, 0);
        String where = switch (p) {
            case Point(int x, int y) when y == 0 -> "trên trục X tại " + x;
            case Point(int x, int y) -> "điểm " + x + ", " + y;
            default -> "không phải Point";
        };
        System.out.println(where);
    }
}

```

Output thật (JDK 21.0.10):

```

length = 5.0
3.14 9.00
trên trục X tại 5

```

9. Code không thể tới (unreachable)

examples/ch02/Ex09_Unreachable.java

```

// objective: 2.1
// expect: compile-error
// Code không thể tới (unreachable) là lỗi biên dịch.
public class Ex09_Unreachable {
    public static void main(String[] args) {
        while (true) {
            break;
            System.out.println("sau break");
        }
        for (int i = 0; ; i++) { }
        System.out.println("sau vòng lặp vô hạn");
    }
}

```

```
}  
}
```

Output thật của `javac` (JDK 21.0.10) — cố ý không biên dịch được:

```
Ex09_Unreachable.java:8: error: unreachable statement  
    System.out.println("sau break");  
    ^  
Ex09_Unreachable.java:11: error: unreachable statement  
    System.out.println("sau vòng lặp vô hạn");  
    ^  
2 errors
```

10. Switch expression không đầy đủ

examples/ch02/Ex10_NotExhaustive.java

```
// objective: 2.1  
// expect: compile-error  
// switch expression trên int phải đầy đủ (exhaustive): thiếu default là lỗi.  
public class Ex10_NotExhaustive {  
    public static void main(String[] args) {  
        int n = 2;  
        String s = switch (n) {  
            case 1 -> "one";  
            case 2 -> "two";  
        };  
    }  
}
```

Output thật của `javac` (JDK 21.0.10) — cố ý không biên dịch được:

```
Ex10_NotExhaustive.java:7: error: the switch expression does not cover all possible input values  
    String s = switch (n) {  
        ^  
1 error
```

11. Kiểu của selector trong switch

examples/ch02/Ex11_SwitchTypes.java

```
// objective: 2.1  
// Kiểu được phép trong switch cổ điển và giá trị null với String/enum.  
public class Ex11_SwitchTypes {  
    enum Color { RED, GREEN }  
  
    public static void main(String[] args) {  
        Color c = Color.GREEN;  
        switch (c) {  
            case RED -> System.out.println("red");           // nhãn enum: không viết Color.RED cũng được  
            case Color.GREEN -> System.out.println("green (Java 21 cho phép tên đầy đủ)");  
        }  
  
        Integer boxed = 2;                                // wrapper được unboxing  
        switch (boxed) { case 1 -> System.out.println("1"); case 2 -> System.out.println("2"); default -> {} }  
  
        String s = null;  
        try {  
            switch (s) { case "a" -> System.out.println("a"); default -> System.out.println("d"); }  
        } catch (NullPointerException e) {  
            System.out.println("NPE: switch cổ điển trên String null");  
        }  
  
        long big = 1L;  
        // switch (big) { case 1L -> ... } // lỗi: long/float/double/boolean không dùng trong switch cổ điển  
        System.out.println("long không dùng làm selector (trừ khi là pattern trên Object) " + big);  
    }  
}
```

Output thật (JDK 21.0.10):

```
green (Java 21 cho phép tên đầy đủ)  
2  
NPE: switch cổ điển trên String null  
long không dùng làm selector (trừ khi là pattern trên Object) 1
```

12. continue trong do-while, phạm vi biến vòng lặp

examples/ch02/Ex12_DoWhileScope.java

```
// objective: 2.1
// Phạm vi biến trong vòng lặp, và continue trong do-while vẫn kiểm tra điều kiện.
public class Ex12_DoWhileScope {
    public static void main(String[] args) {
        int i = 0;
        do {
            i++;
            if (i % 2 == 0) continue; // continue nhảy tới phần kiểm tra điều kiện
            System.out.print(i + " ");
        } while (i < 7);
        System.out.println();

        for (int j = 0; j < 2; j++) { int local = j * 10; System.out.print(local + " "); }
        // System.out.println(j); // lỗi: j chỉ sống trong vòng for
        System.out.println();

        int total = 0;
        for (int a = 1; a <= 3; a++)
            for (int b = 1; b <= a; b++)
                total += b;
        System.out.println("total=" + total);
    }
}
```

Output thật (JDK 21.0.10):

```
1 3 5 7
0 10
total=10
```

13. Case bị “che” (dominance)

examples/ch02/Ex13_Dominance.java

```
// objective: 2.1, 3.5
// expect: compile-error
// Thử tư case trong pattern switch: case rộng hơn đứng trước sẽ "che" (dominate) case hẹp hơn.
public class Ex13_Dominance {
    static String f(Object o) {
        return switch (o) {
            case Number num -> "number";
            case Integer i -> "integer"; // không bao giờ tới được
            default -> "other";
        };
    }

    public static void main(String[] args) {
        System.out.println(f(1));
    }
}
```

Output thật của javac (JDK 21.0.10) — cố ý không biên dịch được:

```
Ex13_Dominance.java:8: error: this case label is dominated by a preceding case label
        case Integer i -> "integer"; // không bao giờ tới được
            ^
1 error
```

Đi sâu

Bốn dạng switch

	Dạng case X:	Dạng case X ->
Statement	Fall-through; break để dừng. Không cần đầy đủ (trừ khi dùng pattern / case null)	Không fall-through. Nhánh là một biểu thức, một khối {} hoặc throw
Expression	Mỗi nhánh kết thúc bằng yield value; (hoặc throw). Phải đầy đủ	Nhánh là biểu thức → giá trị; khối {} phải yield. Phải đầy đủ

Không được trộn `:` và `->` trong cùng một switch.

Kiểu của selector

- **Switch cổ điển** (không có pattern): char, byte, short, int, các wrapper tương ứng, String, enum. **Không** dùng long, float, double, boolean.
- **Switch có pattern** (Java 21): selector là bất kỳ kiểu tham chiếu nào (Object, interface sealed...).
- Nhân case cổ điển phải là hằng số compile-time (literal, biến final gán hằng số, hằng enum) và không trùng nhau.

- Java 21 cho phép viết tên đầy đủ của hằng enum: `case Color.GREEN ->`.

Đầy đủ (exhaustiveness)

Phải đầy đủ khi: switch **expression**, hoặc switch statement dùng pattern hoặc `case null`. Cách đạt: có `default`; hoặc liệt kê hết hằng enum; hoặc phủ hết các lớp con của một **sealed** type; hoặc có một pattern không điều kiện (ví dụ `case Object o`).

Pattern trong switch: thứ tự và `null`

- Case được thử **từ trên xuống**. Case rộng hơn đứng trước case hẹp hơn → lỗi *dominated*. Case có guard `when` phải đứng trước case cùng kiểu không guard.
- Selector là `null` → `NullPointerException`, **trừ khi** có `case null` (có thể viết `case null, default ->`). `default` một mình **không** bắt `null`.
- Guard: `case Integer i when i > 10 ->`. Guard là biểu thức boolean bất kỳ.
- Record pattern có thể lồng nhau và dùng `var`: `case Line(Point(var x1, var y1), Point p2) ->`.

Flow scoping của biến pattern

Biến pattern chỉ có mặt ở nơi compiler **chắc chắn** nó đã được gán:

- `if (o instanceof String s && s.isEmpty())` — OK (vẽ phải chỉ chạy khi vẽ trái đúng).
- `if (o instanceof String s || s.isEmpty())` — lỗi.
- `if (!(o instanceof String s)) return;` → sau câu `if`, `s` dùng được.
- Trong Java 21, `str instanceof String t` (kiểu giống hệt) là hợp lệ; Java 17 thì báo lỗi.

Vòng lặp

- `for (init; condition; update):` cả ba phần đều tùy chọn. `for(;;)` là vòng vô hạn. `init` có thể khai báo nhiều biến **cùng kiểu**: `int i = 0, j = 10`.
- For-each dùng cho mảng và mọi `Iterable`. Biến lặp là **bản sao** giá trị (với object: bản sao tham chiếu).
- `do { } while (cond);` — nhớ dấu `;` cuối; thân chạy ít nhất một lần.
- `continue` trong `for` nhảy tới phần `update`; trong `while/do-while` nhảy tới phần kiểm tra điều kiện.
- `break LABEL / continue LABEL` áp dụng cho vòng lặp (hoặc khối, với `break`) mang nhãn đó.

Unreachable code

Compiler báo lỗi nếu một lệnh chắc chắn không bao giờ chạy: sau `break/continue/return/throw` trong cùng khối; sau `while(true)` không có `break`; thân của `while(false)` hay `for(;;false);`. Lưu ý: `if (false) { ... }` **không** bị coi là unreachable (cố ý, để dùng như “conditional compilation”).

Lỗi và bẫy thường gặp (Exam traps)

1. Quên `break` → fall-through. `default` ở giữa vẫn có thể rơi xuống các case bên dưới.
2. Switch expression trên `int/String` thiếu `default` → lỗi biên dịch.
3. Khối `{ }` trong switch expression thiếu `yield` → lỗi.
4. `case Number n` trước `case Integer i` → lỗi *dominated*.
5. `default` không bắt `null` → `NullPointerException` khi selector là `null`.
6. `else` gắn với `if` **gần nhất**, bất kể thụt lề.
7. `if (x = 5)` với `int x` → lỗi; nhưng với `boolean b`, `if (b = true)` biên dịch được.
8. For-each: gán biến lặp không thay đổi mảng.
9. `while (n++ < 3)` — nhớ lần kiểm tra cuối (sai) vẫn tăng `n`.
10. Biến pattern dùng với `||` → lỗi phạm vi.
11. `break` trong switch nằm trong vòng lặp chỉ thoát switch; `continue` thì áp dụng cho vòng lặp.
12. Selector `long` trong switch cổ điển → lỗi.

Góc nhìn từ TypeScript

TypeScript	Java 21	Ghi chú
<code>if (value)</code> với giá trị <code>truthy/falsy</code>	Điều kiện bắt buộc là <code>boolean</code>	Không có <code>truthy/falsy</code> trong Java
<code>switch</code> chỉ có fall-through, so sánh <code>===</code>	Có cả <code>:</code> (fall-through) và <code>-></code> (không fall-through)	<code>-></code> giống cách viết an toàn hơn
Discriminated union + <code>switch (shape.kind) + never</code> để kiểm tra đầy đủ	<code>sealed interface</code> + <code>record</code> + pattern switch	Java kiểm tra đầy đủ ngay khi biên dịch, không cần mẹo <code>never</code>
Type guard: <code>if (typeof x === "string")</code>	<code>if (x instanceof String s)</code>	Java tạo luôn biến <code>s</code> đã có kiểu

TypeScript	Java 21	Ghi chú
Destructuring <code>const {x, y} = point</code>	Record pattern <code>if (o instanceof Point(int x, int y))</code>	Chỉ dùng trong <code>instanceof</code> / <code>switch</code>
<code>for (const x of arr)</code>	<code>for (int x : arr)</code>	Giống nhau
Label <code>outer: + break outer</code>	Giống hệt	Cả hai ngôn ngữ đều có

Tóm tắt

- `:` → fall-through, cần `break`. `->` → không fall-through.
- Switch expression phải đầy đủ; khối `{}` dùng `yield`.
- Pattern switch: thứ tự từ hẹp tới rộng; guard trước case không guard; `null` cần case `null`.
- Flow scoping: biến pattern có mặt ở chỗ chắc chắn đã gán (`&&`, nhánh đối của `if (!...)` `return`).
- `do-while` chạy ít nhất 1 lần; `for-each` không sửa mảng; nhấn dùng cho vòng lặp lồng nhau.
- Unreachable statement là lỗi biên dịch (trừ `if (false)`).

Bài tập (có lời giải)

Mọi đáp án đã được `tools/book.py` biên dịch và chạy để xác nhận (code: `examples/questions/ch02/`).

Câu hỏi

Câu 02-01 · Dễ · objective 2.1

Chương trình sau in ra gì?

```
public class Fall {
    public static void main(String[] args) {
        int x = 2;
        String r = "";
        switch (x) {
            case 1: r += "a";
            case 2: r += "b";
            case 3: r += "c"; break;
            default: r += "d";
        }
        System.out.println(r);
    }
}
```

- A. b
- B. bc
- C. bcd
- D. abc

Câu 02-02 · Dễ · objective 2.1

Chương trình sau in ra gì?

```
public class Weekend {
    public static void main(String[] args) {
        String day = "SAT";
        int n = switch (day) {
            case "MON", "TUE" -> 1;
            case "SAT" -> {
                int k = 5;
                yield k * 2;
            }
            default -> 0;
        };
        System.out.println(n);
    }
}
```

- A. 5
- B. 0
- C. 10
- D. Không biên dịch được

Câu 02-03 · Vừa · objective 2.1

Dòng nào gây lỗi biên dịch?

```
public class Yield {
    public static void main(String[] args) {
        int v = 3;
        String a = switch (v) { case 1 -> "one"; default -> "other"; }; // L1
        String b = switch (v) { case 1: yield "one"; default: yield "other"; }; // L2
        String c = switch (v) { case 1 -> { yield "one"; } default -> "other"; }; // L3
        String d = switch (v) { case 1 -> "one"; case 2 -> "two"; }; // L4
    }
}
```

- A. L1
- B. L2
- C. L3
- D. L4
- E. Không dòng nào

Câu 02-04 · Khó · objective 2.1

Chương trình sau in ra gì?

```
public class Labels {
    public static void main(String[] args) {
        int count = 0;
        outer:
        for (int i = 0; i < 4; i++) {
            for (int j = 0; j < 4; j++) {
                if (j == i) continue outer;
                if (i + j > 4) break outer;
                count++;
            }
        }
        System.out.println(count);
    }
}
```

- A. 6
- B. 4
- C. 5
- D. 3

Câu 02-05 · Dễ · objective 2.1

Chương trình sau in ra gì?

```
public class DoIt {
    public static void main(String[] args) {
        int i = 5;
        do {
            System.out.print(i + " ");
            i -= 2;
        } while (i > 5);
        System.out.println(i);
    }
}
```

- A. 3
- B. 5 3
- C. 5 3 1
- D. 5

Câu 02-06 · Vừa · objective 2.1, 3.5

Chương trình sau in ra gì?

```
public class Guards {
    static String t(Object o) {
        return switch (o) {
            case Integer i when i > 10 -> "L";
            case Integer i -> "S";
            case String s when s.length() > 2 -> "W";
            case CharSequence cs -> "C";
            default -> "D";
        };
    }

    public static void main(String[] args) {
        System.out.println(t(5) + t(50) + t("hi") + t("hello")
            + t(new StringBuilder("x")) + t(1.0));
    }
}
```

```
}  
}
```

- A. SLCWCD
- B. SLWWCD
- C. LLCWCD
- D. SLCWDD

Câu 02-07 · Khó · objective 2.1, 3.5

Chèn nhóm case nào vào chỗ `// INSERT CODE HERE` thì chương trình biên dịch được? (Chọn 2 đáp án.)

```
public class Order {  
    static String f(Object o) {  
        return switch (o) {  
            // INSERT CODE HERE  
        };  
    }  
  
    public static void main(String[] args) {  
        System.out.println(f("x"));  
    }  
}
```

- A. case String s -> "S"; case CharSequence cs -> "C"; default -> "D";
- B. case CharSequence cs -> "C"; case String s -> "S"; default -> "D";
- C. case String s when s.isEmpty() -> "E"; case String s -> "S"; default -> "D";
- D. case String s -> "S"; case String s when s.isEmpty() -> "E"; default -> "D";
- E. case String s -> "S"; case Integer i -> "I";

Câu 02-08 · Vừa · objective 2.1

Chương trình sau in ra gì?

```
public class Loops {  
    public static void main(String[] args) {  
        int[] a = {1, 2, 3};  
        for (int x : a) {  
            x = x * 2;  
        }  
        int sum = 0;  
        for (int i = 0; i < a.length; i++) {  
            a[i] += i;  
            sum += a[i];  
        }  
        System.out.println(sum);  
    }  
}
```

- A. 9
- B. 12
- C. 18
- D. 6

Câu 02-09 · Khó · objective 2.1

Chương trình sau in ra gì?

```
public class DefaultFirst {  
    static String f(int n) {  
        String r = "";  
        switch (n) {  
            default: r += "d";  
            case 1: r += "1";  
            case 2: r += "2"; break;  
            case 3: r += "3";  
        }  
        return r;  
    }  
  
    public static void main(String[] args) {  
        System.out.println(f(1) + " " + f(3) + " " + f(9));  
    }  
}
```

- A. 12 3 d
- B. 1 3 d12

- C. 12 3 d12
- D. 12 32 d12

Câu 02-10 · Vừa · objective 2.1

Điền biểu thức nào vào chỗ ____ thì chương trình biên dịch được? (Chọn 2 đáp án.)

```
public class Cond {
    public static void main(String[] args) {
        int x = 1;
        Boolean flag = Boolean.TRUE;
        if (____) System.out.println("ok");
    }
}
```

- A. x
- B. x = 1
- C. flag
- D. x == 1 ? true : 0
- E. !(x > 2)

Câu 02-11 · Dễ · objective 2.1

Chương trình sau in ra gì?

```
public class Post {
    public static void main(String[] args) {
        int n = 0;
        while (n++ < 3) { }
        System.out.println(n);
    }
}
```

- A. 3
- B. 2
- C. Vòng lặp vô hạn
- D. 4

Câu 02-12 · Khó · objective 2.1, 3.5

Chương trình sau in ra gì?

```
public class Pairs {
    record Pair(Object a, Object b) {}

    static String m(Object o) {
        return switch (o) {
            case Pair(String s, Integer i) when i > 0 -> "SI+";
            case Pair(String s, Object x) -> "SO";
            case Pair(Object x, Integer i) -> "OI";
            case Pair p -> "P";
            default -> "D";
        };
    }

    public static void main(String[] args) {
        System.out.println(m(new Pair("a", 1)) + m(new Pair("a", -1)) + m(new Pair(1, 2))
            + m(new Pair(1, "b")) + m("z"));
    }
}
```

- A. SOSOIPD
- B. SI+S0OIPD
- C. SI+SOPPD
- D. Không biên dịch được

Câu 02-13 · Vừa · objective 2.1

Hai phát biểu nào đúng? (Chọn 2 đáp án.)

- A. Switch cổ điển (không dùng pattern) có thể dùng selector kiểu long.
- B. Trong switch dạng arrow (->), không có fall-through giữa các nhánh.
- C. Switch **statement** trên enum không bắt buộc liệt kê hết các hằng số.
- D. case null dùng được khi selector có kiểu int.
- E. yield dùng được trong switch **statement** để thoát khỏi switch.

Câu 02-14 · Vừa · objective 2.1, 3.5

Chương trình sau in ra gì?

```
public class Flow {
    static String f(Object o) {
        if (!(o instanceof Integer n) || n < 0) return "no";
        return "int " + (n + 1);
    }

    public static void main(String[] args) {
        System.out.println(f(5) + " | " + f(-1) + " | " + f("x"));
    }
}
```

- A. int 6 | int 0 | no
- B. Không biên dịch được: n không dùng được sau if
- C. int 6 | no | no
- D. Ném ClassCastException với "x"

Câu 02-15 · Khó · objective 2.1, 3.5

Dòng nào gây lỗi biên dịch?

```
public class Scope {
    public static void main(String[] args) {
        Object o = "abc";
        if (o instanceof String s && s.length() > 2) System.out.println(s); // L1
        if (o instanceof String t || t.isEmpty()) System.out.println("?"); // L2
        if (!(o instanceof String u)) return;
        System.out.println(u.length()); // L3
        int s = 5; // L4
    }
}
```

- A. L1
- B. L2
- C. L3
- D. L4
- E. L2 và L4

Câu 02-16 · Dễ · objective 2.1

Chương trình sau in ra gì?

```
public class TwoVars {
    public static void main(String[] args) {
        for (int i = 0, j = 5; i < j; i++, j--)
            System.out.print(i + j + " ");
    }
}
```

- A. 05 14 23
- B. 5 5 5 5
- C. Không biên dịch được
- D. 5 5 5

Câu 02-17 · Khó · objective 2.1

Đoạn code nào, chèn vào chỗ `// INSERT CODE HERE`, in ra đúng 2 4? (Chọn 3 đáp án.)

```
public class Evens {
    public static void main(String[] args) {
        // INSERT CODE HERE
        System.out.println();
    }
}
```

- A. `for (int i = 1; i <= 5; i++) { if (i % 2 != 0) continue; System.out.print(i + " "); }`
- B. `int i = 0; while (i < 5) { i += 2; System.out.print(i + " "); }`
- C. `int i = 0; do { i += 2; if (i > 4) break; System.out.print(i + " "); } while (true);`
- D. `for (int i = 2; i < 5; i += 2) System.out.print(i + " ");`
- E. `for (int i = 0; i < 5; i++) if (i % 2 == 0) System.out.print(i + " ");`

Câu 02-18 · Vừa · objective 2.1

Chương trình sau in ra gì?

```
public class Strings {
    public static void main(String[] args) {
        String s = "Java";
        switch (s.toLowerCase()) {
            case "java" -> System.out.print("A");
            case "java" -> System.out.print("B");
            default -> System.out.print("C");
        }
        switch (s) {
            case "java": System.out.print("D");
            default: System.out.print("E");
            case "Java": System.out.print("F");
        }
    }
}
```

- A. BEF
- B. BF
- C. BDEF
- D. AF

Câu 02-19 · Khó · objective 2.1

Chương trình sau in ra gì?

```
public class LoopSwitch {
    public static void main(String[] args) {
        int total = 0;
        for (int i = 0; i < 5; i++) {
            switch (i) {
                case 1: continue;
                case 3: break;
                default: total += i;
            }
            total += 10;
        }
        System.out.println(total);
    }
}
```

- A. 46
- B. 36
- C. 40
- D. 26

Câu 02-20 · Vừa · objective 2.1

Dòng nào gây lỗi biên dịch?

```
public class Reach {
    public static void main(String[] args) {
        int x = 0;
        while (x < 3) x++; // L1
        for (;;) { if (x > 5) break; x++; } // L2
        do x--; while (x > 0); // L3
        while (false) { x++; } // L4
        System.out.println(x);
    }
}
```

- A. L1
- B. L2
- C. L3
- D. L4
- E. Không có lỗi; in ra 0

Lời giải

Câu 02-01 — Đáp án: B

- **Vì sao đúng:** Switch dạng : cổ điển có **fall-through**: nhảy vào case 2, thêm b, rồi rơi xuống case 3 thêm c, gặp break thì dừng. default không chạy.

- **A sai:** Không có `break` sau case 2, nên tiếp tục rơi xuống case 3.
- **C sai:** `break` ở case 3 dừng switch trước khi tới default.
- **D sai:** Switch nhảy thẳng tới case 2; case 1 không chạy.
- **Kiểm chứng:** `examples/questions/ch02/Q02_01/` — output confirmed (python3 tools/book.py questions ch02).

Câu 02-02 — Đáp án: C

- **Vì sao đúng:** Switch expression trên `String` so khớp bằng `equals`. Nhánh "SAT" là một khối {} nên phải trả về giá trị bằng `yield: 5 * 2 = 10`. Nhánh arrow không có fall-through.
- **A sai:** `yield k * 2` trả về 10, không phải giá trị của `k`.
- **B sai:** "SAT" khớp với case "SAT" nên không vào default.
- **D sai:** Khối {} trong nhánh arrow hợp lệ khi kết thúc bằng `yield`.
- **Kiểm chứng:** `examples/questions/ch02/Q02_02/` — output confirmed (python3 tools/book.py questions ch02).

Câu 02-03 — Đáp án: D

- **Vì sao đúng:** Switch expression phải **đầy đủ (exhaustive)**. Với selector `int`, không thể liệt kê hết giá trị, nên bắt buộc có default. L4 thiếu default → "the switch expression does not cover all possible input values".
- **A sai:** Arrow + default là dạng chuẩn của switch expression.
- **B sai:** Switch expression được phép dùng dạng `:` nếu mỗi nhánh trả về bằng `yield`.
- **C sai:** Khối { `yield ...;` } trong nhánh arrow là hợp lệ.
- **E sai:** L4 lỗi vì thiếu default.
- **Kiểm chứng:** `examples/questions/ch02/Q02_03/` — compile error confirmed at ['L4'] (python3 tools/book.py questions ch02).

Câu 02-04 — Đáp án: C

- **Vì sao đúng:** `continue outer` bỏ phần còn lại của vòng trong và sang `i` tiếp theo. `i=0`: không đếm. `i=1`: đếm `j=0` (1). `i=2`: đếm `j=0`, 1 (3). `i=3`: đếm `j=0` (4), `j=1` (5); tới `j=2` thì `3 + 2 > 4` → `break outer` thoát cả hai vòng.
- **A sai:** Vòng `i=3` dừng ở `j=2` do `break outer`, nên chỉ có 5 lần đếm.
- **B sai:** Với `i=3`, cả `j=0` và `j=1` đều được đếm (`3+1 = 4` không lớn hơn 4).
- **D sai:** Xem cách đếm ở phần giải thích: tổng là 5.
- **Kiểm chứng:** `examples/questions/ch02/Q02_04/` — output confirmed (python3 tools/book.py questions ch02).

Câu 02-05 — Đáp án: B

- **Vì sao đúng:** `do-while` chạy thân vòng lặp ít nhất một lần trước khi kiểm tra điều kiện: in 5, `i` thành 3, điều kiện `3 > 5` sai nên dừng; sau đó in 3.
- **A sai:** Thân `do` luôn chạy lần đầu, nên có in 5.
- **C sai:** Điều kiện `3 > 5` sai nên vòng lặp chỉ chạy một lần.
- **D sai:** Sau vòng lặp còn lệnh `println(i)` in ra 3.
- **Kiểm chứng:** `examples/questions/ch02/Q02_05/` — output confirmed (python3 tools/book.py questions ch02).

Câu 02-06 — Đáp án: A

- **Vì sao đúng:** Các case được thử từ trên xuống. 5 → S; 50 thoả guard → L; "hi" không thoả guard `length() > 2` nên rơi xuống `CharSequence` → C; "hello" → W; `StringBuilder` là `CharSequence` → C; Double → D.
- **B sai:** "hi" có độ dài 2, không thoả `when s.length() > 2`.
- **C sai:** 5 không thoả `i > 10` nên vào case Integer `i` → S.
- **D sai:** `StringBuilder` implements `CharSequence`, nên khớp case `CharSequence cs`.
- **Kiểm chứng:** `examples/questions/ch02/Q02_06/` — output confirmed (python3 tools/book.py questions ch02).

Câu 02-07 — Đáp án: A, C

- **Vì sao đúng:** Case hẹp (cụ thể hơn) phải đứng trước case rộng. A: `String` trước `CharSequence` — đúng thứ tự. C: case có guard đứng trước case cùng kiểu không guard — đúng. Cả hai có default nên đầy đủ.
- **B sai:** `CharSequence` đứng trước sẽ che (dominate) `String` → lỗi "dominated by a preceding case label".
- **D sai:** case `String s` không guard đã bắt mọi `String`, nên case có guard phía sau bị che → lỗi.
- **E sai:** Selector kiểu `Object` mà không có default (hay case `Object`) → switch không đầy đủ → lỗi.
- **Kiểm chứng:** `examples/questions/ch02/Q02_07/` — variants: AC satisfy compiles (python3 tools/book.py questions ch02).

Câu 02-08 — Đáp án: A

- **Vì sao đúng:** Trong for-each, `x` là bản sao của phần tử; gán `x` không đổi mảng. Vòng thứ hai biến mảng thành {1, 3, 5} và tổng là 9.
- **B sai:** 12 là tổng khi mảng đã bị nhân đôi {2, 4, 6} — nhưng for-each không sửa mảng.
- **C sai:** Như B, mảng không bị nhân đôi.
- **D sai:** 6 là tổng ban đầu; vòng thứ hai cộng thêm chỉ số vào từng phần tử.

- **Kiểm chứng:** `examples/questions/ch02/Q02_08/` — output confirmed (python3 tools/book.py questions ch02).

Câu 02-09 — Đáp án: C

- **Vì sao đúng:** `default` được chọn khi không case nào khớp, **bất kể vị trí**; sau đó vẫn fall-through như thường. `f(1)`: “1” rồi rơi xuống “2”, `break`. `f(3)`: “3” là case cuối. `f(9)`: vào `default` “d”, rơi xuống “1”, “2”, `break`.
- **A sai:** Sau `default` không có `break`, nên tiếp tục rơi xuống case 1 và case 2.
- **B sai:** case 1 không có `break`, nên rơi xuống case 2: `f(1)` là “12”.
- **D sai:** case 3 là case cuối cùng; không có gì chạy sau nó, và không quay lại đầu switch.
- **Kiểm chứng:** `examples/questions/ch02/Q02_09/` — output confirmed (python3 tools/book.py questions ch02).

Câu 02-10 — Đáp án: C, E

- **Vì sao đúng:** Điều kiện của `if` phải có kiểu `boolean` (hoặc `Boolean`, sẽ được unboxing). `flag` là `Boolean` → OK. `!(x > 2)` là `boolean` → OK.
- **A sai:** `int` không tự đổi sang `boolean` như trong TypeScript/JavaScript.
- **B sai:** `x = 1` là phép gán, có kiểu `int`, không phải `boolean`.
- **D sai:** Hai nhánh là `boolean` và `int`, kiểu kết quả không phải `boolean` → lỗi incompatible types.
- **Kiểm chứng:** `examples/questions/ch02/Q02_10/` — variants: CE satisfy compiles (python3 tools/book.py questions ch02).

Câu 02-11 — Đáp án: D

- **Vì sao đúng:** `n++ < 3` so sánh giá trị cũ rồi mới tăng. Các lần kiểm tra: `0 < 3` (`n=1`), `1 < 3` (`n=2`), `2 < 3` (`n=3`), `3 < 3` sai (`n` vẫn tăng thành 4). In ra 4.
- **A sai:** Lần kiểm tra cuối cùng (sai) vẫn tăng `n` thêm 1.
- **B sai:** Có 4 lần kiểm tra, mỗi lần tăng `n` một đơn vị.
- **C sai:** `n` tăng mỗi lần kiểm tra, nên điều kiện sẽ sai.
- **Kiểm chứng:** `examples/questions/ch02/Q02_11/` — output confirmed (python3 tools/book.py questions ch02).

Câu 02-12 — Đáp án: B

- **Vì sao đúng:** Record pattern “mở” record và so khớp từng thành phần theo kiểu. (“a”, 1) khớp case đầu (guard đúng). (“a”, -1) guard sai → `Pair(String, Object)` → SO. (1, 2) → `Pair(Object, Integer)` → OI. (1, “b”) → `Pair p` → P. “z” không phải `Pair` → D.
- **A sai:** (“a”, 1) thỏa guard `i > 0`, nên khớp case đầu tiên là SI+.
- **C sai:** (1, 2) có thành phần thứ hai là `Integer`, khớp `Pair(Object x, Integer i)` → OI.
- **D sai:** Không case nào bị che: mỗi case sau rộng hơn hoặc khác case trước, và có `default`.
- **Kiểm chứng:** `examples/questions/ch02/Q02_12/` — output confirmed (python3 tools/book.py questions ch02).

Câu 02-13 — Đáp án: B, C

- **Vì sao đúng:** B: mỗi nhánh arrow chạy độc lập, không rơi xuống nhánh sau. C: switch **statement** kiểu cổ điển trên enum (không có pattern, không có case null) không cần đầy đủ; hằng số không khớp thì đơn giản là không làm gì.
- **A sai:** Switch cổ điển chỉ nhận `char`, `byte`, `short`, `int` (và wrapper), `String`, `enum`. `long` → lỗi.
- **D sai:** case null chỉ dùng được khi selector là kiểu tham chiếu (reference type); `int` không bao giờ null.
- **E sai:** `yield` chỉ dùng để trả giá trị từ switch **expression**; trong switch statement dùng `break`.
- **Kiểm chứng:** `examples/questions/ch02/Q02_13/` — each option proven true/false by a program (python3 tools/book.py questions ch02).

Câu 02-14 — Đáp án: C

- **Vì sao đúng:** Flow scoping: `n` có mặt ở vế phải của `||` (vì chỉ tới đó khi `instanceof` đúng) và cả sau câu `if`, vì nhánh `if` luôn `return`. `f(5)` → `int 6`; `f(-1)` → `n < 0` → no; `f(“x”)` → không phải `Integer` → no.
- **A sai:** -1 thỏa `n < 0` nên trả về no.
- **B sai:** Vì nhánh `if` luôn thoát bằng `return`, compiler biết `n` chắc chắn đã được gán ở phía sau.
- **D sai:** `instanceof` không bao giờ ném `ClassCastException`; nó chỉ trả về `false`.
- **Kiểm chứng:** `examples/questions/ch02/Q02_14/` — output confirmed (python3 tools/book.py questions ch02).

Câu 02-15 — Đáp án: B

- **Vì sao đúng:** Ở L2, vế phải của `||` chỉ chạy khi `instanceof` sai, lúc đó `t` chưa được gán → `t` không nằm trong phạm vi → lỗi “cannot find symbol”. L1 hợp lệ vì `&&`. L3 hợp lệ vì câu `if` phía trên luôn `return`. L4 hợp lệ vì biến pattern `s` ở L1 chỉ sống trong câu `if` đó.
- **A sai:** Với `&&`, vế phải chỉ chạy khi `instanceof` đúng, nên `s` đã được gán.
- **C sai:** `if (!(o instanceof String u)) return;` làm `u` có mặt ở các lệnh sau.
- **D sai:** Phạm vi của `s` (pattern ở L1) kết thúc cùng câu `if` L1, nên khai báo `int s` mới là hợp lệ.
- **E sai:** L4 hợp lệ (xem D).

- **Kiểm chứng:** `examples/questions/ch02/Q02_15/` — compile error confirmed at [‘L2’] (`python3 tools/book.py questions ch02`).

Câu 02-16 — Đáp án: D

- **Vì sao đúng:** `for` cho phép khai báo nhiều biến cùng kiểu và nhiều biểu thức cập nhật cách nhau bởi dấu phẩy. `i + j + " "` tính từ trái: `i + j` là phép cộng số (luôn bằng 5), rồi mới nối chuỗi. Vòng chạy với (0,5), (1,4), (2,3); tới (3,2) thì dừng.
- **A sai:** `i + j` được tính trước khi gặp chuỗi, nên là phép cộng số, không phải nối chuỗi.
- **B sai:** Chỉ có 3 lần lặp: (3,2) không thỏa `i < j`.
- **C sai:** Khai báo `int i = 0, j = 5` và cập nhật `i++`, `j--` đều hợp lệ.
- **Kiểm chứng:** `examples/questions/ch02/Q02_16/` — output confirmed (`python3 tools/book.py questions ch02`).

Câu 02-17 — Đáp án: A, C, D

- **Vì sao đúng:** A bỏ qua số lẻ bằng `continue` → 2 4. C tăng 2 mỗi lần, `break` khi vượt 4 → 2 4. D bắt đầu từ 2, bước 2, dừng trước 5 → 2 4.
- **B sai:** Điều kiện kiểm tra **trước** khi cộng: khi `i = 4` vẫn vào vòng, cộng thành 6 và in 2 4 6.
- **E sai:** 0 cũng là số chẵn, nên in 0 2 4.
- **Kiểm chứng:** `examples/questions/ch02/Q02_17/` — variants: ACD satisfy output (`python3 tools/book.py questions ch02`).

Câu 02-18 — Đáp án: B

- **Vì sao đúng:** Switch trên `String` so khớp bằng `equals` (phân biệt hoa thường). Switch đầu: `"java"` → B. Switch thứ hai: `"Java"` khớp đúng case `"Java"`, là case cuối nên chỉ in F.
- **A sai:** `default` chỉ chạy khi không case nào khớp; ở đây case `"Java"` khớp.
- **C sai:** `"Java"` không bằng `"java"` (phân biệt hoa thường), nên không bắt đầu từ case `"java"`.
- **D sai:** `s.toLowerCase()` là `"java"`, không khớp `"JAVA"`.
- **Kiểm chứng:** `examples/questions/ch02/Q02_18/` — output confirmed (`python3 tools/book.py questions ch02`).

Câu 02-19 — Đáp án: A

- **Vì sao đúng:** `break` trong switch chỉ thoát **switch**, còn `continue` áp dụng cho **vòng lặp**. `i=0: +0 +10; i=1: continue` bỏ qua `+10`; `i=2: +2 +10; i=3: break` rồi `+10`; `i=4: +4 +10`. Tổng `10+12+10+14 = 46`.
- **B sai:** `i=3` vẫn cộng 10 vì `break` chỉ thoát switch, không thoát vòng lặp.
- **C sai:** `i=1` không cộng 10 vì `continue` nhảy sang lần lặp tiếp theo; tổng là 46.
- **D sai:** Tính lại: chỉ có `i=1` bị bỏ `+10`.
- **Kiểm chứng:** `examples/questions/ch02/Q02_19/` — output confirmed (`python3 tools/book.py questions ch02`).

Câu 02-20 — Đáp án: D

- **Vì sao đúng:** `while (false)` có điều kiện là hằng số `false`, nên thân vòng lặp **không thể tới (unreachable)** → lỗi biên dịch. L2 hợp lệ vì có `break`, nên lệnh phía sau vẫn tới được.
- **A sai:** Vòng `while` với điều kiện bình thường là hợp lệ.
- **B sai:** `for(;;)` có `break` bên trong, nên code phía sau vẫn tới được.
- **C sai:** `do x--; while (x > 0);` là cú pháp hợp lệ (thân một lệnh không cần `{}`).
- **E sai:** L4 lỗi biên dịch (unreachable statement).
- **Kiểm chứng:** `examples/questions/ch02/Q02_20/` — compile error confirmed at [‘L4’] (`python3 tools/book.py questions ch02`).

Đọc thêm (link chính thức — bị chặn trong sandbox nên mình chưa mở được)

- JLS §14.11 The switch Statement, §15.28 Switch Expressions, §14.30 Patterns: <https://docs.oracle.com/javase/specs/jls/se21/html/jls-14.html>
- JEP 441 Pattern Matching for switch, JEP 440 Record Patterns: <https://openjdk.org/jeps/441> , <https://openjdk.org/jeps/440>

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-28:

- Kiểm thử của javac cho pattern switch trong mã nguồn JDK 21 (thư mục `test/langtools/tools/javac/patterns`): <https://raw.githubusercontent.com/openjdk/jdk21u/master/test/langtools/tools/javac/patterns/SwitchErrors.java>
- `MatchException.java` (exception khi pattern switch không khớp lúc chạy): <https://raw.githubusercontent.com/openjdk/jdk21u/master/src/java.base/share/classes/java/lang/MatchException.java>
- Mọi hành vi trong chương đều được kiểm chứng bằng cách chạy code với OpenJDK 21.0.10 (xem output ở trên).

Chương 3 — Hướng đối tượng (Using Object-Oriented Concepts in Java)

Mục tiêu

Đây là nhóm mục tiêu **lớn nhất** của đề 1Z0-830 (7 objective). Sau chương này bạn làm được:

- 3.1 Tạo object, dùng lớp lồng nhau (nested class), hiểu vòng đời object và thu gom rác (garbage collection, GC).
- 3.2 Viết class và **record**: field/method static và instance, constructor, khối khởi tạo (initializer), thứ tự khởi tạo.
- 3.3 Nạp chồng method (overloading), kể cả varargs.
- 3.4 Phạm vi biến (scope), đóng gói (encapsulation), lớp bất biến (immutable), `var`.
- 3.5 Kế thừa (inheritance), lớp `abstract` và `sealed`, ghi đè (override) kể cả method của `Object`, đa hình (polymorphism), ép kiểu tham chiếu, `instanceof` và pattern matching.
- 3.6 Interface: method default, static, private; functional interface.
- 3.7 Enum có field, constructor, method.

Giải thích đơn giản

Class là bản thiết kế; **object** là “ngôi nhà” xây từ bản thiết kế đó bằng `new`. Biến kiểu class chỉ giữ **địa chỉ** (tham chiếu) của object. Khi không còn biến nào trỏ tới một object, object đó “đủ điều kiện” bị GC dọn.

Kế thừa (extends) cho lớp con dùng lại code của lớp cha. **Đa hình**: biến kiểu cha có thể trỏ tới object kiểu con, và khi gọi method **instance**, Java chọn bản của **object thật** (lúc chạy). Nhưng field và method **static** thì chọn theo **kiểu của biến** (lúc biên dịch). Đây là bẫy số 1 của chương.

Record là class dữ liệu ngắn gọn: `record Point(int x, int y) {}` tự sinh constructor, accessor `x()`, `equals`, `hashCode`, `toString`. **Sealed** giới hạn ai được kế thừa. **Enum** là tập hằng số cố định nhưng vẫn là class đầy đủ.

Ví dụ

Code trong `examples/ch03/`. Chạy lại: `python3 tools/book.py examples ch03`. Output thật, JDK 21.0.10.

1. Vòng đời object và GC

`examples/ch03/Ex01_Lifecycle.java`

```
// objective: 3.1
// Vòng đời object: tạo bằng new, gán lại tham chiếu, và khi nào object "đủ điều kiện" bị GC.
public class Ex01_Lifecycle {
    static int created = 0;
    final String name;

    Ex01_Lifecycle(String name) {
        this.name = name;
        created++;
    }

    public static void main(String[] args) {
        Ex01_Lifecycle a = new Ex01_Lifecycle("A"); // object A
        Ex01_Lifecycle b = new Ex01_Lifecycle("B"); // object B
        Ex01_Lifecycle c = a;                       // c và a cùng trỏ A (không tạo object mới)
        System.out.println("created=" + created + " a=c? " + (a == c));

        a = b; // A vẫn còn tham chiếu c
        c = null; // bây giờ không còn tham chiếu nào tới A - A đủ điều kiện GC (eligible)
        System.out.println("a=" + a.name + " b=" + b.name + " c=" + c);

        new Ex01_Lifecycle("C"); // tạo ra rồi bỏ ngay - đủ điều kiện GC
        System.out.println("created=" + created);
        System.gc(); // chỉ là "gợi ý"; JVM không bảo đảm sẽ chạy GC
        // Không có cách chắc chắn để biết lúc nào object bị thu hồi; finalize() đã deprecated.
    }
}
```

Output thật (JDK 21.0.10):

```
created=2 a=c? true
a=B b=B c=null
created=3
```

2. Bốn loại lớp lồng nhau

examples/ch03/Ex02_NestedClasses.java

```
// objective: 3.1
// 4 loại lớp lồng nhau: static nested, inner (member), local, anonymous.
public class Ex02_NestedClasses {
    private int value = 10;
    private static int counter = 1;

    static class StaticNested {           // không cần object bên ngoài
        int read() { return counter; }    // chỉ truy cập được thành viên static của lớp ngoài
    }

    class Inner {                         // gắn với một object của lớp ngoài
        int value = 20;
        int read() { return value + Ex02_NestedClasses.this.value; } // 20 + 10
    }

    interface Greeter { String greet(String n); }

    Greeter make(String prefix) {
        int local = 1;                  // effectively final - local/anonymous class dùng được
        class LocalGreeter implements Greeter {
            public String greet(String n) { return prefix + n + local; }
        }
        return new LocalGreeter();
    }

    public static void main(String[] args) {
        StaticNested sn = new StaticNested();
        Ex02_NestedClasses outer = new Ex02_NestedClasses();
        Ex02_NestedClasses.Inner in = outer.new Inner(); // cú pháp đặc biệt: outer.new Inner()
        System.out.println(sn.read() + " " + in.read());

        Greeter anon = new Greeter() { // anonymous class
            @Override public String greet(String n) { return "Hi " + n; }
        };
        System.out.println(anon.greet("Nobin") + " | " + outer.make("Yo ").greet("Java"));
        System.out.println(in.getClass().getName() + " " + anon.getClass().getName());
    }
}
```

Output thật (JDK 21.0.10):

```
1 30
Hi Nobin | Yo Java1
Ex02_NestedClasses$Inner Ex02_NestedClasses$1
```

3. Thứ tự khởi tạo

examples/ch03/Ex03_InitOrder.java

```
// objective: 3.2
// Thứ tự khởi tạo: static (cha -> con) một lần, rồi mỗi object: field/instance initializer -> constructor, cha trước con.
class Parent {
    static { System.out.println("1. Parent static block"); }
    int p = log("3. Parent field");
    { System.out.println("4. Parent instance block"); }
    Parent() { System.out.println("5. Parent constructor"); }
    static int log(String s) { System.out.println(s); return 0; }
}

class Child extends Parent {
    static { System.out.println("2. Child static block"); }
    int c = log("6. Child field");
    { System.out.println("7. Child instance block"); }
    Child() {
        super(); // luôn là lệnh đầu tiên (thêm ngầm nếu không viết)
        System.out.println("8. Child constructor");
    }
    Child(String s) {
        this(); // gọi constructor khác trong cùng lớp
        System.out.println("9. Child(String) " + s);
    }
}

public class Ex03_InitOrder {
    public static void main(String[] args) {
        new Child("x");
        System.out.println("--- object thứ hai: không chạy lại static");
        new Child();
    }
}
```

Output thật (JDK 21.0.10):

```
1. Parent static block
2. Child static block
3. Parent field
4. Parent instance block
5. Parent constructor
6. Child field
7. Child instance block
8. Child constructor
9. Child(String) x
--- object thứ hai: không chạy lại static
3. Parent field
4. Parent instance block
5. Parent constructor
6. Child field
7. Child instance block
8. Child constructor
```

4. Record

examples/ch03/Ex04_Records.java

```
// objective: 3.2, 3.5
// Record: lớp dữ liệu bất biến; compact constructor; accessor x() thay vì getX().
import java.util.List;

public class Ex04_Records {
    record Money(long amount, String currency) {
        static final String DEFAULT = "VND"; // field static: được phép
        // long cents; // field instance: KHÔNG được phép

        Money { // compact constructor: không có tham số, không gán this.x
            if (amount < 0) throw new IllegalArgumentException("amount < 0");
            currency = currency.toUpperCase(); // gán lại THAM SỐ trước khi field được gán
        }

        Money(long amount) { this(amount, DEFAULT); } // constructor khác phải gọi this(...)

        Money plus(Money o) { return new Money(amount + o.amount, currency); }

        @Override public String toString() { return amount + " " + currency; }
    }

    record Team(String name, List<String> members) {
        Team {
            members = List.copyOf(members); // bản sao bất biến: record chỉ "nông" (shallow) immutable
        }
    }

    public static void main(String[] args) {
        Money a = new Money(1000, "usd");
        Money b = new Money(1000, "USD");
        System.out.println(a + " | " + a.amount() + " | equals=" + a.equals(b) + " | sameHash=" + (a.hashCode() == b.hashCode()));
        System.out.println(new Money(5).plus(new Money(7)));
        try {
            new Money(-1, "x");
        } catch (IllegalArgumentException e) {
            System.out.println("IllegalArgumentException: " + e.getMessage());
        }
        var list = new java.util.ArrayList<>(List.of("An"));
        Team t = new Team("dev", list);
        list.add("Binh");
        System.out.println(t + " " + (t instanceof Record));
    }
}
```

Output thật (JDK 21.0.10):

```
1000 USD | 1000 | equals=true | sameHash=true
12 VND
IllegalArgumentException: amount < 0
Team[name=dev, members=[An]] true
```

5. Overloading

examples/ch03/Ex05_Overloading.java

```
// objective: 3.3
// Chọn method overload: đúng kiểu > nới rộng (widening) > boxing > varargs.
public class Ex05_Overloading {
    static void m(long x) { System.out.println("long"); }
    static void m(Integer x) { System.out.println("Integer"); }
}
```

```

static void m(int... x) { System.out.println("int.."); }
static void m(Object x) { System.out.println("Object"); }

static void s(String x) { System.out.println("String"); }
static void s(Object x) { System.out.println("Object"); }

static int sum(int... nums) { // varargs là một mảng
    int t = 0;
    for (int n : nums) t += n;
    return t;
}

public static void main(String[] args) {
    int i = 5;
    m(i); // widening int → long thẳng boxing
    m(Integer.valueOf(5));
    m(); // chỉ varargs khớp
    m(5, 6);
    m("text"); // String → Object
    byte b = 1;
    m(b); // byte → long (widening)
    char c = 'x';
    m(c); // char → long

    s(null); // chọn kiểu CỤ THỂ nhất: String
    s((Object) "a");

    System.out.println(sum() + " " + sum(1, 2, 3) + " " + sum(new int[]{4, 5}));
}
}

```

Output thật (JDK 21.0.10):

```

long
Integer
int...
int...
Object
long
long
String
Object
0 6 9

```

6. Scope, shadowing và var

examples/ch03/Ex06_ScopeAndVar.java

```

// objective: 3.4
// Phạm vi biến (scope), shadowing, và var (local variable type inference).
import java.util.ArrayList;

public class Ex06_ScopeAndVar {
    int count = 100; // instance field

    void show(int count) { // tham số che (shadow) field cùng tên
        System.out.println("param=" + count + " field=" + this.count);
        {
            int inner = count * 2; // chỉ sống trong khối
            System.out.println("inner=" + inner);
        }
        // System.out.println(inner); // lỗi: ngoài phạm vi
    }

    public static void main(String[] args) {
        new Ex06_ScopeAndVar().show(7);

        var n = 10; // int
        var list = new ArrayList<String>(); // ArrayList<String>
        list.add("a");
        var mixed = new ArrayList<>(); // ArrayList<Object> (không có thông tin kiểu)
        mixed.add(1);
        mixed.add("x");
        for (var s : list) System.out.println("for-each var: " + s);
        var arr = new int[]{1, 2};
        System.out.println(n + " " + list + " " + mixed + " " + arr.length);
        // var a; // lỗi: phải khởi tạo
        // var b = null; // lỗi: không suy ra được kiểu
        // var c = 1, d = 2; // lỗi: không khai báo nhiều biến
        // var e = {1, 2}; // lỗi: array initializer cần kiểu rõ ràng
        var var = "var là 'reserved type name', không phải keyword → đặt tên biến được";
        System.out.println(var);
    }
}

```

Output thật (JDK 21.0.10):

```
param=7 field=100
inner=14
for-each var: a
10 [a] [1, x] 2
var là 'reserved type name', không phải keyword → đặt tên biến được
```

7. Lớp bất biến

examples/ch03/Ex07_Immutable.java

```
// objective: 3.4
// Tạo lớp bất biến (immutable) và đóng gói (encapsulation) với defensive copy.
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class Ex07_Immutable {
    static final class Order {                // final: không cho lớp con phá vỡ bất biến
        private final String id;              // private final
        private final List<String> items;

        Order(String id, List<String> items) {
            this.id = id;
            this.items = new ArrayList<>(items); // copy khi nhận vào
        }

        String getId() { return id; }
        List<String> getItems() { return Collections.unmodifiableList(items); } // không trả ra list gốc
        Order withItem(String item) {        // "thay đổi" = tạo object mới
            List<String> copy = new ArrayList<>(items);
            copy.add(item);
            return new Order(id, copy);
        }
    }

    public static void main(String[] args) {
        List<String> src = new ArrayList<>(List.of("pen"));
        Order o = new Order("01", src);
        src.add("hack");                      // không ảnh hưởng o
        Order o2 = o.withItem("book");
        System.out.println(o.getItems() + " " + o2.getItems());
        try {
            o.getItems().add("x");
        } catch (UnsupportedOperationException e) {
            System.out.println("UnsupportedOperationException: không sửa được từ bên ngoài");
        }
    }
}
```

Output thật (JDK 21.0.10):

```
[pen] [pen, book]
UnsupportedOperationException: không sửa được từ bên ngoài
```

8. Đa hình: override vs hiding

examples/ch03/Ex08_Polymorphism.java

```
// objective: 3.5
// Override vs hiding: method instance chọn theo KIỂU OBJECT (lúc chạy);
// field và method static chọn theo KIỂU THAM CHIẾU (lúc biên dịch).
class Animal {
    String name = "animal";
    static String kind() { return "Animal.kind"; }
    String sound() { return "..."; }
    String describe() { return name + " says " + sound(); } // sound() là lời gọi đa hình
}

class Dog extends Animal {
    String name = "dog"; // che (hide) field của cha
    static String kind() { return "Dog.kind"; } // che method static (không phải override)
    @Override String sound() { return "Woof"; }
    String fetch() { return "fetching"; }
}

public class Ex08_Polymorphism {
    public static void main(String[] args) {
        Animal a = new Dog(); // kiểu tham chiếu Animal, kiểu object Dog
        System.out.println(a.sound()); // Woof (override → theo object)
        System.out.println(a.name); // animal (field → theo tham chiếu)
        System.out.println(a.kind()); // Animal.kind (static → theo tham chiếu)
    }
}
```

```

        System.out.println(a.describe());           // "animal says Woof"
        // a.fetch();                               // lỗi: Animal không có fetch()
        System.out.println(((Dog) a).fetch() + " " + ((Dog) a).name);
    }
}

```

Output thật (JDK 21.0.10):

```

Woof
animal
Animal.kind
animal says Woof
fetching dog

```

9. Abstract và sealed

examples/ch03/Ex09_AbstractSealed.java

```

// objective: 3.5
// Lớp abstract và sealed: permits, final / sealed / non-sealed ở lớp con.
public class Ex09_AbstractSealed {
    static abstract sealed class Vehicle permits Car, Truck, Bike {
        abstract int wheels();
        String info() { return getClass().getSimpleName() + " has " + wheels() + " wheels"; }
    }
    static final class Car extends Vehicle { int wheels() { return 4; } }
    static sealed class Truck extends Vehicle permits BigTruck { int wheels() { return 6; } }
    static final class BigTruck extends Truck { @Override int wheels() { return 18; } }
    static non-sealed class Bike extends Vehicle { int wheels() { return 2; } }
    static class EBike extends Bike { } // non-sealed mở lại cho mọi lớp con

    static String tax(Vehicle v) {
        return switch (v) { // sealed → switch đầy đủ không cần default
            case Car c -> "low";
            case Truck t -> "high";
            case Bike b -> "none";
        };
    }

    public static void main(String[] args) {
        Vehicle[] all = {new Car(), new Truck(), new BigTruck(), new Bike(), new EBike()};
        for (Vehicle v : all) System.out.println(v.info() + " / tax " + tax(v));
        // new Vehicle(); // lỗi: abstract
        System.out.println(Vehicle.class.isSealed() + " " + java.util.Arrays.toString(
            java.util.Arrays.stream(Vehicle.class.getPermittedSubclasses()).map(Class::getSimpleName).toArray()));
    }
}

```

Output thật (JDK 21.0.10):

```

Car has 4 wheels / tax low
Truck has 6 wheels / tax high
BigTruck has 18 wheels / tax high
Bike has 2 wheels / tax none
EBike has 2 wheels / tax none
true [Car, Truck, Bike]

```

10. Ép kiểu tham chiếu

examples/ch03/Ex10_Casting.java

```

// objective: 3.5
// Ép kiểu tham chiếu: upcast tự động, downcast cần cast và có thể ném ClassCastException.
public class Ex10_Casting {
    interface Swimmer {}
    static class Fish implements Swimmer {}
    static class Shark extends Fish {}
    static final class Rock {}

    public static void main(String[] args) {
        Fish f = new Shark(); // upcast: không cần cast
        Shark s = (Shark) f; // downcast: OK vì object thật là Shark
        Object o = new Fish();
        System.out.println((o instanceof Shark) + " " + (o instanceof Swimmer) + " " + (null instanceof Object));
        try {
            Shark bad = (Shark) o; // biên dịch được, nhưng lúc chạy object là Fish
        } catch (ClassCastException e) {
            System.out.println("ClassCastException");
        }
        Swimmer sw = (Swimmer) new Object[]{}[0]; // cast sang interface: compiler thường cho phép
        System.out.println(sw.getClass().getSimpleName());
        // Rock r = (Rock) f; // lỗi biên dịch: Fish và Rock không liên quan
    }
}

```

```
// Swimmer x = (Swimmer) new Rock(); // lỗi biên dịch: Rock là final và không implements Swimmer
if (f instanceof Shark sh && sh != null) System.out.println("pattern: " + sh.getClass().getSimpleName());
}
}
```

Output thật (JDK 21.0.10):

```
false true false
ClassCastException
Shark
pattern: Shark
```

11. equals / hashCode / toString

examples/ch03/Ex11_ObjectMethods.java

```
// objective: 3.5
// Override equals/hashCode/toString của Object; ảnh hưởng tới HashSet.
import java.util.HashSet;
import java.util.Objects;
import java.util.Set;

public class Ex11_ObjectMethods {
    static class NoOverride { int id; NoOverride(int id) { this.id = id; } }

    static class User {
        final int id;
        User(int id) { this.id = id; }
        @Override public boolean equals(Object o) {
            return o instanceof User u && u.id == id;
        }
        @Override public int hashCode() { return Objects.hash(id); }
        @Override public String toString() { return "User#" + id; }
    }

    static class BadEquals {
        int id;
        BadEquals(int id) { this.id = id; }
        public boolean equals(BadEquals o) { return o.id == id; } // OVERLOAD, không phải override!
    }

    public static void main(String[] args) {
        Set<Object> set = new HashSet<>();
        set.add(new NoOverride(1)); set.add(new NoOverride(1));
        set.add(new User(1)); set.add(new User(1));
        System.out.println("size=" + set.size());
        System.out.println(new User(7) + " " + new User(7).equals(new User(7)));
        Object x = new BadEquals(1);
        System.out.println(new BadEquals(1).equals(new BadEquals(1)) + " " + x.equals(new BadEquals(1)));
        String s = new NoOverride(3).toString();
        System.out.println(s.startsWith("Ex11_ObjectMethods$NoOverride@"));
    }
}
```

Output thật (JDK 21.0.10):

```
size=3
User#7 true
true false
true
```

12. Interface: default, static, private

examples/ch03/Ex12_Interfaces.java

```
// objective: 3.6
// Interface: method abstract, default, static, private; xung đột default giữa hai interface.
public class Ex12_Interfaces {
    interface Walker {
        int SPEED = 5; // ngầm public static final
        default String move() { return "walk " + helper(); }
        private String helper() { return "at " + SPEED; } // private: chỉ dùng bên trong interface
        static String info() { return "Walker.info"; } // static: gọi qua tên interface
    }

    interface Swimmer {
        default String move() { return "swim"; }
    }

    static class Duck implements Walker, Swimmer {
        @Override public String move() { // BẮT BUỘC override khi hai default trùng
            return Walker.super.move() + " & " + Swimmer.super.move();
        }
    }
}
```

```

    }

    static class Robot implements Walker { } // dùng default có sẵn

    public static void main(String[] args) {
        System.out.println(new Duck().move());
        System.out.println(new Robot().move());
        System.out.println(Walker.info() + " " + Walker.SPEED);
        // new Robot().info(); // lỗi: static method của interface không được kế thừa
    }
}

```

Output thật (JDK 21.0.10):

```

walk at 5 & swim
walk at 5
Walker.info 5

```

13. Functional interface

examples/ch03/Ex13_FunctionalInterface.java

```

// objective: 3.6
// Functional interface: đúng MỘT method abstract (không tính default, static, và method public của Object).
import java.util.function.Function;

public class Ex13_FunctionalInterface {
    @FunctionalInterface
    interface Calculator {
        int apply(int a, int b); // method abstract duy nhất
        default Calculator twice() { return (a, b) -> apply(apply(a, b), b); }
        static Calculator plus() { return (a, b) -> a + b; }
        boolean equals(Object o); // method của Object: không tính
        String toString(); // không tính
    }

    interface NotFunctional { void a(); void b(); } // hai method abstract -> không dùng được với lambda

    public static void main(String[] args) {
        Calculator mul = (a, b) -> a * b;
        System.out.println(mul.apply(3, 4) + " " + Calculator.plus().twice().apply(1, 10));
        Function<String, Integer> len = String::length;
        System.out.println(len.apply("lambda"));
        // NotFunctional nf = () -> {}; // lỗi: không phải functional interface
    }
}

```

Output thật (JDK 21.0.10):

```

12 21
6

```

14. Enum

examples/ch03/Ex14_Enums.java

```

// objective: 3.7
// Enum có field, constructor (luôn private), method, và method riêng cho từng hằng số.
public class Ex14_Enums {
    enum Planet {
        MERCURY(3.7), EARTH(9.8) {
            @Override String note() { return "home"; } // thân riêng của hằng số
        }, MARS(3.7);

        private final double gravity;
        Planet(double gravity) { // constructor enum ngầm private
            this.gravity = gravity;
            System.out.println("init " + name()); // chạy khi enum được nạp lần đầu
        }
        double weight(double mass) { return mass * gravity; }
        String note() { return "-"; }
    }

    enum Op {
        ADD { int apply(int a, int b) { return a + b; } },
        SUB { int apply(int a, int b) { return a - b; } };
        abstract int apply(int a, int b); // mỗi hằng số phải cài đặt
    }

    public static void main(String[] args) {
        System.out.println("start");
    }
}

```

```

Planet p = Planet.valueOf("EARTH");
System.out.println(p + " " + p.ordinal() + " " + p.weight(10) + " " + p.note() + " " + Planet.MARS.note());
System.out.println(Planet.values().length + " " + (Planet.MARS.compareTo(Planet.EARTH) > 0));
for (Op op : Op.values()) System.out.print(op + "=" + op.apply(7, 2) + " ");
System.out.println();
System.out.println(Planet.EARTH.getClass() == Planet.class); // hằng số có thân là lớp con ẩn danh
try {
    Planet.valueOf("earth");
} catch (IllegalArgumentException e) {
    System.out.println("IllegalArgumentException: valueOf phân biệt hoa thường");
}
}
}

```

Output thật (JDK 21.0.10):

```

start
init MERCURY
init EARTH
init MARS
EARTH 1 98.0 home -
3 true
ADD=9 SUB=5
false
IllegalArgumentException: valueOf phân biệt hoa thường

```

15. Quy tắc override (lỗi biên dịch)

examples/ch03/Ex15_OverrideRules.java

```

// objective: 3.5
// expect: compile-error
// Quy tắc override: không giảm quyền truy cập, không ném checked exception rộng hơn,
// kiểu trả về phải "covariant".
import java.io.IOException;

class Base {
    public void a() { }
    void b() throws IOException { }
    Number c() { return 1; }
    final void d() { }
}

class Sub extends Base {
    void a() { } // lỗi: public → package-private (yếu hơn)
    void b() throws Exception { } // lỗi: Exception rộng hơn IOException
    Integer c() { return 2; } // OK: Integer là lớp con của Number (covariant)
    void d() { } // lỗi: không override được method final
}

public class Ex15_OverrideRules {
    public static void main(String[] args) { }
}

```

Output thật của javac (JDK 21.0.10) — cố ý không biên dịch được:

```

Ex15_OverrideRules.java:15: error: a() in Sub cannot override a() in Base
    void a() { } // lỗi: public → package-private (yếu hơn)
    ^
    attempting to assign weaker access privileges; was public
Ex15_OverrideRules.java:16: error: b() in Sub cannot override b() in Base
    void b() throws Exception { } // lỗi: Exception rộng hơn IOException
    ^
    overridden method does not throw Exception
Ex15_OverrideRules.java:18: error: d() in Sub cannot override d() in Base
    void d() { } // lỗi: không override được method final
    ^
    overridden method is final
3 errors

```

Đi sâu

3.1 Object, nested class, GC

Loại	Khai báo	Tạo object	Truy cập lớp ngoài
Static nested	static class N {} trong lớp	new Outer.N()	Chỉ thành viên static
Inner (member)	class I {} trong lớp	outer.new I()	Mọi thành viên, kể cả private; Outer.this.x

Loại	Khai báo	Tạo object	Truy cập lớp ngoài
Local	<code>class L {}</code> trong method	<code>new L()</code> trong method đó	Biến cục bộ phải effectively final
Anonymous	<code>new Type() { ... }</code>	ngay tại chỗ	Như local class

- Một object **đủ điều kiện GC** khi không còn tham chiếu nào **có thể tới được** (reachable) từ code đang chạy. Hai object trỏ nhau nhưng không ai trỏ tới cả hai cũng đủ điều kiện (GC không đếm tham chiếu).
- `System.gc()` chỉ là gợi ý. Không bao giờ chắc chắn object bị dọn lúc nào. `finalize()` đã deprecated.

3.2 Class, record, khởi tạo

Thứ tự khi chạy `new Child()` lần đầu:

- Nạp lớp: static field + static block của **cha**, rồi của **con** (theo thứ tự xuất hiện). Chỉ một lần.
- Constructor con gọi `super(...)` (ngầm nếu không viết) → lên tới lớp cha.
- Ở mỗi lớp (từ cha xuống con): field initializer + instance block (theo thứ tự xuất hiện) → thân constructor.

Quy tắc constructor:

- Không viết constructor nào → compiler thêm **default constructor** không tham số. Viết một constructor bất kỳ → không còn default constructor.
- Dòng đầu của constructor là `this(...)` **hoặc** `super(...)` (không cả hai). Nếu lớp cha không có constructor không tham số, lớp con phải gọi `super(args)` rõ ràng.
- Field **final** phải được gán đúng một lần: tại chỗ khai báo, trong instance block, hoặc trong **mọi** constructor. `static final` phải gán tại chỗ hoặc trong static block.

Record:

- Component → field `private final` + accessor `public` cùng tên (`x()`), không phải `getX()`.
- Không có field instance khác; được có field/method **static**, method instance, nested type, implements interface.
- Record ngầm `final`, ngầm `extends java.lang.Record` → không `extends` được lớp khác.
- Compact constructor** `Point { ... }`: không có danh sách tham số, dùng để kiểm tra/chuẩn hoá; gán lại **tham số** (`x = 0;`), **không** được viết `this.x = ...`.
- Canonical/compact constructor phải có quyền truy cập **ít nhất bằng** record: `public record R(...)` thì constructor phải là `public`.
- Constructor khác (không canonical) phải gọi `this(...)` ở dòng đầu.

3.3 Overloading

Compiler chọn method theo 3 giai đoạn, dừng ở giai đoạn đầu tiên tìm được ứng viên:

- Khớp đúng kiểu hoặc **nới rộng** (widening): `int` → `long` → `float` → `double`.
- Cho phép **boxing/unboxing**.
- Cho phép **varargs**.

Trong một giai đoạn, nếu nhiều ứng viên thì chọn cái **cụ thể nhất** (most specific); không có → *ambiguous*. Không có “nới rộng rồi boxing” (`int` → `Long` là không thể) nhưng có “boxing rồi nới rộng tham chiếu” (`int` → `Integer` → `Object`). Overloading chỉ phụ thuộc **danh sách tham số**; kiểu trả về khác nhau không đủ.

3.4 Scope, encapsulation, immutable, var

- Phạm vi: field (cả lớp), tham số (cả method), biến cục bộ (từ chỗ khai báo tới hết khối `{}`). Biến cục bộ không được trùng tên với biến cục bộ khác còn trong phạm vi, nhưng **được** che field.
- Encapsulation: field `private`, truy cập qua method.
- Lớp bất biến: lớp `final` (hoặc constructor `private`), field `private final`, không có setter, copy dữ liệu mutable khi nhận vào và khi trả ra.
- `var`: chỉ cho **biến cục bộ** có initializer (kể cả trong `for`, `for-each`, `try-with-resources`). Không dùng cho field, tham số method, kiểu trả về. Không `var x = null;`, không `var a = 1, b = 2;`, không `var arr = {1, 2};`, không `var f = () -> 1;`. `var` không phải keyword: đặt tên biến `var` được.

3.5 Kế thừa, override, sealed, casting

Quy tắc **override** (method instance cùng tên, cùng danh sách tham số):

- Quyền truy cập **không hẹp hơn** (`public` > `protected` > `package-private` > `private`).
- Kiểu trả về giống hoặc là **kiểu con** (covariant) — với primitive thì phải giống hệt.
- Không ném checked exception **mới hoặc rộng hơn** (unchecked thì tùy ý).
- Không override được method `final`, `static` (static cùng chữ ký = hiding), hay `private` (không được kế thừa).

`@Override` giúp compiler bắt lỗi (ví dụ viết nhầm `equals(User o)` — là overload, không phải override).

Hợp đồng `equals/hashCode`: hai object `equals` thì phải cùng `hashCode`. Override `equals` mà không override `hashCode` → `HashSet/HashMap` hoạt động sai.

Sealed:

- `sealed class A permits B, C` — chỉ B, C được kế thừa trực tiếp. Lớp con phải là `final`, `sealed` hoặc `non-sealed`. Record và enum ngầm `final`.
- Lớp con phải cùng module (hoặc cùng package nếu không dùng module). Nếu tất cả ở cùng file, có thể bỏ `permits`.

Ép kiểu tham chiếu:

- Upcast (con → cha): tự động. Downcast: cần cast; sai kiểu thật → `ClassCastException` lúc chạy.
- Compiler **báo lỗi** khi hai lớp chắc chắn không liên quan (ví dụ `(String) someInteger`). Cast sang interface thường được cho phép, trừ khi lớp nguồn là `final` và không implements interface đó.
- `x instanceof T` với `x == null` → `false`. `instanceof` giữa hai lớp không liên quan → lỗi biên dịch.

3.6 Interface

Thành viên	Modifier ngầm	Ghi chú
Field	<code>public static final</code>	Phải gán giá trị
Method không thân	<code>public abstract</code>	
default method	<code>public</code>	Có thân; lớp implements có thể override
static method	<code>public</code> (hoặc <code>private static</code>)	Gọi qua tên interface; không kế thừa sang lớp implements
private method	—	Có thân; dùng chung giữa các default/static method

- Hai interface cùng default method → lớp implements **phải** override; gọi bản cụ thể bằng `X.super.m()`.
- Lớp thẳng interface: method của lớp cha (kể cả abstract) được ưu tiên hơn default method.
- Functional interface: đúng một method abstract (không tính default/static/private và method public của `Object`). `@FunctionalInterface` chỉ là kiểm tra thêm, không bắt buộc.

3.7 Enum

- Constructor ngầm `private` (viết `public` là lỗi). Hằng số phải đứng **đầu** thân enum; nếu có thêm thành viên, danh sách hằng số kết thúc bằng `;`.
- `values()`, `valueOf(String)` (phân biệt hoa thường, sai → `IllegalArgumentException`), `name()`, `ordinal()`, `compareTo()` (theo ordinal). Enum so sánh được bằng `==`.
- Enum có thể có method abstract, khi đó mỗi hằng số phải có thân `{ }` cài đặt nó.
- Enum implements interface được; không `extends` được; không `new` được.

Lỗi và bẫy thường gặp (Exam traps)

1. Field/static method theo kiểu **biến**; method instance theo kiểu **object**.
2. Method được gọi từ constructor lớp cha nhưng override ở lớp con → thấy field của con còn giá trị mặc định.
3. `equals(MyType o)` là overload → collection không dùng nó.
4. Instance initializer không chạy lại khi constructor gọi `this(...)`.
5. Viết constructor có tham số → mất default constructor → `new X()` lỗi, và lớp con `super()` ngầm cũng lỗi.
6. Record: không field instance; compact constructor không gán `this.x`; canonical constructor không được hẹp quyền hơn record.
7. `m(1, 2)` với `m(Integer, long)` và `m(long, Integer)` → ambiguous.
8. `var` trong field hoặc tham số → lỗi; `var x = null;` → lỗi.
9. Lớp con của sealed class quên `final/sealed/non-sealed` → lỗi.
10. static method của interface không gọi được qua object hay lớp implements.
11. Hai default method trùng tên mà không override → lỗi.
12. Enum constructor `public` → lỗi; `valueOf("earth")` với hằng `EARTH` → exception.
13. `new Outer.Inner()` từ static context khi `Inner` không static → lỗi; cần `new Outer().new Inner()`.
14. Local/anonymous class dùng biến cục bộ không effectively final → lỗi.

Góc nhìn từ TypeScript

TypeScript	Java	Ghi chú
Structural typing: cùng “hình dạng” là tương thích	Nominal typing : phải <code>extends/implements</code> rõ ràng	Hai lớp giống hệt nhau vẫn không gán cho nhau được
<code>class A { constructor(public x: number) {} }</code>	<code>record A(int x) {}</code>	Record gần với “data class”

TypeScript	Java	Ghi chú
readonly field	final field	final chỉ khoá tham chiếu, giống readonly
Union type <code>Circle Square</code>	sealed interface <code>Shape</code> permits <code>Circle</code> , <code>Square</code>	Kiểm tra đầy đủ bằng switch
Method overloading bằng nhiều chữ ký + một cài đặt	Nhiều method thật sự khác nhau	Java chọn lúc biên dịch theo kiểu tham số
Interface chỉ tồn tại lúc biên dịch	Interface là kiểu thật lúc chạy, có default/static/private method	instanceof interface được
enum là số/chuỗi	enum là class: field, constructor, method	Mạnh hơn nhiều
Mọi property/method đều “virtual”	Field không đa hình; static method không đa hình	Nguồn gốc của nhiều câu bẫy

Tóm tắt

- Tham chiếu vs object; GC khi không còn tham chiếu reachable; `System.gc()` chỉ là gợi ý.
- Khởi tạo: static (cha → con, một lần) → mỗi object: cha (field/block → constructor) → con.
- Overloading: exact/widening → boxing → varargs; không nói rộng-rời-boxing.
- Override: không hẹp quyền, trả về covariant, không thêm checked exception rộng hơn.
- Record: field private final, accessor `x()`, compact constructor gán tham số.
- Sealed: permits + lớp con final/sealed/non-sealed; switch đầy đủ không cần default.
- Interface: default/static/private method; xung đột default → override + `X.super.m()`.
- Enum: constructor private, `values()`, `valueOf()`, `ordinal()`, thân riêng cho từng hằng.

Bài tập (có lời giải)

Mọi đáp án đã được `tools/book.py` biên dịch và chạy để xác nhận (code: `examples/questions/ch03/`). Với câu hỏi “lỗi biên dịch ở dòng nào”, script còn kiểm tra **từng dòng** riêng lẻ: sửa các dòng còn lại thì dòng đó vẫn lỗi, và sửa tất cả thì code biên dịch được.

Câu hỏi

Câu 03-01 · Vừa · objective 3.1

Khi chương trình chạy tới dòng `// line X`, có bao nhiêu object đủ điều kiện bị thu gom rác (eligible for garbage collection)?

```
public class Gc {
    public static void main(String[] args) {
        Object a = new Object(); // obj1
        Object b = new Object(); // obj2
        Object c = new Object(); // obj3
        a = b;
        c = a;
        b = null;
        // line X
    }
}
```

- A. 0
- B. 1
- C. 2
- D. 3

Câu 03-02 · Vừa · objective 3.1

Chèn dòng nào vào chỗ `// INSERT CODE HERE` thì chương trình biên dịch được? (Chọn 2 đáp án.)

```
public class Outer {
    class Inner { }
    static class Nested { }

    public static void main(String[] args) {
        // INSERT CODE HERE
    }
}
```

- A. `Inner i = new Inner();`
- B. `Outer.Inner i = new Outer().new Inner();`
- C. `Nested n = new Outer.Nested();`
- D. `Outer.Nested n = new Outer().new Nested();`
- E. `Inner i = Outer.new Inner();`

Câu 03-03 · Vừa · objective 3.2

Chương trình sau in ra gì?

```
public class Init {
    static String log = "";
    static { log += "S"; }
    { log += "I"; }
    Init() { log += "C"; }
    Init(int x) { this(); log += "X"; }

    public static void main(String[] args) {
        new Init(1);
        new Init();
        System.out.println(log);
    }
}
```

- A. SICXIC
- B. SIICXIC
- C. SICXSIC
- D. SIXCIC

Câu 03-04 · Khó · objective 3.2, 3.5

Chương trình sau in ra gì?

```
class A {
    A() { print(); }
    void print() { System.out.print("A "); }
}

class B extends A {
    int x = 5;
    B() { print(); }
    @Override void print() { System.out.print("B" + x + " "); }
}

public class Ctor {
    public static void main(String[] args) {
        new B();
    }
}
```

- A. A B5
- B. B0 B5
- C. B5 B5
- D. A B0

Câu 03-05 · Vừa · objective 3.2, 3.5

Những dòng nào gây lỗi biên dịch?

```
record Point(int x, int y) {
    static int count;           // L1
    int z;                      // L2
    Point {
        if (x < 0) x = 0;       // L3
    }
    public int x() { return x; } // L4
    void reset() { this.y = 0; } // L5

    public static void main(String[] args) { }
}
```

- A. Chỉ L2
- B. L2 và L5
- C. L3 và L5
- D. L2, L3 và L5
- E. L4 và L5

Câu 03-06 · Vừa · objective 3.3

Chương trình sau in ra gì?

```
public class Over {
    static String f(Object o) { return "0"; }
```

```

static String f(long l) { return "L"; }
static String f(Integer i) { return "I"; }
static String f(int... is) { return "V"; }

public static void main(String[] args) {
    short s = 1;
    Integer boxed = 2;
    System.out.println(f(s) + f(boxed) + f(3L) + f() + f('c') + f(4.0));
}
}

```

- A. OIVLLO
- B. LILVIO
- C. IILVLO
- D. LILVLO

Câu 03-07 · Khó · objective 3.3

Dòng nào gây lỗi biên dịch?

```

public class Amb {
    static void m(Integer a, long b) { }
    static void m(long a, Integer b) { }

    public static void main(String[] args) {
        m(1, 2); // L1
        m(1L, 2); // L2
        m(1, 2L); // L3
    }
}

```

- A. L1
- B. L2
- C. L3
- D. L1 và L2
- E. Không dòng nào

Câu 03-08 · Vừa · objective 3.4

Chèn dòng nào vào chỗ `// INSERT CODE HERE` thì chương trình biên dịch được? (Chọn 2 đáp án.)

```

public class Infer {
    public static void main(String[] args) {
        // INSERT CODE HERE
    }
}

```

- A. `var a = 1, b = 2;`
- B. `var list = new java.util.ArrayList<>();`
- C. `var x; x = 5;`
- D. `var n = null;`
- E. `var arr = new int[]{1, 2};`
- F. `var f = () -> 1;`

Câu 03-09 · Dễ · objective 3.4

Chương trình sau in ra gì?

```

public class Shadow {
    static int x = 1;
    int y = 2;

    void run(int x) {
        int y = x + this.y;
        x = 10;
        System.out.println(x + " " + y + " " + Shadow.x + " " + this.y);
    }

    public static void main(String[] args) {
        new Shadow().run(5);
    }
}

```

- A. 10 7 10 2
- B. 5 7 1 7
- C. 10 12 1 2

- D. 10 7 1 2

Câu 03-10 · Khó · objective 3.4

Hai phát biểu nào đúng? (Chọn 2 đáp án.)

- A. Nội dung của một component kiểu `List` trong record vẫn có thể bị thay đổi qua accessor, nếu record không copy list.
- B. Khai báo `final StringBuilder sb` làm cho nội dung của `sb` không thể thay đổi.
- C. Một lớp có mọi field `private final` nhưng không phải lớp `final` vẫn có thể bị lớp con làm thay đổi hành vi quan sát được.
- D. `String.toUpperCase()` thay đổi chính object `String` gọi nó.
- E. `List.of(...)` trả về list cho phép thêm phần tử.

Câu 03-11 · Vừa · objective 3.5

Chương trình sau in ra gì?

```
class P {
    String n = "P";
    String get() { return n; }
    static String s() { return "sP"; }
}

class C extends P {
    String n = "C";
    @Override String get() { return n; }
    static String s() { return "sC"; }
}

public class Poly {
    public static void main(String[] args) {
        P p = new C();
        System.out.println(p.n + " " + p.get() + " " + p.s() + " " + ((C) p).n);
    }
}
```

- A. C C sC C
- B. P P sP C
- C. P C sC C
- D. P C sP C

Câu 03-12 · Khó · objective 3.5

Dòng nào gây lỗi biên dịch?

```
public class Seal {
    sealed interface Shape permits Circle, Square { }
    record Circle(double r) implements Shape { }           // L1
    final class Square implements Shape { }                 // L2
    class Triangle implements Shape { }                     // L3

    static String name(Shape s) {
        return switch (s) {
            case Circle c -> "circle";
            case Square q -> "square";
        };
    } // L4

    public static void main(String[] args) { }
}
```

- A. L1
- B. L2
- C. L3
- D. L4
- E. L3 và L4

Câu 03-13 · Vừa · objective 3.5

Kết quả của chương trình là gì?

```
public class Cast {
    static class Animal { }
    static class Cat extends Animal { }
    static class Dog extends Animal { }

    public static void main(String[] args) {
        Animal a = new Cat();
    }
}
```

```

Object o = a;
System.out.print((o instanceof Animal) + " ");
Dog d = (Dog) a;
System.out.println("done");
}
}

```

- A. In ra true done
- B. Không biên dịch được ở dòng (Dog) a
- C. In ra false rồi ném ClassCastException
- D. In ra true rồi ném ClassCastException

Câu 03-14 · Vừa · objective 3.5

Chương trình sau in ra gì?

```

import java.util.*;

public class Eq {
    static class Id {
        final int v;
        Id(int v) { this.v = v; }
        public boolean equals(Id o) { return o != null && o.v == v; }
        @Override public int hashCode() { return v; }
    }

    public static void main(String[] args) {
        Id a = new Id(1), b = new Id(1);
        Object ob = b;
        List<Id> list = new ArrayList<>(List.of(a));
        System.out.println(a.equals(b) + " " + a.equals(ob) + " " + list.contains(b));
    }
}

```

- A. true true true
- B. true false true
- C. true false false
- D. false false false

Câu 03-15 · Vừa · objective 3.6

Chương trình sau in ra gì?

```

public class Iface {
    interface A { default String hi() { return "A"; } }
    interface B extends A { default String hi() { return "B" + A.super.hi(); } }
    interface C extends A { }
    static class X implements B, C { }

    public static void main(String[] args) {
        System.out.println(new X().hi());
    }
}

```

- A. A
- B. BA
- C. Không biên dịch được: X phải override hi()
- D. B

Câu 03-16 · Khó · objective 3.6

Khai báo nào, chèn vào chỗ // INSERT CODE HERE, làm cho chương trình biên dịch và in ra 3? (Chọn 3 đáp án.)

```

public class Func {
    // INSERT CODE HERE

    public static void main(String[] args) {
        F f = s -> s.length();
        System.out.println(f.apply("abc"));
    }
}

```

- A. interface F { int apply(String s); }
- B. interface F { int apply(String s); default int twice(String s) { return 2 * apply(s); } }
- C. interface F { int apply(String s); int other(); }
- D. interface F { int apply(String s); boolean equals(Object o); }

- E. `abstract class F { abstract int apply(String s); }`

Câu 03-17 · Vừa · objective 3.6

Dòng nào gây lỗi biên dịch?

```
public class Members {
    interface Shape {
        int SIDES = 0; // L1
        private void log() { } // L2
        protected double area(); // L3
        static Shape unit() { return null; } // L4
        default void print() { log(); } // L5
    }

    public static void main(String[] args) { }
}
```

- A. L1
- B. L2
- C. L3
- D. L4
- E. L2 và L3

Câu 03-18 · Dễ · objective 3.7

Chương trình sau in ra gì?

```
public class Sizes {
    enum Size {
        S(1), M(2), L(3);
        private final int n;
        Size(int n) { this.n = n; }
        Size next() { return values()[ordinal() + 1] % values().length; }
    }

    public static void main(String[] args) {
        Size s = Size.valueOf("M");
        System.out.println(s.next() + " " + s.next().next().n + " "
            + s.compareTo(Size.L) + " " + Size.L.name().toLowerCase());
    }
}
```

- A. L 1 -1 1
- B. L 3 -1 1
- C. L 1 1 L
- D. S 1 -1 1

Câu 03-19 · Khó · objective 3.7

Hai phát biểu nào đúng về enum? (Chọn 2 đáp án.)

- A. Constructor của enum có thể khai báo `public`.
- B. Enum có thể implements interface.
- C. Có thể tạo thêm hằng số enum lúc chạy bằng `new`.
- D. Mỗi hằng số enum có thể có thân lớp riêng để override method.
- E. Enum có thể extends một lớp khác.

Câu 03-20 · Vừa · objective 3.5

Những dòng nào gây lỗi biên dịch?

```
public class Abs {
    abstract static class Shape {
        abstract double area(); // L1
        abstract void draw() { } // L2
        Shape() { } // L3
    }

    static class Sq extends Shape { // L4
        double area() { return 1; }
    }

    public static void main(String[] args) {
        Shape s = new Sq();
    }
}
```

- A. Chỉ L2
- B. L2 và L4
- C. L3 và L4
- D. L1 và L2
- E. Chỉ L4

Lời giải

Câu 03-01 — Đáp án: C

- **Vì sao đúng:** `a = b` làm obj1 mất tham chiếu cuối cùng. `c = a` làm obj3 mất tham chiếu. obj2 vẫn được `a` và `c` trỏ tới (`b = null` chỉ xóa một trong ba tham chiếu). Vậy obj1 và obj3 đủ điều kiện GC. Script kiểm chứng bằng `WeakReference + System.gc()` và đếm được 2 object bị thu hồi.
- **A sai:** obj1 và obj3 không còn biến nào trỏ tới sau hai phép gán.
- **B sai:** Cả obj1 (sau `a = b`) và obj3 (sau `c = a`) đều mất tham chiếu.
- **D sai:** obj2 vẫn được `a` và `c` trỏ tới.
- **Kiểm chứng:** `examples/questions/ch03/Q03_01/` — output confirmed (check_code: bản code có thêm lệnh đo, xem thư mục) (`python3 tools/book.py questions ch03`).

Câu 03-02 — Đáp án: B, C

- **Vì sao đúng:** Inner class (không `static`) cần một object của lớp ngoài: `new Outer().new Inner()`. Static nested class tạo như lớp bình thường: `new Outer.Nested()` (hoặc `new Nested()` bên trong `Outer`).
- **A sai:** `main` là static, không có `this` của `Outer`, nên không tạo trực tiếp `Inner` được.
- **D sai:** Không dùng cú pháp `outer.new` với static nested class → lỗi “qualified new of static class”.
- **E sai:** Cần một **object** trước `.new`, không phải tên lớp.
- **Kiểm chứng:** `examples/questions/ch03/Q03_02/` — variants: BC satisfy compiles (`python3 tools/book.py questions ch03`).

Câu 03-03 — Đáp án: A

- **Vì sao đúng:** Khối static chạy **một lần** khi lớp được nạp (S). `new Init(1)` gọi `this()`; instance initializer chỉ chạy trong constructor gọi `super()` (ở đây là `Init()`), nên chạy đúng một lần: I, C, rồi X. `new Init()` thêm I, C.
- **B sai:** Instance initializer không chạy hai lần khi có `this()`: nó chỉ chạy trong constructor gọi `super()`.
- **C sai:** Khối static chỉ chạy một lần cho cả chương trình.
- **D sai:** Instance initializer chạy **trước** thân constructor `Init()`, và X được thêm sau cùng.
- **Kiểm chứng:** `examples/questions/ch03/Q03_03/` — output confirmed (`python3 tools/book.py questions ch03`).

Câu 03-04 — Đáp án: B

- **Vì sao đúng:** Constructor `A()` chạy trước (do `super()` ngầm). Lời gọi `print()` là đa hình nên gọi bản override của B. Lúc đó field `x` của B chưa được khởi tạo (initializer của B chạy sau `super()`), nên `x = 0`. Sau đó `x = 5` và `B()` in B5.
- **A sai:** Method instance được chọn theo kiểu object thật (B), kể cả khi gọi từ constructor của lớp cha.
- **C sai:** Khi constructor cha chạy, field của lớp con vẫn mang giá trị mặc định 0.
- **D sai:** Như A — bản override của B được gọi.
- **Kiểm chứng:** `examples/questions/ch03/Q03_04/` — output confirmed (`python3 tools/book.py questions ch03`).

Câu 03-05 — Đáp án: B

- **Vì sao đúng:** Record không được khai báo field **instance** ngoài các component (L2). Field của record là `final`, không gán lại được trong method (L5). L3 hợp lệ: trong compact constructor ta gán lại **tham số** `x`, trước khi field được gán tự động. L4 hợp lệ: accessor viết tay phải `public`, đúng tên và kiểu.
- **A sai:** L5 cũng lỗi: `y` là field `final`.
- **C sai:** L3 hợp lệ — gán tham số trong compact constructor là cách chuẩn hoá dữ liệu.
- **D sai:** L3 hợp lệ (xem C).
- **E sai:** L4 là accessor hợp lệ; L2 thì lỗi.
- **Kiểm chứng:** `examples/questions/ch03/Q03_05/` — compile error confirmed at [‘L2’, ‘L5’] (`python3 tools/book.py questions ch03`).

Câu 03-06 — Đáp án: D

- **Vì sao đúng:** Thứ tự ưu tiên: khớp đúng/nới rộng (widening) → boxing → varargs. `short` và `char` nới rộng thành `long` (L). Integer khớp đúng (I). `3L` → L. Không tham số → chỉ varargs (V). `4.0` là `double`: không nới rộng sang `long` được, nên boxing thành `Double` rồi thành `Object` (O).
- **A sai:** `short` nới rộng thành `long` ở bước đầu tiên, không cần boxing thành `Object`.
- **B sai:** `char` nới rộng sang `long` (widening) được ưu tiên hơn boxing sang `Integer` — và `char` cũng không boxing thành `Integer`.
- **C sai:** `short` không boxing thành `Integer`; widening sang `long` thắng.
- **Kiểm chứng:** `examples/questions/ch03/Q03_06/` — output confirmed (`python3 tools/book.py questions ch03`).

Câu 03-07 — Đáp án: A

- **Vì sao đúng:** L1: không method nào khớp mà không cần boxing; ở bước có boxing thì **cả hai** đều khớp và không cái nào cụ thể hơn → “reference to m is ambiguous”. L2: 1L không boxing thành Integer được, nên chỉ m(long, Integer) khớp. L3: tương tự, chỉ m(Integer, long) khớp.
- **B sai:** 1L là long, chỉ khớp tham số long a → chỉ một ứng viên, không mơ hồ.
- **C sai:** 2L chỉ khớp tham số long b → chỉ m(Integer, long).
- **D sai:** L2 có đúng một ứng viên.
- **E sai:** L1 mơ hồ (ambiguous).
- **Kiểm chứng:** examples/questions/ch03/Q03_07/ — compile error confirmed at [‘L1’] (python3 tools/book.py questions ch03).

Câu 03-08 — Đáp án: B, E

- **Vì sao đúng:** var cần một initializer để suy ra kiểu. B suy ra ArrayList<Object>. E suy ra int[].
- **A sai:** var không dùng để khai báo nhiều biến trên một dòng.
- **C sai:** var phải được khởi tạo ngay khi khai báo.
- **D sai:** Không suy ra được kiểu từ null.
- **F sai:** Lambda cần kiểu đích (target type) rõ ràng; var không cung cấp được.
- **Kiểm chứng:** examples/questions/ch03/Q03_08/ — variants: BE satisfy compiles (python3 tools/book.py questions ch03).

Câu 03-09 — Đáp án: D

- **Vì sao đúng:** Tham số x che field static x; biến cục bộ y che field y. y = 5 + this.y = 7. Gán x = 10 chỉ đối tham số. Shadow.x vẫn là 1 và this.y vẫn là 2.
- **A sai:** x = 10 gán cho tham số, không phải field static Shadow.x.
- **B sai:** Tham số x đã bị gán 10 trước khi in; và this.y là field (2), không phải biến cục bộ.
- **C sai:** y được tính **trước** khi gán x = 10, nên y = 5 + 2.
- **Kiểm chứng:** examples/questions/ch03/Q03_09/ — output confirmed (python3 tools/book.py questions ch03).

Câu 03-10 — Đáp án: A, C

- **Vì sao đúng:** A: record chỉ bất biến “nông” (shallow): field items là final, nhưng object list bên trong vẫn sửa được. C: lớp con có thể override getter và trả về giá trị khác, phá vỡ tính bất biến → lớp bất biến nên là final.
- **B sai:** final chỉ khoá **tham chiếu** (không gán sb sang object khác), không khoá nội dung object.
- **D sai:** String bất biến; toUpperCase() trả về chuỗi mới.
- **E sai:** List.of trả về list không sửa được (unmodifiable) → UnsupportedOperationException.
- **Kiểm chứng:** examples/questions/ch03/Q03_10/ — each option proven true/false by a program (python3 tools/book.py questions ch03).

Câu 03-11 — Đáp án: D

- **Vì sao đúng:** Field và method static được chọn theo **kiểu tham chiếu** lúc biên dịch (P): p.n = “P”, p.s() = “sP”. Method instance được override nên chọn theo **kiểu object** lúc chạy (C): p.get() trả về field n của C. Sau khi cast sang C, ((C) p).n = “C”.
- **A sai:** Field không đa hình: p.n dùng kiểu tham chiếu P.
- **B sai:** get() được override trong C và trả về n của C.
- **C sai:** Method static không override được (chỉ bị che - hiding), chọn theo kiểu tham chiếu P.
- **Kiểm chứng:** examples/questions/ch03/Q03_11/ — output confirmed (python3 tools/book.py questions ch03).

Câu 03-12 — Đáp án: C

- **Vì sao đúng:** Shape chỉ cho phép (permits) Circle và Square. Triangle không có trong danh sách nên không được implements Shape. L1: record ngầm là final → hợp lệ. L2: final → hợp lệ. L4: switch đã phủ đủ các lớp được permit nên đầy đủ.
- **A sai:** Record luôn ngầm final, nên là lớp con hợp lệ của sealed interface.
- **B sai:** Lớp con được permit phải là final, sealed hoặc non-sealed — Square là final.
- **D sai:** Switch trên sealed type phủ hết Circle và Square là đầy đủ, không cần default.
- **E sai:** L4 hợp lệ (xem D).
- **Kiểm chứng:** examples/questions/ch03/Q03_12/ — compile error confirmed at [‘L3’] (python3 tools/book.py questions ch03).

Câu 03-13 — Đáp án: D

- **Vì sao đúng:** o trở tới một Cat, là Animal → in true. Cast (Dog) a biên dịch được vì Dog là lớp con của Animal (có thể đúng lúc chạy), nhưng object thật là Cat → ClassCastException.
- **A sai:** Object thật là Cat, không phải Dog, nên cast thất bại lúc chạy.
- **B sai:** Compiler chỉ cấm cast giữa hai lớp không liên quan; Animal → Dog là downcast hợp lệ về cú pháp.
- **C sai:** Cat là lớp con của Animal, nên instanceof Animal là true.
- **Kiểm chứng:** examples/questions/ch03/Q03_13/ — output confirmed (python3 tools/book.py questions ch03).

Câu 03-14 — Đáp án: C

- **Vì sao đúng:** `equals(Id)` là **overload**, không phải override `equals(Object)`. `a.equals(b)`: tham số kiểu `Id` → gọi bản overload → `true`. `a.equals(ob)`: tham số kiểu `Object` → gọi `Object.equals` (so sánh tham chiếu) → `false`. `list.contains` gọi `equals(Object)` → `false`.
- **A sai:** Chỉ lời gọi với tham số kiểu `Id` mới dùng bản overload.
- **B sai:** `Collection` luôn gọi `equals(Object)`, nên `contains` không thấy bản overload.
- **D sai:** `a.equals(b)` với `b` kiểu `Id` chọn bản overload `equals(Id)` → `true`.
- *Kiểm chứng:* `examples/questions/ch03/Q03_14/` — output confirmed (python3 tools/book.py questions ch03).

Câu 03-15 — Đáp án: B

- **Vì sao đúng:** `C` chỉ kế thừa `hi()` của `A`, còn `B` override nó. Bản của `B` cụ thể hơn (`B` là interface con của `A`), nên không có xung đột và `X` dùng bản của `B`. `A.super.hi()` hợp lệ vì `B` kế thừa trực tiếp `A`.
- **A sai:** `B.hi()` override `A.hi()` và được chọn vì cụ thể hơn.
- **C sai:** Chỉ bắt buộc override khi hai default **không liên quan** cùng tên; ở đây `B` đã override `A`.
- **D sai:** `B.hi()` gọi thêm `A.super.hi()` nên kết quả là `BA`.
- *Kiểm chứng:* `examples/questions/ch03/Q03_15/` — output confirmed (python3 tools/book.py questions ch03).

Câu 03-16 — Đáp án: A, B, D

- **Vì sao đúng:** `Lambda` cần một **functional interface**: interface có đúng một method abstract. Method default/static không tính (`B`). Method trùng với method public của `Object` như `equals(Object)` cũng không tính (`D`).
- **C sai:** Có hai method abstract (`apply`, `other`) → không phải functional interface.
- **E sai:** `Lambda` chỉ dùng được với interface, không dùng với abstract class.
- *Kiểm chứng:* `examples/questions/ch03/Q03_16/` — variants: `ABD` satisfy output (python3 tools/book.py questions ch03).

Câu 03-17 — Đáp án: C

- **Vì sao đúng:** Thành viên của interface chỉ có thể là `public` hoặc `private` (method); `protected` không được phép. `L1`: field ngầm `public static final`. `L2`: method `private` có thân (Java 9+). `L4`: method `static`. `L5`: default gọi method `private` → hợp lệ.
- **A sai:** Field trong interface là hằng số `public static final` ngầm định — hợp lệ.
- **B sai:** Interface được có method `private` (có thân) từ Java 9.
- **D sai:** Interface được có method `static` (có thân) từ Java 8.
- **E sai:** `L2` hợp lệ (xem `B`).
- *Kiểm chứng:* `examples/questions/ch03/Q03_17/` — compile error confirmed at ['L3'] (python3 tools/book.py questions ch03).

Câu 03-18 — Đáp án: A

- **Vì sao đúng:** `M.next()` là `L`. `L.next()` quay vòng về `S`, có `n = 1`. `compareTo` so sánh ordinal: `1 - 2 = -1`. `name()` trả về `"L"`, `toLowerCase()` → `"l"`.
- **B sai:** `next()` gọi hai lần từ `M`: `M` → `L` → `S`, và `S.n` là 1.
- **C sai:** `M` đứng trước `L` nên `compareTo` âm; `toLowerCase()` cho `l`.
- **D sai:** `ordinal()` của `M` là 1, nên `next()` là `values()[2] = L`.
- *Kiểm chứng:* `examples/questions/ch03/Q03_18/` — output confirmed (python3 tools/book.py questions ch03).

Câu 03-19 — Đáp án: B, D

- **Vì sao đúng:** `B`: enum là lớp đặc biệt, có thể implements một hay nhiều interface. `D`: hằng số enum có thể có thân `{ ... }` riêng (thực chất là lớp con ẩn danh) để override method.
- **A sai:** Constructor enum ngầm `private`; khai báo `public` hoặc `protected` là lỗi biên dịch.
- **C sai:** Không thể `new` một enum; tập hằng số cố định lúc biên dịch.
- **E sai:** Enum đã ngầm `extends java.lang.Enum`, nên không `extends` được lớp khác.
- *Kiểm chứng:* `examples/questions/ch03/Q03_19/` — each option proven true/false by a program (python3 tools/book.py questions ch03).

Câu 03-20 — Đáp án: B

- **Vì sao đúng:** `L2`: method `abstract` không được có thân `{ }`. `L4`: `Sq` không phải `abstract` nên phải cài đặt **mọi** method `abstract` kế thừa, nhưng thiếu `draw()`. `L1` hợp lệ; `L3` hợp lệ vì lớp `abstract` vẫn có constructor (được gọi qua `super()` từ lớp con).
- **A sai:** `Sq` cũng lỗi vì không cài đặt `draw()`.
- **C sai:** Lớp `abstract` được phép có constructor.
- **D sai:** `L1` là method `abstract` hợp lệ (không có thân).
- **E sai:** `L2` cũng lỗi: method `abstract` không có thân.
- *Kiểm chứng:* `examples/questions/ch03/Q03_20/` — compile error confirmed at ['L2', 'L4'] (python3 tools/book.py questions ch03).

Đọc thêm (link chính thức — bị chặn trong sandbox nên mình chưa mở được)

- JLS Chương 8 (Classes, gồm §8.10 Record Classes), Chương 9 (Interfaces), §8.9 Enum Classes:
<https://docs.oracle.com/javase/specs/jls/se21/html/jls-8.html>
- JLS §15.12 Method Invocation (chọn overload): <https://docs.oracle.com/javase/specs/jls/se21/html/jls-15.html>

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-28:

- `Record.java` (Javadoc về hợp đồng `equals/hashCode` của record):
<https://raw.githubusercontent.com/openjdk/jdk21u/master/src/java.base/share/classes/java/lang/Record.java>
- `Object.java` (hợp đồng `equals/hashCode`):
<https://raw.githubusercontent.com/openjdk/jdk21u/master/src/java.base/share/classes/java/lang/Object.java>
- `Enum.java` (`valueOf`, `ordinal`, `compareTo`):
<https://raw.githubusercontent.com/openjdk/jdk21u/master/src/java.base/share/classes/java/lang/Enum.java>
- `FunctionalInterface.java` (định nghĩa functional interface):
<https://raw.githubusercontent.com/openjdk/jdk21u/master/src/java.base/share/classes/java/lang/FunctionalInterface.java>

Chương 4 — Ngoại lệ (Handling Exceptions)

Mục tiêu

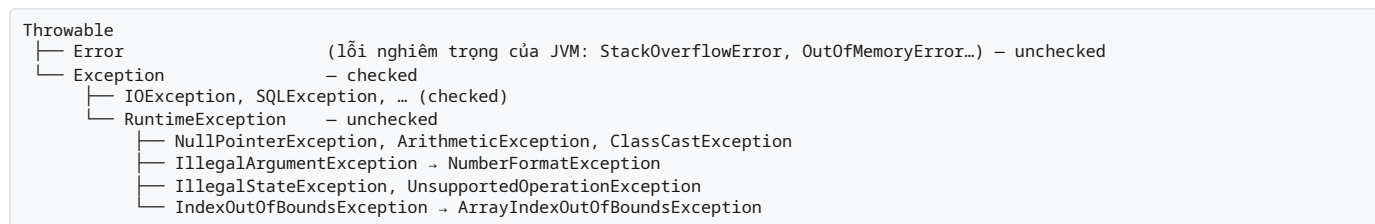
Objective 4.1: xử lý ngoại lệ (exception) bằng `try/catch/finally`, `try-with-resources`, `multi-catch`, và tự định nghĩa exception (custom exception). Cụ thể:

- Phân biệt **checked** và **unchecked** exception, và **Error**.
- Dự đoán đúng thứ tự chạy của `try / catch / finally`, kể cả khi có `return` hoặc exception lồng nhau.
- Dùng `try-with-resources`: thứ tự đóng, **suppressed exception**.
- Biết các lỗi biên dịch hay gặp: `catch` sai thứ tự, `catch checked exception` không thể xảy ra, chưa “handle or declare”.

Giải thích đơn giản

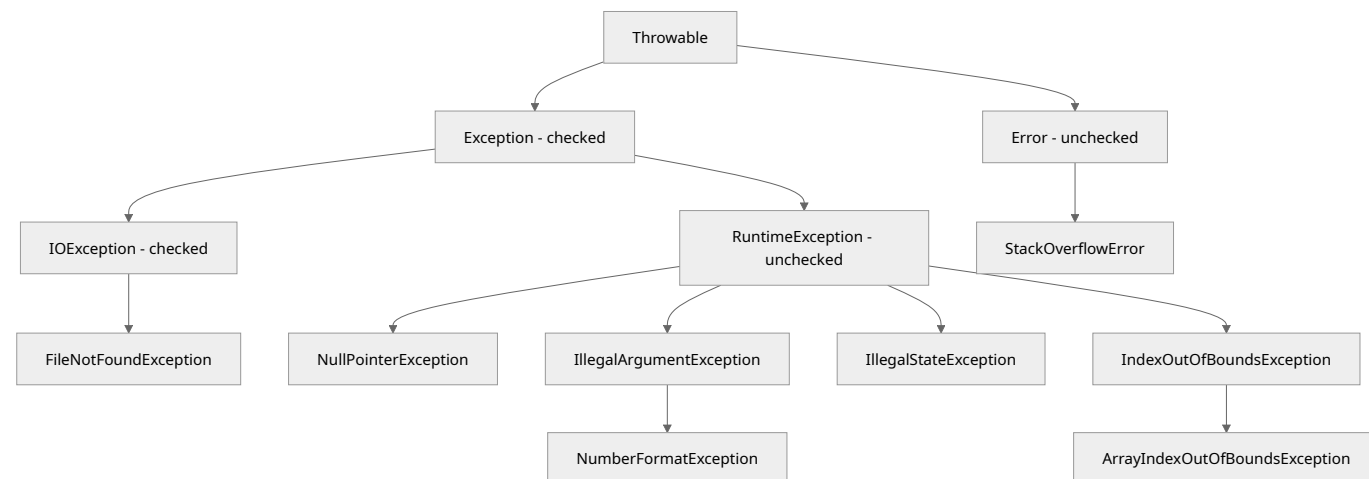
Exception là một **object** mô tả sự cố. Khi có sự cố, Java “ném” (throw) object đó; nó bay ngược lên theo chuỗi lời gọi method cho đến khi gặp một khối `catch` phù hợp. Không ai bắt → chương trình dừng và in **stack trace**.

Cây kế thừa rút gọn:



Checked exception (nhánh `Exception` trừ `RuntimeException`) buộc bạn phải **xử lý hoặc khai báo** (handle or declare): hoặc `catch`, hoặc ghi `throws` ở method. **Unchecked** thì không bắt buộc.

Sơ đồ (Mermaid):



Ví dụ

Code trong `examples/ch04/`. Chạy lại: `python3 tools/book.py examples ch04`. Output thật, JDK 21.0.10.

1. Cây kế thừa: checked hay unchecked?

`examples/ch04/Ex01_Hierarchy.java`

```
// objective: 4.1
// Cây kế thừa của exception: checked (phải khai báo/bắt) vs unchecked (RuntimeException, Error).
import java.io.FileNotFoundException;
import java.io.IOException;

public class Ex01_Hierarchy {
    static void chain(Class<?> c) {
        StringBuilder sb = new StringBuilder(c.getSimpleName());
        for (Class<?> p = c.getSuperclass(); p != null; p = p.getSuperclass()) sb.append(" -> ").append(p.getSimpleName());
    }
}
```

```

        boolean unchecked = RuntimeException.class.isAssignableFrom(c) || Error.class.isAssignableFrom(c);
        System.out.println(sb + (unchecked ? "    [unchecked]" : "    [checked]"));
    }

    public static void main(String[] args) {
        chain(FileNotFoundException.class);
        chain(IOException.class);
        chain(Exception.class);
        chain(NullPointerException.class);
        chain(NumberFormatException.class);
        chain(ArrayIndexOutOfBoundsException.class);
        chain(StackOverflowError.class);
        chain(ExceptionInInitializerError.class);
    }
}

```

Output thật (JDK 21.0.10):

```

FileNotFoundException -> IOException -> Exception -> Throwable -> Object    [checked]
IOException -> Exception -> Throwable -> Object    [checked]
Exception -> Throwable -> Object    [checked]
NullPointerException -> RuntimeException -> Exception -> Throwable -> Object    [unchecked]
NumberFormatException -> IllegalArgumentException -> RuntimeException -> Exception -> Throwable -> Object    [unchecked]
ArrayIndexOutOfBoundsException -> IndexOutOfBoundsException -> RuntimeException -> Exception -> Throwable -> Object    [unchecked]
StackOverflowError -> VirtualMachineError -> Error -> Throwable -> Object    [unchecked]
ExceptionInInitializerError -> LinkageError -> Error -> Throwable -> Object    [unchecked]

```

2. try / catch / finally

examples/ch04/Ex02_TryCatchFinally.java

```

// objective: 4.1
// Luồng chạy của try / catch / finally; catch đầu tiên khớp sẽ được chọn.
public class Ex02_TryCatchFinally {
    static void run(String input) {
        System.out.print(input + ": ");
        try {
            System.out.print("try ");
            int n = Integer.parseInt(input);
            System.out.print("parsed ");
            System.out.print(10 / n + " ");
        } catch (NumberFormatException e) {
            System.out.print("NFE ");
        } catch (RuntimeException e) {                // lớp cha phải đứng SAU lớp con
            System.out.print(e.getClass().getSimpleName() + " ");
        } finally {
            System.out.print("finally");              // luôn chạy
        }
        System.out.println();
    }

    public static void main(String[] args) {
        run("5");
        run("abc");
        run("");
    }
}

```

Output thật (JDK 21.0.10):

```

5: try parsed 2 finally
abc: try NFE finally
: try parsed ArithmeticException finally

```

3. Multi-catch

examples/ch04/Ex03_MultiCatch.java

```

// objective: 4.1
// Multi-catch: một khối catch cho nhiều loại exception không liên quan kế thừa.
import java.io.IOException;

public class Ex03_MultiCatch {
    static void risky(int k) throws IOException {
        switch (k) {
            case 1 -> throw new IOException("disk");
            case 2 -> throw new IllegalStateException("state");
            case 3 -> throw new ArrayIndexOutOfBoundsException(9);
            default -> System.out.println("ok " + k);
        }
    }
}

```

```

public static void main(String[] args) {
    for (int k = 0; k <= 3; k++) {
        try {
            risky(k);
        } catch (IOException | IllegalStateException e) {
            // e = new IOException(); // lỗi: biến của multi-catch ngấm final
            System.out.println("multi-catch: " + e.getMessage());
        } catch (RuntimeException e) {
            System.out.println("runtime: " + e);
        }
    }
    // catch (FileNotFoundException | IOException e) → lỗi: hai kiểu có quan hệ cha-con
}
}

```

Output thật (JDK 21.0.10):

```

ok 0
multi-catch: disk
multi-catch: state
runtime: java.lang.ArrayIndexOutOfBoundsException: Array index out of range: 9

```

4. try-with-resources

examples/ch04/Ex04_TryWithResources.java

```

// objective: 4.1
// try-with-resources: đóng tài nguyên theo thứ tự NGƯỢC, trước khi catch/finally chạy.
public class Ex04_TryWithResources {
    record Res(String name) implements AutoCloseable {
        Res { System.out.println("open " + name); }
        @Override public void close() { System.out.println("close " + name); }
    }

    public static void main(String[] args) {
        try (Res a = new Res("A"); Res b = new Res("B")) {
            System.out.println("body");
            throw new IllegalStateException("boom");
        } catch (IllegalStateException e) {
            System.out.println("catch " + e.getMessage());
        } finally {
            System.out.println("finally");
        }

        Res c = new Res("C"); // Java 9+: dùng biến effectively final có sẵn
        try (c) {
            System.out.println("using " + c.name());
        }
        // c = null; // nếu có dòng này, try (c) sẽ lỗi biên dịch
    }
}

```

Output thật (JDK 21.0.10):

```

open A
open B
body
close B
close A
catch boom
finally
open C
using C
close C

```

5. Suppressed exception vs finally ném exception

examples/ch04/Ex05_Suppressed.java

```

// objective: 4.1
// Suppressed exception: lỗi khi close() được "đính kèm" vào exception chính.
// So sánh với finally ném exception: exception gốc bị MẤT.
public class Ex05_Suppressed {
    static class Bad implements AutoCloseable {
        @Override public void close() throws Exception { throw new Exception("close failed"); }
    }

    public static void main(String[] args) {
        try (Bad b = new Bad()) {
            throw new RuntimeException("body failed");
        } catch (Exception e) {
            System.out.println("primary: " + e.getMessage());
        }
    }
}

```

```

        for (Throwable s : e.getSuppressed()) System.out.println("suppressed: " + s.getMessage());
    }

    try {
        try {
            throw new RuntimeException("try failed");
        } finally {
            throw new IllegalStateException("finally failed");    // thay thế exception gốc
        }
    } catch (RuntimeException e) {
        System.out.println("caught: " + e.getMessage() + ", suppressed=" + e.getSuppressed().length);
    }
}
}

```

Output thật (JDK 21.0.10):

```

primary: body failed
suppressed: close failed
caught: finally failed, suppressed=0

```

6. Custom exception

examples/ch04/Ex06_CustomException.java

```

// objective: 4.1
// Exception tự định nghĩa (custom): checked extends Exception, unchecked extends RuntimeException; gói cause.
public class Ex06_CustomException {
    static class InsufficientFundsException extends Exception {    // checked
        private final long missing;
        InsufficientFundsException(String msg, long missing) { super(msg); this.missing = missing; }
        long getMissing() { return missing; }
    }

    static class AccountLockedException extends RuntimeException { // unchecked
        AccountLockedException(String msg, Throwable cause) { super(msg, cause); }
    }

    static long balance = 100;

    static void withdraw(long amount) throws InsufficientFundsException {
        if (amount > balance) throw new InsufficientFundsException("need more money", amount - balance);
        balance -= amount;
    }

    public static void main(String[] args) {
        try {
            withdraw(30);
            withdraw(500);
        } catch (InsufficientFundsException e) {
            System.out.println(e.getMessage() + ", missing " + e.getMissing() + ", balance " + balance);
            try {
                throw new AccountLockedException("locked", e);
            } catch (AccountLockedException ex) {
                System.out.println(ex + " / cause: " + ex.getCause().getClass().getSimpleName());
            }
        }
    }
}

```

Output thật (JDK 21.0.10):

```

need more money, missing 430, balance 70
Ex06_CustomException$AccountLockedException: locked / cause: InsufficientFundsException

```

7. return và finally

examples/ch04/Ex07_FinallyReturn.java

```

// objective: 4.1
// return trong try và finally: giá trị trả về được "chốt" trước khi finally chạy; return trong finally thắng.
public class Ex07_FinallyReturn {
    static int a() {
        int x = 1;
        try {
            return x;    // giá trị 1 được chốt
        } finally {
            x = 99;    // không ảnh hưởng giá trị đã chốt
        }
    }

    static StringBuilder b() {

```

```

        StringBuilder sb = new StringBuilder("v1");
        try {
            return sb;           // chốt THAM CHIẾU
        } finally {
            sb.append("+finally"); // object vẫn bị sửa
        }
    }

    @SuppressWarnings("finally")
    static int c() {
        try {
            throw new RuntimeException("lost");
        } finally {
            return 3;           // nuốt luôn exception
        }
    }

    public static void main(String[] args) {
        System.out.println(a() + " " + b() + " " + c());
    }
}

```

Output thật (JDK 21.0.10):

```
1 v1+finally 3
```

8. Lỗi biên dịch hay gặp

examples/ch04/Ex08_CatchOrder.java

```

// objective: 4.1
// expect: compile-error
// Lỗi biên dịch: catch lớp cha trước lớp con; catch checked exception không thể xảy ra; chưa xử lý checked.
import java.io.FileNotFoundException;
import java.io.IOException;

public class Ex08_CatchOrder {
    static void read() throws IOException { }

    public static void main(String[] args) {
        try {
            read();
        } catch (IOException e) {
        } catch (FileNotFoundException e) {           // đã bị catch IOException bắt trước
        }

        try {
            System.out.println("no IO here");
        } catch (IOException e) {                     // khối try không thể ném IOException
        }

        read();                                       // checked exception chưa được bắt hay khai báo
    }
}

```

Output thật của javac (JDK 21.0.10) — cố ý không biên dịch được:

```

Ex08_CatchOrder.java:14: error: exception FileNotFoundException has already been caught
    } catch (FileNotFoundException e) {           // đã bị catch IOException bắt trước
      ^
Ex08_CatchOrder.java:19: error: exception IOException is never thrown in body of corresponding try statement
    } catch (IOException e) {                     // khối try không thể ném IOException
      ^
Ex08_CatchOrder.java:22: error: unreported exception IOException; must be caught or declared to be thrown
        read();
        ^
3 errors

```

9. Các exception hay gặp và message thật

examples/ch04/Ex09_CommonExceptions.java

```

// objective: 4.1
// Những exception hay gặp trong đề và thông điệp thật của chúng (JDK 21).
import java.util.List;

public class Ex09_CommonExceptions {
    interface Action { void run() throws Exception; }

    static void show(Action a) {
        try {
            a.run();
        }
    }
}

```

```

    } catch (Exception e) {
        System.out.println(e.getClass().getName() + ": " + e.getMessage());
    }
}

public static void main(String[] args) {
    show(() -> { String s = null; s.length(); });
    show(() -> { int[] arr = new int[2]; arr[2] = 1; });
    show(() -> Integer.parseInt("12a"));
    show(() -> { Object o = "x"; Integer i = (Integer) o; });
    show(() -> List.of(1).add(2));
    show(() -> List.of(1, 2).get(5));
    show(() -> { throw new IllegalArgumentException("bad arg"); });
    show(() -> new java.util.ArrayList<Integer>().iterator().next());
}
}

```

Output thật (JDK 21.0.10):

```

java.lang.NullPointerException: Cannot invoke "String.length()" because "<local0>" is null
java.lang.ArrayIndexOutOfBoundsException: Index 2 out of bounds for length 2
java.lang.NumberFormatException: For input string: "12a"
java.lang.ClassCastException: class java.lang.String cannot be cast to class java.lang.Integer (java.lang.String and
java.lang.Integer are in module java.base of loader 'bootstrap')
java.lang.UnsupportedOperationException: null
java.lang.IndexOutOfBoundsException: Index: 5 Size: 2
java.lang.IllegalArgumentException: bad arg
java.util.NoSuchElementException: null

```

10. Exception trong khởi tạo static

examples/ch04/Ex10_InitializerError.java

```

// objective: 4.1
// Exception trong khối static → ExceptionInInitializerError (một Error), lần sau → NoClassDefFoundError.
public class Ex10_InitializerError {
    static class Config {
        static int value = Integer.parseInt("not a number");
    }

    public static void main(String[] args) {
        for (int i = 0; i < 2; i++) {
            try {
                System.out.println(Config.value);
            } catch (Throwable t) {
                System.out.println(t.getClass().getSimpleName()
                    + (t.getCause() != null ? " caused by " + t.getCause().getClass().getSimpleName() : ""));
            }
        }
    }
}

```

Output thật (JDK 21.0.10):

```

ExceptionInInitializerError caused by NumberFormatException
NoClassDefFoundError caused by ExceptionInInitializerError

```

11. Rethrow và gói exception

examples/ch04/Ex11_Rethrow.java

```

// objective: 4.1
// Ném lại (rethrow) chính xác: catch (Exception e) rồi throw e – compiler biết kiểu thật là IOException.
import java.io.IOException;

public class Ex11_Rethrow {
    static void io(boolean fail) throws IOException {
        if (fail) throw new IOException("io");
    }

    static void precise(boolean fail) throws IOException { // KHÔNG cần "throws Exception"
        try {
            io(fail);
        } catch (Exception e) {
            System.out.println("log: " + e.getMessage());
            throw e; // e effectively final → rethrow chính xác
        }
    }

    static void wrap() {
        try {
            io(true);
        }
    }
}

```

```

    } catch (IOException e) {
        throw new RuntimeException("wrapped", e); // đổi checked thành unchecked
    }
}

public static void main(String[] args) {
    try {
        precise(true);
    } catch (IOException e) {
        System.out.println("main caught " + e.getMessage());
    }
    try {
        wrap();
    } catch (RuntimeException e) {
        System.out.println(e.getMessage() + " <- " + e.getCause().getMessage());
    }
}
}

```

Output thật (JDK 21.0.10):

```

log: io
main caught io
wrapped <- io

```

12. Exception không được bắt

examples/ch04/Ex12_StackTrace.java

```

// objective: 4.1
// expect: exception
// Exception không được bắt: JVM in stack trace ra System.err và kết thúc với exit code 1.
public class Ex12_StackTrace {
    static int depth(int n) {
        if (n == 0) throw new IllegalStateException("reached bottom");
        return depth(n - 1);
    }

    public static void main(String[] args) {
        System.out.println("before");
        depth(2);
        System.out.println("never printed");
    }
}

```

Output thật (JDK 21.0.10):

```

before
Exception in thread "main" java.lang.IllegalStateException: reached bottom
    at Ex12_StackTrace.depth(Ex12_StackTrace.java:6)
    at Ex12_StackTrace.depth(Ex12_StackTrace.java:7)
    at Ex12_StackTrace.depth(Ex12_StackTrace.java:7)
    at Ex12_StackTrace.main(Ex12_StackTrace.java:12)

```

Đi sâu

Cú pháp hợp lệ

- try { } catch (...) { }, try { } finally { }, try { } catch { } finally { } — try thường phải có ít nhất một catch hoặc finally.
- try-with-resources try (R r = ...) { } **không** cần catch/finally.
- Dấu ngoặc {} là **bắt buộc** cho try, catch, finally (khác if).
- Thứ tự catch: lớp con trước lớp cha. Catch lớp con sau lớp cha → lỗi “has already been caught”.
- Catch một **checked** exception mà khối try không thể ném → lỗi (trừ Exception và Throwable, vì chúng còn bao gồm unchecked).

Multi-catch

- catch (IOException | IllegalStateException e) — các kiểu không được có quan hệ cha-con.
- Biến e ngầm final; kiểu tĩnh của e là kiểu chung gần nhất (ở đây Exception).

try-with-resources

- Tài nguyên phải implements AutoCloseable (hoặc Closeable, là interface con).
- Mở theo thứ tự khai báo; đóng theo thứ tự **ngược lại**, ngay khi rời khỏi try, **trước** catch và finally.
- Biến tài nguyên ngầm final, phạm vi chỉ trong khối try.

- Java 9+: `try (r1; r2)` với biến có sẵn, miễn là effectively final.
- Nếu thân `try` ném exception và `close()` cũng ném → exception của thân là **chính**, exception của `close()` được thêm vào `getSuppressed()`.
- `AutoCloseable.close()` khai báo `throws Exception`; nếu lớp của bạn khai báo `close()` `throws Exception` thì code dùng nó phải xử lý `Exception`. `Closeable.close()` khai báo `throws IOException`.

finally

- Luôn chạy (trừ khi JVM dừng: `System.exit`, `crash`).
- `return` trong `try`: giá trị được chốt trước khi `finally` chạy. Primitive không đổi; object thì `finally` vẫn sửa được nội dung.
- `return` hoặc `throw` trong `finally` **thay thế** kết quả/exception của `try/catch` — exception gốc mất hẳn (không thành suppressed).

Custom exception

- Checked: extends `Exception`. Unchecked: extends `RuntimeException`.
- Nên có constructor (`String message`) và (`String message, Throwable cause`) gọi `super(...)`.
- `getMessage()` trả về `message`; `toString()` = tên lớp đầy đủ + ": " + `message`; `getCause()` trả về exception gốc.

Exception và override

Method override được: bỏ `throws`, ném checked exception **hẹp hơn**, thêm bất kỳ unchecked exception nào. Không được: thêm checked exception mới hoặc rộng hơn (xem thêm chương 3).

Rethrow chính xác (precise rethrow)

`catch (Exception e) { throw e; }` — nếu `e` effectively final, compiler chỉ coi nó là các checked exception mà khối `try` thực sự có thể ném. Gán lại `e` → mất tính năng này.

Lỗi và bẫy thường gặp (Exam traps)

- Catch lớp cha trước lớp con → lỗi biên dịch.
- Catch `IOException` khi `try` không thể ném nó → lỗi; catch `Exception` thì luôn được.
- Multi-catch với hai kiểu cha-con → lỗi; gán lại biến multi-catch → lỗi.
- `try-with-resources`: đóng **trước** `catch/finally`, thứ tự ngược.
- Exception từ `close()` là suppressed, không thay thế exception chính.
- `return/throw` trong `finally` nuốt exception.
- `finally` không chạy khi gọi `System.exit`.
- `NumberFormatException` là `IllegalArgumentException`; `ArrayIndexOutOfBoundsException` là `IndexOutOfBoundsException`.
- Exception trong khối tạo static → `ExceptionInInitializerError` (lần sau truy cập: `NoClassDefFoundError`).
- Override thêm checked exception rộng hơn → lỗi.
- Biến tài nguyên dùng ngoài khối `try` → lỗi phạm vi.
- Class kế thừa trực tiếp `Throwable` là **checked**.

Góc nhìn từ TypeScript

TypeScript	Java	Ghi chú
throw được mọi giá trị (throw "oops")	Chỉ throw object là Throwable	
catch (e) — e là unknown, một khối catch duy nhất	Nhiều khối catch theo kiểu, multi-catch	Java chọn catch theo kiểu, không cần if (e instanceof ...)
Không có checked exception	Checked exception: compiler bắt buộc handle or declare	Khác biệt lớn nhất
try/finally + using (TS 5.2, explicit resource management)	try-with-resources + AutoCloseable	Ý tưởng giống using / Symbol.dispose
new Error("msg", { cause })	new RuntimeException("msg", cause)	Cùng khái niệm “cause”
Promise bị reject không ai bắt → cảnh báo	Exception không bắt → luồng (thread) đó kết thúc, in stack trace	

Tóm tắt

- Checked = `Exception` trừ nhánh `RuntimeException`; phải handle or declare. `Error` là unchecked.
- Catch: lớp con trước lớp cha; multi-catch không cha-con, biến ngầm final.
- `try-with-resources`: đóng ngược, trước `catch/finally`; exception của `close()` là suppressed.
- `finally` luôn chạy; `return` trong `try` chốt giá trị; `return` trong `finally` nuốt exception.

- Custom exception: chọn `Exception` hoặc `RuntimeException`, truyền message và cause.

Bài tập (có lời giải)

Mọi đáp án đã được `tools/book.py` biên dịch và chạy để xác nhận (code: `examples/questions/ch04/`).

Câu hỏi

Câu 04-01 · Dễ · objective 4.1

Chương trình sau in ra gì?

```
public class Flow {
    public static void main(String[] args) {
        try {
            System.out.print("A");
            int x = 5 / 0;
            System.out.print("B");
        } catch (ArithmeticException e) {
            System.out.print("C");
        } finally {
            System.out.print("D");
        }
        System.out.print("E");
    }
}
```

- A. ACDE
- B. ABCDE
- C. ACD
- D. ADE

Câu 04-02 · Vừa · objective 4.1

Chương trình sau in ra gì?

```
public class Ret {
    static int f() {
        int x = 10;
        try {
            x++;
            return x;
        } finally {
            x += 100;
            System.out.print(x + " ");
        }
    }

    public static void main(String[] args) {
        System.out.println(f());
    }
}
```

- A. 111 111
- B. 111 11
- C. 11 11
- D. 11 111

Câu 04-03 · Vừa · objective 4.1

Dòng nào gây lỗi biên dịch?

```
import java.io.*;

public class Catches {
    static void load() throws FileNotFoundException { }

    public static void main(String[] args) {
        try { load(); }
        catch (FileNotFoundException e) { } // L1
        catch (IOException e) { } // L2
        try { load(); }
        catch (Exception e) { } // L3
        catch (RuntimeException e) { } // L4
        try { System.out.print(""); }
        catch (RuntimeException e) { } // L5
    }
}
```

```
}  
}
```

- A. L2
- B. L4
- C. L2 và L4
- D. L4 và L5
- E. Không dòng nào

Câu 04-04 · Vừa · objective 4.1

Chương trình sau in ra gì?

```
public class Twr {  
    static class R implements AutoCloseable {  
        final String n;  
        R(String n) { this.n = n; System.out.print("o" + n + " "); }  
        public void close() { System.out.print("c" + n + " "); }  
    }  
  
    public static void main(String[] args) {  
        try (R a = new R("1"); R b = new R("2")) {  
            System.out.print("body ");  
            throw new RuntimeException();  
        } catch (RuntimeException e) {  
            System.out.print("catch ");  
        } finally {  
            System.out.print("fin");  
        }  
    }  
}
```

- A. o1 o2 body c1 c2 catch fin
- B. o1 o2 body catch c2 c1 fin
- C. o1 o2 body c2 c1 catch fin
- D. o1 o2 body catch fin c2 c1

Câu 04-05 · Khó · objective 4.1

Kết quả của chương trình là gì?

```
public class Supp {  
    static class R implements AutoCloseable {  
        public void close() { throw new IllegalStateException("close"); }  
    }  
  
    public static void main(String[] args) {  
        try (R r = new R()) {  
            throw new IllegalArgumentException("body");  
        } catch (RuntimeException e) {  
            System.out.println(e.getMessage() + " " + e.getSuppressed().length + " "  
                               + e.getSuppressed()[0].getMessage());  
        }  
    }  
}
```

- A. In ra close 1 body
- B. Ném `ArrayIndexOutOfBoundsException` vì mảng suppressed rỗng
- C. Không biên dịch được vì `close()` không khai báo `throws`
- D. In ra body 1 close

Câu 04-06 · Vừa · objective 4.1

Chèn khối catch nào vào chỗ `// INSERT CODE HERE` thì chương trình biên dịch được? (Chọn 2 đáp án.)

```
import java.io.*;  
  
public class Multi {  
    static void work() throws IOException { }  
  
    public static void main(String[] args) {  
        try {  
            work();  
        } // INSERT CODE HERE  
    }  
}
```

- A. `catch (FileNotFoundException | IOException e) { }`

- B. `catch (IOException | RuntimeException e) { }`
- C. `catch (IOException | IllegalStateException e) { e = null; }`
- D. `catch (IOException e1 | RuntimeException e2) { }`
- E. `catch (RuntimeException | IOException e) { System.out.println(e); }`

Câu 04-07 · Dễ · objective 4.1

Hai phát biểu nào đúng? (Chọn 2 đáp án.)

- A. `NumberFormatException` là checked exception.
- B. `IOException` là checked exception.
- C. `Error` và các lớp con của nó là unchecked.
- D. Method ném `RuntimeException` phải khai báo `throws RuntimeException`.
- E. Lớp `Exception` là unchecked exception.

Câu 04-08 · Khó · objective 4.1

Chương trình sau in ra gì?

```
public class Nested {
    public static void main(String[] args) {
        StringBuilder sb = new StringBuilder();
        try {
            try {
                sb.append("1");
                throw new IllegalStateException("x");
            } catch (IllegalArgumentException e) {
                sb.append("2");
            } finally {
                sb.append("3");
            }
            sb.append("4");
        } catch (RuntimeException e) {
            sb.append("5");
        } finally {
            sb.append("6");
        }
        System.out.println(sb);
    }
}
```

- A. 13456
- B. 12356
- C. 136
- D. 1356

Câu 04-09 · Vừa · objective 4.1, 3.5

Dòng nào gây lỗi biên dịch?

```
import java.io.*;

public class Throws {
    static class Reader { void read() throws IOException { } }
    static class A extends Reader { void read() throws FileNotFoundException { } } // L1
    static class B extends Reader { void read() { } } // L2
    static class C extends Reader { void read() throws Exception { } } // L3
    static class D extends Reader { void read() throws IllegalStateException { } } // L4

    public static void main(String[] args) { }
}
```

- A. L1
- B. L2
- C. L3
- D. L4
- E. L3 và L4

Câu 04-10 · Vừa · objective 4.1

Chương trình sau in ra gì?

```
public class Custom {
    static class AppException extends Exception {
        AppException(String m) { super(m); }
    }
}
```

```

public static void main(String[] args) {
    try {
        throw new AppException("oops");
    } catch (AppException e) {
        System.out.println(e.getMessage() + " | " + e);
    }
}
}

```

- A. oops | oops
- B. oops | AppException: oops
- C. oops | Custom\$AppException: oops
- D. oops | Custom.AppException: oops

Câu 04-11 · Khó · objective 4.1

Kết quả của chương trình là gì?

```

public class Swallow {
    @SuppressWarnings("finally")
    static String f() {
        try {
            throw new RuntimeException("A");
        } catch (RuntimeException e) {
            throw new IllegalStateException("B");
        } finally {
            return "C";
        }
    }

    public static void main(String[] args) {
        try {
            System.out.println(f());
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }
    }
}

```

- A. In ra A
- B. In ra B
- C. Không biên dịch được: không có lệnh return tới được
- D. In ra C

Câu 04-12 · Vừa · objective 4.1

Chèn dòng nào vào chỗ `// INSERT CODE HERE` thì chương trình biên dịch được? (Chọn 3 đáp án.)

```

public class Res {
    static class R implements AutoCloseable { public void close() { } }

    public static void main(String[] args) {
        R outer = new R();
        // INSERT CODE HERE
    }
}

```

- A. try (outer) { }
- B. try (R r = new R()) { }
- C. try (R r = new R()); { }
- D. try (Object o = new Object()) { }
- E. try (R r = new R()) { } r.close();
- F. try { }

Câu 04-13 · Vừa · objective 4.1

Chương trình sau in ra gì?

```

public class Boot {
    static class Cfg {
        static final int N = compute();
        static int compute() {
            System.out.print("compute ");
            return 1 / 0;
        }
    }
}

```

```

public static void main(String[] args) {
    try {
        System.out.print(Cfg.N);
    } catch (ExceptionInInitializerError e) {
        System.out.print("EIIIE:" + e.getCause().getClass().getSimpleName());
    }
}
}

```

- A. compute EIIIE:ArithmeticException
- B. EIIIE:ArithmeticException
- C. compute 0
- D. compute EIIIE:ExceptionInInitializerError

Câu 04-14 · Vừa · objective 4.1

Hai phát biểu nào đúng? (Chọn 2 đáp án.)

- A. Khối finally vẫn chạy khi khối try thực hiện return.
- B. try-with-resources đóng tài nguyên sau khi khối catch chạy xong.
- C. Mỗi khối try (không có tài nguyên) phải có ít nhất một catch.
- D. Biến của multi-catch ngầm final.
- E. Nếu try gọi System.exit(0) thì finally vẫn chạy.

Câu 04-15 · Dễ · objective 4.1

Chương trình sau in ra gì?

```

public class Parse {
    public static void main(String[] args) {
        try {
            int n = Integer.parseInt("3.5");
            System.out.println(n);
        } catch (IllegalArgumentException e) {
            System.out.println("IAE");
        } catch (RuntimeException e) {
            System.out.println("RE");
        }
    }
}

```

- A. 3
- B. RE
- C. IAE
- D. Không biên dịch được vì thứ tự các khối catch

Câu 04-16 · Khó · objective 4.1

Dòng nào gây lỗi biên dịch?

```

import java.io.*;

public class Re {
    static void io() throws IOException { }

    static void a() throws IOException {
        try { io(); } catch (Exception e) { throw e; } // L1
    }
    static void b() throws IOException {
        try { io(); } catch (Exception e) { e = new Exception(); throw e; } // L2
    }
    static void c() {
        try { io(); } catch (IOException e) { throw new RuntimeException(e); } // L3
    }

    public static void main(String[] args) { }
}

```

- A. L1
- B. L2
- C. L3
- D. L1 và L2
- E. Không dòng nào

Câu 04-17 · Vừa · objective 4.1

Chương trình sau in ra gì?

```
public class Order {
    static String log = "";

    static void m() {
        try {
            log += "t";
            throw new RuntimeException();
        } catch (RuntimeException e) {
            log += "c";
            throw new IllegalStateException();
        } finally {
            log += "f";
        }
    }

    public static void main(String[] args) {
        try { m(); } catch (IllegalStateException e) { log += "m"; }
        System.out.println(log);
    }
}
```

- A. tcm
- B. tcmf
- C. tfm
- D. tcfm

Câu 04-18 · Khó · objective 4.1

Khai báo nào, chèn vào chỗ `// INSERT CODE HERE`, làm chương trình biên dịch và in ra `ok`? (Chọn 3 đáp án.)

```
public class Declare {
    // INSERT CODE HERE

    static void check(int v) throws BadValue {
        if (v < 0) throw new BadValue("negative");
    }

    public static void main(String[] args) throws Throwable {
        check(1);
        System.out.println("ok");
    }
}
```

- A. static class BadValue extends Exception { BadValue(String m) { super(m); } }
- B. static class BadValue extends RuntimeException { BadValue(String m) { super(m); } }
- C. static class BadValue extends Throwable { BadValue(String m) { super(m); } }
- D. static class BadValue extends Exception { }
- E. static class BadValue implements Exception { BadValue(String m) { } }

Câu 04-19 · Vừa · objective 4.1

Kết quả của chương trình là gì?

```
public class Cause {
    public static void main(String[] args) {
        try {
            try {
                Object o = null;
                o.toString();
            } catch (NullPointerException e) {
                throw new IllegalStateException("wrapped", e);
            }
        } catch (IllegalStateException e) {
            System.out.println(e.getMessage() + " / " + (e.getCause() instanceof NullPointerException));
        }
    }
}
```

- A. In ra wrapped / true
- B. In ra wrapped / false
- C. In ra null / true
- D. Ném NullPointerException ra khỏi main

Câu 04-20 · Dễ · objective 4.1

Dòng nào gây lỗi biên dịch?

```
import java.io.*;

public class Unreported {
    static void save() throws IOException { }
    static void run() { save(); } // L1
    static void run2() throws Exception { save(); } // L2
    static void run3() { try { save(); } catch (Exception e) { } } // L3

    public static void main(String[] args) { }
}
```

- A. L1
- B. L2
- C. L3
- D. L1 và L3
- E. Không dòng nào

Lời giải

Câu 04-01 — Đáp án: A

- **Vì sao đúng:** 5 / 0 ném `ArithmeticException` nên B bị bỏ qua. Catch khớp → in C. `finally` luôn chạy → D. Exception đã được xử lý nên chương trình chạy tiếp → E.
- **B sai:** Lệnh sau dòng ném exception trong khối `try` không chạy.
- **C sai:** Exception đã được bắt, nên code sau `try` vẫn chạy và in E.
- **D sai:** Catch `ArithmeticException` khớp nên C được in.
- **Kiểm chứng:** [examples/questions/ch04/Q04_01/](#) — output confirmed (python3 tools/book.py questions ch04).

Câu 04-02 — Đáp án: B

- **Vì sao đúng:** `return x` chốt giá trị 11 trước khi `finally` chạy. `finally` đổi biến `x` thành 111 và in ra, nhưng giá trị trả về (kiểu primitive) đã được chốt là 11.
- **A sai:** Với primitive, thay đổi biến trong `finally` không đổi giá trị đã chốt để trả về.
- **C sai:** `finally` in giá trị của `x` sau khi cộng 100, tức là 111.
- **D sai:** Thứ tự in: `finally` in trước (111), rồi `main` in giá trị trả về (11).
- **Kiểm chứng:** [examples/questions/ch04/Q04_02/](#) — output confirmed (python3 tools/book.py questions ch04).

Câu 04-03 — Đáp án: B

- **Vì sao đúng:** `RuntimeException` là lớp con của `Exception`; catch `Exception` đứng trước đã bắt hết → L4 lỗi “exception `RuntimeException` has already been caught”. L2 hợp lệ vì `IOException` rộng hơn (không phải lớp con của) `FileNotFoundException`. L5 hợp lệ vì unchecked exception luôn được phép catch.
- **A sai:** Catch lớp cha sau lớp con là hợp lệ; chỉ lớp con sau lớp cha mới lỗi.
- **C sai:** L2 hợp lệ (xem A).
- **D sai:** Catch unchecked exception (L5) luôn hợp lệ, kể cả khi khối try không ném gì.
- **E sai:** L4 bị catch `Exception` phía trên che mất.
- **Kiểm chứng:** [examples/questions/ch04/Q04_03/](#) — compile error confirmed at [‘L4’] (python3 tools/book.py questions ch04).

Câu 04-04 — Đáp án: C

- **Vì sao đúng:** Tài nguyên mở theo thứ tự khai báo và đóng theo thứ tự ngược lại (b rồi a). Việc đóng xảy ra ngay khi rời khỏi `try`, trước khi `catch` và `finally` chạy.
- **A sai:** Đóng theo thứ tự ngược: c2 trước c1.
- **B sai:** Tài nguyên được đóng trước khi vào `catch`.
- **D sai:** Tài nguyên được đóng trước cả `catch` và `finally`.
- **Kiểm chứng:** [examples/questions/ch04/Q04_04/](#) — output confirmed (python3 tools/book.py questions ch04).

Câu 04-05 — Đáp án: D

- **Vì sao đúng:** Exception chính là exception ném trong thân `try` (body). Exception do `close()` ném sau đó không làm mất exception chính mà được thêm vào danh sách **suppressed** của nó. Vậy có 1 suppressed với message `close`.
- **A sai:** Exception từ thân `try` là exception chính; exception từ `close()` mới là suppressed.
- **B sai:** Exception của `close()` được thêm vào mảng suppressed, nên mảng có 1 phần tử.
- **C sai:** `close()` ném unchecked exception, không cần `throws`; và override được phép bỏ `throws Exception` của `AutoCloseable.close()`.

- **Kiểm chứng:** `examples/questions/ch04/Q04_05/` — output confirmed (python3 tools/book.py questions ch04).

Câu 04-06 — Đáp án: B, E

- **Vì sao đúng:** Multi-catch dùng **một** biến cho các kiểu **không có quan hệ cha-con**. `IOException` và `RuntimeException` không liên quan nên B và E hợp lệ (thứ tự các kiểu không quan trọng).
- **A sai:** `FileNotFoundException` là lớp con của `IOException` → lỗi “Alternatives in a multi-catch statement cannot be related by subclassing”.
- **C sai:** Biến của multi-catch ngầm `final`, không gán lại được.
- **D sai:** Multi-catch chỉ có một tên biến ở cuối.
- **Kiểm chứng:** `examples/questions/ch04/Q04_06/` — variants: BE satisfy compiles (python3 tools/book.py questions ch04).

Câu 04-07 — Đáp án: B, C

- **Vì sao đúng:** Checked exception = `Exception` và các lớp con, **trừ** nhánh `RuntimeException`. `IOException` là checked. `Error` (và lớp con) là unchecked, giống `RuntimeException`.
- **A sai:** `NumberFormatException` → `IllegalArgumentException` → `RuntimeException`: unchecked.
- **D sai:** Unchecked exception không bắt buộc khai báo `throws` (khai báo vẫn được phép).
- **E sai:** `Exception` là checked: ném nó mà không khai báo `throws` là lỗi biên dịch.
- **Kiểm chứng:** `examples/questions/ch04/Q04_07/` — each option proven true/false by a program (python3 tools/book.py questions ch04).

Câu 04-08 — Đáp án: D

- **Vì sao đúng:** `IllegalStateException` không phải `IllegalArgumentException` nên catch trong không bắt; `finally` trong vẫn chạy (3), rồi exception thoát ra ngoài, bỏ qua 4. Catch ngoài (`RuntimeException`) bắt (5), `finally` ngoài (6).
- **A sai:** Exception chưa được xử lý khi rời khỏi try trong, nên `sb.append("4")` bị bỏ qua.
- **B sai:** `IllegalStateException` và `IllegalArgumentException` là hai lớp anh em, không khớp nhau.
- **C sai:** Catch ngoài bắt `RuntimeException`, nên có in 5.
- **Kiểm chứng:** `examples/questions/ch04/Q04_08/` — output confirmed (python3 tools/book.py questions ch04).

Câu 04-09 — Đáp án: C

- **Vì sao đúng:** Method override không được ném checked exception **rộng hơn** hoặc mới so với method bị override. Exception rộng hơn `IOException` → L3 lỗi.
- **A sai:** Ném checked exception **hẹp hơn** (`FileNotFoundException`) là được phép.
- **B sai:** Bỏ hẳn `throws` là được phép.
- **D sai:** Unchecked exception (`IllegalStateException`) luôn được phép thêm.
- **E sai:** L4 hợp lệ vì `IllegalStateException` là unchecked.
- **Kiểm chứng:** `examples/questions/ch04/Q04_09/` — compile error confirmed at ['L3'] (python3 tools/book.py questions ch04).

Câu 04-10 — Đáp án: C

- **Vì sao đúng:** `getMessage()` trả về message. `toString()` của `Throwable` là `getClass().getName() + ": " + message`. Tên (binary name) của lớp lồng là `Custom$AppException`.
- **A sai:** `toString()` có thêm tên lớp phía trước message.
- **B sai:** `getName()` trả về tên đầy đủ, gồm cả lớp bao ngoài.
- **D sai:** Tên lớp lồng lúc chạy dùng dấu `$`, không phải dấu chấm.
- **Kiểm chứng:** `examples/questions/ch04/Q04_10/` — output confirmed (python3 tools/book.py questions ch04).

Câu 04-11 — Đáp án: D

- **Vì sao đúng:** `return` trong `finally` làm method kết thúc bình thường và **huỷ** exception đang bay (B). Vì vậy `f()` trả về "C". Đây là lý do không nên `return` trong `finally`.
- **A sai:** Exception A đã được catch.
- **B sai:** Exception B bị `return` trong `finally` nuốt mất.
- **C sai:** `return "C"` trong `finally` là lệnh `return` hợp lệ; code biên dịch được (chỉ có cảnh báo).
- **Kiểm chứng:** `examples/questions/ch04/Q04_11/` — output confirmed (python3 tools/book.py questions ch04).

Câu 04-12 — Đáp án: A, B, C

- **Vì sao đúng:** A: từ Java 9 dùng được biến effectively final có sẵn. B: try-with-resources không bắt buộc có `catch/finally`. C: dấu `;` thừa sau tài nguyên cuối cùng được cho phép.
- **D sai:** Tài nguyên phải implements `AutoCloseable`; `Object` thì không.
- **E sai:** Biến `r` chỉ có phạm vi trong khối try.
- **F sai:** try thường (không có tài nguyên) phải có ít nhất một `catch` hoặc `finally`.
- **Kiểm chứng:** `examples/questions/ch04/Q04_12/` — variants: ABC satisfy compiles (python3 tools/book.py questions ch04).

Câu 04-13 — Đáp án: A

- **Vì sao đúng:** Truy cập `Cfg.N` lần đầu làm lớp `Cfg` được khởi tạo; `compute()` in `compute` rồi ném `ArithmeticException`. Exception trong khởi tạo static được gói vào `ExceptionInInitializerError`, với `getCause()` là exception gốc.
- **B sai:** `compute()` chạy và in trước khi ném exception.
- **C sai:** `1 / 0` với số nguyên ném exception, không trả về 0.
- **D sai:** `getCause()` trả về exception gốc (`ArithmeticException`).
- **Kiểm chứng:** `examples/questions/ch04/Q04_13/` — output confirmed (`python3 tools/book.py questions ch04`).

Câu 04-14 — Đáp án: A, D

- **Vì sao đúng:** A: `finally` chạy trước khi method thực sự trả về. D: biến trong `catch (A | B e)` nằm gần `final`.
- **B sai:** Tài nguyên được đóng **trước** khi `catch` chạy.
- **C sai:** `try-finally` (không có `catch`) là hợp lệ.
- **E sai:** `System.exit` dừng JVM ngay; `finally` không chạy.
- **Kiểm chứng:** `examples/questions/ch04/Q04_14/` — each option proven true/false by a program (`python3 tools/book.py questions ch04`).

Câu 04-15 — Đáp án: C

- **Vì sao đúng:** "3.5" không phải số nguyên → `NumberFormatException`, là lớp con của `IllegalArgumentException`. Catch đầu tiên khớp được chọn → IAE.
- **A sai:** `parseInt` không làm tròn; chuỗi có dấu chấm là không hợp lệ.
- **B sai:** Catch đầu tiên khớp được chọn; `IllegalArgumentException` đứng trước.
- **D sai:** Lớp con (`IllegalArgumentException`) đứng trước lớp cha (`RuntimeException`) là đúng thứ tự.
- **Kiểm chứng:** `examples/questions/ch04/Q04_15/` — output confirmed (`python3 tools/book.py questions ch04`).

Câu 04-16 — Đáp án: B

- **Vì sao đúng:** L1 dùng **rethrow chính xác** (precise rethrow): `e` effectively final nên compiler biết chỉ có thể là `IOException` (hoặc unchecked) → khớp `throws IOException`. Ở L2, `e` bị gán lại nên mất tính effectively final; `throw e` bị coi là ném `Exception` → "unreported exception Exception".
- **A sai:** Rethrow chính xác cho phép `throw e` với `e` kiểu `Exception` khi `e` không bị gán lại.
- **C sai:** Gói checked exception vào `RuntimeException` là hợp lệ và không cần `throws`.
- **D sai:** L1 hợp lệ (xem A).
- **E sai:** L2 lỗi vì `e` không còn effectively final.
- **Kiểm chứng:** `examples/questions/ch04/Q04_16/` — compile error confirmed at ['L2'] (`python3 tools/book.py questions ch04`).

Câu 04-17 — Đáp án: D

- **Vì sao đúng:** `try` ném exception → `catch` chạy (c) và ném exception mới. Trước khi exception rời method, `finally` chạy (f). Sau đó `main` bắt exception (m).
- **A sai:** `finally` luôn chạy, kể cả khi `catch` ném exception mới.
- **B sai:** `finally` của `m()` chạy **trước** khi exception tới `main`.
- **C sai:** `catch` trong `m()` có chạy (bắt `RuntimeException`).
- **Kiểm chứng:** `examples/questions/ch04/Q04_17/` — output confirmed (`python3 tools/book.py questions ch04`).

Câu 04-18 — Đáp án: A, B, C

- **Vì sao đúng:** Có thể ném bất kỳ lớp con nào của `Throwable`. A (checked), B (unchecked) và C (kế thừa trực tiếp `Throwable`, được coi là checked) đều hợp lệ. `main` khai báo `throws Throwable` nên phủ cả trường hợp C. Lưu ý: nếu `main` chỉ khai báo `throws Exception` thì C sẽ lỗi, vì `Throwable` rộng hơn `Exception`.
- **D sai:** `new BadValue("negative")` cần một constructor nhận `String`; lớp chỉ có default constructor.
- **E sai:** `Exception` là class, phải dùng `extends`, không phải `implements`.
- **Kiểm chứng:** `examples/questions/ch04/Q04_18/` — variants: ABC satisfy output (`python3 tools/book.py questions ch04`).

Câu 04-19 — Đáp án: A

- **Vì sao đúng:** `o.toString()` ném `NullPointerException`, được catch trong và gói vào `IllegalStateException` với cause là NPE. Catch ngoài bắt `IllegalStateException`: message là `wrapped`, cause đúng là NPE.
- **B sai:** Constructor (`String, Throwable`) lưu exception gốc làm cause.
- **C sai:** Message là chuỗi truyền vào constructor: `wrapped`.
- **D sai:** NPE đã được catch trong và gói lại; exception mới được catch ngoài bắt.
- **Kiểm chứng:** `examples/questions/ch04/Q04_19/` — output confirmed (`python3 tools/book.py questions ch04`).

Câu 04-20 — Đáp án: A

- **Vì sao đúng:** Checked exception phải được **bắt** (catch) hoặc **khai báo** (throws) — quy tắc “handle or declare”. L1 không làm cả hai. L2 khai báo `throws Exception` (rộng hơn) → được. L3 bắt `Exception` → được.
- **B sai:** Khai báo `throws` một lớp cha của exception là hợp lệ.
- **C sai:** Catch `Exception` bắt được `IOException`.
- **D sai:** L3 hợp lệ (xem C).
- **E sai:** L1 vi phạm quy tắc handle-or-declare.
- *Kiểm chứng:* `examples/questions/ch04/Q04_20/` — compile error confirmed at `['L1']` (`python3 tools/book.py questions ch04`).

Đọc thêm (link chính thức — bị chặn trong sandbox nên mình chưa mở được)

- JLS Chương 11 Exceptions và §14.20 The try statement: <https://docs.oracle.com/javase/specs/jls/se21/html/jls-11.html>
- Javadoc `Throwable`: <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/Throwable.html>

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-28:

- `Throwable.java` (`getSuppressed`, `addSuppressed`, `toString`):
<https://raw.githubusercontent.com/openjdk/jdk21u/master/src/java.base/share/classes/java/lang/Throwable.java>
- `AutoCloseable.java` (hợp đồng `close()`):
<https://raw.githubusercontent.com/openjdk/jdk21u/master/src/java.base/share/classes/java/lang/AutoCloseable.java>
- `ExceptionInInitializerError.java`:
<https://raw.githubusercontent.com/openjdk/jdk21u/master/src/java.base/share/classes/java/lang/ExceptionInInitializerError.java>

Chương 5 — Mảng và Collections (Working with Arrays and Collections)

Mục tiêu

Objective 5.1: tạo mảng (array), List, Set, Map, Deque; thêm, xoá, sửa, lấy và sắp xếp phần tử. Để làm tốt phần này bạn cũng cần **generics** (kiểu tham số hoá) và Comparable/Comparator.

Giải thích đơn giản

- **Mảng** có kích thước **cố định**, chứa primitive hoặc object, truy cập bằng chỉ số `a[i]`.
- **Collections Framework** là các cấu trúc dữ liệu co giãn được, chỉ chứa **object** (primitive được autoboxing):
 - List — có thứ tự, cho phép trùng (như Array trong TypeScript).
 - Set — không trùng (như Set trong JS).
 - Map — cặp key → value (như Map trong JS). Map **không** kế thừa Collection.
 - Deque — hàng đợi hai đầu: dùng làm stack (LIFO) hoặc queue (FIFO).
- **Generics** `List<String>` cho compiler biết kiểu phần tử, để kiểm tra lúc biên dịch.

Interface	Cài đặt hay gặp	Thứ tự	null?
List	ArrayList, LinkedList	Theo chỉ số	Có
Set	HashSet	Không bảo đảm	Một null
	LinkedHashSet	Thứ tự thêm vào	Một null
	TreeSet	Sắp xếp (tự nhiên hoặc Comparator)	Không (với thứ tự tự nhiên)
Map	HashMap	Không bảo đảm	Một key null, value null
	LinkedHashMap	Thứ tự thêm vào	Như trên
	TreeMap	Sắp xếp theo key	Key null: không
Deque	ArrayDeque	Hai đầu	Không
	LinkedList	Hai đầu	Có

Ví dụ

Code trong `examples/ch05/`. Chạy lại: `python3 tools/book.py examples ch05`. Output thật, JDK 21.0.10.

1. Mảng

`examples/ch05/Ex01_Arrays.java`

```
// objective: 5.1
// Khai báo, khởi tạo mảng; giá trị mặc định; mảng nhiều chiều (có thể "răng cưa" - jagged).
import java.util.Arrays;

public class Ex01_Arrays {
    public static void main(String[] args) {
        int[] a = new int[3];           // mặc định 0
        int b[] = {5, 6, 7};           // cú pháp C, vẫn hợp lệ
        String[] names = new String[2]; // mặc định null
        int[] c = new int[]{1, 2};      // không được ghi kích thước khi có {...}
        System.out.println(Arrays.toString(a) + " " + Arrays.toString(b) + " "
            + Arrays.toString(names) + " " + c.length);

        int[][] grid = new int[2][3];
        int[][] jagged = {{1}, {2, 3}, {4, 5, 6}};
        int[][] lazy = new int[2][];    // chiều thứ hai chưa tạo
        lazy[0] = new int[]{9};
        System.out.println(grid[1].length + " " + jagged[2][1] + " " + Arrays.deepToString(jagged)
            + " " + lazy[1]);

        int[] ids, other;               // cả hai là int[]
        int[] x, y[];                   // x là int[], y là int[][]
        y = new int[1][1];
        System.out.println(y[0][0] + " " + a.getClass().getSimpleName() + " " + (b instanceof Object));
        try {
            System.out.println(b[3]);
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

```
}  
}
```

Output thật (JDK 21.0.10):

```
[0, 0, 0] [5, 6, 7] [null, null] 2  
3 5 [[1], [2, 3], [4, 5, 6]] null  
0 int[] true  
Index 3 out of bounds for length 3
```

2. Lớp tiện ích Arrays

examples/ch05/Ex02_ArraysUtil.java

```
// objective: 5.1  
// Lớp tiện ích Arrays: sort, binarySearch, fill, copyOf, equals, compare, mismatch, asList.  
import java.util.Arrays;  
import java.util.List;  
  
public class Ex02_ArraysUtil {  
    public static void main(String[] args) {  
        int[] nums = {40, 10, 30, 20};  
        Arrays.sort(nums);  
        System.out.println(Arrays.toString(nums));  
        System.out.println(Arrays.binarySearch(nums, 30) + " " + Arrays.binarySearch(nums, 25)  
            + " " + Arrays.binarySearch(nums, 5)); // không thấy: -(vị trí chèn) - 1  
  
        String[] words = {"banana", "Apple", "cherry", "apple"};  
        Arrays.sort(words); // chữ HOA đứng trước chữ thường (theo Unicode)  
        System.out.println(Arrays.toString(words));  
  
        int[] copy = Arrays.copyOf(nums, 6);  
        int[] range = Arrays.copyOfRange(nums, 1, 3);  
        int[] filled = new int[3];  
        Arrays.fill(filled, 7);  
        System.out.println(Arrays.toString(copy) + " " + Arrays.toString(range) + " " + Arrays.toString(filled));  
  
        int[] p = {1, 2, 3}, q = {1, 2, 3}, r = {1, 2, 4};  
        System.out.println((p == q) + " " + p.equals(q) + " " + Arrays.equals(p, q)  
            + " " + Arrays.compare(p, r) + " " + Arrays.mismatch(p, r) + " " + Arrays.mismatch(p, q));  
  
        String[] backing = {"x", "y"};  
        List<String> view = Arrays.asList(backing); // list cố định kích thước, "nhìn" vào mảng  
        view.set(0, "CHANGED");  
        System.out.println(backing[0]);  
        try {  
            view.add("z");  
        } catch (UnsupportedOperationException e) {  
            System.out.println("asList: không add/remove được");  
        }  
    }  
}
```

Output thật (JDK 21.0.10):

```
[10, 20, 30, 40]  
2 -3 -1  
[Apple, apple, banana, cherry]  
[10, 20, 30, 40, 0, 0] [20, 30] [7, 7, 7]  
false false true -1 2 -1  
CHANGED  
asList: không add/remove được
```

3. List

examples/ch05/Ex03_Lists.java

```
// objective: 5.1  
// ArrayList: add, set, get, remove(int) vs remove(Object), indexOf, contains, subList.  
import java.util.ArrayList;  
import java.util.List;  
  
public class Ex03_Lists {  
    public static void main(String[] args) {  
        List<String> list = new ArrayList<>();  
        list.add("a");  
        list.add("c");  
        list.add(1, "b"); // chèn tại chỉ số 1  
        System.out.println(list + " size=" + list.size());  
        String old = list.set(2, "C"); // set trả về phần tử cũ  
        System.out.println(old + " -> " + list + " " + list.get(0) + " " + list.indexOf("z"));  
        list.remove("a"); // remove(Object)
```

```

list.remove(0); // remove(int index)
System.out.println(list + " " + list.contains("C") + " " + list.isEmpty());

List<Integer> nums = new ArrayList<>(List.of(10, 20, 30, 40));
nums.remove(1); // xóa CHỈ SỐ 1 (giá trị 20)!
nums.remove(Integer.valueOf(40)); // xóa GIÁ TRỊ 40
System.out.println(nums);

List<Integer> big = new ArrayList<>(List.of(1, 2, 3, 4, 5));
List<Integer> sub = big.subList(1, 3); // view [2, 3]
sub.set(0, 99);
System.out.println(sub + " " + big);
try {
    list.get(5);
} catch (IndexOutOfBoundsException e) {
    System.out.println(e.getMessage());
}
}
}

```

Output thật (JDK 21.0.10):

```

[a, b, c] size=3
c -> [a, b, c] a -1
[C] true false
[10, 30]
[99, 3] [1, 99, 3, 4, 5]
Index 5 out of bounds for length 1

```

4. Sửa list khi đang duyệt

examples/ch05/Ex04_ModifyWhileIterating.java

```

// objective: 5.1
// Sửa list khi đang duyệt bằng for-each -> ConcurrentModificationException; dùng removeIf hoặc Iterator.
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

public class Ex04_ModifyWhileIterating {
    public static void main(String[] args) {
        List<String> names = new ArrayList<>(List.of("An", "Binh", "Chi", "Dung"));
        try {
            for (String n : names) {
                if (n.startsWith("B")) names.remove(n);
            }
        } catch (java.util.ConcurrentModificationException e) {
            System.out.println("ConcurrentModificationException");
        }
        System.out.println(names);

        names.removeIf(n -> n.length() == 3); // cách an toàn
        System.out.println(names);

        Iterator<String> it = names.iterator();
        while (it.hasNext()) {
            if (it.next().equals("An")) it.remove(); // xóa qua iterator: an toàn
        }
        System.out.println(names);

        List<Integer> nums = new ArrayList<>(List.of(1, 2, 3, 4));
        nums.replaceAll(x -> x * 10);
        System.out.println(nums);
    }
}

```

Output thật (JDK 21.0.10):

```

ConcurrentModificationException
[An, Chi, Dung]
[An, Dung]
[Dung]
[10, 20, 30, 40]

```

5. Set

examples/ch05/Ex05_Sets.java

```

// objective: 5.1
// Set: không trùng lặp. HashSet (không thứ tự), LinkedHashSet (thứ tự thêm), TreeSet (sắp xếp).
import java.util.*;

```

```

public class Ex05_Sets {
    public static void main(String[] args) {
        List<String> data = List.of("pear", "apple", "fig", "apple", "kiwi");
        Set<String> linked = new LinkedHashSet<>(data);
        TreeSet<String> tree = new TreeSet<>(data);
        System.out.println(linked + " " + tree);

        Set<Integer> hs = new HashSet<>();
        System.out.println(hs.add(5) + " " + hs.add(5) + " " + hs.size()); // add trả về false khi đã có

        TreeSet<Integer> t = new TreeSet<>(List.of(10, 20, 30, 40));
        System.out.println(t.first() + " " + t.last() + " " + t.floor(25) + " " + t.ceiling(25)
            + " " + t.lower(10) + " " + t.higher(40));
        System.out.println(t.headSet(30) + " " + t.tailSet(30) + " " + t.subSet(15, 35) + " " + t.descendingSet());

        TreeSet<String> byLength = new TreeSet<>(Comparator.comparing(String::length));
        byLength.addAll(List.of("aa", "b", "cc", "ddd")); // "cc" bị coi là trùng "aa" (cùng độ dài)
        System.out.println(byLength);
        try {
            new TreeSet<Object>().add(null);
        } catch (NullPointerException e) {
            System.out.println("TreeSet không nhận null");
        }
    }
}

```

Output thật (JDK 21.0.10):

```

[pear, apple, fig, kiwi] [apple, fig, kiwi, pear]
true false 1
10 40 20 30 null null
[10, 20] [30, 40] [20, 30] [40, 30, 20, 10]
[b, aa, ddd]
TreeSet không nhận null

```

6. Map

examples/ch05/Ex06_Maps.java

```

// objective: 5.1
// Map: put/get/getOrDefault/putIfAbsent/merge/compute; HashMap vs LinkedHashMap vs TreeMap.
import java.util.*;

public class Ex06_Maps {
    public static void main(String[] args) {
        Map<String, Integer> stock = new HashMap<>();
        System.out.println(stock.put("apple", 3) + " " + stock.put("apple", 5)); // null, rồi giá trị cũ 3
        stock.putIfAbsent("apple", 100); // đã có → không đổi
        stock.putIfAbsent("pear", 1);
        System.out.println(stock.get("apple") + " " + stock.get("none") + " " + stock.getOrDefault("none", 0));

        Map<String, Integer> counts = new TreeMap<>();
        for (String w : "b a c a b a".split(" ")) counts.merge(w, 1, Integer::sum);
        System.out.println(counts); // TreeMap: sắp xếp theo key

        counts.compute("a", (k, v) -> v == null ? 1 : v * 10);
        counts.computeIfAbsent("z", k -> 0);
        counts.computeIfPresent("b", (k, v) -> null); // trả về null → xóa key
        System.out.println(counts + " " + counts.containsKey("b") + " " + counts.containsValue(0));

        for (Map.Entry<String, Integer> e : counts.entrySet()) System.out.print(e.getKey() + "=" + e.getValue() + ";");
        System.out.println(" keys=" + counts.keySet() + " values=" + counts.values());

        TreeMap<Integer, String> tm = new TreeMap<>(Map.of(1, "one", 5, "five", 9, "nine"));
        System.out.println(tm.firstKey() + " " + tm.floorKey(6) + " " + tm.headMap(5) + " " + tm.tailMap(5, true));

        Map<String, Integer> fixed = Map.of("k", 1);
        try {
            fixed.put("x", 2);
        } catch (UnsupportedOperationException e) {
            System.out.println("Map.of: không sửa được");
        }
        HashMap<String, String> nulls = new HashMap<>();
        nulls.put(null, "null key OK"); nulls.put("v", null);
        System.out.println(nulls.get(null) + " " + nulls.containsKey("v"));
    }
}

```

Output thật (JDK 21.0.10):

```

null 3
5 null 0
{a=3, b=2, c=1}
{a=30, c=1, z=0} false true

```

```
a=30;c=1;z=0; keys=[a, c, z] values=[30, 1, 0]
1 5 {1=one} {5=five, 9=nine}
Map.of: không sửa được
null key OK true
```

7. Deque: stack và queue

examples/ch05/Ex07_Deque.java

```
// objective: 5.1
// Deque (ArrayDeque): dùng như ngăn xếp (stack, LIFO) và hàng đợi (queue, FIFO).
import java.util.ArrayDeque;
import java.util.Deque;
import java.util.Queue;

public class Ex07_Deque {
    public static void main(String[] args) {
        Deque<String> stack = new ArrayDeque<>();
        stack.push("a"); stack.push("b"); stack.push("c");           // push = addFirst
        System.out.println(stack + " peek=" + stack.peek() + " pop=" + stack.pop() + " " + stack);

        Queue<String> queue = new ArrayDeque<>();
        queue.offer("1"); queue.offer("2"); queue.add("3");           // offer/add = addLast
        System.out.println(queue + " peek=" + queue.peek() + " poll=" + queue.poll() + " " + queue);

        Deque<Integer> d = new ArrayDeque<>();
        d.offerFirst(2); d.offerLast(3); d.addFirst(1);
        System.out.println(d + " " + d.peekFirst() + " " + d.peekLast() + " " + d.pollLast() + " " + d);

        Deque<Integer> empty = new ArrayDeque<>();
        System.out.println(empty.poll() + " " + empty.peek());        // null: "special value"
        try {
            empty.pop();                                               // hoặc remove()/element(): ném exception
        } catch (java.util.NoSuchElementException e) {
            System.out.println("NoSuchElementException");
        }
        try {
            empty.offer(null);
        } catch (NullPointerException e) {
            System.out.println("ArrayDeque không nhận null");
        }
    }
}
```

Output thật (JDK 21.0.10):

```
[c, b, a] peek=c pop=c [b, a]
[1, 2, 3] peek=1 poll=1 [2, 3]
[1, 2, 3] 1 3 3 [1, 2]
null null
NoSuchElementException
ArrayDeque không nhận null
```

8. Sắp xếp với Comparable và Comparator

examples/ch05/Ex08_Sorting.java

```
// objective: 5.1
// Sắp xếp: Comparable (thứ tự tự nhiên) và Comparator (thứ tự tùy chọn, nối chuỗi được).
import java.util.*;

public class Ex08_Sorting {
    record Student(String name, int score) implements Comparable<Student> {
        public int compareTo(Student o) { return name.compareTo(o.name); } // tự nhiên: theo tên
    }

    public static void main(String[] args) {
        List<Student> list = new ArrayList<>(List.of(
            new Student("Chi", 8), new Student("An", 9), new Student("Binh", 8), new Student("Dung", 7)));
        Collections.sort(list);
        System.out.println(list.stream().map(Student::name).toList());

        list.sort(Comparator.comparingInt(Student::score).reversed().thenComparing(Student::name));
        System.out.println(list);

        List<String> words = new ArrayList<>(List.of("b", "B", "a", "A", "10", "9"));
        words.sort(null); // null = thứ tự tự nhiên
        System.out.println(words);
        words.sort(String.CASE_INSENSITIVE_ORDER);
        System.out.println(words);
        words.sort(Comparator.reverseOrder());
        System.out.println(words);
    }
}
```

```

List<String> withNull = new ArrayList<>(Arrays.asList("b", null, "a"));
withNull.sort(Comparator.nullsFirst(Comparator.naturalOrder()));
System.out.println(withNull);
System.out.println(Collections.max(List.of(3, 9, 2)) + " " + Collections.min(List.of("pear", "fig"),
Comparator.comparing(String::length)));
}
}

```

Output thật (JDK 21.0.10):

```

[An, Binh, Chi, Dung]
[Student[name=An, score=9], Student[name=Binh, score=8], Student[name=Chi, score=8], Student[name=Dung, score=7]]
[10, 9, A, B, a, b]
[10, 9, A, a, B, b]
[b, a, B, A, 9, 10]
[null, a, b]
9 fig

```

9. Generics và wildcard

examples/ch05/Ex09_Generics.java

```

// objective: 5.1
// Generics: lớp và method generic, wildcard ? extends (đọc) và ? super (ghi) - "PECS".
import java.util.ArrayList;
import java.util.List;

public class Ex09_Generics {
    static class Box<T> {
        private T value;
        Box(T value) { this.value = value; }
        T get() { return value; }
        <R> Box<R> map(java.util.function.Function<T, R> f) { return new Box<>(f.apply(value)); }
    }

    static <T extends Comparable<T>> T max(List<T> list) { // bounded type parameter
        T best = list.get(0);
        for (T t : list) if (t.compareTo(best) > 0) best = t;
        return best;
    }

    static double sum(List<? extends Number> nums) { // Producer Extends: chỉ đọc
        double s = 0;
        for (Number n : nums) s += n.doubleValue();
        return s;
    }

    static void fill(List<? super Integer> out) { // Consumer Super: ghi Integer vào được
        for (int i = 1; i <= 3; i++) out.add(i);
    }

    public static void main(String[] args) {
        Box<String> b = new Box<>("hello");
        Box<Integer> len = b.map(String::length);
        System.out.println(b.get() + " " + len.get());
        System.out.println(max(List.of(3, 7, 5)) + " " + max(List.of("pear", "apple")));
        System.out.println(sum(List.of(1, 2.5, 3L)));
        List<Number> numbers = new ArrayList<>();
        List<Object> objects = new ArrayList<>();
        fill(numbers);
        fill(objects);
        System.out.println(numbers + " " + objects);
        List<?> unknown = List.of("x", 1);
        Object first = unknown.get(0); // đọc ra kiểu Object
        System.out.println(first + " " + (new ArrayList<String>().getClass() == new ArrayList<Integer>().getClass()));
    }
}

```

Output thật (JDK 21.0.10):

```

hello 5
7 pear
6.5
[1, 2, 3] [1, 2, 3]
x true

```

10. Sequenced collections (Java 21)

examples/ch05/Ex10_Sequenced.java

```

// objective: 5.1
// Sequenced collections (Java 21): getFirst/getLast/reversed/addFirst cho List, Deque, LinkedHashSet, LinkedHashMap.

```

```
import java.util.*;

public class Ex10_Sequenced {
    public static void main(String[] args) {
        List<Integer> list = new ArrayList<>(List.of(1, 2, 3));
        list.addFirst(0);
        list.addLast(4);
        System.out.println(list + " " + list.getFirst() + " " + list.getLast() + " " + list.reversed());

        LinkedHashSet<String> set = new LinkedHashSet<>(List.of("b", "c"));
        set.addFirst("a");
        set.addFirst("c"); // đã có → chuyển lên đầu
        System.out.println(set + " " + set.reversed() + " " + set.removeLast());

        LinkedHashMap<String, Integer> map = new LinkedHashMap<>();
        map.put("x", 1); map.put("y", 2);
        map.putFirst("w", 0);
        System.out.println(map + " " + map.firstEntry() + " " + map.lastEntry() + " " + map.sequencedKeySet().reversed());

        List<Integer> view = list.reversed(); // reversed() là một VIEW, không phải bản sao
        list.set(0, 100);
        System.out.println(view);
        try {
            List.of(1, 2).addFirst(0);
        } catch (UnsupportedOperationException e) {
            System.out.println("List.of vẫn bất biến");
        }
    }
}
```

Output thật (JDK 21.0.10):

```
[0, 1, 2, 3, 4] 0 4 [4, 3, 2, 1, 0]
[c, a, b] [b, a, c] b
{w=0, x=1, y=2} w=0 y=2 [y, x, w]
[4, 3, 2, 1, 100]
List.of vẫn bất biến
```

11. Collection bất biến

examples/ch05/Ex11_Unmodifiable.java

```
// objective: 5.1
// So sánh List.of, List.copyOf, Collections.unmodifiableList (view), Arrays.asList.
import java.util.*;

public class Ex11_Unmodifiable {
    public static void main(String[] args) {
        List<String> src = new ArrayList<>(List.of("a", "b"));
        List<String> view = Collections.unmodifiableList(src); // view chỉ đọc: vẫn thấy thay đổi của src
        List<String> copy = List.copyOf(src); // bản sao bất biến
        src.add("c");
        System.out.println(view + " " + copy);

        try {
            List.of("a", null);
        } catch (NullPointerException e) {
            System.out.println("List.of không nhận null");
        }

        List<String> asList = Arrays.asList("x", null); // asList nhận null
        System.out.println(asList + " contains null? " + asList.contains(null));

        Set<Integer> s = Set.of(3, 1, 2);
        System.out.println(s.size() + " " + s.contains(2));
        try {
            Set.of(1, 1);
        } catch (IllegalArgumentException e) {
            System.out.println("Set.of trùng phần tử → IllegalArgumentException");
        }
    }
}
```

Output thật (JDK 21.0.10):

```
[a, b, c] [a, b]
List.of không nhận null
[x, null] contains null? true
3 true
Set.of trùng phần tử → IllegalArgumentException
```

12. Lỗi generics hay gặp

examples/ch05/Ex12_GenericsErrors.java

```
// objective: 5.1
// expect: compile-error
// Generics không "hiệp biến" (invariant): List<String> KHÔNG phải List<Object>; không thêm được vào ? extends.
import java.util.ArrayList;
import java.util.List;

public class Ex12_GenericsErrors {
    public static void main(String[] args) {
        List<String> strings = new ArrayList<>();
        List<Object> objects = strings;
        List<? extends Number> nums = new ArrayList<Integer>();
        nums.add(1);
        List<int> primitives = new ArrayList<>();
    }
}
```

Output thật của javac (JDK 21.0.10) — cố ý không biên dịch được:

```
Ex12_GenericsErrors.java:10: error: incompatible types: List<String> cannot be converted to List<Object>
    List<Object> objects = strings;
                        ^
Ex12_GenericsErrors.java:12: error: incompatible types: int cannot be converted to CAP#1
    nums.add(1);
            ^
    where CAP#1 is a fresh type-variable:
      CAP#1 extends Number from capture of ? extends Number
Ex12_GenericsErrors.java:13: error: unexpected type
    List<int> primitives = new ArrayList<>();
    ^
    required: reference
    found:    int
Note: Some messages have been simplified; recompile with -Xdiags:verbose to get full output
3 errors
```

Đi sâu

Mảng

- Khai báo: `int[] a`, `int a[]`, `int[] a, b` (cả hai là mảng), `int[] a, b[]` (b là mảng 2 chiều).
- Tạo: `new int[3]` (phải có kích thước), `new int[]{1, 2}` (không được có kích thước), `{1, 2}` (chỉ khi khai báo).
- Mảng là object: `a.length` (thuộc tính, không phải method), `a.equals(b)` là so sánh tham chiếu.
- `Arrays.sort` (số tăng dần; `String` theo Unicode: số < chữ HOA < chữ thường), `Arrays.binarySearch` (mảng phải đã sắp xếp; không thấy → -(điểm chèn) - 1), `Arrays.equals`, `Arrays.compare`, `Arrays.mismatch` (chỉ số khác đầu tiên, hoặc -1), `Arrays.fill`, `Arrays.copyOf`, `Arrays.asList` (fixed-size, ghi xuống mảng gốc).

List

Method	Ghi chú
<code>add(e)</code> , <code>add(i, e)</code>	<code>add(i, e)</code> chèn, đẩy phần tử sau lùi lại
<code>set(i, e)</code>	Thay thế, trả về phần tử cũ
<code>remove(int i)</code> vs <code>remove(Object o)</code>	Với <code>List<Integer></code> , <code>remove(1)</code> xóa chỉ số 1
<code>get(i)</code> , <code>indexOf</code> , <code>lastIndexOf</code> , <code>contains</code>	<code>indexOf</code> không thấy → -1
<code>removeIf(pred)</code> , <code>replaceAll(op)</code> , <code>sort(cmp)</code>	<code>sort(null)</code> = thứ tự tự nhiên
<code>subList(from, to)</code>	View: sửa view là sửa list gốc

Sửa cấu trúc list (add/remove) trong for-each → thường là `ConcurrentModificationException`. Dùng `removeIf` hoặc `Iterator.remove()`.

Set và TreeSet

`add` trả về `false` nếu phần tử đã có. `HashSet` dùng `hashCode` + `equals`; `TreeSet` dùng `compareTo/compare` — hai phần tử so sánh bằng 0 bị coi là trùng, kể cả khi `equals` khác.

`TreeSet/NavigableSet`: `first`, `last`, `floor(x)` ($\leq$), `ceiling(x)` ($\geq$), `lower(x)` ($<$), `higher(x)` ($>$), `pollFirst`, `pollLast`, `headSet(x)` ($< x$), `tailSet(x)` ($\geq x$), `subSet(a, b)` ([a, b)), `descendingSet`.

Map

Method	Trả về / hành vi
<code>put(k, v)</code>	Giá trị cũ hoặc <code>null</code>
<code>get(k)</code> , <code>getOrDefault(k, d)</code>	<code>null</code> nếu không có (hoặc giá trị là <code>null</code>)

Method	Trả về / hành vi
<code>putIfAbsent(k, v)</code>	Chỉ thêm khi chưa có (hoặc đang là <code>null</code>); trả về giá trị hiện tại
<code>merge(k, v, f)</code>	Chưa có $\rightarrow$ đặt <code>v</code> ; có $\rightarrow f(cũ, v)$; kết quả <code>null</code> $\rightarrow$ xoá key
<code>compute, computeIfAbsent, computeIfPresent</code>	Hàm trả về <code>null</code> $\rightarrow$ xoá / không thêm
<code>keySet(), values(), entrySet()</code>	View của map
<code>containsKey, containsValue, remove(k)</code>	

`TreeMap`: `firstKey, lastKey, floorKey, ceilingKey, headMap(k) (< k), tailMap(k) ( $\geq k$ )`.

Deque

	Ném exception khi rỗng	Trả về giá trị đặc biệt (null / false)
Thêm đầu	<code>addFirst, push</code>	<code>offerFirst</code>
Thêm cuối	<code>addLast, add</code>	<code>offerLast, offer</code>
Lấy + xoá đầu	<code>removeFirst, remove, pop</code>	<code>pollFirst, poll</code>
Lấy + xoá cuối	<code>removeLast</code>	<code>pollLast</code>
Xem đầu	<code>getFirst, element</code>	<code>peekFirst, peek</code>
Xem cuối	<code>getLast</code>	<code>peekLast</code>

Stack: `push/pop/peek` đều làm việc ở **đầu**. Queue: `offer` ở **cuối**, `poll/peek` ở **đầu**.

Comparable vs Comparator

- `Comparable<T>` (trong lớp): `int compareTo(T o)` — “thứ tự tự nhiên”. `String, wrapper, LocalDate...` đều có.
- `Comparator<T>` (bên ngoài): `int compare(T a, T b)`. Factory: `Comparator.comparing(keyExtractor), comparingInt, thenComparing, reversed()` (đảo **toàn bộ** chuỗi so sánh đứng trước nó), `naturalOrder(), reverseOrder(), nullsFirst/nullsLast`.
- Quy ước: âm nếu `a` đứng trước, 0 nếu bằng, dương nếu đứng sau.

Generics

- `class Box<T>, method generic static <T> T first(List<T> l)`, giới hạn `<T extends Comparable<T>>`.
- Bất biến (invariant)**: `List<Integer>` không phải `List<Number>`.
- Wildcard: `List<?>` (chỉ đọc ra `Object`), `List<? extends Number>` (đọc `Number`, không thêm được trừ `null`), `List<? super Integer>` (thêm `Integer` được, đọc ra `Object`). Ghi nhớ **PECS: Producer Extends, Consumer Super**.
- Không dùng primitive làm tham số kiểu (`List<int>` lỗi). Diamond `<>` chỉ ở sau `new`.
- Type erasure**: thông tin kiểu generic bị xoá lúc chạy $\rightarrow$ `new ArrayList<String>().getClass() == new ArrayList<Integer>().getClass()`.

Collection bất biến

Tạo bằng	Sửa được?	null?	Ghi chú
<code>List.of, Set.of, Map.of</code>	Không	Không	<code>Set.of/Map.of</code> trùng phần tử/key $\rightarrow$ <code>IllegalArgumentException</code>
<code>List.copyOf</code>	Không	Không	Bản sao
<code>Collections.unmodifiableList(l)</code>	Không (qua view)	Theo list gốc	View : thấy thay đổi của <code>l</code>
<code>Arrays.asList(...)</code>	set được, add/remove không	Có	Ghi xuống mảng gốc

Sequenced collections (Java 21)

Interface mới `SequencedCollection` (`List, Deque, LinkedHashSet, SortedSet...`) có `addFirst, addLast, getFirst, getLast, removeFirst, removeLast, reversed()` (view). `SequencedMap` (`LinkedHashMap, TreeMap...`) có `putFirst, putLast, firstEntry, lastEntry, pollFirstEntry, sequencedKeySet()` ... Với collection bất biến, các method thay đổi ném `UnsupportedOperationException`.

Lỗi và bẫy thường gặp (Exam traps)

- `List<Integer>.remove(1)` xoá chỉ số, không xoá giá trị 1.
- `Arrays.asList(...).add(...)` $\rightarrow$ `UnsupportedOperationException`; set thì được.
- `binarySearch` trên mảng chưa sắp xếp $\rightarrow$ kết quả không xác định.
- Thứ tự `String`: chữ hoa trước chữ thường.
- `TreeSet/TreeMap` với `comparator`: “bằng 0” là trùng.
- `ArrayDeque, TreeMap(key), List.of` không nhận `null`.
- `push` thêm **đầu**, `offer/add` thêm **cuối**.

8. `reversed()` của `Comparator` đảo cả phần phía trước nó; `thenComparing` sau `reversed()` không bị đảo.
9. `Collections.unmodifiableList` là view, `List.copyOf` là bản sao.
10. `List<Number> = new ArrayList<Integer>()` → lỗi; thêm vào `List<? extends ...>` → lỗi.
11. `new int[3]{1,2,3}` và `new int[]` → lỗi.
12. `Map` không phải `Collection`: không có `add`, không dùng `for-each` trực tiếp trên `Map`.

Góc nhìn từ TypeScript

TypeScript	Java	Ghi chú
<code>const a: number[] = []</code> (co giãn)	<code>int[]</code> (cố định) hoặc <code>List<Integer></code> (co giãn)	Dùng <code>List</code> khi cần thêm/xoá
<code>arr.push(x)</code> , <code>arr.splice(i, 1)</code>	<code>list.add(x)</code> , <code>list.remove(i)</code>	
<code>arr.sort()</code> sắp xếp theo chuỗi mặc định	<code>Collections.sort</code> theo thứ tự tự nhiên (số là số)	<code>[10, 9].sort()</code> trong JS cho <code>[10, 9]</code>
<code>arr.sort((a, b) => a - b)</code>	<code>list.sort(Comparator.naturalOrder())</code> hoặc <code>(a, b) -> a - b</code>	Cẩn thận tràn số với <code>a - b</code>
<code>new Set()</code> , <code>new Map()</code> giữ thứ tự thêm	<code>LinkedHashSet</code> , <code>LinkedHashMap</code>	<code>HashSet</code> / <code>HashMap</code> không giữ thứ tự
<code>readonly T[]</code> , <code>ReadonlyArray<T></code> (chỉ lúc biên dịch)	<code>List.of(...)</code> (bất biến thật lúc chạy)	
Generics bị xoá khi biên dịch	Generics cũng bị xoá (type erasure)	Gống nhau về ý tưởng
<code>Array<T></code> hiệp biến (covariant)	<code>List<T></code> bất biến (invariant)	Java dùng wildcard <code>? extends</code>

Tóm tắt

- Mảng cố định, `length`; `Arrays` cho sort/search/compare.
- List có chỉ số; Set không trùng; Map key → value; Deque hai đầu.
- Hash (không thứ tự), Linked (thứ tự thêm), Tree (sắp xếp).
- `Comparable` bên trong lớp; `Comparator` bên ngoài, nối chuỗi bằng `thenComparing`, `reversed`.
- Generics bất biến; PECS; không primitive; diamond chỉ sau `new`.
- Java 21: sequenced collections với `getFirst`/`getLast`/`reversed`.

Bài tập (có lời giải)

Mọi đáp án đã được `tools/book.py` biên dịch và chạy để xác nhận (code: `examples/questions/ch05/`).

Câu hỏi

Câu 05-01 · Dễ · objective 5.1

Chương trình sau in ra gì?

```
import java.util.*;

public class Remove {
    public static void main(String[] args) {
        List<Integer> list = new ArrayList<>(List.of(5, 1, 3, 1));
        list.remove(1);
        list.remove(Integer.valueOf(1));
        System.out.println(list);
    }
}
```

- A. `[5, 3]`
- B. `[5, 3, 1]`
- C. `[3, 1]`
- D. `[5, 1]`

Câu 05-02 · Vừa · objective 5.1

Chương trình sau in ra gì?

```
import java.util.*;

public class Nav {
    public static void main(String[] args) {
        TreeSet<Integer> t = new TreeSet<>(List.of(8, 2, 6, 4));
        System.out.println(t.floor(5) + " " + t.higher(6) + " " + t.headSet(6) + " "
            + t.pollFirst() + " " + t);
    }
}
```

```
}  
}
```

- A. 4 8 [2, 4, 6] 2 [4, 6, 8]
- B. 6 8 [2, 4] 2 [2, 4, 6, 8]
- C. 4 6 [2, 4] 2 [4, 6, 8]
- D. 4 8 [2, 4] 2 [4, 6, 8]

Câu 05-03 · Vừa · objective 5.1

Chương trình sau in ra gì?

```
import java.util.*;  
  
public class Puts {  
    public static void main(String[] args) {  
        Map<String, Integer> m = new HashMap<>();  
        m.put("a", 1);  
        Integer r1 = m.put("a", 2);  
        Integer r2 = m.putIfAbsent("a", 3);  
        Integer r3 = m.putIfAbsent("b", 4);  
        m.merge("a", 10, (x, y) -> x + y);  
        System.out.println(r1 + " " + r2 + " " + r3 + " " + m.get("a") + " " + m.get("b"));  
    }  
}
```

- A. 1 null null 12 4
- B. null 2 null 13 4
- C. 1 2 null 12 4
- D. 1 2 4 12 4

Câu 05-04 · Vừa · objective 5.1

Chương trình sau in ra gì?

```
import java.util.*;  
  
public class Dq {  
    public static void main(String[] args) {  
        Deque<Integer> d = new ArrayDeque<>();  
        d.push(1);  
        d.offer(2);  
        d.push(3);  
        d.offerFirst(4);  
        System.out.println(d.pop() + " " + d.pollLast() + " " + d.peek() + " " + d);  
    }  
}
```

- A. 1 4 2 [2, 3]
- B. 4 2 1 [3, 1]
- C. 2 4 3 [3, 1]
- D. 4 2 3 [3, 1]

Câu 05-05 · Khó · objective 5.1

Chương trình sau in ra gì?

```
import java.util.*;  
  
public class Search {  
    public static void main(String[] args) {  
        String[] arr = {"kiwi", "Fig", "apple", "Date"};  
        Arrays.sort(arr);  
        System.out.println(Arrays.toString(arr) + " " + Arrays.binarySearch(arr, "banana"));  
    }  
}
```

- A. [apple, Date, Fig, kiwi] -2
- B. [Date, Fig, apple, kiwi] -4
- C. [Date, Fig, apple, kiwi] -3
- D. [apple, Date, Fig, kiwi] -3

Câu 05-06 · Vừa · objective 5.1

Chèn dòng nào vào chỗ // INSERT CODE HERE thì chương trình biên dịch được? (Chọn 3 đáp án.)

```
import java.util.*;

public class Wildcards {
    public static void main(String[] args) {
        // INSERT CODE HERE
    }
}
```

- A. `List<Number> a = new ArrayList<Integer>();`
- B. `List<? extends Number> b = new ArrayList<Integer>();`
- C. `List<? super Integer> c = new ArrayList<Number>();`
- D. `List<Object> d = new ArrayList<String>();`
- E. `ArrayList<> e = new ArrayList<String>();`
- F. `var f = new ArrayList<>();`

Câu 05-07 · Khó · objective 5.1

Kết quả của chương trình là gì?

```
import java.util.*;

public class Cme {
    public static void main(String[] args) {
        List<String> list = new ArrayList<>(List.of("a", "b", "c"));
        for (String s : list) {
            if (s.equals("b")) list.remove(s);
        }
        System.out.println(list);
    }
}
```

- A. In ra `[a, c]`
- B. Ném `ConcurrentModificationException`
- C. In ra `[a, b, c]`
- D. Không biên dịch được

Câu 05-08 · Vừa · objective 5.1

Chương trình sau in ra gì?

```
import java.util.*;

public class Sorter {
    record P(String name, int age) { }

    public static void main(String[] args) {
        List<P> ps = new ArrayList<>(List.of(new P("Lan", 30), new P("An", 25),
            new P("Minh", 30), new P("Bao", 25)));
        ps.sort(Comparator.comparing(P::age).reversed().thenComparing(P::name));
        System.out.println(ps.stream().map(P::name).toList());
    }
}
```

- A. `[Minh, Lan, Bao, An]`
- B. `[An, Bao, Lan, Minh]`
- C. `[Lan, Minh, An, Bao]`
- D. `[Lan, Minh, Bao, An]`

Câu 05-09 · Dễ · objective 5.1

Kết quả của chương trình là gì?

```
public class Jagged {
    public static void main(String[] args) {
        int[][] a = new int[3][];
        a[0] = new int[2];
        System.out.println(a.length + " " + a[0].length + " " + a[0][1] + " " + a[1]);
    }
}
```

- A. In ra `3 0 0 null`
- B. In ra `3 2 null null`
- C. Ném `NullPointerException`
- D. In ra `3 2 0 null`

Câu 05-10 · Vừa · objective 5.1

Hai phát biểu nào đúng? (Chọn 2 đáp án.)

- A. `HashMap` cho phép một key `null`.
- B. `TreeMap` (thứ tự tự nhiên) cho phép key `null`.
- C. `ArrayDeque` cho phép phần tử `null`.
- D. `List.of(...)` trả về list không thay đổi được.
- E. `LinkedHashSet` sắp xếp phần tử theo thứ tự tự nhiên.

Câu 05-11 · Khó · objective 5.1

Những dòng nào gây lỗi biên dịch?

```
import java.util.*;

public class Wild {
    static void read(List<? extends Number> in) {
        Number n = in.get(0);           // L1
        in.add(null);                    // L2
        in.add(Integer.valueOf(1));      // L3
    }

    static void write(List<? super Integer> out) {
        out.add(5);                      // L4
        Integer i = out.get(0);          // L5
    }

    public static void main(String[] args) { }
```

- A. L2 và L3
- B. L3 và L5
- C. Chỉ L3
- D. L2, L3 và L5
- E. L4 và L5

Câu 05-12 · Vừa · objective 5.1

Chương trình sau in ra gì?

```
import java.util.*;

public class Seq {
    public static void main(String[] args) {
        List<String> list = new ArrayList<>(List.of("b", "c"));
        list.addFirst("a");
        List<String> rev = list.reversed();
        list.addLast("d");
        System.out.println(rev + " " + rev.getFirst() + " " + list.getLast());
    }
}
```

- A. [c, b, a] c d
- B. [d, c, b, a] a d
- C. [d, c, b, a] d d
- D. [a, b, c, d] a d

Câu 05-13 · Vừa · objective 5.1

Chương trình sau in ra gì?

```
import java.util.*;

public class Rev {
    public static void main(String[] args) {
        TreeMap<String, Integer> m = new TreeMap<>(Comparator.reverseOrder());
        m.put("b", 2);
        m.put("a", 1);
        m.put("c", 3);
        System.out.println(m + " " + m.firstKey() + " " + m.headMap("b"));
    }
}
```

- A. {a=1, b=2, c=3} a {a=1}
- B. {c=3, b=2, a=1} c {a=1}

- C. {c=3, b=2, a=1} a {c=3}
- D. {c=3, b=2, a=1} c {c=3}

Câu 05-14 · Dễ · objective 5.1

Những dòng nào gây lỗi biên dịch?

```
public class Decl {
    public static void main(String[] args) {
        int[] a = new int[3];           // L1
        int[] b = new int[3]{1, 2, 3};   // L2
        int c[] = {1, 2, 3};             // L3
        int[] d = new int[];             // L4
        int[][] e = new int[2][];        // L5
    }
}
```

- A. Chỉ L2
- B. L2 và L4
- C. L4 và L5
- D. L2, L4 và L5
- E. L3 và L4

Câu 05-15 · Khó · objective 5.1

Chương trình sau in ra gì?

```
import java.util.*;

public class Custom {
    public static void main(String[] args) {
        TreeSet<String> set = new TreeSet<>(
            Comparator.comparing(String::length).thenComparing(Comparator.reverseOrder()));
        set.addAll(List.of("bb", "a", "ccc", "aa", "b"));
        System.out.println(set);
    }
}
```

- A. [b, a, bb, aa, ccc]
- B. [a, b, aa, bb, ccc]
- C. [ccc, bb, aa, b, a]
- D. [a, aa, b, bb, ccc]

Câu 05-16 · Vừa · objective 5.1

Hai phát biểu nào đúng về lớp Arrays? (Chọn 2 đáp án.)

- A. Arrays.asList trả về list cố định kích thước nhưng vẫn gọi set() được.
- B. Arrays.equals(a, b) so sánh tham chiếu của hai mảng.
- C. Arrays.binarySearch trên mảng chưa sắp xếp luôn trả về -1.
- D. Arrays.compare trả về số âm nếu mảng thứ nhất nhỏ hơn theo thứ tự từ điển.
- E. Sau `int[] a = {};` thì `a.length` bằng 1.

Câu 05-17 · Vừa · objective 5.1

Chương trình sau in ra gì?

```
import java.util.*;

public class Views {
    public static void main(String[] args) {
        List<String> src = new ArrayList<>(List.of("x"));
        List<String> v = Collections.unmodifiableList(src);
        List<String> c = List.copyOf(src);
        src.add("y");
        System.out.println(v.size() + " " + c.size());
    }
}
```

- A. 1 1
- B. 2 2
- C. 2 1
- D. 1 2

Câu 05-18 · Khó · objective 5.1

Đoạn code nào, chèn vào chỗ `// INSERT CODE HERE`, in ra `[3, 2, 1]`? (Chọn 3 đáp án.)

```
import java.util.*;

public class Reverse {
    public static void main(String[] args) {
        List<Integer> list = new ArrayList<>(List.of(1, 2, 3));
        // INSERT CODE HERE
    }
}
```

- A. `Collections.reverse(list); System.out.println(list);`
- B. `System.out.println(list.reversed());`
- C. `list.sort(Comparator.reverseOrder()); System.out.println(list);`
- D. `Collections.sort(list); System.out.println(list);`
- E. `Deque<Integer> d = new ArrayDeque<>(list); System.out.println(d);`

Câu 05-19 · Vừa · objective 5.1

Những dòng nào gây lỗi biên dịch?

```
import java.util.*;

public class Gen {
    static <T> T first(List<T> list) { return list.get(0); }

    public static void main(String[] args) {
        List<String> names = new ArrayList<>();           // L1
        String s = first(names);                          // L2
        Integer i = first(names);                        // L3
        Map<String, List<Integer>> m = new HashMap<>();    // L4
        List<> bad = new ArrayList<String>();             // L5
    }
}
```

- A. Chỉ L3
- B. L3 và L5
- C. L2 và L3
- D. L4 và L5
- E. Chỉ L5

Câu 05-20 · Dễ · objective 5.1

Chương trình sau in ra gì?

```
import java.util.*;

public class Adds {
    public static void main(String[] args) {
        List<String> l = new ArrayList<>();
        l.add("x");
        l.add(0, "y");
        l.add("z");
        l.set(1, "w");
        System.out.println(l + " " + l.indexOf("z"));
    }
}
```

- A. `[y, w, z] 2`
- B. `[w, x, z] 2`
- C. `[y, x, w] -1`
- D. `[y, w, x, z] 3`

Lời giải

Câu 05-01 — Đáp án: A

- **Vì sao đúng:** `remove(1)` với tham số `int` gọi `remove(int index)` → xoá phần tử ở chỉ số 1 (giá trị 1) → `[5, 3, 1]`.
`remove(Integer.valueOf(1))` gọi `remove(Object)` → xoá lần xuất hiện đầu tiên của giá trị 1 → `[5, 3]`.
- **B sai:** Lỗi gọi thứ hai xoá giá trị 1 còn lại.
- **C sai:** `remove(1)` xoá theo chỉ số, không xoá phần tử đầu tiên.
- **D sai:** `remove(1)` xoá phần tử ở chỉ số 1 chứ không phải giá trị 3.

- **Kiểm chứng:** `examples/questions/ch05/Q05_01/` — output confirmed (python3 tools/book.py questions ch05).

Câu 05-02 — Đáp án: D

- **Vì sao đúng:** `floor(5)` = phần tử lớn nhất $\leq 5 \rightarrow 4$. `higher(6)` = nhỏ nhất $> 6 \rightarrow 8$. `headSet(6)` = các phần tử **nhỏ hơn** 6 (không gồm 6) $\rightarrow [2, 4]$. `pollFirst()` lấy và **xoá** 2, nên set còn `[4, 6, 8]`. Biểu thức tính từ trái sang phải nên `headSet` được in trước khi `pollFirst` chạy.
- **A sai:** `headSet(6)` mặc định không gồm 6.
- **B sai:** `floor(5)` phải ≤ 5 ; và `pollFirst()` đã xoá 2 khỏi set.
- **C sai:** `higher(6)` là phần tử lớn hơn hẳn 6, tức 8.
- **Kiểm chứng:** `examples/questions/ch05/Q05_02/` — output confirmed (python3 tools/book.py questions ch05).

Câu 05-03 — Đáp án: C

- **Vì sao đúng:** `put` trả về giá trị cũ (1). `putIfAbsent` khi key đã có: không đổi và trả về giá trị hiện tại (2). Khi key chưa có: thêm và trả về `null`. `merge("a", 10, sum) \rightarrow 2 + 10 = 12`.
- **A sai:** `putIfAbsent` trả về giá trị đang có (2) khi key đã tồn tại.
- **B sai:** `put` lần hai trả về giá trị cũ 1; `putIfAbsent("a", 3)` không ghi đè nên `merge` tính `2 + 10`.
- **D sai:** `putIfAbsent` trả về `null` khi vừa thêm key mới.
- **Kiểm chứng:** `examples/questions/ch05/Q05_03/` — output confirmed (python3 tools/book.py questions ch05).

Câu 05-04 — Đáp án: D

- **Vì sao đúng:** `push = addFirst`, `offer = addLast`. Sau 4 lệnh: `[4, 3, 1, 2]`. `pop()` lấy đầu $\rightarrow 4$. `pollLast()` lấy cuối $\rightarrow 2$. `peek()` xem đầu (không xoá) $\rightarrow 3$. Còn lại `[3, 1]`.
- **A sai:** `push` thêm vào **đầu** deque, không phải cuối.
- **B sai:** Sau khi `pop()` lấy 4, phần tử đầu là 3.
- **C sai:** `pop()` lấy từ đầu (4), `pollLast()` lấy từ cuối (2).
- **Kiểm chứng:** `examples/questions/ch05/Q05_04/` — output confirmed (python3 tools/book.py questions ch05).

Câu 05-05 — Đáp án: B

- **Vì sao đúng:** Thứ tự tự nhiên của `String` theo mã Unicode: chữ HOA (D, F) đứng trước chữ thường. "banana" sẽ chen vào giữa apple (chỉ số 2) và kiwi (chỉ số 3), nên vị trí chen là 3; không tìm thấy $\rightarrow -(3) - 1 = -4$.
- **A sai:** Sắp xếp `String` phân biệt hoa thường: chữ hoa trước.
- **C sai:** Công thức khi không thấy là `-(insertion point) - 1`, tức -4 chứ không phải -3.
- **D sai:** Sai cả thứ tự sắp xếp lẫn giá trị trả về.
- **Kiểm chứng:** `examples/questions/ch05/Q05_05/` — output confirmed (python3 tools/book.py questions ch05).

Câu 05-06 — Đáp án: B, C, F

- **Vì sao đúng:** Generics bất biến (invariant): `List<Number>` không nhận `ArrayList<Integer>`. Wildcard nói lỏng: `? extends Number` nhận list của bất kỳ lớp con nào của `Number` (B); `? super Integer` nhận list của `Integer` hoặc lớp cha (C). var với diamond suy ra `ArrayList<Object>` (F).
- **A sai:** `ArrayList<Integer>` không phải `List<Number>` dù `Integer` là `Number`.
- **D sai:** `ArrayList<String>` không phải `List<Object>`.
- **E sai:** Diamond `<>` chỉ dùng ở vế phải (sau `new`).
- **Kiểm chứng:** `examples/questions/ch05/Q05_06/` — variants: BCF satisfy compiles (python3 tools/book.py questions ch05).

Câu 05-07 — Đáp án: A

- **Vì sao đúng:** Bẫy khó: xoá phần tử **áp chót** (b) thì không có exception. Sau khi xoá, list còn 2 phần tử và vị trí con trỏ của iterator là 2, nên `hasNext()` trả về `false` và vòng lặp kết thúc trước khi `next()` kịp kiểm tra thay đổi. Đây là hành vi của cài đặt `ArrayList` — **đừng dựa vào nó** trong code thật; hãy dùng `removeIf` hoặc `Iterator.remove`.
- **B sai:** `ConcurrentModificationException` chỉ ném khi `next()` được gọi sau khi list bị sửa; ở đây vòng lặp đã dừng.
- **C sai:** `list.remove(s)` có chạy và xoá b.
- **D sai:** Sửa list trong for-each là lỗi lúc chạy (nếu có), không phải lỗi biên dịch.
- **Kiểm chứng:** `examples/questions/ch05/Q05_07/` — output confirmed (python3 tools/book.py questions ch05).

Câu 05-08 — Đáp án: C

- **Vì sao đúng:** `comparing(P::age).reversed()` $\rightarrow$ tuổi giảm dần (30 trước 25). `thenComparing(P::name)` áp dụng **sau** `reversed()` nên tên vẫn tăng dần: Lan, Minh (30), rồi An, Bao (25).
- **A sai:** `reversed()` chỉ đảo phần so sánh theo tuổi đứng trước nó; tên vẫn tăng dần.
- **B sai:** `reversed()` làm tuổi giảm dần, nên nhóm 30 đứng trước.
- **D sai:** Trong nhóm 25, tên tăng dần: An trước Bao.
- **Kiểm chứng:** `examples/questions/ch05/Q05_08/` — output confirmed (python3 tools/book.py questions ch05).

Câu 05-09 — Đáp án: D

- **Vì sao đúng:** `new int[3][]` tạo mảng ngoài 3 phần tử, mỗi phần tử (mảng con) mặc định `null`. `a[0]` được gán mảng 2 phần tử `int` (mặc định 0). In `a[1]` (là `null`) không gây NPE vì chỉ nối chuỗi, không truy cập phần tử.
- **A sai:** `a[0]` là mảng 2 phần tử nên `a[0].length` là 2.
- **B sai:** Phần tử của `int[]` mặc định là 0, không phải `null`.
- **C sai:** Nối `null` vào chuỗi in ra chữ `null`; không có truy cập `a[1][...]`.
- *Kiểm chứng:* `examples/questions/ch05/Q05_09/` — output confirmed (python3 tools/book.py questions ch05).

Câu 05-10 — Đáp án: A, D

- **Vì sao đúng:** A: `HashMap` cho phép một key `null` (và nhiều value `null`). D: `List.of` trả về list bất biến; add ném `UnsupportedOperationException`.
- **B sai:** `TreeMap` phải so sánh key; so sánh `null` theo thứ tự tự nhiên ném `NullPointerException`.
- **C sai:** `ArrayDeque` dùng `null` làm tín hiệu “rỗng”, nên cấm phần tử `null`.
- **E sai:** `LinkedHashSet` giữ **thứ tự thêm vào**; sắp xếp là việc của `TreeSet`.
- *Kiểm chứng:* `examples/questions/ch05/Q05_10/` — each option proven true/false by a program (python3 tools/book.py questions ch05).

Câu 05-11 — Đáp án: B

- **Vì sao đúng:** `? extends Number`: đọc ra được `Number` (L1) nhưng **không thêm** được gì ngoài `null` (L2 hợp lệ, L3 lỗi), vì list thật có thể là `List<Double>`. `? super Integer`: thêm `Integer` được (L4), nhưng đọc ra chỉ biết là `Object` (L5 lỗi).
- **A sai:** Thêm `null` luôn hợp lệ vì `null` thuộc mọi kiểu tham chiếu.
- **C sai:** L5 cũng lỗi: `out.get(0)` có kiểu `Object`.
- **D sai:** L2 hợp lệ (xem A).
- **E sai:** `? super Integer` cho phép thêm `Integer` → L4 hợp lệ.
- *Kiểm chứng:* `examples/questions/ch05/Q05_11/` — compile error confirmed at ['L3', 'L5'] (python3 tools/book.py questions ch05).

Câu 05-12 — Đáp án: C

- **Vì sao đúng:** Java 21 thêm `SequencedCollection`: `addFirst`, `addLast`, `getFirst`, `getLast`, `reversed`. `reversed()` trả về một **view** đảo ngược, nên thay đổi sau đó (`addLast("d")`) vẫn thấy được. `rev` là `[d, c, b, a]`.
- **A sai:** `reversed()` là view, không phải bản sao; nó thấy cả `d` được thêm sau.
- **B sai:** Phần tử đầu của view đảo ngược là phần tử cuối của list: `d`.
- **D sai:** `rev` là thứ tự đảo ngược của list.
- *Kiểm chứng:* `examples/questions/ch05/Q05_12/` — output confirmed (python3 tools/book.py questions ch05).

Câu 05-13 — Đáp án: D

- **Vì sao đúng:** `TreeMap` sắp xếp theo comparator được truyền vào: ngược thứ tự tự nhiên → `c, b, a`. `firstKey()` là key đầu theo thứ tự đó (`c`). `headMap("b")` = các key đứng **trước** `b` theo thứ tự của map → chỉ `c`.
- **A sai:** Comparator `reverseOrder()` đảo thứ tự: `c` đứng đầu.
- **B sai:** `headMap` dùng thứ tự của map (`c, b, a`), nên phần đứng trước `b` là `c`.
- **C sai:** `firstKey()` là `c` theo thứ tự của map.
- *Kiểm chứng:* `examples/questions/ch05/Q05_13/` — output confirmed (python3 tools/book.py questions ch05).

Câu 05-14 — Đáp án: B

- **Vì sao đúng:** Khi có danh sách giá trị `{...}` thì **không** được ghi kích thước (L2 lỗi). Khi không có danh sách giá trị thì **phải** ghi kích thước (L4 lỗi). L3 dùng cú pháp kiểu C (`int c[]`) vẫn hợp lệ. L5 chỉ cần kích thước chiều đầu.
- **A sai:** L4 cũng lỗi: `new int[]` thiếu kích thước.
- **C sai:** L5 hợp lệ: mảng nhiều chiều chỉ bắt buộc kích thước chiều đầu tiên.
- **D sai:** L5 hợp lệ (xem C).
- **E sai:** `int c[] = {...}` là cú pháp hợp lệ.
- *Kiểm chứng:* `examples/questions/ch05/Q05_14/` — compile error confirmed at ['L2', 'L4'] (python3 tools/book.py questions ch05).

Câu 05-15 — Đáp án: A

- **Vì sao đúng:** So sánh theo độ dài trước (1, 2, 3), cùng độ dài thì theo thứ tự chữ **ngược** (`b` trước `a`, `bb` trước `aa`).
- **B sai:** Tiêu chí thứ hai là `reverseOrder()`, nên cùng độ dài thì `b` đứng trước `a`.
- **C sai:** Độ dài được so sánh tăng dần (không có `reversed()` cho độ dài).
- **D sai:** Tiêu chí đầu tiên là độ dài, không phải chữ cái.
- *Kiểm chứng:* `examples/questions/ch05/Q05_15/` — output confirmed (python3 tools/book.py questions ch05).

Câu 05-16 — Đáp án: A, D

- **Vì sao đúng:** A: list của `asList` “nhìn” vào mảng gốc: `set` được (ghi xuống mảng), `add/remove` thì không. D: `Arrays.compare` so sánh từng phần tử theo thứ tự từ điển, trả về âm/0/dương.
- **B sai:** `Arrays.equals` so sánh **nội dung** từng phần tử; `==` mới so sánh tham chiếu.
- **C sai:** Với mảng chưa sắp xếp, kết quả là **không xác định (undefined)**, không phải luôn -1 (ví dụ ở đây là -4).
- **E sai:** `{}` là mảng rỗng, độ dài 0.
- **Kiểm chứng:** `examples/questions/ch05/Q05_16/` — each option proven true/false by a program (`python3 tools/book.py questions ch05`).

Câu 05-17 — Đáp án: C

- **Vì sao đúng:** `Collections.unmodifiableList` trả về một **view** chỉ đọc của list gốc: list gốc đổi thì view thấy. `List.copyOf` tạo **bản sao** bất biến tại thời điểm gọi.
- **A sai:** View `v` phản ánh thay đổi của `src`.
- **B sai:** `List.copyOf` là bản sao, không thấy phần tử thêm sau.
- **D sai:** Ngược lại: view thay đổi theo, bản sao thì không.
- **Kiểm chứng:** `examples/questions/ch05/Q05_17/` — output confirmed (`python3 tools/book.py questions ch05`).

Câu 05-18 — Đáp án: A, B, C

- **Vì sao đúng:** A đảo list tại chỗ. B dùng view đảo ngược của Java 21. C sắp xếp giảm dần.
- **D sai:** `Collections.sort` sắp xếp tăng dần → `[1, 2, 3]`.
- **E sai:** `ArrayDeque` giữ thứ tự của list nguồn khi in → `[1, 2, 3]`.
- **Kiểm chứng:** `examples/questions/ch05/Q05_18/` — variants: ABC satisfy output (`python3 tools/book.py questions ch05`).

Câu 05-19 — Đáp án: B

- **Vì sao đúng:** Với `List<String>`, `T` được suy ra là `String`, nên kết quả không gán cho `Integer` được (L3). Diamond `<>` chỉ được dùng sau `new`, không dùng ở kiểu khai báo (L5). L4 dùng diamond đúng cách.
- **A sai:** L5 cũng lỗi: `List<>` ở vế trái không hợp lệ.
- **C sai:** L2 hợp lệ: `T = String`.
- **D sai:** L4 hợp lệ: diamond suy ra `HashMap<String, List<Integer>>`.
- **E sai:** L3 cũng lỗi: không gán `String` cho `Integer`.
- **Kiểm chứng:** `examples/questions/ch05/Q05_19/` — compile error confirmed at `['L3', 'L5']` (`python3 tools/book.py questions ch05`).

Câu 05-20 — Đáp án: A

- **Vì sao đúng:** `[x] → add(0, "y")` chèn đầu → `[y, x] → thêm cuối → [y, x, z] → set(1, "w")` thay phần tử ở chỉ số 1 → `[y, w, z]`. `indexOf("z") = 2`.
- **B sai:** `add(0, "y")` chèn `y` vào đầu, sau đó `set(1, ...)` thay `x` chứ không thay `y`.
- **C sai:** `set(1, ...)` thay chỉ số 1 (là `x`), không phải chỉ số 2.
- **D sai:** `set` thay thế, không chèn thêm.
- **Kiểm chứng:** `examples/questions/ch05/Q05_20/` — output confirmed (`python3 tools/book.py questions ch05`).

Đọc thêm (link chính thức — bị chặn trong sandbox nên mình chưa mở được)

- Javadoc `java.util` (Collections Framework): <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/package-summary.html>
- JEP 431 Sequenced Collections: <https://openjdk.org/jeps/431>

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-28:

- `SequencedCollection.java`:
<https://raw.githubusercontent.com/openjdk/jdk21u/master/src/java.base/share/classes/java/util/SequencedCollection.java>
- `Arrays.java` (`binarySearch`, `asList`, `mismatch`):
<https://raw.githubusercontent.com/openjdk/jdk21u/master/src/java.base/share/classes/java/util/Arrays.java>
- `ArrayDeque.java` (cấm phần tử null):
<https://raw.githubusercontent.com/openjdk/jdk21u/master/src/java.base/share/classes/java/util/ArrayDeque.java>
- `Deque.java` (bảng method ném exception / trả giá trị đặc biệt):
<https://raw.githubusercontent.com/openjdk/jdk21u/master/src/java.base/share/classes/java/util/Deque.java>

Chương 6 — Lambda và Stream (Working with Streams and Lambda expressions)

Mục tiêu

- 6.1 Dùng stream object và stream primitive, cùng biểu thức lambda cài đặt functional interface, để **tạo, lọc, biến đổi, xử lý và sắp xếp** dữ liệu.
- 6.2 **Phân rã** (decomposition: `flatMap`), **nối** (concatenation: `concat`), **rút gọn** (reduction: `reduce`, `count`, `sum...`), **gom nhóm** (grouping) và **chia hai** (partitioning) trên stream tuần tự và song song.

Giải thích đơn giản

Lambda là một hàm không tên, viết gọn: `x -> x * 2`. Trong Java, lambda luôn “đóng vai” một **functional interface** (interface có đúng một method abstract), ví dụ `Function<Integer, Integer>`.

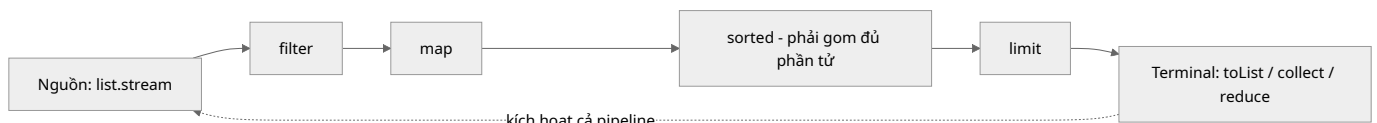
Stream là một dây chuyền xử lý dữ liệu:

nguồn (source) → 0..n thao tác trung gian (intermediate) → 1 thao tác kết thúc (terminal)
`List.stream()` `filter / map / sorted / distinct / limit` `toList / collect / reduce / forEach / count`

Ba đặc điểm cần nhớ:

- Lười (lazy):** thao tác trung gian chỉ chạy khi có terminal operation.
- Từng phần tử một:** mỗi phần tử đi hết dây chuyền rồi phần tử sau mới vào (trừ `sorted` phải gom hết).
- Dùng một lần:** sau terminal operation, stream đóng; dùng lại → `IllegalStateException`.

Stream **không** thay đổi collection nguồn.



Ví dụ

Code trong `examples/ch06/`. Chạy lại: `python3 tools/book.py examples ch06`. Output thật, JDK 21.0.10.

1. Cú pháp lambda và method reference

`examples/ch06/Ex01_LambdaSyntax.java`

```
// objective: 6.1, 3.6
// Cú pháp lambda và 4 loại method reference.
import java.util.*;
import java.util.function.*;

public class Ex01_LambdaSyntax {
    static boolean isShort(String s) { return s.length() < 4; }

    public static void main(String[] args) {
        Predicate<String> p1 = s -> s.isEmpty(); // 1 tham số: bỏ được ()
        Predicate<String> p2 = (String s) -> s.isEmpty(); // ghi kiểu rõ
        Predicate<String> p3 = (var s) -> { return s.isEmpty(); }; // var + khối lệnh cần return
        BiFunction<Integer, Integer, Integer> add = (a, b) -> a + b;
        Supplier<List<String>> maker = () -> new ArrayList<>();
        System.out.println(p1.test("") + " " + p2.test("x") + " " + p3.test("") + " " + add.apply(2, 3) + " " + maker.get());

        Predicate<String> r1 = Ex01_LambdaSyntax::isShort; // static method
        String prefix = "Mr. ";
        Function<String, String> r2 = prefix::concat; // method của một object cụ thể
        Function<String, Integer> r3 = String::length; // method instance, object là tham số đầu
        Supplier<StringBuilder> r4 = StringBuilder::new; // constructor
        Function<Integer, int[]> r5 = int[]::new; // tạo mảng
        System.out.println(r1.test("abc") + " " + r2.apply("Nobin") + " " + r3.apply("lambda")
            + " " + r4.get().append("sb") + " " + r5.apply(3).length);

        int base = 10;
        IntUnaryOperator plusBase = x -> x + base; // effectively final
        // base++; // nếu có dòng này → lambda trên lỗi biên dịch
        System.out.println(plusBase.applyAsInt(5));
    }
}
```

```
}  
}
```

Output thật (JDK 21.0.10):

```
true false true 5 []  
true Mr. Nobin 6 sb 3  
15
```

2. Functional interface có sẵn

examples/ch06/Ex02_FunctionalInterfaces.java

```
// objective: 6.1, 3.6  
// Các functional interface có sẵn trong java.util.function và cách kết hợp chúng.  
import java.util.function.*;  
  
public class Ex02_FunctionalInterfaces {  
    public static void main(String[] args) {  
        Supplier<String> sup = () -> "hi";  
        Consumer<String> con = s -> System.out.print("[" + s + "]);  
        BiConsumer<String, Integer> bi = (k, v) -> System.out.print(k + "=" + v + " ");  
        con.andThen(s -> System.out.println(" again " + s)).accept(sup.get());  
        bi.accept("a", 1);  
        System.out.println();  
  
        Predicate<Integer> even = n -> n % 2 == 0;  
        Predicate<Integer> big = n -> n > 10;  
        System.out.println(even.and(big).test(12) + " " + even.or(big).test(3) + " " + even.negate().test(3)  
            + " " + Predicate.not(even).test(4) + " " + Predicate.isEqual("x").test("x"));  
  
        Function<Integer, Integer> times2 = x -> x * 2;  
        Function<Integer, Integer> plus3 = x -> x + 3;  
        System.out.println(times2.andThen(plus3).apply(5) + " " + times2.compose(plus3).apply(5)  
            + " " + Function.<Integer>identity().apply(7));  
  
        UnaryOperator<String> upper = String::toUpperCase;  
        BinaryOperator<Integer> max = BinaryOperator.maxBy(Integer::compare);  
        System.out.println(upper.apply("java") + " " + max.apply(4, 9));  
  
        IntPredicate odd = i -> i % 2 != 0; // primitive: tránh boxing  
        ToIntFunction<String> len = String::length;  
        IntFunction<String> stars = n -> "***".repeat(n);  
        DoubleUnaryOperator half = d -> d / 2;  
        IntBinaryOperator mul = (a, b) -> a * b;  
        BooleanSupplier yes = () -> true;  
        System.out.println(odd.test(3) + " " + len.applyAsInt("abcd") + " " + stars.apply(3) + " "  
            + half.applyAsDouble(5) + " " + mul.applyAsInt(6, 7) + " " + yes.getAsBoolean());  
    }  
}
```

Output thật (JDK 21.0.10):

```
[hi] again hi  
a=1  
true false true false true  
13 16 7  
JAVA 9  
true 4 *** 2.5 42 true
```

3. Tính lười của stream

examples/ch06/Ex03_Laziness.java

```
// objective: 6.1  
// Stream "lười" (lazy): không có terminal operation thì không chạy gì; phần tử đi qua pipeline TỪNG CÁI MỘT.  
import java.util.List;  
import java.util.stream.Stream;  
  
public class Ex03_Laziness {  
    public static void main(String[] args) {  
        Stream<String> s = Stream.of("a", "bb", "ccc")  
            .peek(x -> System.out.println("peek " + x))  
            .filter(x -> x.length() > 1);  
        System.out.println("chưa có terminal → chưa in peek");  
        System.out.println(s.toList());  
  
        List<String> result = Stream.of("one", "two", "three", "four")  
            .filter(x -> { System.out.println("filter " + x); return x.length() == 3; })  
            .map(x -> { System.out.println("map " + x); return x.toUpperCase(); })  
            .limit(1) // đủ 1 phần tử thì dừng luôn  
            .toList();  
    }  
}
```

```

    System.out.println(result);

    Stream<Integer> once = Stream.of(1, 2);
    once.count();
    try {
        once.count(); // stream chỉ dùng được một lần
    } catch (IllegalStateException e) {
        System.out.println("IllegalStateException: " + e.getMessage());
    }
}
}

```

Output thật (JDK 21.0.10):

```

chưa có terminal → chưa in peek
peek a
peek bb
peek ccc
[bb, ccc]
filter one
map one
[ONE]
IllegalStateException: stream has already been operated upon or closed

```

4. filter, map, sorted, distinct, limit, skip

examples/ch06/Ex04_FilterMapSort.java

```

// objective: 6.1
// Tạo, lọc, biến đổi, sắp xếp dữ liệu với stream.
import java.util.*;
import java.util.stream.*;

public class Ex04_FilterMapSort {
    record Emp(String name, String dept, int salary) { }

    public static void main(String[] args) {
        List<Emp> emps = List.of(
            new Emp("An", "IT", 3000), new Emp("Binh", "HR", 2000),
            new Emp("Chi", "IT", 4000), new Emp("Dung", "Sales", 2500), new Emp("Em", "IT", 3000));

        System.out.println(emps.stream()
            .filter(e -> e.dept().equals("IT"))
            .sorted(Comparator.comparingInt(Emp::salary).reversed().thenComparing(Emp::name))
            .map(Emp::name)
            .collect(Collectors.joining(", ", "[", "]")));

        System.out.println(Stream.of(5, 3, 5, 1, 3, 9).distinct().sorted().skip(1).limit(2).toList());
        System.out.println(Stream.of("b", "a", "c").sorted(Comparator.reverseOrder()).toList());
        System.out.println(Stream.iterate(1, x -> x * 2).limit(6).toList());
        System.out.println(Stream.iterate(1, x -> x < 50, x -> x * 3).toList()); // iterate có điều kiện dừng
        System.out.println(Stream.generate(() -> "x").limit(3).collect(Collectors.joining()));
        System.out.println(Arrays.stream(new int[]{3, 1, 2}).map(x -> x * 10).boxed().toList());
        System.out.println(Stream.ofNullable(null).count() + " " + Stream.empty().findFirst());
    }
}

```

Output thật (JDK 21.0.10):

```

[Chi, An, Em]
[3, 5]
[c, b, a]
[1, 2, 4, 8, 16, 32]
[1, 3, 9, 27]
xxx
[30, 10, 20]
0 Optional.empty

```

5. flatMap và concat

examples/ch06/Ex05_FlatMapConcat.java

```

// objective: 6.2
// Phân rã (decomposition) bằng flatMap, nối (concatenation) bằng Stream.concat.
import java.util.*;
import java.util.stream.*;

public class Ex05_FlatMapConcat {
    public static void main(String[] args) {
        List<List<Integer>> nested = List.of(List.of(1, 2), List.of(3), List.of());
        System.out.println(nested.stream().flatMap(List::stream).map(x -> x * x).toList());
    }
}

```

```

List<String> lines = List.of("to be", "or not", "to be");
System.out.println(lines.stream().flatMap(l -> Arrays.stream(l.split(" "))).distinct().toList());

Stream<String> a = Stream.of("x", "y");
Stream<String> b = Stream.of("z");
System.out.println(Stream.concat(a, b).toList());

System.out.println(IntStream.concat(IntStream.range(0, 2), IntStream.of(9)).boxed().toList());
System.out.println(Stream.of("ab", "cde").flatMapToInt(String::chars).mapToObj(c -> (char) c).toList());

List<Object> expanded = Stream.of(1, 2, 3)
    .<Object>mapMulti((n, sink) -> { if (n != 2) { sink.accept(n); sink.accept(-n); } })
    .toList();
System.out.println(expanded);
}
}

```

Output thật (JDK 21.0.10):

```

[1, 4, 9]
[to, be, or, not]
[x, y, z]
[0, 1, 9]
[a, b, c, d, e]
[1, -1, 3, -3]

```

6. Reduction

examples/ch06/Ex06_Reduction.java

```

// objective: 6.2
// Rút gọn (reduction): reduce, count, min, max, sum, average.
import java.util.*;
import java.util.stream.*;

public class Ex06_Reduction {
    public static void main(String[] args) {
        List<Integer> nums = List.of(4, 8, 15, 16, 23, 42);
        int sum1 = nums.stream().reduce(0, Integer::sum); // identity + accumulator
        Optional<Integer> sum2 = nums.stream().reduce(Integer::sum); // không identity -> Optional
        int totalLen = Stream.of("ab", "cde").reduce(0, (acc, s) -> acc + s.length(), Integer::sum); // 3 tham số
        System.out.println(sum1 + " " + sum2 + " " + totalLen);

        System.out.println(nums.stream().count() + " " + nums.stream().max(Integer::compare).get()
            + " " + nums.stream().min(Comparator.naturalOrder()).orElse(-1));
        System.out.println(nums.stream().mapToInt(Integer::intValue).sum() + " "
            + nums.stream().mapToInt(i -> i).average().getAsDouble());
        System.out.println(Stream.<Integer>empty().reduce(Integer::sum) + " "
            + Stream.<Integer>empty().reduce(100, Integer::sum) + " "
            + IntStream.empty().average() + " " + IntStream.empty().sum());
        String joined = Stream.of("a", "b", "c").reduce("", (x, y) -> x + y);
        System.out.println(joined);
    }
}

```

Output thật (JDK 21.0.10):

```

108 Optional[108] 5
6 42 4
108 18.0
Optional.empty 100 OptionalDouble.empty 0
abc

```

7. Optional

examples/ch06/Ex07_Optional.java

```

// objective: 6.1
// Optional: chứa 0 hoặc 1 giá trị; tránh null.
import java.util.Optional;

public class Ex07_Optional {
    static String expensive() {
        System.out.print("(expensive called) ");
        return "fallback";
    }

    public static void main(String[] args) {
        Optional<String> some = Optional.of("java");
        Optional<String> none = Optional.empty();
        Optional<String> maybe = Optional.ofNullable(null);
        System.out.println(some + " " + none + " " + maybe.isPresent() + " " + none.isEmpty());
    }
}

```

```

System.out.println(some.orElse(expensive())); // orElse LUÔN tính tham số
System.out.println(some.orElseGet(Ex07_Optional::expensive)); // orElseGet chỉ gọi khi rỗng
System.out.println(some.map(String::length).filter(n -> n > 3).orElse(0));
some.ifPresentOrElse(v -> System.out.println("có " + v), () -> System.out.println("rỗng"));
none.ifPresentOrElse(v -> System.out.println("có " + v), () -> System.out.println("rỗng"));
System.out.println(some.flatMap(v -> Optional.of(v + "!")).get() + " " + none.or(() -> Optional.of("B")).get());
try {
    none.get();
} catch (java.util.NoSuchElementException e) {
    System.out.println("NoSuchElementException: " + e.getMessage());
}
try {
    Optional.of(null);
} catch (NullPointerException e) {
    System.out.println("Optional.of(null) -> NPE");
}
try {
    none.orElseThrow();
} catch (java.util.NoSuchElementException e) {
    System.out.println("orElseThrow() -> NoSuchElementException");
}
}
}

```

Output thật (JDK 21.0.10):

```

Optional[java] Optional.empty false true
(expensive called) java
java
4
có java
rỗng
java! B
NoSuchElementException: No value present
Optional.of(null) -> NPE
orElseThrow() -> NoSuchElementException

```

8. Stream primitive

examples/ch06/Ex08_PrimitiveStreams.java

```

// objective: 6.1
// IntStream, LongStream, DoubleStream: range, sum, average, summaryStatistics, boxed, mapToObj.
import java.util.*;
import java.util.stream.*;

public class Ex08_PrimitiveStreams {
    public static void main(String[] args) {
        System.out.println(IntStream.range(1, 5).boxed().toList() + " " + IntStream.rangeClosed(1, 5).sum());
        IntSummaryStatistics st = IntStream.of(4, 9, 2).summaryStatistics();
        System.out.println(st.getMin() + " " + st.getMax() + " " + st.getAverage() + " " + st.getSum() + " " + st.getCount());
        OptionalDouble avg = IntStream.of(1, 2).average();
        OptionalInt max = IntStream.empty().max();
        System.out.println(avg + " " + avg.getAsDouble() + " " + max + " " + max.orElse(-1));
        System.out.println(DoubleStream.of(1.5, 2.5).map(d -> d * 2).sum() + " " + LongStream.rangeClosed(1, 20).reduce(1, (a, b) ->
a * b));
        List<String> labels = IntStream.range(0, 3).mapToObj(i -> "#" + i).toList();
        System.out.println(labels + " " + Stream.of("a", "bb").mapToInt(String::length).max().getAsInt());
        double[] ds = Stream.of(1, 2).mapToDouble(Integer::doubleValue).toArray();
        System.out.println(Arrays.toString(ds) + " " + IntStream.of(3, 4).asLongStream().sum() + " " + IntStream.of(1, 2).mapToLong(i
-> i * 10L).sum());
    }
}

```

Output thật (JDK 21.0.10):

```

[1, 2, 3, 4] 15
2 9 5.0 15 3
OptionalDouble[1.5] 1.5 OptionalInt.empty -1
8.0 2432902008176640000
[#0, #1, #2] 2
[1.0, 2.0] 7 30

```

9. Collectors: groupingBy, partitioningBy, toMap

examples/ch06/Ex09_Collectors.java

```

// objective: 6.2
// Collectors: toList, joining, toMap, groupingBy (gom nhóm), partitioningBy (chia hai), counting, mapping...
import java.util.*;
import java.util.stream.*;

```

```

public class Ex09_Collectors {
    record Emp(String name, String dept, int salary) { }

    public static void main(String[] args) {
        List<Emp> emps = List.of(new Emp("An", "IT", 3000), new Emp("Binh", "HR", 2000),
            new Emp("Chi", "IT", 4000), new Emp("Dung", "Sales", 2500), new Emp("Em", "HR", 2200));

        Map<String, List<String>> byDept = emps.stream().collect(
            Collectors.groupingBy(Emp::dept, TreeMap::new, Collectors.mapping(Emp::name, Collectors.toList())));
        System.out.println(byDept);

        Map<String, Long> count = emps.stream().collect(Collectors.groupingBy(Emp::dept, TreeMap::new, Collectors.counting()));
        Map<String, Double> avg = emps.stream().collect(Collectors.groupingBy(Emp::dept, TreeMap::new,
            Collectors.averagingInt(Emp::salary)));
        System.out.println(count + " " + avg);

        Map<Boolean, List<String>> rich = emps.stream().collect(
            Collectors.partitioningBy(e -> e.salary() >= 3000, Collectors.mapping(Emp::name, Collectors.toList())));
        System.out.println(rich);
        System.out.println(Stream.of("x").collect(Collectors.partitioningBy(s -> s.isEmpty()))); // luôn có cả false và true

        Map<String, Integer> nameToSalary = emps.stream().collect(Collectors.toMap(Emp::name, Emp::salary));
        System.out.println(new TreeMap<>(nameToSalary));
        Map<String, Integer> maxByDept = emps.stream().collect(
            Collectors.toMap(Emp::dept, Emp::salary, Integer::max, TreeMap::new));
        System.out.println(maxByDept);
        try {
            emps.stream().collect(Collectors.toMap(Emp::dept, Emp::name)); // key trùng, không có merge
        } catch (IllegalStateException e) {
            System.out.println("IllegalStateException: " + e.getMessage());
        }

        System.out.println(emps.stream().map(Emp::name).collect(Collectors.joining("|")) + " "
            + emps.stream().collect(Collectors.summingInt(Emp::salary)) + " "
            + emps.stream().collect(Collectors.teeing(Collectors.counting(), Collectors.summingInt(Emp::salary),
                (n, total) -> total / n)));
        Optional<Emp> top = emps.stream().collect(Collectors.maxBy(Comparator.comparingInt(Emp::salary)));
        System.out.println(top.map(Emp::name).orElse("-") + " "
            + emps.stream().map(Emp::dept).collect(Collectors.toCollection(TreeSet::new)));
    }
}

```

Output thật (JDK 21.0.10):

```

{HR=[Binh, Em], IT=[An, Chi], Sales=[Dung]}
{HR=2, IT=2, Sales=1} {HR=2100.0, IT=3500.0, Sales=2500.0}
{false=[Binh, Dung, Em], true=[An, Chi]}
{false=[x], true=[]}
{An=3000, Binh=2000, Chi=4000, Dung=2500, Em=2200}
{HR=2200, IT=4000, Sales=2500}
IllegalStateException: Duplicate key IT (attempted merging values An and Chi)
An|Binh|Chi|Dung|Em 13700 2740
Chi [HR, IT, Sales]

```

10. Short-circuit, takeWhile, dropWhile

examples/ch06/Ex10_ShortCircuit.java

```

// objective: 6.1
// Terminal operation "ngắn mạch" (short-circuit): anyMatch, allMatch, noneMatch, findFirst; takeWhile, dropWhile.
import java.util.stream.*;

public class Ex10_ShortCircuit {
    public static void main(String[] args) {
        System.out.println(Stream.of(1, 2, 3).anyMatch(x -> x > 2) + " " + Stream.of(1, 2, 3).allMatch(x -> x > 2)
            + " " + Stream.of(1, 2, 3).noneMatch(x -> x > 5));
        System.out.println("empty: any=" + Stream.empty().anyMatch(x -> true) + " all=" + Stream.empty().allMatch(x -> false)
            + " none=" + Stream.empty().noneMatch(x -> true));

        boolean found = Stream.iterate(1, x -> x + 1) // stream vô hạn
            .peek(x -> System.out.print(x + " "))
            .anyMatch(x -> x % 7 == 0); // dừng ở phần tử đầu tiên thoả
        System.out.println("-> " + found);

        System.out.println(Stream.of(1, 3, 5, 6, 7, 9).takeWhile(x -> x % 2 == 1).toList() + " "
            + Stream.of(1, 3, 5, 6, 7, 9).dropWhile(x -> x % 2 == 1).toList());
        System.out.println(Stream.of("a", "b", "c").findFirst().get() + " "
            + IntStream.iterate(10, x -> x - 3).filter(x -> x < 0).findFirst().getAsInt());
    }
}

```

Output thật (JDK 21.0.10):

```
true false true
empty: any=false all=true none=true
1 2 3 4 5 6 7 → true
[1, 3, 5] [6, 7, 9]
a -2
```

11. Parallel stream và reduce

Kết quả dòng `par reduce10` và `par minus` là output thật trên máy kiểm tra (4 CPU); nó **phụ thuộc cách stream được chia nhỏ**, nên không được coi là giá trị bảo đảm. Bài học: identity phải là phần tử trung hoà thật và hàm phải có tính kết hợp.

examples/ch06/Ex11_ParallelReduce.java

```
// objective: 6.2, 8.3
// Parallel stream: kết quả reduce đúng chỉ khi identity đúng nghĩa và hàm có tính kết hợp (associative).
import java.util.List;
import java.util.stream.IntStream;

public class Ex11_ParallelReduce {
    public static void main(String[] args) {
        List<Integer> nums = List.of(1, 2, 3, 4);
        System.out.println("seq sum      = " + nums.stream().reduce(0, Integer::sum));
        System.out.println("par sum      = " + nums.parallelStream().reduce(0, Integer::sum));
        System.out.println("seq reduce10 = " + nums.stream().reduce(10, Integer::sum));
        // 10 không phải identity của phép cộng → song song cộng 10 ở MỖI phần nhỏ (kết quả phụ thuộc cách chia)
        System.out.println("par reduce10 = " + nums.parallelStream().reduce(10, Integer::sum));
        // Phép trừ không kết hợp → song song cho kết quả khác tuần tự
        System.out.println("seq minus   = " + IntStream.rangeClosed(1, 4).reduce(0, (a, b) -> a - b));
        System.out.println("par minus   = " + IntStream.rangeClosed(1, 4).parallel().reduce(0, (a, b) -> a - b));
        System.out.println("isParallel = " + nums.parallelStream().isParallel() + " / "
            + nums.stream().parallel().sequential().isParallel());
        // forEachOrdered giữ thứ tự nguồn kể cả khi song song
        StringBuilder sb = new StringBuilder();
        nums.parallelStream().map(x -> x * 10).forEachOrdered(x -> sb.append(x).append(' '));
        System.out.println("forEachOrdered: " + sb.toString().trim());
        System.out.println("findFirst trên parallel vẫn là phần tử đầu: " + nums.parallelStream().findFirst().get());
    }
}
```

Output thật (JDK 21.0.10):

```
seq sum      = 10
par sum      = 10
seq reduce10 = 20
par reduce10 = 50
seq minus    = -10
par minus    = 0
isParallel   = true / false
forEachOrdered: 10 20 30 40
findFirst trên parallel vẫn là phần tử đầu: 1
```

12. Lỗi lambda hay gặp

examples/ch06/Ex12_LambdaErrors.java

```
// objective: 6.1, 3.6
// expect: compile-error
// Lỗi lambda hay gặp: tham số trùng tên biến cục bộ; khối lệnh thiếu return.
import java.util.function.*;

public class Ex12_LambdaErrors {
    public static void main(String[] args) {
        String s = "x";
        Predicate<String> p = s -> s.isEmpty();
        Function<Integer, Integer> f = x -> { int y = x * 2; };
    }
}
```

Output thật của `javac` (JDK 21.0.10) — cố ý không biên dịch được:

```
Ex12_LambdaErrors.java:9: error: variable s is already defined in method main(String[])
        Predicate<String> p = s -> s.isEmpty();
                               ^
Ex12_LambdaErrors.java:10: error: incompatible types: bad return type in lambda expression
        Function<Integer, Integer> f = x -> { int y = x * 2; };
                                         ^
        missing return value
2 errors
```

13. Effectively final

examples/ch06/Ex13_EffectivelyFinal.java

```
// objective: 6.1, 3.6
// expect: compile-error
// Lambda chỉ dùng được biến cục bộ final hoặc effectively final (không bị gán lại ở bất kỳ đâu).
public class Ex13_EffectivelyFinal {
    public static void main(String[] args) {
        int count = 0;
        Runnable r = () -> System.out.println(count);
        count++; // gán lại SAU lambda vẫn làm count mất tính effectively final
        int[] box = {0};
        Runnable ok = () -> box[0]++; // OK: tham chiếu box không đổi, chỉ nội dung mảng đổi
    }
}
```

Output thật của javac (JDK 21.0.10) — cố ý không biên dịch được:

```
Ex13_EffectivelyFinal.java:7: error: local variables referenced from a lambda expression must be final or effectively final
    Runnable r = () -> System.out.println(count);
                        ^
1 error
```

Đi sâu

Cú pháp lambda

Dạng	Ví dụ
Không tham số	() -> 42
Một tham số, không kiểu	x -> x + 1 (bỏ được ngoặc)
Có kiểu hoặc var	(String s) -> s.length(), (var s) -> s.length() — phải có ngoặc; không trộn kiểu/var/không kiểu
Khối lệnh	(a, b) -> { int c = a + b; return c; } — cần return nếu có giá trị, và dấu ;

Lambda dùng được: tham số của nó, field (không giới hạn), biến cục bộ **final** hoặc **effectively final**. Tham số lambda không được trùng tên biến cục bộ đang trong phạm vi.

Method reference (4 loại)

Loại	Cú pháp	Lambda tương đương
Static method	Integer::parseInt	s -> Integer.parseInt(s)
Method của object cụ thể	System.out::println, prefix::concat	x -> System.out.println(x)
Method instance, object là tham số đầu	String::length, String::equals	s -> s.length(), (a, b) -> a.equals(b)
Constructor	ArrayList::new, int[]::new	() -> new ArrayList<>(), n -> new int[n]

Functional interface trong java.util.function

Interface	Method	Ý nghĩa
Supplier<T>	T get()	Tạo giá trị
Consumer<T> / BiConsumer<T,U>	void accept(T) / (T, U)	Dùng giá trị, không trả về
Predicate<T> / BiPredicate<T,U>	boolean test(T)	Điều kiện; and, or, negate, Predicate.not
Function<T,R> / BiFunction<T,U,R>	R apply(T)	Biến đổi; andThen, compose, identity
UnaryOperator<T>	T apply(T)	Function<T,T>
BinaryOperator<T>	T apply(T, T)	BiFunction<T,T,T>; minBy, maxBy

Phiên bản primitive (tránh boxing): IntPredicate, IntFunction<R>, ToIntFunction<T>, IntUnaryOperator, IntBinaryOperator, IntSupplier (getAsInt), IntConsumer, BooleanSupplier (getAsBoolean), ObjIntConsumer<T>, tương tự cho Long, Double.

Thao tác trung gian (intermediate)

filter, map, mapToInt/mapToObj/..., flatMap, mapMulti, distinct, sorted() / sorted(cmp), peek, limit, skip, takeWhile, dropWhile, boxed, parallel, sequential, unordered.

Thao tác kết thúc (terminal)

Method	Trả về	Ghi chú
forEach, forEachOrdered	void	forEach trên parallel không giữ thứ tự
toList() (Java 16+)	List<T> bất biến	Khác collect(Collectors.toList()) (không bảo đảm bất biến)
collect(collector)	tuỳ	
reduce(identity, op)	T	
reduce(op)	Optional<T>	
reduce(identity, accumulator, combiner)	U	Đổi kiểu kết quả
count()	long	
min(cmp), max(cmp)	Optional<T>	Trên primitive stream: OptionalInt... không cần comparator
findFirst(), findAny()	Optional<T>	Short-circuit
anyMatch, allMatch, noneMatch	boolean	Short-circuit; stream rỗng: false, true, true
toArray()	Object[]	toArray(String[]::new)

Stream primitive

- Tạo: IntStream.range(a, b) (không gồm b), rangeClosed(a, b), IntStream.of(...), Arrays.stream(int[]), "abc".chars().
- Có sẵn: sum() (trả về int/long/double), average() (luôn OptionalDouble), max()/min() (OptionalInt...), summaryStatistics().
- Chuyển đổi: boxed(), mapToObj, mapToLong, asDoubleStream; từ object: mapToInt(ToIntFunction).

Optional

Optional.of(v) (v không được null), ofNullable(v), empty(). Dùng: isPresent, isEmpty, get / orElseThrow() (rỗng → NoSuchElementException), orElse(v) (luôn tính v), orElseGet(supplier), orElseThrow(supplier), map, flatMap, filter, ifPresent, ifPresentOrElse, or.

Collectors hay ra đề

Collector	Kết quả
toList(), toSet(), toCollection(TreeSet::new)	Collection
joining(), joining(", "), joining(", ", "[" , "]")	String
toMap(keyFn, valueFn)	Trùng key → IllegalStateException
toMap(keyFn, valueFn, merge), toMap(..., merge, TreeMap::new)	Có hàm merge, chọn loại map
groupingBy(classifier)	Map<K, List<T>> (HashMap)
groupingBy(classifier, downstream)	ví dụ counting() → Map<K, Long>
groupingBy(classifier, mapFactory, downstream)	ví dụ TreeMap::new
partitioningBy(predicate[, downstream])	Map<Boolean, ...> luôn có cả false và true
counting(), summingInt, averagingInt (→ Double), minBy, maxBy (→ Optional)	
mapping(fn, downstream), filtering, flatMapping	Collector phụ
teeing(c1, c2, merger)	Kết hợp hai collector

Parallel stream

- Tạo: collection.parallelStream() hoặc stream.parallel(). isParallel() kiểm tra; lệnh gọi cuối cùng (parallel/sequential) quyết định cả pipeline.
- reduce song song đúng khi: identity là phần tử trung hoà (0 cho +, 1 cho *, "" cho nối chuỗi), accumulator **kết hợp (associative)** và **không trạng thái (stateless)**.
- Với nguồn có thứ tự: findFirst, limit, skip, forEachOrdered, collect(toList()) giữ thứ tự gặp; findAny, forEach thì không.
- Tránh lambda có tác dụng phụ (ghi vào list chung không đồng bộ) — xem chương 8.

Lỗi và bẫy thường gặp (Exam traps)

1. Không có terminal operation → không có gì chạy (kể cả peek).
2. Dùng lại stream → IllegalStateException.
3. orElse(f()) luôn gọi f().
4. Optional.of(null) → NPE; get() trên rỗng → NoSuchElementException.

- 5. `average()` trả `OptionalDouble`; `sum()` của `IntStream` trả `int`; `count()` trả `long`.
- 6. `range(1, 4)` không gồm 4.
- 7. `toMap` trùng key không có `merge` → `IllegalStateException`.
- 8. `partitioningBy` luôn có hai khoá, kể cả nhóm rỗng.
- 9. `allMatch` trên stream rỗng → `true`.
- 10. `takeWhile` dừng ở phần tử sai đầu tiên, khác `filter`.
- 11. Lambda dùng biến cục bộ bị gán lại (ở bất kỳ đâu) → lỗi.
- 12. Method reference có `()` → lỗi.
- 13. `reduce(10, Integer::sum)` song song có thể khác tuần tự.
- 14. `sorted()` trên object không `Comparable` → `ClassCastException` lúc chạy.

Góc nhìn từ TypeScript

TypeScript	Java	Ghi chú
<code>arr.filter(...).map(...)</code> chạy ngay, tạo mảng trung gian	<code>list.stream().filter(...).map(...)</code> lười, không tạo list trung gian	Cần terminal operation như <code>toList()</code>
<code>arr.reduce((a, b) => a + b, 0)</code>	<code>stream.reduce(0, Integer::sum)</code>	Thứ tự tham số khác nhau
<code>arr.flatMap(x => x.items)</code>	<code>stream.flatMap(x -> x.items().stream())</code>	Java cần trả về một <code>Stream</code>
<code>arr.some</code> , <code>arr.every</code>	<code>anyMatch</code> , <code>allMatch</code>	
<code>arr.find(...)</code> trả <code>undefined</code> nếu không thấy	<code>filter(...).findFirst()</code> trả <code>Optional</code>	
Arrow function <code>(x) => x * 2</code>	Lambda <code>x -> x * 2</code>	Dấu <code>-></code> thay vì <code>=></code>
Closure bắt được mọi biến, kể cả <code>let</code> bị gán lại	Chỉ biến <code>effectively final</code>	
<code>Object.groupBy(arr, fn)</code> (ES2024)	<code>Collectors.groupingBy(fn)</code>	
Mảng dùng lại nhiều lần	<code>Stream</code> chỉ dùng một lần	

Tóm tắt

- Lambda = cài đặt functional interface; biến cục bộ phải `effectively final`.
- 4 loại method reference; biết interface nào có method gì (`get`, `accept`, `test`, `apply`).
- Stream: nguồn → intermediate (lười) → một terminal; dùng một lần.
- Reduction: `reduce` (3 dạng), `count`, `sum`, `average` (`OptionalDouble`), `min/max` (`Optional`).
- Collectors: `groupingBy` (+ downstream + map factory), `partitioningBy`, `toMap` (`merge`), `joining`.
- Parallel: identity đúng + hàm kết hợp; thứ tự chỉ được giữ với các thao tác “ordered”.

Bài tập (có lời giải)

Mọi đáp án đã được `tools/book.py` biên dịch và chạy để xác nhận (code: `examples/questions/ch06/`).

Câu hỏi

Câu 06-01 · Dễ · objective 6.1

Chương trình sau in ra gì?

```
import java.util.*;
import java.util.stream.*;

public class Basic {
    public static void main(String[] args) {
        List<String> r = Stream.of("pear", "fig", "apple", "kiwi")
            .filter(s -> s.length() > 3)
            .map(String::toUpperCase)
            .sorted()
            .toList();
        System.out.println(r);
    }
}
```

- A. [PEAR, APPLE, KIWI]
- B. [APPLE, FIG, KIWI, PEAR]
- C. [APPLE, KIWI, PEAR]
- D. [apple, kiwi, pear]

Câu 06-02 · Vừa · objective 6.1

Chương trình sau in ra gì?

```
import java.util.stream.*;

public class Lazy {
    public static void main(String[] args) {
        Stream.of(1, 2, 3, 4, 5)
            .peek(x -> System.out.print("p" + x + " "))
            .filter(x -> x % 2 == 0)
            .limit(1)
            .forEach(x -> System.out.print("f" + x + " "));
    }
}
```

- A. p1 p2 p3 p4 p5 f2
- B. p1 p2 f2
- C. p1 p2 f2 p3 p4 f4
- D. f2

Câu 06-03 · Vừa · objective 6.2

Chương trình sau in ra gì?

```
import java.util.*;
import java.util.stream.*;

public class Reduce {
    public static void main(String[] args) {
        int a = Stream.of(1, 2, 3).reduce(10, Integer::sum);
        Optional<Integer> b = Stream.<Integer>empty().reduce(Integer::sum);
        int c = Stream.of("a", "bb", "ccc").reduce(0, (acc, s) -> acc + s.length(), Integer::sum);
        System.out.println(a + " " + b + " " + c);
    }
}
```

- A. 6 0 6
- B. 16 null 6
- C. 16 Optional[0] 6
- D. 16 Optional.empty 6

Câu 06-04 · Khó · objective 6.2

Chương trình sau in ra gì?

```
import java.util.*;
import java.util.stream.*;

public class Groups {
    record Item(String cat, int price) { }

    public static void main(String[] args) {
        Map<Boolean, Map<String, Long>> m = Stream.of(
            new Item("a", 5), new Item("b", 20), new Item("a", 30), new Item("c", 1))
            .collect(Collectors.partitioningBy(i -> i.price() > 10,
                Collectors.groupingBy(Item::cat, TreeMap::new, Collectors.counting())));
        System.out.println(m);
    }
}
```

- A. {true={a=1, b=1}, false={a=1, c=1}}
- B. {false={a=1, c=1}, true={a=1, b=1}}
- C. {false={a=2, c=1}, true={b=1}}
- D. {false={a=1, c=1}, true={a=1, b=1}, null={}}

Câu 06-05 · Vừa · objective 6.1

Chương trình sau in ra gì?

```
import java.util.*;

public class Opt {
    static String load() {
        System.out.print("load ");
        return "db";
    }
}
```

```

public static void main(String[] args) {
    Optional<String> o = Optional.of("cache");
    String a = o.orElse(load());
    String b = o.orElseGet(() -> load());
    String c = Optional.<String>empty().map(String::toUpperCase).orElseGet(() -> "none");
    System.out.println(a + " " + b + " " + c);
}
}

```

- A. cache cache none
- B. load load cache cache none
- C. load cache cache none
- D. load cache db NONE

Câu 06-06 · Vừa · objective 6.1, 3.6

Chèn dòng nào vào chỗ `// INSERT CODE HERE` thì chương trình biên dịch được? (Chọn 3 đáp án.)

```

import java.util.function.*;

public class Syntax {
    public static void main(String[] args) {
        // INSERT CODE HERE
    }
}

```

- A. `Function<String, Integer> f = s -> s.length();`
- B. `BiFunction<String, String, Boolean> b = (x, y) -> { x.equals(y); };`
- C. `Supplier<String> s = () -> "hi";`
- D. `Predicate<String> p = (var x, y) -> true;`
- E. `Consumer<String> c = (String x) -> System.out.println(x);`
- F. `Runnable r = x -> {};`

Câu 06-07 · Vừa · objective 6.2

Chương trình sau in ra gì?

```

import java.util.*;
import java.util.stream.*;

public class Flat {
    public static void main(String[] args) {
        List<List<String>> data = List.of(List.of("a", "b"), List.of("b", "c"), List.of());
        long n = data.stream().flatMap(List::stream).distinct().count();
        String s = data.stream().map(List::size).map(String::valueOf)
            .collect(Collectors.joining("-", "<", ">"));
        System.out.println(n + " " + s);
    }
}

```

- A. 3 <2-2-0>
- B. 4 <2-2-0>
- C. 3 <2-2>
- D. 3 2-2-0

Câu 06-08 · Vừa · objective 6.1

Hai phát biểu nào đúng? (Chọn 2 đáp án.)

- A. Một stream có thể dừng lại sau khi đã chạy terminal operation.
- B. `allMatch` trên stream rỗng trả về `true`.
- C. `peek` là một terminal operation.
- D. `IntStream.average()` trả về `OptionalDouble`.
- E. `Collectors.toMap` tự bỏ qua các key bị trùng.

Câu 06-09 · Khó · objective 6.2, 8.3

Điều gì đúng về kết quả của chương trình?

```

import java.util.*;
import java.util.stream.*;

public class Par {
    public static void main(String[] args) {

```

```

List<Integer> r = IntStream.rangeClosed(1, 10).parallel()
    .filter(i -> i % 3 == 0).boxed().collect(Collectors.toList());
int first = IntStream.rangeClosed(1, 10).parallel().filter(i -> i > 4).findFirst().getAsInt();
System.out.println(r + " " + first);
}
}

```

- A. Luôn in [3, 6, 9] 5
- B. In [3, 6, 9] và một số bất kỳ lớn hơn 4
- C. Thứ tự trong list không xác định, còn số luôn là 5
- D. Không biên dịch được

Câu 06-10 · Vừa · objective 6.1

Chương trình sau in ra gì?

```

import java.util.*;
import java.util.stream.*;

public class Prim {
    public static void main(String[] args) {
        IntSummaryStatistics s = IntStream.of(3, 8, 1).summaryStatistics();
        double avg = IntStream.rangeClosed(1, 4).average().orElse(0);
        System.out.println(s.getMax() + " " + s.getAverage() + " " + avg + " " + IntStream.range(1, 4).sum());
    }
}

```

- A. 8 4 2.5 10
- B. 8 4.0 2.5 10
- C. 8 4.0 2.0 6
- D. 8 4.0 2.5 6

Câu 06-11 · Vừa · objective 6.1, 3.6

Những dòng nào gây lỗi biên dịch?

```

import java.util.function.*;

public class Capture {
    int field = 0;

    void run() {
        int local = 1;
        int other = 2;
        Supplier<Integer> a = () -> field++; // L1
        Supplier<Integer> b = () -> local + 1; // L2
        Supplier<Integer> c = () -> other++; // L3
        IntUnaryOperator d = local -> local * 2; // L4
    }

    public static void main(String[] args) { }
}

```

- A. Chỉ L3
- B. L1 và L3
- C. L3 và L4
- D. L1, L3 và L4
- E. Chỉ L4

Câu 06-12 · Khó · objective 6.2

Kết quả của chương trình là gì?

```

import java.util.*;
import java.util.stream.*;

public class ToMap {
    public static void main(String[] args) {
        Map<Integer, String> m = Stream.of("aa", "b", "cc", "ddd", "e")
            .collect(Collectors.toMap(String::length, s -> s, (x, y) -> x + y, TreeMap::new));
        System.out.println(m);
    }
}

```

- A. In ra {1=e, 2=cc, 3=ddd}
- B. In ra {1=b, 2=aa, 3=ddd}
- C. In ra {1=be, 2=aacc, 3=ddd}

- D. Ném `IllegalStateException` vì trùng key

Câu 06-13 · Vừa · objective 6.1

Chương trình sau in ra gì?

```
import java.util.function.*;

public class Compose {
    public static void main(String[] args) {
        Function<Integer, Integer> inc = x -> x + 1;
        Function<Integer, Integer> dbl = x -> x * 2;
        System.out.println(inc.andThen(dbl).apply(3) + " " + inc.compose(dbl).apply(3) + " "
            + dbl.andThen(dbl).compose(inc).apply(1));
    }
}
```

- A. 7 8 8
- B. 8 7 6
- C. 8 8 8
- D. 8 7 8

Câu 06-14 · Dễ · objective 6.1

Chương trình sau in ra gì?

```
import java.util.stream.*;

public class While {
    public static void main(String[] args) {
        System.out.println(Stream.of(2, 4, 5, 6).takeWhile(x -> x % 2 == 0).toList() + " "
            + Stream.of(2, 4, 5, 6).dropWhile(x -> x % 2 == 0).toList());
    }
}
```

- A. [2, 4] [5, 6]
- B. [2, 4, 6] [5]
- C. [2, 4] [5]
- D. [5, 6] [2, 4]

Câu 06-15 · Khó · objective 6.2

Đoạn code nào, chèn vào chỗ `// INSERT CODE HERE`, in ra đúng `{false=[1, 3], true=[2, 4]}`? (Chọn 2 đáp án.)

```
import java.util.*;
import java.util.stream.*;

public class Part {
    public static void main(String[] args) {
        List<Integer> nums = List.of(1, 2, 3, 4);
        // INSERT CODE HERE
    }
}
```

- A. `System.out.println(nums.stream().collect(Collectors.partitioningBy(n -> n % 2 == 0)));`
- B. `System.out.println(nums.stream().collect(Collectors.partitioningBy(n -> n % 2 == 1)));`
- C. `Map<Boolean, List<Integer>> m = nums.stream().collect(Collectors.groupingBy(n -> n % 2 == 0, TreeMap::new, Collectors.toList())); System.out.println(m);`
- D. `System.out.println(nums.stream().collect(Collectors.partitioningBy(n -> n % 2 == 0, Collectors.counting())));`
- E. `System.out.println(nums.stream().filter(n -> n % 2 == 0).toList());`

Câu 06-16 · Vừa · objective 6.1

Chương trình sau in ra gì?

```
import java.util.*;
import java.util.stream.*;

public class Sort {
    public static void main(String[] args) {
        List<Integer> r = Stream.of("10", "9", "100").sorted().map(Integer::parseInt).toList();
        List<Integer> s = Stream.of("10", "9", "100").map(Integer::parseInt)
            .sorted(Comparator.reverseOrder()).toList();
        System.out.println(r + " " + s);
    }
}
```

- A. [9, 10, 100] [100, 10, 9]
- B. [10, 100, 9] [100, 10, 9]
- C. [10, 100, 9] [9, 100, 10]
- D. [10, 9, 100] [100, 10, 9]

Câu 06-17 · Vừa · objective 6.1, 3.6

Dòng nào gây lỗi biên dịch?

```
import java.util.function.*;

public class Refs {
    static int twice(int x) { return 2 * x; }

    public static void main(String[] args) {
        Function<String, Integer> a = String::length;           // L1
        Supplier<String> b = String::new;                       // L2
        IntUnaryOperator c = Refs::twice;                       // L3
        Function<String, String> d = String::toUpperCase();    // L4
        BiFunction<String, String, Boolean> e = String::equals; // L5
    }
}
```

- A. L2
- B. L4
- C. L2 và L4
- D. L4 và L5
- E. Không dòng nào

Câu 06-18 · Khó · objective 6.1

Hai phát biểu nào đúng về Optional? (Chọn 2 đáp án.)

- A. Optional.ofNullable(null).isPresent() trả về false.
- B. Optional.of(null) trả về một Optional rỗng.
- C. orElse(x) chỉ tính biểu thức x khi Optional rỗng.
- D. Optional.empty().map(f) không gọi hàm f.
- E. get() trên Optional rỗng trả về null.

Câu 06-19 · Vừa · objective 6.2

Chương trình sau in ra gì?

```
import java.util.*;
import java.util.stream.*;

public class Join {
    public static void main(String[] args) {
        Map<Integer, String> m = Stream.of("sun", "moon", "sky", "star")
            .collect(Collectors.groupingBy(String::length, TreeMap::new, Collectors.joining("+")));
        System.out.println(m);
    }
}
```

- A. {3=sun+sky, 4=moon+star}
- B. {3=[sun, sky], 4=[moon, star]}
- C. {4=moon+star, 3=sun+sky}
- D. {3=sun+sky, 4=moon+star}

Câu 06-20 · Dễ · objective 6.1

Kết quả của chương trình là gì?

```
import java.util.*;

public class Empty {
    public static void main(String[] args) {
        List<String> empty = List.of();
        System.out.println(empty.stream().count() + " "
            + empty.stream().allMatch(s -> s.length() > 5) + " "
            + empty.stream().anyMatch(s -> true));
    }
}
```

- A. In ra 0 false false

- **B.** In ra `0 true true`
- **C.** Ném `NoSuchElementException`
- **D.** In ra `0 true false`

Lời giải

Câu 06-01 — Đáp án: C

- **Vì sao đúng:** `filter` bỏ `fig` (độ dài 3, không > 3). `map` đổi sang chữ hoa. `sorted()` sắp xếp theo thứ tự tự nhiên.
- **A sai:** `sorted()` sắp xếp lại theo bảng chữ cái, không giữ thứ tự ban đầu.
- **B sai:** `fig` có độ dài 3, bị `filter(s -> s.length() > 3)` loại.
- **D sai:** `map(String::toUpperCase)` đã đổi sang chữ hoa.
- **Kiểm chứng:** [examples/questions/ch06/Q06_01/](#) — output confirmed (python3 tools/book.py questions ch06).

Câu 06-02 — Đáp án: B

- **Vì sao đúng:** Stream xử lý từng phần tử đi hết pipeline (không phải từng bước cho cả danh sách). 1 qua `peek` rồi bị `filter` loại; 2 qua `peek`, qua `filter`, tới `forEach`. `limit(1)` đã đủ nên stream dừng, 3, 4, 5 không được xử lý.
- **A sai:** Stream không chạy `peek` cho tất cả trước; `limit` làm dừng sớm (short-circuit).
- **C sai:** `limit(1)` chỉ cho một phần tử đi qua.
- **D sai:** `peek` vẫn chạy cho các phần tử đã được kéo qua pipeline (1 và 2).
- **Kiểm chứng:** [examples/questions/ch06/Q06_02/](#) — output confirmed (python3 tools/book.py questions ch06).

Câu 06-03 — Đáp án: D

- **Vì sao đúng:** `reduce(10, sum)` bắt đầu từ 10 (tuần tự) $\rightarrow$ 16. `reduce(accumulator)` không có identity trả về `Optional`; stream rỗng $\rightarrow$ `Optional.empty`. Dạng 3 tham số (identity, accumulator, combiner) cho phép đổi kiểu: cộng độ dài $\rightarrow$ 6.
- **A sai:** Giá trị khởi đầu 10 được cộng vào; và dạng không identity trả về `Optional`.
- **B sai:** `reduce` không bao giờ trả về `null`; nó trả về `Optional.empty`.
- **C sai:** Không có phần tử thì không có giá trị nào, nên là `Optional.empty`, không phải 0.
- **Kiểm chứng:** [examples/questions/ch06/Q06_03/](#) — output confirmed (python3 tools/book.py questions ch06).

Câu 06-04 — Đáp án: B

- **Vì sao đúng:** `partitioningBy` chia thành hai nhóm `false` và `true` (in `false` trước). Trong mỗi nhóm, `groupingBy` đếm theo cat vào `TreeMap`. Giá ≤ 10 : a(5), c(1). Giá > 10 : b(20), a(30).
- **A sai:** Map của `partitioningBy` in khoá `false` trước `true`.
- **C sai:** a có một món giá 5 (nhóm `false`) và một món giá 30 (nhóm `true`), nên mỗi nhóm chỉ đếm 1.
- **D sai:** `partitioningBy` chỉ có đúng hai khoá: `false` và `true`.
- **Kiểm chứng:** [examples/questions/ch06/Q06_04/](#) — output confirmed (python3 tools/book.py questions ch06).

Câu 06-05 — Đáp án: C

- **Vì sao đúng:** Tham số của `orElse` là một biểu thức bình thường, nên `load()` luôn được gọi (in `load`), dù `Optional` có giá trị. `orElseGet` nhận `Supplier`, chỉ gọi khi rỗng $\rightarrow$ không in. `map` trên `Optional` rỗng vẫn rỗng $\rightarrow$ "none".
- **A sai:** `orElse(load())` luôn tính `load()` trước khi gọi `orElse`.
- **B sai:** `orElseGet` chỉ gọi `supplier` khi `Optional` rỗng; ở đây có giá trị.
- **D sai:** `Optional` có giá trị `cache`, nên b là `cache`; và c là "none" do `supplier` trả về (không đi qua `map`).
- **Kiểm chứng:** [examples/questions/ch06/Q06_05/](#) — output confirmed (python3 tools/book.py questions ch06).

Câu 06-06 — Đáp án: A, C, E

- **Vì sao đúng:** A: một tham số, bỏ ngoặc và kiểu. C: không tham số dùng `()`. E: tham số có kiểu rõ ràng phải đặt trong ngoặc.
- **B sai:** Khối `{ }` trả về giá trị phải có `return: { return x.equals(y); }`.
- **D sai:** `Predicate` chỉ có một tham số; hơn nữa không được trộn `var` với tham số không có kiểu.
- **F sai:** `Runnable.run()` không có tham số.
- **Kiểm chứng:** [examples/questions/ch06/Q06_06/](#) — variants: ACE satisfy compiles (python3 tools/book.py questions ch06).

Câu 06-07 — Đáp án: A

- **Vì sao đúng:** `flatMap` "làm phẳng" các list con thành một stream: a, b, b, c $\rightarrow$ `distinct` còn 3. `joining(delimiter, prefix, suffix)` nối kích thước các list (2, 2, 0) với - và bọc bởi `< >`.
- **B sai:** `distinct()` loại b trùng.
- **C sai:** List rỗng vẫn là một phần tử của stream ngoài, kích thước 0.
- **D sai:** `joining` với 3 tham số thêm prefix `<` và suffix `>`.
- **Kiểm chứng:** [examples/questions/ch06/Q06_07/](#) — output confirmed (python3 tools/book.py questions ch06).

Câu 06-08 — Đáp án: B, D

- **Vì sao đúng:** B: “mọi phần tử đều thỏa” là đúng một cách hiển nhiên khi không có phần tử nào (vacuous truth). D: `average()` của primitive stream trả về `OptionalDouble` (có thể rỗng).
- **A sai:** Stream chỉ dùng được một lần; lần thứ hai ném `IllegalStateException`.
- **C sai:** `peek` là intermediate operation; không có terminal thì nó không chạy (không in 1, 2).
- **E sai:** Key trùng mà không có hàm merge → `IllegalStateException: Duplicate key`.
- **Kiểm chứng:** `examples/questions/ch06/Q06_08/` — each option proven true/false by a program (`python3 tools/book.py questions ch06`).

Câu 06-09 — Đáp án: A

- **Vì sao đúng:** `IntStream.rangeClosed` là nguồn **có thứ tự (ordered)**. Với stream có thứ tự, `collect(toList())` giữ đúng thứ tự gặp (encounter order) và `findFirst()` luôn trả về phần tử **đầu tiên** thỏa điều kiện, kể cả khi chạy song song.
- **B sai:** Đó là hành vi của `findAny()`, không phải `findFirst()`.
- **C sai:** `collect` trên stream có thứ tự giữ thứ tự gặp, kể cả song song.
- **D sai:** Code hợp lệ: `boxed()` đổi `IntStream` thành `Stream<Integer>` trước khi `collect`.
- **Kiểm chứng:** `examples/questions/ch06/Q06_09/` — output confirmed (`python3 tools/book.py questions ch06`).

Câu 06-10 — Đáp án: D

- **Vì sao đúng:** `getAverage()` trả về `double: (3 + 8 + 1) / 3 = 4.0`. `rangeClosed(1, 4) = 1..4`, trung bình 2.5. `range(1, 4)` **không gồm 4**: $1 + 2 + 3 = 6$.
- **A sai:** `getAverage()` là `double` nên in `4.0`; `range(1, 4)` không gồm 4.
- **B sai:** `range` loại trừ đầu cuối: tổng là 6.
- **C sai:** `average()` tính bằng số thực: $10 / 4 = 2.5$.
- **Kiểm chứng:** `examples/questions/ch06/Q06_10/` — output confirmed (`python3 tools/book.py questions ch06`).

Câu 06-11 — Đáp án: C

- **Vì sao đúng:** Lambda chỉ dùng được biến **cục bộ** final hoặc effectively final; `other++` sửa biến cục bộ → L3 lỗi. Tham số lambda không được trùng tên với biến cục bộ đang trong phạm vi → L4 lỗi. L1 hợp lệ vì `field` là field (qua `this`), không bị giới hạn. L2 chỉ đọc local.
- **A sai:** L4 cũng lỗi: tham số `local` trùng tên biến cục bộ.
- **B sai:** Field của object không bị quy tắc effectively final.
- **D sai:** L1 hợp lệ (xem B).
- **E sai:** L3 cũng lỗi: sửa biến cục bộ trong lambda.
- **Kiểm chứng:** `examples/questions/ch06/Q06_11/` — compile error confirmed at ['L3', 'L4'] (`python3 tools/book.py questions ch06`).

Câu 06-12 — Đáp án: C

- **Vì sao đúng:** `toMap` với 4 tham số: key mapper, value mapper, **hàm merge** khi trùng key, và nhà máy tạo map (`TreeMap`). Trùng key thì nối chuỗi theo thứ tự gặp: độ dài 1 → `b + e`, độ dài 2 → `aa + cc`.
- **A sai:** Hàm merge `(x, y) -> x + y` giữ cả hai giá trị, không chỉ giá trị sau.
- **B sai:** Hàm merge nối cả hai, không chỉ giữ giá trị đầu.
- **D sai:** Có hàm merge nên key trùng không gây exception.
- **Kiểm chứng:** `examples/questions/ch06/Q06_12/` — output confirmed (`python3 tools/book.py questions ch06`).

Câu 06-13 — Đáp án: D

- **Vì sao đúng:** `f.andThen(g)` = chạy f rồi g: $(3 + 1) * 2 = 8$. `f.compose(g)` = chạy g rồi f: $3 * 2 + 1 = 7$. `dbl.andThen(dbl).compose(inc) : inc` trước ($1 \rightarrow 2$), rồi `dbl`, `dbl` ($2 \rightarrow 4 \rightarrow 8$).
- **A sai:** Nhằm `andThen` và `compose`: `andThen` chạy hàm hiện tại **trước**.
- **B sai:** `compose(inc)` chạy `inc` trước: $1 + 1 = 2$, rồi nhân đôi hai lần → 8.
- **C sai:** `inc.compose(dbl)` nhân trước rồi cộng: 7.
- **Kiểm chứng:** `examples/questions/ch06/Q06_13/` — output confirmed (`python3 tools/book.py questions ch06`).

Câu 06-14 — Đáp án: A

- **Vì sao đúng:** `takeWhile` lấy phần tử **cho tới khi** điều kiện sai lần đầu (dừng ở 5). `dropWhile` bỏ phần tử cho tới khi điều kiện sai lần đầu, rồi giữ **tất cả** phần còn lại (5, 6), kể cả 6 là số chẵn.
- **B sai:** `takeWhile` không phải `filter`: nó dừng hẳn ở phần tử đầu tiên không thỏa.
- **C sai:** `dropWhile` giữ toàn bộ phần còn lại sau phần tử đầu tiên không thỏa, gồm cả 6.
- **D sai:** Thứ tự in là kết quả `takeWhile` trước, `dropWhile` sau.
- **Kiểm chứng:** `examples/questions/ch06/Q06_14/` — output confirmed (`python3 tools/book.py questions ch06`).

Câu 06-15 — Đáp án: A, C

- **Vì sao đúng:** A: `partitioningBy` theo “là số chẵn” → `false = [1, 3]`, `true = [2, 4]`. C: `groupingBy` với khoá `Boolean` vào `TreeMap` (thứ tự tự nhiên của `Boolean`: `false < true`) cho cùng kết quả.
- **B sai:** Điều kiện “là số lẻ” đảo ngược hai nhóm: `{false=[2, 4], true=[1, 3]}`.
- **D sai:** `Collector` phụ `counting()` đếm số phần tử: `{false=2, true=2}`.
- **E sai:** `filter` chỉ giữ số chẵn: `[2, 4]`.
- *Kiểm chứng:* `examples/questions/ch06/Q06_15/` — variants: AC satisfy output (`python3 tools/book.py questions ch06`).

Câu 06-16 — Đáp án: B

- **Vì sao đúng:** Stream đầu sắp xếp **chuỗi** (theo từng ký tự: “10” < “100” < “9”) rồi mới đổi sang số. Stream thứ hai đổi sang số trước rồi sắp xếp giảm dần theo giá trị số.
- **A sai:** `sorted()` chạy trên `String` trước `map`, nên so sánh theo chữ, không theo số.
- **C sai:** Stream thứ hai sắp xếp **số** giảm dần: 100, 10, 9.
- **D sai:** “100” < “9” theo thứ tự chuỗi vì ‘1’ < ‘9’.
- *Kiểm chứng:* `examples/questions/ch06/Q06_16/` — output confirmed (`python3 tools/book.py questions ch06`).

Câu 06-17 — Đáp án: B

- **Vì sao đúng:** Method reference **không** có dấu ngoặc `()`: phải viết `String::toUpperCase`. Các dòng khác hợp lệ: L2 dùng constructor không tham số của `String`; L5: object đầu tiên là “người nhận” (receiver), tham số thứ hai truyền vào `equals`.
- **A sai:** `String::new` khớp `Supplier<String>` qua constructor `String()`.
- **C sai:** L2 hợp lệ (xem A).
- **D sai:** `String::equals` khớp `BiFunction<String, String, Boolean>`: `(x, y) -> x.equals(y)`.
- **E sai:** L4 có dấu `()` sau tên method → lỗi cú pháp.
- *Kiểm chứng:* `examples/questions/ch06/Q06_17/` — compile error confirmed at [‘L4’] (`python3 tools/book.py questions ch06`).

Câu 06-18 — Đáp án: A, D

- **Vì sao đúng:** A: `ofNullable(null)` tạo `Optional` rỗng. D: `map` trên `Optional` rỗng trả về `Optional` rỗng mà không gọi hàm.
- **B sai:** `Optional.of(null)` ném `NullPointerException`; muốn chấp nhận `null` phải dùng `ofNullable`.
- **C sai:** Tham số của `orElse` luôn được tính trước khi gọi method (dùng `orElseGet` để trì hoãn).
- **E sai:** `get()` trên `Optional` rỗng ném `NoSuchElementException`.
- *Kiểm chứng:* `examples/questions/ch06/Q06_18/` — each option proven true/false by a program (`python3 tools/book.py questions ch06`).

Câu 06-19 — Đáp án: A

- **Vì sao đúng:** Gom nhóm theo độ dài vào `TreeMap` (khoá tăng dần). `Collector` phụ `joining("+")` nối các phần tử của từng nhóm theo thứ tự gặp.
- **B sai:** `Collector` phụ là `joining`, không phải `toList`.
- **C sai:** `TreeMap` sắp xếp khoá tăng dần: 3 trước 4.
- **D sai:** `joining` giữ thứ tự gặp: sun trước sky.
- *Kiểm chứng:* `examples/questions/ch06/Q06_19/` — output confirmed (`python3 tools/book.py questions ch06`).

Câu 06-20 — Đáp án: D

- **Vì sao đúng:** Stream rỗng: `count() = 0`; `allMatch = true` (không có phần tử nào vi phạm); `anyMatch = false` (không có phần tử nào thoả).
- **A sai:** `allMatch` trên stream rỗng là `true`.
- **B sai:** `anyMatch` cần ít nhất một phần tử thoả; stream rỗng → `false`.
- **C sai:** Các method này không ném exception với stream rỗng.
- *Kiểm chứng:* `examples/questions/ch06/Q06_20/` — output confirmed (`python3 tools/book.py questions ch06`).

Đọc thêm (link chính thức — bị chặn trong sandbox nên mình chưa mở được)

- Javadoc gói `java.util.stream`: <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/stream/package-summary.html>
- JLS §15.27 Lambda Expressions, §15.13 Method Reference Expressions: <https://docs.oracle.com/javase/specs/jls/se21/html/jls-15.html>

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-28:

- `package-info.java` của `java.util.stream` (giải thích laziness, stateless, associativity, ordering): <https://raw.githubusercontent.com/openjdk/jdk21u/master/src/java.base/share/classes/java/util/stream/package-info.java>

- `Collectors.java`:
<https://raw.githubusercontent.com/openjdk/jdk21u/master/src/java.base/share/classes/java/util/stream/Collectors.java>
- `Optional.java`: <https://raw.githubusercontent.com/openjdk/jdk21u/master/src/java.base/share/classes/java/util/Optional.java>

Chương 7 — Module, đóng gói và triển khai (Packaging and Deploying Java Code)

Mục tiêu

- 7.1 Định nghĩa module và “mở” nội dung của nó (kể cả cho reflection); khai báo phụ thuộc; định nghĩa **service**, **provider** và **consumer**.
- 7.2 Biên dịch code, tạo JAR modular và non-modular, tạo **runtime image** (jlink), và chuyển đổi (migration) sang module bằng **unnamed module** và **automatic module**.

Giải thích đơn giản

Trước Java 9, mọi lớp `public` đều dùng được từ bất cứ đâu trên classpath. **Java Platform Module System (JPMS)** thêm một lớp nữa phía trên package: **module** = một nhóm package + file mô tả `module-info.java`.

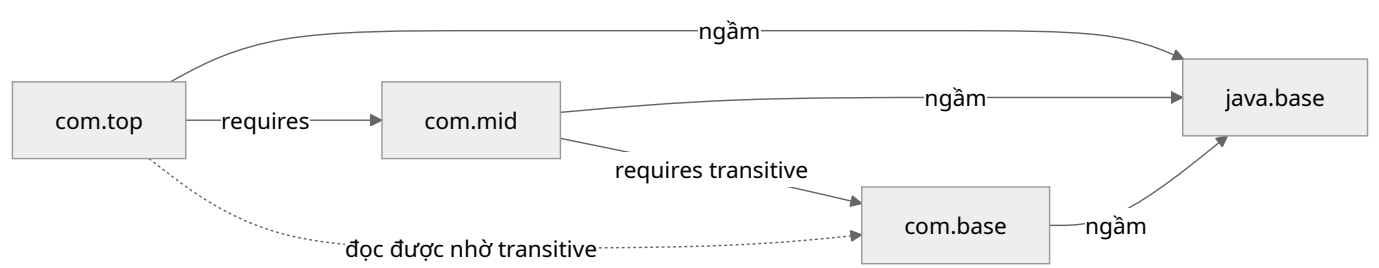
```
module com.shop {
    requires java.sql;    // tên module
                        // tôi cần module này
    exports com.shop.api; // package này cho người khác dùng
}
```

Ba lợi ích: **đóng gói mạnh** (package không `exports` thì bên ngoài không dùng được, dù lớp là `public`), **phụ thuộc rõ ràng** (thiếu module → lỗi ngay khi khởi động, không đợi `ClassNotFoundException`), và **runtime nhỏ** (jlink chỉ gói các module cần).

Ba “vùng” code:

Loại	Ở đâu	Có tên?	Độc được gì	Export gì
Named module	Module path, có <code>module-info</code>	Có	Chỉ module được <code>requires</code> (+ transitive)	Chỉ package <code>exports</code>
Automatic module	Module path, JAR không có <code>module-info</code>	Có (suy ra từ tên file hoặc MANIFEST)	Mọi module + unnamed module	Tất cả package
Unnamed module	Classpath	Không	Mọi module	Tất cả package, nhưng named module không <code>requires</code> được nó

Ví dụ về readability với `requires transitive` (ví dụ 3 bên dưới):



Ví dụ

Mỗi ví dụ là một thư mục trong `examples/ch07/` có script `run.sh`. Output bên dưới là output thật của script (JDK 21.0.10); đường dẫn tạm của máy được thay bằng `..`. Chạy lại: `python3 tools/book.py examples ch07`.

1. Module đầu tiên

Ex01_FirstModule/common.sh

```
# Helper sourced by every run.sh in ch07: prints each command, then its output with the temp path hidden.
unset JAVA_TOOL_OPTIONS
run() {
    echo "$ $*"
    "$@" 2>&1 | sed "s|$PWD|.|g"
    return 0
}
```

Ex01_FirstModule/src/com.greet/com/greet/Main.java

```
package com.greet;

public class Main {
    public static void main(String[] args) {
        Module m = Main.class.getModule();
        System.out.println("Hello from module " + m.getName());
        System.out.println("named? " + m.isNamed() + ", requires java.logging? "
            + m.getDescriptor().requires().stream().anyMatch(r -> r.name().equals("java.logging")));
    }
}
```

Ex01_FirstModule/src/com.greet/module-info.java

```
// objective: 7.1, 7.2
module com.greet {
    requires java.logging;    // tên module: thường theo tên package gốc
                             // java.base luôn được requires ngầm
}
```

Ex01_FirstModule/run.sh

```
#!/usr/bin/env bash
# objective: 7.1, 7.2
# Biên dịch và chạy một module đơn giản.
source ./common.sh
run javac -d out --module-source-path src -m com.greet
run find out -name "*.class" | sort
run java --module-path out --module com.greet/com.greet.Main
run java -p out -m com.greet/com.greet.Main
run java -p out --describe-module com.greet
```

Output thật (chạy bash run.sh, JDK 21.0.10):

```
$ javac -d out --module-source-path src -m com.greet
$ find out -name *.class
out/com.greet/com/greet/Main.class
out/com.greet/module-info.class
$ java --module-path out --module com.greet/com.greet.Main
Hello from module com.greet
named? true, requires java.logging? true
$ java -p out -m com.greet/com.greet.Main
Hello from module com.greet
named? true, requires java.logging? true
$ java -p out --describe-module com.greet
com.greet file:///out/com.greet/
requires java.base mandated
requires java.logging
contains com.greet
```

2. requires / exports

Ex02_RequiresExports/bad/com.app/com/app/App.java

```
package com.app;

import com.util.internal.Helper;    // package không được export

public class App {
    public static void main(String[] args) {
        System.out.println(Helper.clean(" x "));
    }
}
```

Ex02_RequiresExports/bad/com.app/module-info.java

```
module com.app {
    requires com.util;
}
```

Ex02_RequiresExports/common.sh

```
# Helper sourced by every run.sh in ch07: prints each command, then its output with the temp path hidden.
unset JAVA_TOOL_OPTIONS
run() {
    echo "\$ $*"
    "$@" 2>&1 | sed "s|${PWD}|.|g"
    return 0
}
```

Ex02_RequiresExports/src/com.app/com/app/App.java

```
package com.app;

import com.util.api.TextUtil;

public class App {
    public static void main(String[] args) {
        System.out.println(TextUtil.shout(" modules "));
    }
}
```

Ex02_RequiresExports/src/com.app/module-info.java

```
module com.app {
    requires com.util;
}
```

Ex02_RequiresExports/src/com.util/com/util/api/TextUtil.java

```
package com.util.api;

import com.util.internal.Helper;

public class TextUtil {
    public static String shout(String s) { return Helper.clean(s).toUpperCase() + "!"; }
}
```

Ex02_RequiresExports/src/com.util/com/util/internal/Helper.java

```
package com.util.internal;

public class Helper {
    public static String clean(String s) { return s.strip(); }
}
```

Ex02_RequiresExports/src/com.util/module-info.java

```
module com.util {
    exports com.util.api;           // chỉ package này được module khác dùng
    // com.util.internal KHÔNG export → bị "đóng gói mạnh" (strong encapsulation)
}
```

Ex02_RequiresExports/run.sh

```
#!/usr/bin/env bash
# objective: 7.1
# requires + exports; và lỗi khi dùng package KHÔNG được export.
source ./common.sh
run javac -d out --module-source-path src -m com.util,com.app
run java -p out -m com.app/com.app.App
echo "--- dùng package com.util.internal (không export):"
mkdir -p bad-src && cp -r src/com.util bad-src/ && cp -r bad/com.app bad-src/
run javac -d out2 --module-source-path bad-src -m com.util,com.app
```

Output thật (chạy bash run.sh, JDK 21.0.10):

```
$ javac -d out --module-source-path src -m com.util,com.app
$ java -p out -m com.app/com.app.App
MODULES!
--- dùng package com.util.internal (không export):
$ javac -d out2 --module-source-path bad-src -m com.util,com.app
bad-src/com.app/com/app/App.java:3: error: package com.util.internal is not visible
import com.util.internal.Helper;    // package không được export
    ^
(package com.util.internal is declared in module com.util, which does not export it)
1 error
```

3. requires transitive

Ex03_Transitive/common.sh

```
# Helper sourced by every run.sh in ch07: prints each command, then its output with the temp path hidden.
unset JAVA_TOOL_OPTIONS
run() {
    echo "\$ $*"
    "$@" 2>&1 | sed "s|${PWD}|.|g"
    return 0
}
```

Ex03_Transitive/src/com.base/com/base/Money.java

```
package com.base;

public record Money(long amount) { }
```

Ex03_Transitive/src/com.base/module-info.java

```
module com.base {
    exports com.base;
}
```

Ex03_Transitive/src/com.mid/com/mid/Wallet.java

```
package com.mid;

import com.base.Money;

public class Wallet {
    public static Money balance() { return new Money(500); } // API trả về kiểu của com.base
}
```

Ex03_Transitive/src/com.mid/module-info.java

```
module com.mid {
    requires transitive com.base; // ai requires com.mid cũng tự đọc được com.base
    exports com.mid;
}
```

Ex03_Transitive/src/com.top/com/top/Main.java

```
package com.top;

import com.base.Money;
import com.mid.Wallet;

public class Main {
    public static void main(String[] args) {
        Money m = Wallet.balance();
        System.out.println("balance = " + m.amount());
        System.out.println("com.top reads com.base? " + Main.class.getModule()
            .canRead(Money.class.getModule()));
    }
}
```

Ex03_Transitive/src/com.top/module-info.java

```
module com.top {
    requires com.mid; // KHÔNG cần requires com.base nhờ "transitive"
}
```

Ex03_Transitive/run.sh

```
#!/usr/bin/env bash
# objective: 7.1
# requires transitive (implied readability).
source ./common.sh
run javac -d out --module-source-path src -m com.top
run java -p out -m com.top/com.top.Main
run java -p out --describe-module com.mid
```

Output thật (chạy bash run.sh, JDK 21.0.10):

```
$ javac -d out --module-source-path src -m com.top
$ java -p out -m com.top/com.top.Main
balance = 500
com.top reads com.base? true
$ java -p out --describe-module com.mid
com.mid file:///out/com.mid/
exports com.mid
requires com.base transitive
requires java.base mandated
```

4. exports vs opens (reflection)

Ex04_OpensReflection/common.sh

```
# Helper sourced by every run.sh in ch07: prints each command, then its output with the temp path hidden.
unset JAVA_TOOL_OPTIONS
run() {
    echo "\$ $*"
    "$@" 2>&1 | sed "s|${PWD}|.|g"
    return 0
}
```

Ex04_OpensReflection/src/com.inspect/com/inspect/Main.java

```
package com.inspect;

import com.model.Secret;
import java.lang.reflect.Field;

public class Main {
    public static void main(String[] args) throws Exception {
        Secret s = new Secret();
        System.out.println(s.hint());
        Field f = Secret.class.getDeclaredField("code");
        try {
            f.setAccessible(true); // đòi hỏi package phải được "opens"
            System.out.println("private code = " + f.get(s));
        } catch (RuntimeException e) {
            System.out.println(e.getClass().getSimpleName());
        }
    }
}
```

Ex04_OpensReflection/src/com.inspect/module-info.java

```
module com.inspect {
    requires com.model;
}
```

Ex04_OpensReflection/src/com.model/com/model/Secret.java

```
package com.model;

public class Secret {
    private String code = "42";
    public String hint() { return "public hint"; }
}
```

Ex04_OpensReflection/src/com.model/module-info.java

```
module com.model {
    exports com.model; // compile-time + public members lúc chạy
    // "opens com.model;" sẽ cho phép reflection vào cả thành viên private
}
```

Ex04_OpensReflection/run.sh

```
#!/usr/bin/env bash
# objective: 7.1
# exports vs opens: reflection sâu (private) cần "opens".
source ./common.sh
run javac -d out --module-source-path src -m com.model,com.inspect
echo "--- chỉ exports:"
run java -p out -m com.inspect/com.inspect.Main
echo "--- thêm 'opens com.model;' vào module-info của com.model:"
sed -i 's|// "opens com.model;".*|opens com.model;|' src/com.model/module-info.java
run cat src/com.model/module-info.java
run javac -d out --module-source-path src -m com.model,com.inspect
run java -p out -m com.inspect/com.inspect.Main
```

Output thật (chạy bash run.sh, JDK 21.0.10):

```
$ javac -d out --module-source-path src -m com.model,com.inspect
--- chỉ exports:
$ java -p out -m com.inspect/com.inspect.Main
public hint
InaccessibleObjectException
--- thêm 'opens com.model;' vào module-info của com.model:
$ cat src/com.model/module-info.java
module com.model {
    exports com.model; // compile-time + public members lúc chạy
    opens com.model;
}
$ javac -d out --module-source-path src -m com.model,com.inspect
$ java -p out -m com.inspect/com.inspect.Main
```

```
public hint
private code = 42
```

5. Service: provider và consumer

Ex05_Services/common.sh

```
# Helper sourced by every run.sh in ch07: prints each command, then its output with the temp path hidden.
unset JAVA_TOOL_OPTIONS
run() {
    echo "\$ $*"
    "$@" 2>&1 | sed "s|${PWD}|.|g"
    return 0
}
```

Ex05_Services/src/com.shop.api/com/shop/api/PaymentService.java

```
package com.shop.api;

public interface PaymentService {
    String name();
    String pay(long amount);
}
```

Ex05_Services/src/com.shop.api/module-info.java

```
module com.shop.api {
    exports com.shop.api;           // service interface
}
```

Ex05_Services/src/com.shop.app/com/shop/app/Checkout.java

```
package com.shop.app;

import com.shop.api.PaymentService;
import java.util.ServiceLoader;

public class Checkout {
    public static void main(String[] args) {
        ServiceLoader<PaymentService> loader = ServiceLoader.load(PaymentService.class);
        long count = loader.stream().count();
        System.out.println("providers found: " + count);
        loader.findFirst().ifPresentOrElse(
            p -> System.out.println(p.pay(150_000)),
            () -> System.out.println("no payment provider"));
    }
}
```

Ex05_Services/src/com.shop.app/module-info.java

```
module com.shop.app {
    requires com.shop.api;
    uses com.shop.api.PaymentService; // consumer khai báo sẽ tìm service này
}
```

Ex05_Services/src/com.shop.vnpay/com/shop/vnpay/VnPay.java

```
package com.shop.vnpay;

import com.shop.api.PaymentService;

public class VnPay implements PaymentService {
    public VnPay() { } // provider cần constructor public không tham số (hoặc method static provider())
    public String name() { return "VNPay"; }
    public String pay(long amount) { return "paid " + amount + " VND via " + name(); }
}
```

Ex05_Services/src/com.shop.vnpay/module-info.java

```
module com.shop.vnpay {
    requires com.shop.api;
    provides com.shop.api.PaymentService with com.shop.vnpay.VnPay; // service provider
    // KHÔNG cần exports com.shop.vnpay: consumer không biết lớp cài đặt
}
```

Ex05_Services/run.sh

```
#!/usr/bin/env bash
# objective: 7.1
# Service: interface (api) + provider (provides ... with ...) + consumer (uses + ServiceLoader).
source ./common.sh
run javac -d out --module-source-path src -m com.shop.api,com.shop.vnpay,com.shop.app
echo "--- chạy KHÔNG có provider trên module path:"
mkdir -p only && cp -r out/com.shop.api out/com.shop.app only/
run java -p only -m com.shop.app/com.shop.app.Checkout
echo "--- chạy CÓ provider:"
run java -p out -m com.shop.app/com.shop.app.Checkout
run java -p out --describe-module com.shop.vnpay
```

Output thật (chạy bash run.sh, JDK 21.0.10):

```
$ javac -d out --module-source-path src -m com.shop.api,com.shop.vnpay,com.shop.app
--- chạy KHÔNG có provider trên module path:
$ java -p only -m com.shop.app/com.shop.app.Checkout
providers found: 0
no payment provider
--- chạy CÓ provider:
$ java -p out -m com.shop.app/com.shop.app.Checkout
providers found: 1
paid 150000 VND via VNPay
$ java -p out --describe-module com.shop.vnpay
com.shop.vnpay file:///out/com.shop.vnpay/
requires java.base mandated
requires com.shop.api
provides com.shop.api.PaymentService with com.shop.vnpay.VnPay
contains com.shop.vnpay
```

6. JAR modular và non-modular

Ex06_Jars/common.sh

```
# Helper sourced by every run.sh in ch07: prints each command, then its output with the temp path hidden.
unset JAVA_TOOL_OPTIONS
run() {
  echo "\$ $*"
  "$@" 2>&1 | sed "s|$PWD|.|g"
  return 0
}
```

Ex06_Jars/legacy/org/old/Tool.java

```
package org.old;

public class Tool {
    public static void main(String[] args) {
        Module m = Tool.class.getModule();
        System.out.println("legacy tool, named module? " + m.isNamed() + " name=" + m.getName());
    }
}
```

Ex06_Jars/src/com.calc/com/calc/Calc.java

```
package com.calc;

public class Calc {
    public static int add(int a, int b) { return a + b; }
    public static void main(String[] args) {
        System.out.println("2 + 3 = " + add(2, 3) + " (module " + Calc.class.getModule().getName() + ")");
    }
}
```

Ex06_Jars/src/com.calc/module-info.java

```
module com.calc {
    exports com.calc;
}
```

Ex06_Jars/run.sh

```
#!/usr/bin/env bash
# objective: 7.2
# Tạo JAR modular (có module-info.class) và JAR thường (non-modular), rồi chạy.
source ./common.sh
run javac -d out/com.calc src/com.calc/module-info.java src/com.calc/com/calc/Calc.java
mkdir -p mods
run jar --create --file mods/calc.jar --main-class com.calc.Calc -C out/com.calc .
run jar --describe-module --file mods/calc.jar
```

```
run java -p mods -m com.calc # có Main-Class nên không cần ghi tên lớp
echo "--- JAR non-modular chạy trên classpath:"
run javac -d legacy-out legacy/org/old/Tool.java
run jar --create --file tool.jar -C legacy-out .
run java -cp tool.jar org.old.Tool
run jar --list --file mods/calc.jar
```

Output thật (chạy bash run.sh, JDK 21.0.10):

```
$ javac -d out/com.calc src/com.calc/module-info.java src/com.calc/com/calc/Calc.java
$ jar --create --file mods/calc.jar --main-class com.calc.Calc -C out/com.calc .
$ jar --describe-module --file mods/calc.jar
com.calc jar:file:///./mods/calc.jar!/module-info.class
exports com.calc
requires java.base mandated
main-class com.calc.Calc

$ java -p mods -m com.calc
2 + 3 = 5 (module com.calc)
--- JAR non-modular chạy trên classpath:
$ javac -d legacy-out legacy/org/old/Tool.java
$ jar --create --file tool.jar -C legacy-out .
$ java -cp tool.jar org.old.Tool
legacy tool, named module? false name=null
$ jar --list --file mods/calc.jar
META-INF/
META-INF/MANIFEST.MF
module-info.class
com/
com/calc/
com/calc/Calc.class
```

7. Automatic module và unnamed module

Ex07_AutomaticModules/common.sh

```
# Helper sourced by every run.sh in ch07: prints each command, then its output with the temp path hidden.
unset JAVA_TOOL_OPTIONS
run() {
  echo "\$ $*"
  "$@" 2>&1 | sed "s|${PWD}|.|g"
  return 0
}
```

Ex07_AutomaticModules/lib/org/json/lite/Json.java

```
package org.json.lite;

// Thư viện "cũ" không có module-info.java
public class Json {
  public static String quote(String s) { return "\"" + s + "\""; }
  public static String whoAmI() {
    Module m = Json.class.getModule();
    return "Json is in module: " + (m.isNamed() ? m.getName() : "UNNAMED");
  }
}
```

Ex07_AutomaticModules/src/com.report/com/report/Report.java

```
package com.report;

import org.json.lite.Json;

public class Report {
  public static void main(String[] args) {
    System.out.println(Json.quote("report") + " | " + Json.whoAmI());
  }
}
```

Ex07_AutomaticModules/src/com.report/module-info.java

```
module com.report {
  requires json.lite; // tên automatic module suy ra từ tên file json-lite-1.2.jar
}
```

Ex07_AutomaticModules/run.sh

```
#!/usr/bin/env bash
# objective: 7.2
# Migration: JAR không có module-info đặt trên MODULE PATH → automatic module; trên CLASSPATH → unnamed module.
source ./common.sh
```

```

run javac -d lib-out lib/org/json/lite/Json.java
mkdir -p mods
run jar --create --file mods/json-lite-1.2.jar -C lib-out .
run jar --describe-module --file mods/json-lite-1.2.jar
run javac -d out -p mods --module-source-path src -m com.report
run java -p mods:out -m com.report/com.report.Report
echo "--- cùng thư viện nhưng đặt trên classpath (unnamed module), chạy code không modular:"
cat > Plain.java <<'JAVA'
public class Plain {
    public static void main(String[] args) {
        System.out.println(org.json.lite.Json.whoAmI());
    }
}
JAVA
run javac -cp mods/json-lite-1.2.jar -d plain-out Plain.java
run java -cp mods/json-lite-1.2.jar:plain-out Plain
echo "--- đặt Automatic-Module-Name trong MANIFEST để chọn tên ổn định:"
printf 'Automatic-Module-Name: org.json.lite\n' > manifest.txt
run jar --create --file mods/json-lite-1.2.jar --manifest manifest.txt -C lib-out .
run jar --describe-module --file mods/json-lite-1.2.jar

```

Output thật (chạy bash run.sh, JDK 21.0.10):

```

$ javac -d lib-out lib/org/json/lite/Json.java
$ jar --create --file mods/json-lite-1.2.jar -C lib-out .
$ jar --describe-module --file mods/json-lite-1.2.jar
No module descriptor found. Derived automatic module.

json.lite@1.2 automatic
requires java.base mandated
contains org.json.lite

$ javac -d out -p mods --module-source-path src -m com.report
$ java -p mods:out -m com.report/com.report.Report
"report" | Json is in module: json.lite
--- cùng thư viện nhưng đặt trên classpath (unnamed module), chạy code không modular:
$ javac -cp mods/json-lite-1.2.jar -d plain-out Plain.java
$ java -cp mods/json-lite-1.2.jar:plain-out Plain
Json is in module: UNNAMED
--- đặt Automatic-Module-Name trong MANIFEST để chọn tên ổn định:
$ jar --create --file mods/json-lite-1.2.jar --manifest manifest.txt -C lib-out .
$ jar --describe-module --file mods/json-lite-1.2.jar
No module descriptor found. Derived automatic module.

org.json.lite@1.2 automatic
requires java.base mandated
contains org.json.lite

```

8. jlink: runtime image

Ex08_Jlink/common.sh

```

# Helper sourced by every run.sh in ch07: prints each command, then its output with the temp path hidden.
unset JAVA_TOOL_OPTIONS
run() {
    echo "\$ $*"
    "$@" 2>&1 | sed "s|${PWD}|.|g"
    return 0
}

```

Ex08_Jlink/src/com.hello/com/hello/Hello.java

```

package com.hello;

public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello from a custom runtime image! modules in boot layer: "
            + ModuleLayer.boot().modules().size());
    }
}

```

Ex08_Jlink/src/com.hello/module-info.java

```

module com.hello {
}

```

Ex08_Jlink/run.sh

```

#!/usr/bin/env bash
# objective: 7.2
# jlink: tạo runtime image nhỏ chỉ chứa các module cần thiết.
source ./common.sh

```

```
run javac -d out --module-source-path src -m com.hello
run jlink --module-path out --add-modules com.hello --output image --strip-debug --no-header-files --no-man-pages
run image/bin/java --list-modules
run image/bin/java -m com.hello/com.hello.Hello
echo "\$ ls image"
ls image | sort
```

Output thật (chạy bash run.sh, JDK 21.0.10):

```
$ javac -d out --module-source-path src -m com.hello
$ jlink --module-path out --add-modules com.hello --output image --strip-debug --no-header-files --no-man-pages
$ image/bin/java --list-modules
com.hello
java.base@21.0.10
$ image/bin/java -m com.hello/com.hello.Hello
Hello from a custom runtime image! modules in boot layer: 2
$ ls image
bin
conf
legal
lib
release
```

9. jdeps

Chú ý dòng cuối: `--print-module-deps` chỉ in `java.base,java.sql` vì `java.sql` đã requires transitive `java.logging`.

Ex09_Jdeps/app/com/legacy/Db.java

```
package com.legacy;

import java.sql.DriverManager;
import java.util.logging.Logger;

public class Db {
    static final Logger LOG = Logger.getLogger("db");
    public static void main(String[] args) {
        int timeout = DriverManager.getLoginTimeout(); // dùng một lớp của java.sql
        LOG.fine("timeout " + timeout);
        System.out.println("needs java.sql and java.logging");
    }
}
```

Ex09_Jdeps/common.sh

```
# Helper sourced by every run.sh in ch07: prints each command, then its output with the temp path hidden.
unset JAVA_TOOL_OPTIONS
run() {
    echo "\$ $*"
    "$@" 2>&1 | sed "s|$PWD|.|g"
    return 0
}
```

Ex09_Jdeps/run.sh

```
#!/usr/bin/env bash
# objective: 7.2
# jdeps: xem JAR phụ thuộc module nào của JDK (bước đầu khi chuyển sang module).
source ./common.sh
run javac -d app-out app/com/legacy/Db.java
run jar --create --file legacy.jar -C app-out .
run jdeps -summary legacy.jar
run jdeps --list-deps legacy.jar
run jdeps --print-module-deps legacy.jar
```

Output thật (chạy bash run.sh, JDK 21.0.10):

```
$ javac -d app-out app/com/legacy/Db.java
$ jar --create --file legacy.jar -C app-out .
$ jdeps -summary legacy.jar
legacy.jar -> java.base
legacy.jar -> java.logging
legacy.jar -> java.sql
$ jdeps --list-deps legacy.jar
java.base
java.logging
java.sql
$ jdeps --print-module-deps legacy.jar
java.base,java.sql
```

10. Module của JDK

Ex10_JdkModules/common.sh

```
# Helper sourced by every run.sh in ch07: prints each command, then its output with the temp path hidden.
unset JAVA_TOOL_OPTIONS
run() {
    echo "\$ $*"
    "$@" 2>&1 | sed "s|$PWD|.|g"
    return 0
}
```

Ex10_JdkModules/run.sh

```
#!/usr/bin/env bash
# objective: 7.1, 7.2
# Các module của JDK: java.base là gốc; mọi module đều requires java.base ngầm.
source ./common.sh
echo "\$ java --list-modules | grep -E '^java\.(base|sql|logging|se)@"
java --list-modules | grep -E '^java\.(base|sql|logging|se)@"
run java --describe-module java.sql
echo "\$ java --describe-module java.base | head -5"
java --describe-module java.base | head -5
```

Output thật (chạy bash run.sh, JDK 21.0.10):

```
$ java --list-modules | grep -E '^java\.(base|sql|logging|se)@"
java.base@21.0.10
java.logging@21.0.10
java.se@21.0.10
java.sql@21.0.10
$ java --describe-module java.sql
java.sql@21.0.10
exports java.sql
exports javax.sql
requires java.base mandated
requires java.transaction.xa transitive
requires java.logging transitive
requires java.xml transitive
uses java.sql.Driver
$ java --describe-module java.base | head -5
java.base@21.0.10
exports java.io
exports java.lang
exports java.lang.annotation
exports java.lang.constant
```

11. Lỗi module hay gặp

Ex11_ModuleErrors/common.sh

```
# Helper sourced by every run.sh in ch07: prints each command, then its output with the temp path hidden.
unset JAVA_TOOL_OPTIONS
run() {
    echo "\$ $*"
    "$@" 2>&1 | sed "s|$PWD|.|g"
    return 0
}
```

Ex11_ModuleErrors/cycle/mod.a/a/A.java

```
package a;
public class A { }
```

Ex11_ModuleErrors/cycle/mod.a/module-info.java

```
module mod.a { requires mod.b; exports a; }
```

Ex11_ModuleErrors/cycle/mod.b/b/B.java

```
package b;
public class B { }
```

Ex11_ModuleErrors/cycle/mod.b/module-info.java

```
module mod.b { requires mod.a; exports b; }
```

Ex11_ModuleErrors/missing/mod.c/c/C.java

```
package c;
public class C { }
```

Ex11_ModuleErrors/missing/mod.c/module-info.java

```
module mod.c { requires does.not.exist; }
```

Ex11_ModuleErrors/run.sh

```
#!/usr/bin/env bash
# objective: 7.1
# Lỗi hay gặp: vòng phụ thuộc (cycle) giữa các module; requires module không tồn tại; chạy module không có trên path.
source ./common.sh
run javac -d out1 --module-source-path cycle -m mod.a,mod.b
run javac -d out2 --module-source-path missing -m mod.c
run java -p out1 -m mod.a/a.A
```

Output thật (chạy bash run.sh, JDK 21.0.10):

```
$ javac -d out1 --module-source-path cycle -m mod.a,mod.b
cycle/mod.b/module-info.java:1: error: cyclic dependence involving mod.a
module mod.b { requires mod.a; exports b; }
                  ^
cycle/mod.a/module-info.java:1: error: cyclic dependence involving mod.b
module mod.a { requires mod.b; exports a; }
                  ^
2 errors
$ javac -d out2 --module-source-path missing -m mod.c
missing/mod.c/module-info.java:1: error: module not found: does.not.exist
module mod.c { requires does.not.exist; }
                  ^
1 error
$ java -p out1 -m mod.a/a.A
Error occurred during initialization of boot layer
java.lang.module.FindException: Module mod.a not found
```

Đi sâu

Các directive trong module-info.java

Directive	Ý nghĩa
requires M;	Module này đọc (reads) M — lúc biên dịch và lúc chạy
requires transitive M;	Như trên, và ai requires module này cũng đọc M (implied readability)
requires static M;	Chỉ cần M lúc biên dịch; lúc chạy là tùy chọn
exports P;	Package P dùng được (kiểu public) lúc biên dịch và lúc chạy
exports P to M1, M2;	Qualified export: chỉ M1, M2 dùng được
opens P; / opens P to M;	Cho phép deep reflection (cả private) vào P lúc chạy; không mở lúc biên dịch
open module X { ... }	Mở mọi package cho reflection; bên trong không được viết opens
uses S;	Module này tiêu thụ (consume) service S qua <code>ServiceLoader</code>
provides S with Impl1, Impl2;	Module này cung cấp cài đặt cho service S

Quy tắc: java.base luôn được requires ngầm (mandated). Không có vòng requires. Một package chỉ thuộc một module (không split package). exports package rỗng/không tồn tại là lỗi. module-info.java nằm ở gốc thư mục module, không khai báo package.

Service (4 thành phần)

1. **Service interface** — trong module “api”, được exports.
2. **Service provider** — lớp cài đặt, module của nó requires api và provides Service with Impl;. Lớp cài đặt không cần export; cần constructor public không tham số hoặc method public static provider().
3. **Service locator / consumer** — module requires api và uses Service;; gọi `ServiceLoader.load(Service.class)`. Thiếu uses trong named module → `ServiceConfigurationError` lúc chạy.
4. **ServiceLoader** — `iterator()`, `stream()` (trả về Provider → `.get()`), `findFirst()` (trả về Optional). Không có provider → rỗng, không lỗi.

Thêm provider mới chỉ cần đặt module provider lên module path — consumer không phải biên dịch lại.

Lệnh cần thuộc

Việc	Lệnh
Biên dịch một module	<code>javac -d out/com.x src/com.x/module-info.java</code> <code>src/com.x/com/x/*.java</code>
Biên dịch nhiều module	<code>javac -d out --module-source-path src -m com.a,com.b</code>
Biên dịch với module đã có	thêm <code>-p mods</code> (<code>--module-path</code>)
Chạy	<code>java -p out -m com.x/com.x.Main</code> (<code>--module-path</code> , <code>--module</code>)
Tạo JAR modular	<code>jar --create --file mods/x.jar [--main-class com.x.Main] -C</code> <code>out/com.x .</code> (<code>-cvf</code> dạng ngắn)
Chạy JAR có Main-Class	<code>java -p mods -m com.x</code>
Xem mô tả module	<code>java -p mods --describe-module com.x (-d)</code> , <code>jar --describe-module</code> <code>--file x.jar (-d)</code>
Liệt kê module JDK	<code>java --list-modules</code>
Xem phụ thuộc	<code>jdeps -summary x.jar (-s)</code> , <code>jdeps --list-deps</code> , <code>jdeps --print-</code> <code>module-deps</code> , <code>jdeps --jdk-internals</code>
Tạo runtime image	<code>jlink --module-path out --add-modules com.x --output image</code>
Chẩn đoán lúc chạy	<code>java --show-module-resolution</code> , <code>java -p mods --validate-modules</code>

Migration (chuyển đổi sang module)

- **Bottom-up**: chuyển các thư viện “lá” (không phụ thuộc ai) thành named module trước, rồi đi dần lên. Code chưa chuyển vẫn chạy trên classpath (unnamed module đọc được mọi module).
- **Top-down**: chuyển ứng dụng thành named module trước; các thư viện chưa có `module-info` được đặt lên module path thành **automatic module** để ứng dụng `requires` được.
- Tên automatic module: `Automatic-Module-Name` trong MANIFEST nếu có; nếu không, suy ra từ tên file (`my-utils-2.0.1.jar` → `my.utils`, version 2.0.1).
- Named module **không** `requires` được unnamed module. Automatic module đọc được unnamed module (cầu nối).
- JAR có `module-info.class` nhưng đặt trên **classpath** → `module-info` bị bỏ qua, lớp thuộc unnamed module.

Lỗi và bẫy thường gặp (Exam traps)

1. `public` không đủ: package phải được `exports`.
2. `exports` ≠ `opens`: reflection vào private cần `opens` (hoặc `open module`).
3. `requires` không truyền tiếp; cần `requires transitive`.
4. `provides Impl` with Interface (đảo thứ tự) → lỗi. Đúng: `provides Interface` with `Impl`.
5. Consumer thiếu `uses` → lỗi lúc **chạy** (`ServiceConfigurationError`), không phải lúc biên dịch.
6. Không có provider → `ServiceLoader` rỗng, không lỗi.
7. Vòng `requires` → lỗi biên dịch “cyclic dependence”.
8. `open module` + `opens` bên trong → lỗi.
9. `-m` của `java` nhận module/lớp; thư mục module đưa bằng `-p`, không phải `-cp`.
10. `javac -m` cần `--module-source-path`.
11. `jlink` cần `--add-modules`; image gồm cả phụ thuộc bắc cầu (qua `requires transitive`).
12. Không có directive `imports`, `requires public`, `exports ... with`.

Góc nhìn từ TypeScript

TypeScript / npm	Java module	Ghi chú
<code>package.json</code> → <code>dependencies</code>	<code>requires</code>	Nhưng Java không tải thư viện; chỉ khai báo phụ thuộc giữa module có sẵn
<code>package.json</code> → <code>exports</code> (điểm vào công khai)	<code>exports</code>	Rất giống: chặn import “sâu” vào file nội bộ
Import file nội bộ <code>lib/dist/internal/x</code> (khi không có <code>exports</code>)	Không thể: package không export → lỗi biên dịch	Java chặt hơn
<code>peerDependencies</code>	Gần với <code>requires transitive</code> (cho người dùng thấy kiểu của phụ thuộc)	Chỉ tương đồng ý tưởng
Dependency injection / plugin của Angular (<code>InjectionToken</code> , <code>multi: true</code>)	<code>ServiceLoader</code> + <code>uses</code> / <code>provides</code>	Nhiều provider cho một token
Bundle tree-shaking (esbuild, webpack)	<code>jlink</code> (chỉ giữ module cần thiết)	Ở mức module, không mức hàm

Tóm tắt

- `module-info.java`: `requires` (transitive/static), `exports` (to), `opens` (to), `uses`, `provides ... with`.
- Named / automatic / unnamed: ai đọc ai, ai export gì.
- Service = interface + provider + consumer + `ServiceLoader`.
- Lệnh: `javac --module-source-path -m, java -p -m, jar --create --main-class, jar/java --describe-module, jdeps, jlink --add-modules --output`.
- Migration: bottom-up (thư viện trước), top-down (ứng dụng trước + automatic module).

Bài tập (có lời giải)

Câu hỏi của chương này dùng nhiều file, nên script kiểm tra tạo đúng cấu trúc thư mục, chạy `javac/java/jar/jlink` thật và so sánh output (code: `examples/questions/ch07/`).

Câu hỏi

Câu 07-01 · Dễ · objective 7.1

Hai module nằm trong thư mục `src` như dưới đây. Chạy lệnh `javac -d out --module-source-path src -m com.lib,com.app` thì điều gì xảy ra?

`src/com.lib/module-info.java`

```
module com.lib {
    exports com.lib.api;
}
```

`src/com.lib/com/lib/api/Api.java`

```
package com.lib.api;
public class Api { public static String hi() { return "hi"; } }
```

`src/com.lib/com/lib/impl/Impl.java`

```
package com.lib.impl;
public class Impl { public static String secret() { return "secret"; } }
```

`src/com.app/module-info.java`

```
module com.app {
    requires com.lib;
}
```

`src/com.app/com/app/Main.java`

```
package com.app;
import com.lib.impl.Impl;
public class Main {
    public static void main(String[] args) { System.out.println(Impl.secret()); }
}
```

- A. Biên dịch thành công; chạy in ra `secret`
- B. Lỗi biên dịch: package `com.lib.impl` không nhìn thấy được (not visible)
- C. Biên dịch thành công nhưng ném `IllegalAccessError` lúc chạy
- D. Biên dịch thành công nhưng ném `ClassNotFoundException` lúc chạy

Câu 07-02 · Vừa · objective 7.1

Module `com.top` dùng kiểu `com.base.Money` do method của `com.mid` trả về, nhưng **không** `requires com.base`. Điền directive nào vào chỗ `// INSERT CODE HERE` trong `module-info.java` của `com.mid` thì cả ba module biên dịch được?

`src/com.base/module-info.java`

```
module com.base { exports com.base; }
```

`src/com.base/com/base/Money.java`

```
package com.base;
public record Money(long amount) { }
```

src/com.mid/module-info.java

```
module com.mid {  
    // INSERT CODE HERE  
    exports com.mid;  
}
```

src/com.mid/com/mid/Wallet.java

```
package com.mid;  
public class Wallet { public static com.base.Money balance() { return new com.base.Money(5); } }
```

src/com.top/module-info.java

```
module com.top { requires com.mid; }
```

src/com.top/com/top/Main.java

```
package com.top;  
import com.base.Money;  
public class Main {  
    public static void main(String[] args) { Money m = com.mid.Wallet.balance(); System.out.println(m); }  
}
```

- A. requires com.base;
- B. requires transitive com.base;
- C. requires static com.base;
- D. exports com.base;

Câu 07-03 · Vừa · objective 7.1

Hai phát biểu nào đúng về module? (Chọn 2 đáp án.)

- A. Mọi module đều ngầm requires java.base.
- B. Hai module khác nhau có thể cùng chứa một package trên module path.
- C. exports một package cho phép reflection truy cập cả thành viên private của nó.
- D. Tên module có thể chứa dấu chấm, ví dụ com.shop.api.
- E. module-info.java có thể khai báo package ở dòng đầu như lớp bình thường.

Câu 07-04 · Vừa · objective 7.1

Cho hai module dưới đây. Sau khi biên dịch, chạy `java -p out -m m.user/m.user.Main` thì in ra gì?

src/m.model/module-info.java

```
module m.model {  
    exports m.model;  
    opens m.model;  
}
```

src/m.model/m/model/Config.java

```
package m.model;  
public class Config { private String env = "prod"; }
```

src/m.user/module-info.java

```
module m.user { requires m.model; }
```

src/m.user/m/user/Main.java

```
package m.user;  
import java.lang.reflect.Field;  
public class Main {  
    public static void main(String[] args) throws Exception {  
        Field f = m.model.Config.class.getDeclaredField("env");  
        f.setAccessible(true);  
        System.out.println("env=" + f.get(new m.model.Config()));  
    }  
}
```

- A. env=null
- B. Ném InaccessibleObjectException

- C. `env=prod`
- D. Lỗi biên dịch: không được vừa `exports` vừa `opens` cùng một package

Câu 07-05 · Khó · objective 7.1

Điền directive nào vào `module-info.java` của module provider (chỗ `// INSERT CODE HERE`) để `ServiceLoader` trong module `com.app` tìm được đúng một provider?

`src/com.api/module-info.java`

```
module com.api { exports com.api; }
```

`src/com.api/com/api/Greeter.java`

```
package com.api;
public interface Greeter { String greet(); }
```

`src/com.impl/module-info.java`

```
module com.impl {
    requires com.api;
    // INSERT CODE HERE
}
```

`src/com.impl/com/impl/Hi.java`

```
package com.impl;
public class Hi implements com.api.Greeter { public String greet() { return "hi"; } }
```

`src/com.app/module-info.java`

```
module com.app {
    requires com.api;
    uses com.api.Greeter;
}
```

`src/com.app/com/app/Main.java`

```
package com.app;
import java.util.ServiceLoader;
public class Main {
    public static void main(String[] args) {
        System.out.println("providers=" + ServiceLoader.load(com.api.Greeter.class).stream().count());
    }
}
```

- A. provides `com.api.Greeter` with `com.impl.Hi`;
- B. exports `com.impl`;
- C. uses `com.api.Greeter`;
- D. provides `com.impl.Hi` with `com.api.Greeter`;

Câu 07-06 · Vừa · objective 7.1

Module `com.app` dưới đây gọi `ServiceLoader.load(...)` nhưng quên khai báo `uses`. Điều gì xảy ra khi chạy `java -p out -m com.app/com.app.Main`?

`src/com.api/module-info.java`

```
module com.api { exports com.api; }
```

`src/com.api/com/api/Greeter.java`

```
package com.api;
public interface Greeter { String greet(); }
```

`src/com.app/module-info.java`

```
module com.app {
    requires com.api;
}
```

`src/com.app/com/app/Main.java`

```
package com.app;
import java.util.ServiceLoader;
public class Main {
    public static void main(String[] args) {
        System.out.println("count=" + ServiceLoader.load(com.api.Greeter.class).stream().count());
    }
}
```

- A. In ra count=0
- B. Lỗi biên dịch: thiếu uses
- C. Ném java.util.ServiceConfigurationError lúc chạy
- D. In ra count=1

Câu 07-07 · Dễ · objective 7.2

File my-utils-2.0.1.jar **không** có module-info.class và không có Automatic-Module-Name trong MANIFEST. Khi đặt nó trên module path, tên automatic module là gì?

lib/org/util/Tool.java

```
package org.util;
public class Tool { }
```

- A. my-utils
- B. my.utils
- C. my.utils.2.0.1
- D. myutils

Câu 07-08 · Vừa · objective 7.2

Lớp Probe (không thuộc module nào) được biên dịch và chạy trên **classpath** bằng `java -cp out Probe`. Chương trình in ra gì?

Probe.java

```
public class Probe {
    public static void main(String[] args) {
        Module m = Probe.class.getModule();
        System.out.println(m.isNamed() + " " + m.getName() + " "
            + m.canRead(java.sql.Connection.class.getModule()));
    }
}
```

- A. true Probe true
- B. false null false
- C. false unnamed true
- D. false null true

Câu 07-09 · Vừa · objective 7.2

Module com.x (lớp chính com.x.Main, in ra hi) đã được biên dịch vào thư mục mods. Lệnh nào chạy được chương trình? (Chọn 2 đáp án.)

- A. `java -p mods -m com.x/com.x.Main`
- B. `java --module-path mods --module com.x/com.x.Main`
- C. `java -cp mods -m com.x/com.x.Main`
- D. `java -p mods com.x.Main`
- E. `java -m mods/com.x/com.x.Main`

Câu 07-10 · Khó · objective 7.2

Module com.app chỉ có requires java.sql;. Sau khi chạy `jlink --module-path out --add-modules com.app --output image`, lệnh `image/bin/java --list-modules` liệt kê những module nào?

src/com.app/module-info.java

```
module com.app { requires java.sql; }
```

src/com.app/com/app/Main.java

```
package com.app;
public class Main { public static void main(String[] args) { } }
```

- A. Chỉ com.app

- B. com.app và java.base
- C. com.app, java.base, java.logging, java.sql, java.transaction.xa, java.xml
- D. Toàn bộ module của JDK

Câu 07-11 · Vừa · objective 7.1

Chèn dòng nào vào chỗ `// INSERT CODE HERE` trong `module-info.java` thì làm module **không** biên dịch được? (Chọn 2 đáp án.)

`src/com.shop/module-info.java`

```
module com.shop {
    // INSERT CODE HERE
}
```

`src/com.shop/com/shop/api/Cart.java`

```
package com.shop.api;
public class Cart { }
```

- A. requires java.sql;
- B. exports com.shop.api to java.sql;
- C. imports com.shop.api;
- D. opens com.shop.api;
- E. requires public java.sql;

Câu 07-12 · Khó · objective 7.1

Khai báo module nào (thay cho `// INSERT CODE HERE`) làm cho việc biên dịch **thất bại**? (Chọn 2 đáp án.)

`src/com.m/module-info.java`

```
// INSERT CODE HERE
```

`src/com.m/com/m/p/Data.java`

```
package com.m.p;
public class Data { }
```

- A. open module com.m { exports com.m.p; }
- B. open module com.m { opens com.m.p; }
- C. module com.m { opens com.m.p; exports com.m.p; }
- D. module com.m { opens com.m.p to java.base; }
- E. module com.m { exports com.m.missing; }

Câu 07-13 · Khó · objective 7.2

Hai phát biểu nào đúng về unnamed module và automatic module? (Chọn 2 đáp án.)

- A. Code trên classpath (unnamed module) dùng được các package được export của named module.
- B. Named module có thể requires code nằm trên classpath.
- C. Automatic module export **tất cả** package của nó.
- D. Một JAR có `module-info.class` đặt trên **classpath** vẫn chạy như named module.
- E. Tên automatic module luôn giống hệt tên file JAR.

Câu 07-14 · Dễ · objective 7.2

`legacy.jar` chứa một lớp dùng `java.sql.DriverManager` và `java.util.logging.Logger`. Lệnh `jdeps --print-module-deps legacy.jar` in ra gì?

`app/com/legacy/Db.java`

```
package com.legacy;
public class Db {
    public static void main(String[] args) {
        java.util.logging.Logger.getLogger("db").info("t=" + java.sql.DriverManager.getLoginTimeout());
    }
}
```

- A. java.base, java.sql
- B. java.sql, java.logging
- C. java.base, java.logging, java.sql

- D. legacy.jar -> java.sql

Câu 07-15 · Vừa · objective 7.1

Module `com.core` có exports `com.core.api` to `com.friend`; Hai module `com.friend` và `com.other` đều requires `com.core` và dùng `com.core.api`. Kết quả khi biên dịch từng module là gì?

`src/com.core/module-info.java`

```
module com.core {
    exports com.core.api to com.friend;
}
```

`src/com.core/com/core/api/Api.java`

```
package com.core.api;
public class Api { }
```

`src/com.friend/module-info.java`

```
module com.friend { requires com.core; }
```

`src/com.friend/com/friend/F.java`

```
package com.friend;
public class F { com.core.api.Api api; }
```

`src/com.other/module-info.java`

```
module com.other { requires com.core; }
```

`src/com.other/com/other/O.java`

```
package com.other;
public class O { com.core.api.Api api; }
```

- A. Cả hai biên dịch được
- B. `com.friend` lỗi, `com.other` được
- C. Cả hai đều lỗi
- D. `com.friend` được, `com.other` lỗi

Câu 07-16 · Vừa · objective 7.2

Mã nguồn module `com.x` nằm ở `src/com.x/` (gồm `module-info.java` và `com.x/Main.java`, in ra hi). Lệnh biên dịch nào (thay cho <LỆNH>) cho phép sau đó chạy `java -p out -m com.x/com.x.Main`? (Chọn 2 đáp án.)

- A. `javac -d out --module-source-path src -m com.x`
- B. `javac -d out --module-path src -m com.x`
- C. `javac -d out --module-source-path src com.x`
- D. `javac --module-source-path src -d out $(find src -name '*.java')`
- E. `javac -cp src -d out -m com.x`

Câu 07-17 · Vừa · objective 7.1

`mod.a` có requires `mod.b`; và `mod.b` có requires `mod.a`; Chạy `javac -d out --module-source-path src -m mod.a,mod.b` thì sao?

`src/mod.a/module-info.java`

```
module mod.a { requires mod.b; exports a; }
```

`src/mod.a/a/A.java`

```
package a;
public class A { }
```

`src/mod.b/module-info.java`

```
module mod.b { requires mod.a; exports b; }
```

`src/mod.b/b/B.java`

```
package b;
public class B { }
```

- A. Biên dịch thành công
- B. Biên dịch thành công nhưng chạy báo `FindException`
- C. Chỉ biên dịch được khi thêm `--add-reads`
- D. Lỗi biên dịch: phụ thuộc vòng (cyclic dependence)

Câu 07-18 · Vừa · objective 7.1

Module `com.q` chỉ có `requires java.sql`; nhưng code của nó dùng `java.util.logging.Logger`. Điều gì xảy ra?

`src/com.q/module-info.java`

```
module com.q {
    requires java.sql;
}
```

`src/com.q/com/q/Main.java`

```
package com.q;
import java.util.logging.Logger;
public class Main {
    public static void main(String[] args) {
        System.out.println(Logger.getLogger("q").getName());
    }
}
```

- A. Biên dịch và chạy được, in ra `q`
- B. Lỗi biên dịch: package `java.util.logging` không nhìn thấy được
- C. Biên dịch được nhưng lỗi lúc chạy
- D. Chỉ biên dịch được khi thêm `--add-modules java.logging`

Câu 07-19 · Dễ · objective 7.1

Consumer `com.app` có `uses com.api.Greeter`; nhưng khi chạy chỉ có `com.api` và `com.app` trên module path (không có provider nào). Chương trình in ra gì?

`src/com.api/module-info.java`

```
module com.api { exports com.api; }
```

`src/com.api/com/api/Greeter.java`

```
package com.api;
public interface Greeter { String greet(); }
```

`src/com.app/module-info.java`

```
module com.app {
    requires com.api;
    uses com.api.Greeter;
}
```

`src/com.app/com/app/Main.java`

```
package com.app;
import java.util.ServiceLoader;
public class Main {
    public static void main(String[] args) {
        var first = ServiceLoader.load(com.api.Greeter.class).findFirst();
        System.out.println(first.isPresent() ? first.get().greet() : "no provider");
    }
}
```

- A. Ném `ServiceConfigurationError`
- B. `no provider`
- C. Lỗi biên dịch vì không có module nào `provides Greeter`
- D. Ném `NullPointerException`

Câu 07-20 · Vừa · objective 7.2

Module `com.app` (lớp `com.app.Main` in ra hi) đã được biên dịch vào `out`. Lệnh nào (thay cho `<LỆNH>`) tạo được runtime image `image` sao cho `image/bin/java -m com.app/com.app.Main` chạy được? (Chọn 2 đáp án.)

- A. `jlink --module-path out --add-modules com.app --output image`
- B. `jlink -p out --add-modules com.app --output image`
- C. `jlink --module-path out --output image`
- D. `jlink -p out -m com.app --output image`
- E. `jar --create --file image -p out com.app`

Lời giải

Câu 07-01 — Đáp án: B

- **Vì sao đúng:** Module chỉ cho module khác dùng các package được `exports`. `com.lib.impl` không được export nên `com.app` không thể import nó, dù lớp `Impl` là `public` — đây là **đóng gói mạnh (strong encapsulation)**. Lỗi xảy ra ngay lúc biên dịch.
- **A sai:** `public` không còn đủ: package phải được `exports`.
- **C sai:** Hệ thống module kiểm tra ngay lúc biên dịch, không đợi tới lúc chạy.
- **D sai:** Code không biên dịch được nên không có bước chạy.
- *Kiểm chứng:* `examples/questions/ch07/Q07_01/` — script output confirmed (python3 `tools/book.py questions ch07`).

Câu 07-02 — Đáp án: B

- **Vì sao đúng:** `requires transitive com.base` tạo **implied readability**: module nào đọc `com.mid` cũng tự động đọc `com.base`. Vì vậy `com.top` dùng được `Money` mà không cần tự `requires com.base`.
- **A sai:** `requires` thường chỉ cho `com.mid` đọc `com.base`; `com.top` vẫn không đọc được → lỗi ở `com.top`.
- **C sai:** `requires static` là phụ thuộc chỉ lúc biên dịch của riêng `com.mid`, không truyền sang `com.top`.
- **D sai:** `exports` nhận tên **package** thuộc chính module; `com.mid` không chứa package `com.base` → lỗi.
- *Kiểm chứng:* `examples/questions/ch07/Q07_02/` — script variants: B in ra 'OK' (python3 `tools/book.py questions ch07`).

Câu 07-03 — Đáp án: A, D

- **Vì sao đúng:** A: `java.base` được `requires` ngầm (hiện là `requires java.base mandated`). D: tên module theo quy tắc giống tên package, dấu chấm là bình thường (thường dùng tên “reverse DNS”).
- **B sai:** Một package chỉ được thuộc **một** module (cấm “split package”): ở đây `javac` chưa bắt lỗi (package không được export), nhưng JVM từ chối khởi động với `LayerInstantiationException`.
- **C sai:** `exports` chỉ mở các kiểu `public` lúc biên dịch/chạy; reflection vào `private` cần `opens` → `InaccessibleObjectException`.
- **E sai:** `module-info.java` không thuộc package nào; khai báo package là lỗi biên dịch.
- *Kiểm chứng:* `examples/questions/ch07/Q07_03/` — each option proven true/false by a program (python3 `tools/book.py questions ch07`).

Câu 07-04 — Đáp án: C

- **Vì sao đúng:** `opens m.model` cho phép **deep reflection** (kể cả `private`) lúc chạy, nên `setAccessible(true)` thành công và đọc được giá trị `prod`. Một package có thể vừa `exports` (dùng lúc biên dịch) vừa `opens` (reflection).
- **A sai:** `Field` được khởi tạo “`prod`” khi tạo object; reflection đọc đúng giá trị đó.
- **B sai:** Exception này chỉ xảy ra khi package **không** được `opens`.
- **D sai:** `exports` và `opens` cùng một package là hợp lệ.
- *Kiểm chứng:* `examples/questions/ch07/Q07_04/` — script output confirmed (python3 `tools/book.py questions ch07`).

Câu 07-05 — Đáp án: A

- **Vì sao đúng:** Provider đăng ký cài đặt bằng `provides <service interface> with <lớp cài đặt>;`. Consumer (`com.app`) đã có `uses com.api.Greeter;`, nên `ServiceLoader` tìm thấy `Hi`. Lớp cài đặt **không** cần được export.
- **B sai:** `exports` chỉ làm package nhìn thấy được; không đăng ký service nào → `providers=0`.
- **C sai:** `uses` là việc của consumer; provider không đăng ký gì → `providers=0`.
- **D sai:** Đảo ngược thứ tự: phải là `provides Interface with Implementation` → lỗi biên dịch.
- *Kiểm chứng:* `examples/questions/ch07/Q07_05/` — script variants: A in ra ‘providers=1’ (python3 `tools/book.py questions ch07`).

Câu 07-06 — Đáp án: C

- **Vì sao đúng:** Code trong một **named module** chỉ được tải service mà module đó đã khai báo bằng `uses`. Compiler không kiểm tra điều này (lời gọi `ServiceLoader.load` là code bình thường), nên lỗi xảy ra lúc chạy: `ServiceConfigurationError` (“module `com.app` does not declare `uses`”).
- **A sai:** Không có `uses` thì `ServiceLoader` không trả về rỗng mà ném lỗi.
- **B sai:** `javac` không bắt lỗi này; nó chỉ xảy ra lúc chạy.

- **D sai:** Không có provider nào, và thiếu `uses` còn gây lỗi.
- *Kiểm chứng:* `examples/questions/ch07/Q07_06/` — script output confirmed (`python3 tools/book.py questions ch07`).

Câu 07-07 — Đáp án: B

- **Vì sao đúng:** Tên automatic module suy ra từ tên file: bỏ `.jar`, bỏ phần phiên bản (`-2.0.1`), rồi thay ký tự không phải chữ/số (như `-`) bằng dấu chấm → `my.utils` (phiên bản 2.0.1). Nếu MANIFEST có `Automatic-Module-Name` thì dùng tên đó.
- **A sai:** Dấu `-` không hợp lệ trong tên module, nên được thay bằng `.`
- **C sai:** Phần phiên bản ở cuối tên file bị tách ra thành version, không nằm trong tên.
- **D sai:** Ký tự không phải chữ/số được thay bằng dấu chấm, không bị xoá.
- *Kiểm chứng:* `examples/questions/ch07/Q07_07/` — script output confirmed (`python3 tools/book.py questions ch07`).

Câu 07-08 — Đáp án: D

- **Vì sao đúng:** Code trên classpath thuộc **unnamed module**: `isNamed()` là `false`, `getName()` là `null`. Unnamed module đọc được (reads) mọi module trong boot layer, nên đọc được `java.sql`.
- **A sai:** Classpath không tạo named module.
- **B sai:** Unnamed module đọc được mọi module, kể cả `java.sql`.
- **C sai:** Tên của unnamed module là `null`, không phải chuỗi `"unnamed"`.
- *Kiểm chứng:* `examples/questions/ch07/Q07_08/` — script output confirmed (`python3 tools/book.py questions ch07`).

Câu 07-09 — Đáp án: A, B

- **Vì sao đúng:** `-p` là dạng ngắn của `--module-path`, `-m` là dạng ngắn của `--module`; tham số của `-m` là `tênModule/tênLớp`.
- **C sai:** `-cp` đặt mods vào classpath; module `com.x` không được tìm trên module path → lỗi khởi động.
- **D sai:** Thiếu `-m: com.x.Main` được tìm trên classpath (không có) → không chạy được.
- **E sai:** Thư mục module phải đưa bằng `-p`; `-m` chỉ nhận `module/lớp`.
- *Kiểm chứng:* `examples/questions/ch07/Q07_09/` — script variants: AB in ra 'hi' (`python3 tools/book.py questions ch07`).

Câu 07-10 — Đáp án: C

- **Vì sao đúng:** Jlink thêm module gốc và **toàn bộ phụ thuộc bắc cầu** của nó: `java.base` (luôn có), `java.sql`, và các module mà `java.sql` requires transitive: `java.logging`, `java.transaction.xa`, `java.xml`. Không thêm gì khác.
- **A sai:** Runtime image luôn cần `java.base` và các module được requires.
- **B sai:** Thiếu `java.sql` và các phụ thuộc của nó.
- **D sai:** Mục đích của Jlink là tạo image **nhỏ**, chỉ gồm module cần thiết.
- *Kiểm chứng:* `examples/questions/ch07/Q07_10/` — script output confirmed (`python3 tools/book.py questions ch07`).

Câu 07-11 — Đáp án: C, E

- **Vì sao đúng:** Các directive hợp lệ: `requires` (kèm `transitive/static`), `exports` (có thể `to` module cụ thể), `opens` (có thể `to`), `uses`, `provides ... with`. Không có directive `imports`, và `requires public` không tồn tại (đúng là `requires transitive`).
- **A sai:** `requires java.sql`; là hợp lệ.
- **B sai:** Qualified export `exports ... to <module>` là hợp lệ.
- **D sai:** `opens` một package của chính module là hợp lệ.
- *Kiểm chứng:* `examples/questions/ch07/Q07_11/` — script variants: CE in ra 'FAIL' (`python3 tools/book.py questions ch07`).

Câu 07-12 — Đáp án: B, E

- **Vì sao đúng:** `open` module đã mở **mọi** package cho reflection, nên viết thêm `opens` bên trong là lỗi (B). `exports` một package rỗng hoặc không tồn tại là lỗi (E).
- **A sai:** `open` module vẫn có thể `exports` package.
- **C sai:** Một package vừa `opens` vừa `exports` là hợp lệ.
- **D sai:** Qualified `opens opens ... to <module>` là hợp lệ.
- *Kiểm chứng:* `examples/questions/ch07/Q07_12/` — script variants: BE in ra 'FAIL' (`python3 tools/book.py questions ch07`).

Câu 07-13 — Đáp án: A, C

- **Vì sao đúng:** A: unnamed module đọc mọi module và thấy mọi package được export. C: automatic module export (và open) mọi package trong JAR, nên module khác dùng được cả `org.a` lẫn `org.b`.
- **B sai:** Named module không đọc unnamed module (không có tên để requires) → `package org.cp does not exist`.
- **D sai:** Trên classpath, `module-info.class` bị bỏ qua: mọi lớp thuộc unnamed module → `isNamed()` là `false`.
- **E sai:** Tên được chuẩn hoá: `my-lib-1.0.jar` → `my.lib` (version 1.0).
- *Kiểm chứng:* `examples/questions/ch07/Q07_13/` — each option proven true/false by a program (`python3 tools/book.py questions ch07`).

Câu 07-14 — Đáp án: A

- **Vì sao đúng:** `--print-module-deps` in danh sách module **tối thiểu** (cách nhau dấu phẩy), dùng được cho `jlink --add-modules java.logging` không cần ghi vì `java.sql` đã `requires transitive java.logging`.
- **B sai:** Danh sách luôn có `java.base`; và `java.logging` được suy ra từ `java.sql`.
- **C sai:** Đây là kiểu output của `--list-deps`; `--print-module-deps` rút gọn các module đã được kéo theo bắc cầu.
- **D sai:** Đó là dạng output của `jdeps -summary`.
- *Kiểm chứng:* `examples/questions/ch07/Q07_14/` — script output confirmed (python3 tools/book.py questions ch07).

Câu 07-15 — Đáp án: D

- **Vì sao đúng:** **Qualified export** `exports <package> to <module>` chỉ mở package cho các module được liệt kê. `com.other` không có trong danh sách nên không thấy `com.core.api`.
- **A sai:** `to com.friend` giới hạn chỉ `com.friend` được dùng package.
- **B sai:** Ngược lại: chỉ module được nêu tên sau `to` mới dùng được.
- **C sai:** `com.friend` được phép vì có tên trong qualified export.
- *Kiểm chứng:* `examples/questions/ch07/Q07_15/` — script output confirmed (python3 tools/book.py questions ch07).

Câu 07-16 — Đáp án: A, D

- **Vì sao đúng:** Với chế độ nhiều module, `--module-source-path src` cho javac biết mỗi thư mục con của `src` là một module; có thể chọn module bằng `-m com.x` (A) hoặc liệt kê các file nguồn (D). Kết quả nằm ở `out/com.x/`.
- **B sai:** `--module-path` dành cho module **đã biên dịch**, không phải mã nguồn; `-m` cần `--module-source-path`.
- **C sai:** Không có `-m`, `com.x` bị hiểu là tên file nguồn → lỗi.
- **E sai:** `-m` (module) cần `--module-source-path`; `-cp` không thay thế được.
- *Kiểm chứng:* `examples/questions/ch07/Q07_16/` — script variants: AD in ra 'hi' (python3 tools/book.py questions ch07).

Câu 07-17 — Đáp án: D

- **Vì sao đúng:** Đồ thị phụ thuộc giữa các module **không được có vòng** (lúc biên dịch). javac báo “cyclic dependence involving mod.a”. Cách sửa thường gặp: tách phần dùng chung ra module thứ ba.
- **A sai:** Hệ thống module cấm vòng `requires`.
- **B sai:** Lỗi được phát hiện ngay lúc biên dịch.
- **C sai:** Không có tùy chọn nào làm vòng `requires` hợp lệ khi biên dịch.
- *Kiểm chứng:* `examples/questions/ch07/Q07_17/` — script output confirmed (python3 tools/book.py questions ch07).

Câu 07-18 — Đáp án: A

- **Vì sao đúng:** `java.sql` khai báo `requires transitive java.logging`, nên module nào `requires java.sql` cũng đọc được `java.logging` (implied readability).
- **B sai:** Nhờ `requires transitive` trong `java.sql`, `com.q` đọc được `java.logging`.
- **C sai:** Readability được giải quyết giống nhau lúc biên dịch và lúc chạy; không có lỗi.
- **D sai:** Không cần tùy chọn thêm.
- *Kiểm chứng:* `examples/questions/ch07/Q07_18/` — script output confirmed (python3 tools/book.py questions ch07).

Câu 07-19 — Đáp án: B

- **Vì sao đúng:** `uses` chỉ khai báo ý định dùng service. Không có provider thì `ServiceLoader` đơn giản là không tìm thấy gì: `findFirst()` trả về `Optional` rỗng. Provider có thể được thêm lúc chạy mà không cần biên dịch lại consumer.
- **A sai:** Lỗi này chỉ xảy ra khi **thiếu uses**, không phải khi thiếu provider.
- **C sai:** Consumer không phụ thuộc lúc biên dịch vào provider nào.
- **D sai:** `findFirst()` trả về `Optional`, không trả về `null`.
- *Kiểm chứng:* `examples/questions/ch07/Q07_19/` — script output confirmed (python3 tools/book.py questions ch07).

Câu 07-20 — Đáp án: A, B

- **Vì sao đúng:** `jlink` cần: nơi tìm module (`--module-path/-p`), module gốc (`--add-modules`) và thư mục đích (`--output`).
- **C sai:** Thiếu `--add-modules`: `jlink` không biết đưa module nào vào → lỗi.
- **D sai:** `jlink` không có tùy chọn `-m` để chọn module gốc; phải dùng `--add-modules`.
- **E sai:** `jar` tạo file JAR, không tạo runtime image.
- *Kiểm chứng:* `examples/questions/ch07/Q07_20/` — script variants: AB in ra 'hi' (python3 tools/book.py questions ch07).

Đọc thêm (link chính thức — bị chặn trong sandbox nên mình chưa mở được)

- JLS §7.7 Module Declarations: <https://docs.oracle.com/javase/specs/jls/se21/html/jls-7.html>
- JEP 261 Module System: <https://openjdk.org/jeps/261>

- Hướng dẫn công cụ `jlink`, `jdeps`, `jar`: <https://docs.oracle.com/en/java/javase/21/docs/specs/man/index.html>

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-28:

- `ModuleFinder.java` (quy tắc tên automatic module, `Automatic-Module-Name`):
<https://raw.githubusercontent.com/openjdk/jdk21u/master/src/java.base/share/classes/java/lang/module/ModuleFinder.java>
- `ServiceLoader.java` (yêu cầu `uses`, `provider constructor` / `provider()` method):
<https://raw.githubusercontent.com/openjdk/jdk21u/master/src/java.base/share/classes/java/util/ServiceLoader.java>
- `module-info.java` của `java.sql` (các `requires transitive`):
<https://raw.githubusercontent.com/openjdk/jdk21u/master/src/java.sql/share/classes/module-info.java>
- Mọi lệnh trong chương đều được chạy thật bằng JDK 21.0.10 (xem output ở trên).

Chương 8 — Đồng thời (Managing Concurrent Code Execution)

Mục tiêu

- 8.1 Tạo **platform thread** và **virtual thread**; dùng `Runnable` và `Callable`; quản lý vòng đời thread; dùng các `ExecutorService` và concurrent API để chạy task.
- 8.2 Viết code an toàn luồng (thread-safe) bằng cơ chế khoá (locking) và concurrent API.
- 8.3 Xử lý collection đồng thời và dùng parallel stream.

Giải thích đơn giản

Thread (luồng) là một dòng chạy code độc lập trong cùng chương trình. Nhiều thread chạy “cùng lúc” và **dùng chung bộ nhớ** (heap). Điều đó nhanh, nhưng nguy hiểm: hai thread cùng sửa một biến có thể làm mất dữ liệu (race condition).

Java 21 có hai loại thread:

	Platform thread	Virtual thread
Là gì	Bọc một thread của hệ điều hành	Thread nhẹ do JVM quản lý, chạy “trên” vài platform thread (carrier)
Chi phí	Đắt (≈ MB bộ nhớ stack), vài nghìn là nhiều	Rẻ, có thể tạo hàng trăm nghìn
Hợp với	Việc nặng CPU	Việc chờ I/O (gọi HTTP, DB, sleep)
Tạo	<code>new Thread(r), Thread.ofPlatform()</code>	<code>Thread.ofVirtual(), Thread.startVirtualThread(r), Executors.newVirtualThreadPerTaskExecutor()</code>
Daemon	Tuỳ chọn	Luôn là daemon

Thay vì tự quản lý thread, ta giao **task** (`Runnable` hoặc `Callable`) cho một **ExecutorService**.

Ví dụ

Code trong `examples/ch08/`. Chạy lại: `python3 tools/book.py examples ch08`. Output thật, JDK 21.0.10. Các ví dụ được viết để output **luôn giống nhau** (dùng `join`, `Future.get`, `latch...`); chỗ nào kết quả thật sự thay đổi mỗi lần chạy, ví dụ chỉ in điều luôn đúng và ghi chú rõ.

1. Tạo thread

`examples/ch08/Ex01_CreateThreads.java`

```
// objective: 8.1
// 5 cách tạo luồng (thread): extends Thread, Runnable, Thread.Builder (platform/virtual), startVirtualThread.
public class Ex01_CreateThreads {
    static class Worker extends Thread {
        Worker() { super("worker-1"); }
        @Override public void run() { System.out.println("extends Thread: " + getName()); }
    }

    public static void main(String[] args) throws InterruptedException {
        Thread t1 = new Worker();
        t1.start();
        t1.join(); // đợi t1 kết thúc

        Thread t2 = new Thread(() -> System.out.println("Runnable lambda: " + Thread.currentThread().getName()), "task-2");
        t2.start();
        t2.join();

        Thread t3 = Thread.ofPlatform().name("platform-3").start(
            () -> System.out.println("platform: virtual? " + Thread.currentThread().isVirtual());
        t3.join();

        Thread t4 = Thread.ofVirtual().name("virtual-4").start(
            () -> System.out.println("virtual: virtual? " + Thread.currentThread().isVirtual()
                + ", daemon? " + Thread.currentThread().isDaemon());
        t4.join();

        Thread t5 = Thread.startVirtualThread(() -> System.out.println("startVirtualThread, name="
            + Thread.currentThread().getName() + "")); // virtual thread mặc định không có tên
        t5.join();

        Thread unstarted = Thread.ofVirtual().unstarted(() -> { });
        System.out.println("unstarted state: " + unstarted.getState());
        System.out.println("main is virtual? " + Thread.currentThread().isVirtual());
    }
}
```

```
}  
}
```

Output thật (JDK 21.0.10):

```
extends Thread: worker-1  
Runnable lambda: task-2  
platform: virtual? false  
virtual: virtual? true, daemon? true  
startVirtualThread, name=''  
unstarted state: NEW  
main is virtual? false
```

2. Vòng đời thread

examples/ch08/Ex02_Lifecycle.java

```
// objective: 8.1  
// Vòng đời thread: NEW → RUNNABLE → (BLOCKED/WAITING/TIMED_WAITING) → TERMINATED; run() vs start().  
import java.util.concurrent.CountDownLatch;  
  
public class Ex02_Lifecycle {  
    public static void main(String[] args) throws Exception {  
        CountDownLatch started = new CountDownLatch(1);  
        Thread t = new Thread(() -> {  
            started.countDown();  
            try { Thread.sleep(2_000); } catch (InterruptedException e) {  
                System.out.println("bị interrupt khi đang sleep → InterruptedException");  
            }  
        }, "sleeper");  
        System.out.println("1: " + t.getState());           // NEW  
        t.start();  
        started.await();  
        while (t.getState() != Thread.State.TIMED_WAITING) Thread.onSpinWait();  
        System.out.println("2: " + t.getState());           // TIMED_WAITING (đang sleep)  
        t.interrupt();                                       // đánh thức sớm  
        t.join();  
        System.out.println("3: " + t.getState() + ", alive? " + t.isAlive());  
  
        Thread r = new Thread(() -> System.out.println("run() chạy trên thread: " + Thread.currentThread().getName()), "other");  
        r.run();                                             // KHÔNG tạo thread mới  
        r.start();  
        r.join();  
        try {  
            r.start();                                       // start lần 2  
        } catch (IllegalThreadStateException e) {  
            System.out.println("IllegalThreadStateException: không start() hai lần");  
        }  
    }  
}
```

Output thật (JDK 21.0.10):

```
1: NEW  
2: TIMED_WAITING  
bị interrupt khi đang sleep → InterruptedException  
3: TERMINATED, alive? false  
run() chạy trên thread: main  
run() chạy trên thread: other  
IllegalThreadStateException: không start() hai lần
```

3. ExecutorService, Callable, Future

examples/ch08/Ex03_ExecutorCallable.java

```
// objective: 8.1  
// ExecutorService: submit Runnable/Callable, Future.get, invokeAll, shutdown, awaitTermination.  
import java.util.List;  
import java.util.concurrent.*;  
  
public class Ex03_ExecutorCallable {  
    public static void main(String[] args) throws Exception {  
        ExecutorService pool = Executors.newFixedThreadPool(2);  
        Callable<Integer> sum = () -> {  
            int s = 0;  
            for (int i = 1; i <= 100; i++) s += i;  
            return s;  
        };  
        Future<Integer> f = pool.submit(sum);  
        Future<?> r = pool.submit(() -> System.out.println("Runnable không trả về giá trị"));  
        System.out.println("sum = " + f.get() + ", runnable result = " + r.get() + ", done? " + f.isDone());  
    }  
}
```

```

List<Callable<String>> tasks = List.of(() -> "a", () -> "b", () -> "c");
StringBuilder sb = new StringBuilder();
for (Future<String> fu : pool.invokeAll(tasks)) sb.append(fu.get()); // kết quả theo đúng thứ tự task
System.out.println("invokeAll: " + sb);

Future<Integer> failing = pool.submit(() -> 1 / 0);
try {
    failing.get();
} catch (ExecutionException e) {
    System.out.println("ExecutionException, cause: " + e.getCause());
}
Future<String> slow = pool.submit(() -> { Thread.sleep(3_000); return "late"; });
try {
    slow.get(100, TimeUnit.MILLISECONDS);
} catch (TimeoutException e) {
    System.out.println("TimeoutException; cancel -> " + slow.cancel(true) + ", cancelled? " + slow.isCancelled());
}

pool.shutdown(); // không nhận task mới, chạy nốt task cũ
System.out.println("isShutdown=" + pool.isShutdown() + " terminated=" + pool.awaitTermination(5, TimeUnit.SECONDS));
try {
    pool.submit(() -> "x");
} catch (RejectedExecutionException e) {
    System.out.println("RejectedExecutionException sau shutdown");
}
}
}

```

Output thật (JDK 21.0.10):

```

Runnable không trả về giá trị
sum = 5050, runnable result = null, done? true
invokeAll: abc
ExecutionException, cause: java.lang.ArithmeticException: / by zero
TimeoutException; cancel -> true, cancelled? true
isShutdown=true terminated=true
RejectedExecutionException sau shutdown

```

4. Virtual thread executor

examples/ch08/Ex04_VirtualExecutor.java

```

// objective: 8.1
// Virtual threads: một thread cho mỗi task, rất rẻ; ExecutorService là AutoCloseable (close() đợi task xong).
import java.time.Duration;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.*;

public class Ex04_VirtualExecutor {
    public static void main(String[] args) throws Exception {
        List<Future<Integer>> results = new ArrayList<>();
        try (ExecutorService ex = Executors.newVirtualThreadPerTaskExecutor()) {
            for (int i = 1; i <= 10_000; i++) {
                int n = i;
                results.add(ex.submit(() -> {
                    Thread.sleep(Duration.ofMillis(10)); // "chặn" (blocking) nhưng virtual thread nhả carrier thread
                    return n;
                }));
            }
            // close() = shutdown + đợi mọi task kết thúc
            long total = 0;
            for (Future<Integer> f : results) total += f.get();
            System.out.println("10000 virtual tasks, total = " + total);

            try (var ex = Executors.newVirtualThreadPerTaskExecutor()) {
                Future<Boolean> isVirtual = ex.submit(() -> Thread.currentThread().isVirtual());
                System.out.println("task chạy trên virtual thread? " + isVirtual.get());
            }
        }
    }
}

```

Output thật (JDK 21.0.10):

```

10000 virtual tasks, total = 50005000
task chạy trên virtual thread? true

```

5. ScheduledExecutorService

examples/ch08/Ex05_Scheduled.java

```
// objective: 8.1
// ScheduledExecutorService: chạy sau một khoảng trễ, hoặc lặp lại định kỳ.
import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicInteger;

public class Ex05_Scheduled {
    public static void main(String[] args) throws Exception {
        ScheduledExecutorService ses = Executors.newSingleThreadScheduledExecutor();
        ScheduledFuture<String> later = ses.schedule(() -> "chạy sau 100ms", 100, TimeUnit.MILLISECONDS);
        System.out.println(later.get());

        AtomicInteger ticks = new AtomicInteger();
        CountDownLatch three = new CountDownLatch(3);
        ScheduledFuture<?> repeat = ses.scheduleAtFixedRate(() -> {
            ticks.incrementAndGet();
            three.countDown();
        }, 0, 50, TimeUnit.MILLISECONDS);
        three.await();
        repeat.cancel(false);
        System.out.println("ticks >= 3 ? " + (ticks.get() >= 3) + ", cancelled? " + repeat.isCancelled());
        ses.shutdown();
        System.out.println("terminated: " + ses.awaitTermination(2, TimeUnit.SECONDS));
    }
}
```

Output thật (JDK 21.0.10):

```
chạy sau 100ms
ticks >= 3 ? true, cancelled? true
terminated: true
```

6. Race condition

examples/ch08/Ex06_RaceCondition.java

```
// objective: 8.2
// Race condition: count++ không nguyên tử (atomic). So sánh với synchronized và AtomicInteger.
import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicInteger;

public class Ex06_RaceCondition {
    static int unsafe = 0;
    static int safe = 0;
    static final Object LOCK = new Object();
    static final AtomicInteger atomic = new AtomicInteger();

    public static void main(String[] args) throws Exception {
        int threads = 4, perThread = 100_000;
        try (ExecutorService ex = Executors.newFixedThreadPool(threads)) {
            for (int t = 0; t < threads; t++) {
                ex.submit(() -> {
                    for (int i = 0; i < perThread; i++) {
                        unsafe++;
                        synchronized (LOCK) { safe++; }
                        atomic.incrementAndGet();
                    }
                });
            }
        }
        int expected = threads * perThread;
        System.out.println("expected = " + expected);
        System.out.println("synchronized= " + safe);
        System.out.println("atomic = " + atomic.get());
        System.out.println("unsafe <= expected? " + (unsafe <= expected) + " (giá trị thật thay đổi mỗi lần chạy)");
    }
}
```

Output thật (JDK 21.0.10):

```
expected = 400000
synchronized= 400000
atomic = 400000
unsafe <= expected? true (giá trị thật thay đổi mỗi lần chạy)
```

7. Lock

examples/ch08/Ex07_Locks.java

```
// objective: 8.2
// ReentrantLock: lock()/unlock() trong finally, tryLock(), khoá "reentrant", ReadWriteLock.
import java.util.concurrent.TimeUnit;
import java.util.concurrent.locks.*;
```

```

public class Ex07_Locks {
    static final ReentrantLock lock = new ReentrantLock();
    static int balance = 100;

    static void withdraw(int amount) {
        lock.lock();
        try {
            if (balance >= amount) balance -= amount;
            nested(); // cùng thread lấy lại khoá được (reentrant)
        } finally {
            lock.unlock(); // LUÔN mở khoá trong finally
        }
    }

    static void nested() {
        lock.lock();
        try { System.out.println("hold count = " + lock.getHoldCount()); } finally { lock.unlock(); }
    }

    public static void main(String[] args) throws Exception {
        withdraw(30);
        System.out.println("balance = " + balance + ", locked now? " + lock.isLocked());

        lock.lock(); // main giữ khoá
        Thread other = new Thread(() -> {
            try {
                boolean got = lock.tryLock(100, TimeUnit.MILLISECONDS);
                System.out.println("other tryLock: " + got);
                if (got) lock.unlock();
            } catch (InterruptedException e) { }
        });
        other.start();
        other.join();
        lock.unlock();

        try {
            lock.unlock(); // mở khoá khi không giữ
        } catch (IllegalMonitorStateException e) {
            System.out.println("IllegalMonitorStateException");
        }

        ReadWriteLock rw = new ReentrantReadWriteLock();
        rw.readLock().lock();
        rw.readLock().lock(); // nhiều reader cùng lúc được
        System.out.println("read locks = " + ((ReentrantReadWriteLock) rw).getReadLockCount());
        rw.readLock().unlock();
        rw.readLock().unlock();
    }
}

```

Output thật (JDK 21.0.10):

```

hold count = 2
balance = 70, locked now? false
other tryLock: false
IllegalMonitorStateException
read locks = 2

```

8. Concurrent collections

examples/ch08/Ex08_ConcurrentCollections.java

```

// objective: 8.2, 8.3
// Collection an toàn cho đa luồng: ConcurrentHashMap, CopyOnWriteArrayList, BlockingQueue.
import java.util.*;
import java.util.concurrent.*;

public class Ex08_ConcurrentCollections {
    public static void main(String[] args) throws Exception {
        ConcurrentHashMap<String, Integer> hits = new ConcurrentHashMap<>();
        try (ExecutorService ex = Executors.newFixedThreadPool(4)) {
            for (int i = 0; i < 1000; i++) {
                String page = (i % 2 == 0) ? "home" : "about";
                ex.submit(() -> hits.merge(page, 1, Integer::sum)); // merge là nguyên tử
            }
        }
        System.out.println(new TreeMap<>(hits));

        List<String> cow = new CopyOnWriteArrayList<>(List.of("a", "b"));
        for (String s : cow) cow.add(s + "!"); // không có ConcurrentModificationException
        System.out.println(cow); // vòng lặp duyệt "ảnh chụp" (snapshot) cũ

        BlockingQueue<Integer> queue = new ArrayBlockingQueue<>(2);
        Thread producer = new Thread(() -> {

```

```

        try {
            for (int i = 1; i <= 5; i++) queue.put(i);    // put chặn khi hàng đợi đầy
            queue.put(-1);                               // tín hiệu kết thúc
        } catch (InterruptedException e) { }
    });
    producer.start();
    StringBuilder got = new StringBuilder();
    for (int v; (v = queue.take()) != -1; ) got.append(v).append(' '); // take chặn khi rỗng
    System.out.println("consumed: " + got.toString().trim());
    System.out.println("offer on full: " + new ArrayBlockingQueue<Integer>(1) {{ offer(1); }}.offer(2));

    Map<String, Integer> sync = Collections.synchronizedMap(new HashMap<>());
    sync.put("x", 1);
    System.out.println(sync + " " + new ConcurrentSkipListSet<>(List.of(3, 1, 2)));
}
}

```

Output thật (JDK 21.0.10):

```

{about=500, home=500}
[a, b, a!, b!]
consumed: 1 2 3 4 5
offer on full: false
{x=1} [1, 2, 3]

```

9. Atomic

examples/ch08/Ex09_Atomics.java

```

// objective: 8.2
// Lớp atomic: incrementAndGet vs getAndIncrement, compareAndSet, updateAndGet, accumulateAndGet.
import java.util.concurrent.atomic.*;

public class Ex09_Atomics {
    public static void main(String[] args) {
        AtomicInteger a = new AtomicInteger(5);
        System.out.println(a.getAndIncrement() + " " + a.get() + " " + a.incrementAndGet() + " " + a.get());
        System.out.println(a.compareAndSet(7, 100) + " " + a.get() + " " + a.compareAndSet(7, 200) + " " + a.get());
        System.out.println(a.updateAndGet(x -> x * 2) + " " + a.getAndUpdate(x -> x + 1) + " " + a.get());
        System.out.println(a.accumulateAndGet(10, Math::max) + " " + a.addAndGet(-1));

        AtomicLong big = new AtomicLong();
        big.addAndGet(5_000_000_000L);
        AtomicBoolean flag = new AtomicBoolean();
        System.out.println(big + " " + flag.getAndSet(true) + " " + flag.get());

        LongAdder adder = new LongAdder(); // nhanh hơn khi nhiều thread cùng cộng
        adder.increment(); adder.add(10);
        AtomicReference<String> ref = new AtomicReference<>("v1");
        ref.set("v2");
        System.out.println(adder.sum() + " " + ref.get());
    }
}

```

Output thật (JDK 21.0.10):

```

5 6 7 7
true 100 false 100
200 200 201
201 200
5000000000 false true
11 v2

```

10. CountdownLatch, CyclicBarrier

examples/ch08/Ex10_Coordination.java

```

// objective: 8.2
// Phối hợp thread: CountdownLatch (đợi N việc xong), CyclicBarrier (đợi nhau tại một điểm).
import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicInteger;

public class Ex10_Coordination {
    public static void main(String[] args) throws Exception {
        CountdownLatch done = new CountdownLatch(3);
        AtomicInteger work = new AtomicInteger();
        for (int i = 0; i < 3; i++) {
            Thread.ofVirtual().start(() -> { work.addAndGet(10); done.countDown(); });
        }
        done.await(); // chặn cho tới khi count = 0
        System.out.println("latch released, work = " + work.get() + ", count = " + done.getCount());
    }
}

```

```

CyclicBarrier barrier = new CyclicBarrier(3, () -> System.out.println("barrier action: cả 3 đã tới"));
try (ExecutorService ex = Executors.newFixedThreadPool(3)) {
    for (int i = 0; i < 3; i++) ex.submit(() -> { barrier.await(); return null; });
}
System.out.println("parties = " + barrier.getParties() + ", broken? " + barrier.isBroken());
}
}

```

Output thật (JDK 21.0.10):

```

latch released, work = 30, count = 0
barrier action: cả 3 đã tới
parties = 3, broken? false

```

11. Collection song song và parallel stream

examples/ch08/Ex11_ParallelCollections.java

```

// objective: 8.3
// Xử lý collection song song: parallel stream, groupingByConcurrent, và bẫy tác dụng phụ (side effect).
import java.util.*;
import java.util.concurrent.*;
import java.util.stream.*;

public class Ex11_ParallelCollections {
    public static void main(String[] args) {
        List<Integer> nums = IntStream.rangeClosed(1, 10_000).boxed().toList();
        System.out.println("sum = " + nums.parallelStream().mapToLong(Integer::longValue).sum());

        List<Integer> good = nums.parallelStream().filter(n -> n % 1000 == 0).toList(); // giữ thứ tự gặp
        System.out.println("collected (ordered) = " + good);

        List<Integer> bad = new ArrayList<>(); // KHÔNG an toàn khi ghi từ nhiều thread
        List<Integer> safe = Collections.synchronizedList(new ArrayList<>());
        nums.parallelStream().forEach(safe::add);
        System.out.println("synchronizedList size = " + safe.size() + " (thứ tự có thể lộn xộn)");

        ConcurrentMap<Boolean, Long> evenOdd = nums.parallelStream()
            .collect(Collectors.groupingByConcurrent(n -> n % 2 == 0, Collectors.counting()));
        System.out.println("groupingByConcurrent = " + new TreeMap<>(evenOdd));

        System.out.println("parallelism of common pool >= 1 ? " + (ForkJoinPool.commonPool().getParallelism() >= 1)
            + ", bad.size() = " + bad.size());
    }
}

```

Output thật (JDK 21.0.10):

```

sum = 50005000
collected (ordered) = [1000, 2000, 3000, 4000, 5000, 6000, 7000, 8000, 9000, 10000]
synchronizedList size = 10000 (thứ tự có thể lộn xộn)
groupingByConcurrent = {false=5000, true=5000}
parallelism of common pool >= 1 ? true, bad.size() = 0

```

12. shutdownNow

examples/ch08/Ex12_ShutdownNow.java

```

// objective: 8.1
// shutdown() vs shutdownNow(): shutdownNow interrupt các task đang chạy và trả về task chưa chạy.
import java.util.List;
import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicBoolean;

public class Ex12_ShutdownNow {
    public static void main(String[] args) throws Exception {
        ExecutorService single = Executors.newSingleThreadExecutor();
        CountDownLatch running = new CountDownLatch(1);
        AtomicBoolean interrupted = new AtomicBoolean();
        single.submit(() -> {
            running.countDown();
            try { Thread.sleep(10_000); } catch (InterruptedException e) {
                interrupted.set(true); // shutdownNow() đã interrupt task này
            }
        });
        for (int i = 0; i < 3; i++) single.submit(() -> System.out.println("never runs"));
        running.await();
        List<Runnable> notStarted = single.shutdownNow();
        System.out.println("chưa chạy: " + notStarted.size());
        System.out.println("terminated: " + single.awaitTermination(2, TimeUnit.SECONDS) + ", isTerminated: " +
            single.isTerminated());
        System.out.println("task đang chạy bị interrupt? " + interrupted.get());
    }
}

```

```
}
}
```

Output thật (JDK 21.0.10):

```
chưa chạy: 3
terminated: true, isTerminated: true
task đang chạy bị interrupt? true
```

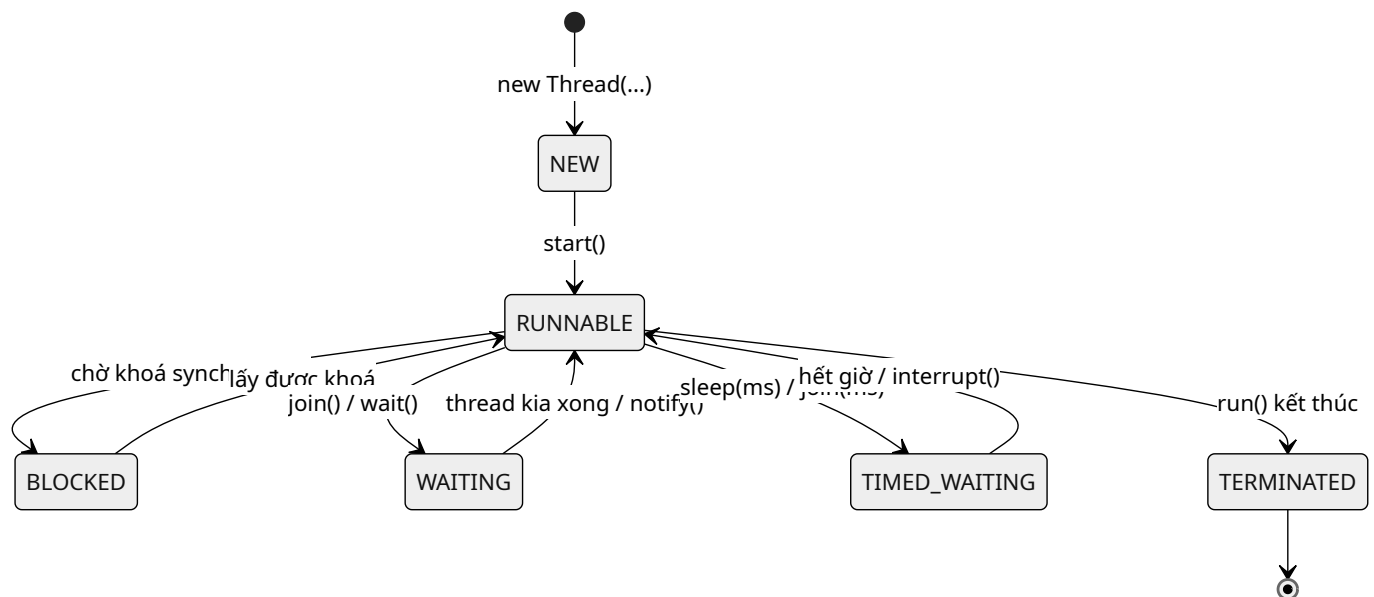
Đi sâu

Runnable vs Callable

	Runnable	Callable<V>
Method	void run()	V call() throws Exception
Trả về	không	có
Checked exception	không được ném	được ném
Dùng với	new Thread(r), execute(r), submit(r) → Future<?>	submit(c) → Future<V>, invokeAll, invokeAny

Lambda () -> "x" khớp Callable; () -> System.out.println() khớp cả hai (compiler chọn tùy ngữ cảnh); () -> { return; } chỉ khớp Runnable.

Vòng đời thread (Thread.State)



NEW (tạo, chưa start) → RUNNABLE (đang chạy hoặc sẵn sàng) → có thể vào BLOCKED (chờ khoá synchronized), WAITING (join(), wait(), LockSupport.park), TIMED_WAITING (sleep(ms), join(ms)) → TERMINATED.

- start() hai lần → IllegalStateException. Gọi run() trực tiếp không tạo thread.
- interrupt() đánh thức thread đang sleep/wait/join bằng InterruptedException; nếu thread đang chạy thì chỉ đặt cờ (isInterrupted()).
- JVM kết thúc khi mọi thread **không phải daemon** đã kết thúc.

ExecutorService

Tạo	Ghi chú
Executors.newSingleThreadExecutor()	Một thread, task chạy tuần tự theo thứ tự gửi
Executors.newFixedThreadPool(n)	n thread
Executors.newCachedThreadPool()	Tạo thêm khi cần, dùng lại thread rảnh
Executors.newScheduledThreadPool(n) / newSingleThreadScheduledExecutor()	schedule, scheduleAtFixedRate, scheduleWithFixedDelay
Executors.newVirtualThreadPerTaskExecutor()	Một virtual thread mới cho mỗi task

Method	Ý nghĩa
<code>execute(Runnable)</code>	<code>void</code> , từ <code>Executor</code>
<code>submit(Runnable/Callable)</code>	<code>Future</code>
<code>invokeAll(tasks)</code>	Đợi tất cả; <code>List<Future></code> theo thứ tự task
<code>invokeAny(tasks)</code>	Đợi một task thành công; trả về kết quả trực tiếp, huỷ các task còn lại
<code>shutdown()</code>	Không nhận task mới; task cũ chạy nốt. Gửi thêm → <code>RejectedExecutionException</code>
<code>shutdownNow()</code>	Interrupt task đang chạy, trả về <code>List<Runnable></code> chưa chạy
<code>awaitTermination(t, unit)</code>	Đợi tối đa <code>t</code> ; trả về <code>true</code> nếu đã kết thúc
<code>isShutdown(), isTerminated()</code>	
<code>close()</code> (Java 19+)	<code>ExecutorService</code> là <code>AutoCloseable</code> : <code>shutdown</code> + đợi → dùng được với <code>try-with-resources</code>

`Future`: `get()` (chặn; lỗi trong task → `ExecutionException` với `getCause()`), `get(t, unit)` (→ `TimeoutException`), `isDone()`, `cancel(mayInterrupt)`, `isCancelled()`.

Thread-safety

- **Race condition**: `count++` = đọc, cộng, ghi — không nguyên tử (atomic).
- `synchronized` method (khóa `this`, hoặc `Class` nếu `static`) / khối `synchronized (obj)` (`obj` phải là tham chiếu). Constructor không `synchronized` được.
- `Lock` / `ReentrantLock`: `lock()` + `unlock()` trong `finally`; `tryLock()` / `tryLock(t, unit)` trả về `boolean`; `reentrant` (hold count); `unlock()` khi không giữ → `IllegalMonitorStateException`. `ReentrantReadWriteLock`: nhiều reader hoặc một writer.
- `Atomic`: `AtomicInteger`, `AtomicLong`, `AtomicBoolean`, `AtomicReference`: `get`, `set`, `getAndIncrement` (trả cũ) / `incrementAndGet` (trả mới), `compareAndSet`, `updateAndGet`, `accumulateAndGet`. `LongAdder` cho bộ đếm nhiều thread.
- Phối hợp: `CountDownLatch` (đếm ngược một lần, không âm), `CyclicBarrier` (đợi nhau, dùng lại được).
- Vấn đề liveness: **deadlock** (hai thread giữ khoá và chờ khoá của nhau), **starvation** (một thread không bao giờ được chạy), **livelock** (hai thread liên tục nhường nhau, không tiến triển).

Concurrent collections

Lớp	Ghi chú
<code>ConcurrentHashMap</code>	Không nhận <code>key/value null</code> ; <code>merge</code> , <code>compute</code> , <code>putIfAbsent</code> nguyên tử
<code>CopyOnWriteArrayList</code> / <code>CopyOnWriteArraySet</code>	Ghi = sao chép mảng; iterator duyệt snapshot, không CME
<code>ConcurrentLinkedQueue</code> , <code>ConcurrentSkipListMap/Set</code>	Không khoá / có sắp xếp
<code>ArrayBlockingQueue</code> , <code>LinkedBlockingQueue</code>	<code>put/take</code> chặn; <code>offer/poll</code> không chặn (hoặc có timeout)
<code>Collections.synchronizedList/Map(...)</code>	Bọc đồng bộ từng method; duyệt vẫn phải tự <code>synchronized</code>

Parallel stream (8.3)

- Chạy trên `ForkJoinPool.commonPool()`. Chỉ có lợi với dữ liệu lớn, việc nặng CPU, không tác dụng phụ.
- Ghi vào collection thường từ `forEach` song song là lỗi; hãy dùng `collect`, `toList()`, `groupingByConcurrent`, `toConcurrentMap`, hoặc collection đồng bộ.
- `forEach` không giữ thứ tự; `forEachOrdered` giữ thứ tự gặp. `findAny` có thể trả phần tử bất kỳ.
- `reduce` cần identity đúng + hàm kết hợp (xem chương 6).

Lỗi và bẫy thường gặp (Exam traps)

1. `t.run()` không tạo thread; `start()` hai lần → exception.
2. Lambda `Runnable` gọi `Thread.sleep` mà không bắt `InterruptedException` → lỗi biên dịch; `Callable` thì được.
3. `execute` trả về `void`; `submit(Runnable)` trả `Future<?>` với `get() = null`.
4. Exception trong task → `ExecutionException` khi `get()`.
5. `invokeAll` giữ thứ tự task; `invokeAny` trả kết quả, không trả `Future`.
6. Sau `shutdown()` gửi task → `RejectedExecutionException`; task đang chạy vẫn tiếp tục.
7. `synchronized (primitive)` và constructor `synchronized` → lỗi biên dịch.
8. `unlock()` không giữ khoá → `IllegalMonitorStateException`; quên `unlock()` → thread khác chờ mãi.
9. `getAndIncrement` vs `incrementAndGet`.
10. `ConcurrentHashMap.put(null, ...)` → NPE.
11. `offer` không chặn (trả `false`), `put` chặn.
12. Virtual thread luôn `daemon`; `setDaemon(false)` → `IllegalArgumentException`.
13. `forEach` trên parallel stream không có thứ tự.

Góc nhìn từ TypeScript

TypeScript / Node.js	Java	Ghi chú
Một thread chạy event loop; async/await không chạy song song code JS	Nhiều thread thật, dùng chung bộ nhớ	Race condition là chuyện có thật trong Java
<code>Promise<T></code>	<code>Future<T></code> / <code>CompletableFuture<T></code>	<code>future.get()</code> chặn thread; <code>await</code> thì không chặn event loop
<code>Promise.all([...])</code>	<code>invokeAll(tasks)</code>	
<code>Promise.any([...])</code>	<code>invokeAny(tasks)</code>	
Worker threads (không chung bộ nhớ, trừ <code>SharedArrayBuffer</code>)	Thread chung heap	Cần <code>synchronized</code> , <code>Lock</code> , <code>atomic</code>
<code>Atomics.add</code> trên <code>SharedArrayBuffer</code>	<code>AtomicInteger.addAndGet</code>	
async I/O không chặn	Virtual thread: viết code chặn bình thường, JVM tự “nhả” carrier thread khi chờ	Code đơn giản như code đồng bộ

Tóm tắt

- Tạo thread: `extends Thread`, `new Thread(Runnable)`, `Thread.ofPlatform()/ofVirtual()`, `startVirtualThread`.
- Vòng đời: `NEW` → `RUNNABLE` → `BLOCKED/WAITING/TIMED_WAITING` → `TERMINATED`.
- `ExecutorService`: `submit/execute/invokeAll/invokeAny`, `shutdown/shutdownNow/awaitTermination`, `close()`.
- Thread-safe: `synchronized`, `ReentrantLock` (unlock trong `finally`), `atomic`, `concurrent collections`.
- Parallel stream: không side effect, dùng `collector`; `forEachOrdered` để giữ thứ tự.

Bài tập (có lời giải)

Mọi đáp án đã được `tools/book.py` biên dịch và chạy để xác nhận (code: `examples/questions/ch08/`). Các câu hỏi được thiết kế để có output xác định; bộ kiểm tra đã được chạy lặp lại nhiều lần với cùng kết quả.

Câu hỏi

Câu 08-01 · Dễ · objective 8.1

Chương trình sau in ra gì?

```
public class RunStart {
    public static void main(String[] args) throws InterruptedException {
        Thread t = new Thread(() -> System.out.print(Thread.currentThread().getName() + " "), "worker");
        t.run();
        t.start();
        t.join();
    }
}
```

- A. `main worker`
- B. `worker worker`
- C. `main main`
- D. `worker main`

Câu 08-02 · Vừa · objective 8.1

Chương trình sau in ra gì?

```
import java.util.concurrent.*;

public class Fail {
    public static void main(String[] args) {
        ExecutorService ex = Executors.newSingleThreadExecutor();
        Future<Integer> f = ex.submit(() -> Integer.parseInt("x1"));
        try {
            System.out.println(f.get());
        } catch (ExecutionException e) {
            System.out.println("EE:" + e.getCause().getClass().getSimpleName());
        } catch (InterruptedException e) {
            System.out.println("IE");
        } finally {
            ex.shutdown();
        }
    }
}
```

- A. Ném `NumberFormatException` ra khỏi `main`
- B. `EE:ExecutionException`
- C. `null`
- D. `EE:NumberFormatException`

Câu 08-03 · Vừa · objective 8.1

Những dòng nào gây lỗi biên dịch?

```
import java.util.concurrent.*;

public class Tasks {
    public static void main(String[] args) throws Exception {
        ExecutorService ex = Executors.newFixedThreadPool(1);
        Future<?> a = ex.submit(() -> System.out.println("a"));           // L1
        Future<String> b = ex.submit(() -> "b");                         // L2
        Callable<Void> c = () -> { Thread.sleep(10); return null; };     // L3
        Runnable d = () -> { Thread.sleep(10); };                       // L4
        Future<Integer> e = ex.submit(() -> { return; });                // L5
        ex.shutdown();
    }
}
```

- A. Chỉ L4
- B. L4 và L5
- C. L3 và L4
- D. L3, L4 và L5
- E. Chỉ L5

Câu 08-04 · Vừa · objective 8.1

Chương trình sau in ra gì?

```
public class States {
    public static void main(String[] args) throws InterruptedException {
        Thread t = new Thread(() -> { });
        System.out.print(t.getState() + " ");
        t.start();
        t.join();
        System.out.print(t.getState() + " " + t.isAlive());
    }
}
```

- A. `NEW RUNNABLE true`
- B. `CREATED TERMINATED false`
- C. `NEW TERMINATED false`
- D. `NEW DEAD false`

Câu 08-05 · Vừa · objective 8.1

Hai phát biểu nào đúng về virtual thread (Java 21)? (Chọn 2 đáp án.)

- A. Virtual thread luôn là daemon thread.
- B. `Thread.ofVirtual().start(x)` trả về một thread đã được start.
- C. `new Thread(x)` tạo ra một virtual thread.
- D. Có thể gọi `setDaemon(false)` trên virtual thread.
- E. `Executors.newVirtualThreadPerTaskExecutor()` giới hạn số thread chạy cùng lúc bằng số CPU.

Câu 08-06 · Vừa · objective 8.1

Chương trình sau in ra gì?

```
import java.util.*;
import java.util.concurrent.*;

public class AllOf {
    public static void main(String[] args) throws Exception {
        ExecutorService ex = Executors.newFixedThreadPool(3);
        List<Callable<Integer>> tasks = List.of(
            () -> { Thread.sleep(200); return 1; },
            () -> 2,
            () -> { Thread.sleep(100); return 3; });
        StringBuilder sb = new StringBuilder();
        for (Future<Integer> f : ex.invokeAll(tasks)) sb.append(f.get());
        ex.shutdown();
        System.out.println(sb);
    }
}
```

```
}  
}
```

- A. 213
- B. 231
- C. 123
- D. Thứ tự không xác định

Câu 08-07 · Khó · objective 8.2

Chương trình sau in ra gì?

```
import java.util.concurrent.atomic.AtomicInteger;  
  
public class Atom {  
    public static void main(String[] args) {  
        AtomicInteger a = new AtomicInteger(10);  
        int x = a.getAndAdd(5);  
        int y = a.incrementAndGet();  
        boolean z = a.compareAndSet(15, 0);  
        int w = a.updateAndGet(v -> v * 2);  
        System.out.println(x + " " + y + " " + z + " " + w);  
    }  
}
```

- A. 15 16 true 0
- B. 10 16 true 0
- C. 15 17 false 34
- D. 10 16 false 32

Câu 08-08 · Vừa · objective 8.2

Những dòng nào gây lỗi biên dịch?

```
public class Sync {  
    private final Object lock = new Object();  
    private int count;  
  
    synchronized void a() { count++; } // L1  
    void b() { synchronized (lock) { count++; } } // L2  
    void c() { synchronized (count) { count++; } } // L3  
    static synchronized void d() { } // L4  
    synchronized Sync() { } // L5  
  
    public static void main(String[] args) { }  
}
```

- A. Chỉ L3
- B. L3 và L5
- C. L4 và L5
- D. L1 và L3
- E. Chỉ L5

Câu 08-09 · Vừa · objective 8.2

Chương trình sau in ra gì?

```
import java.util.concurrent.locks.ReentrantLock;  
  
public class Holds {  
    public static void main(String[] args) {  
        ReentrantLock lock = new ReentrantLock();  
        lock.lock();  
        lock.lock();  
        System.out.print(lock.getHoldCount() + " ");  
        lock.unlock();  
        System.out.print(lock.isLocked() + " ");  
        lock.unlock();  
        System.out.print(lock.isLocked() + " ");  
        try {  
            lock.unlock();  
        } catch (IllegalMonitorStateException e) {  
            System.out.print("IMSE");  
        }  
    }  
}
```

- A. 2 true false IMSE

- B. 2 false false IMSE
- C. 1 true false IMSE
- D. 2 true false

Câu 08-10 · Khó · objective 8.2

Hai phát biểu nào đúng? (Chọn 2 đáp án.)

- A. ConcurrentHashMap không cho phép key null.
- B. CopyOnWriteArrayList cho phép thêm phần tử trong lúc đang duyệt bằng for-each mà không ném ConcurrentModificationException.
- C. Gọi unlock() trên ReentrantLock mà thread không giữ thì không có gì xảy ra.
- D. AtomicInteger.getAndIncrement() trả về giá trị sau khi tăng.
- E. ArrayBlockingQueue.offer(e) chặn (block) khi hàng đợi đầy.

Câu 08-11 · Vừa · objective 8.3

Điều gì đúng về kết quả của chương trình?

```
import java.util.*;

public class Ordered {
    public static void main(String[] args) {
        StringBuilder sb = new StringBuilder();
        List.of(1, 2, 3, 4, 5).parallelStream()
            .map(x -> x * x)
            .forEachOrdered(x -> sb.append(x).append(' '));
        System.out.println(sb);
    }
}
```

- A. Luôn in 1,4,9,16,25,
- B. In các số theo thứ tự bất kỳ
- C. Ném ConcurrentModificationException
- D. Không biên dịch được vì sb được sửa trong lambda

Câu 08-12 · Vừa · objective 8.2

Chương trình sau in ra gì?

```
import java.util.concurrent.CountDownLatch;

public class Latch {
    public static void main(String[] args) throws InterruptedException {
        CountDownLatch latch = new CountDownLatch(2);
        latch.countDown();
        latch.countDown();
        latch.countDown();
        latch.await();
        System.out.println(latch.getCount());
    }
}
```

- A. -1
- B. Ném IllegalStateException
- C. Chương trình bị treo (chờ mãi ở await())
- D. 0

Câu 08-13 · Vừa · objective 8.1

Những dòng nào gây lỗi biên dịch?

```
import java.util.concurrent.*;

public class Exec {
    public static void main(String[] args) throws Exception {
        ExecutorService ex = Executors.newCachedThreadPool();
        ex.execute(() -> System.out.println("x")); // L1
        Future<?> f = ex.execute(() -> System.out.println("y")); // L2
        Future<String> g = ex.submit(() -> "z"); // L3
        ex.execute(() -> "w"); // L4
        ex.shutdown();
    }
}
```

- A. Chỉ L2

- B. Chỉ L4
- C. L2 và L4
- D. L3 và L4
- E. L1 và L2

Câu 08-14 · Vừa · objective 8.1

Chương trình sau in ra gì?

```
import java.util.concurrent.*;

public class Shut {
    public static void main(String[] args) throws Exception {
        ExecutorService ex = Executors.newSingleThreadExecutor();
        ex.submit(() -> System.out.print("A ")).get();
        ex.shutdown();
        try {
            ex.submit(() -> System.out.print("B "));
        } catch (RejectedExecutionException e) {
            System.out.print("R ");
        }
        ex.awaitTermination(1, TimeUnit.SECONDS);
        System.out.print(ex.isTerminated());
    }
}
```

- A. A B true
- B. A R true
- C. A R false
- D. R A true

Câu 08-15 · Khó · objective 8.1

Chèn dòng nào vào chỗ `// INSERT CODE HERE` thì chương trình biên dịch và in ra 4? (Chọn 3 đáp án.)

```
import java.util.*;
import java.util.concurrent.*;

public class Wait {
    public static void main(String[] args) throws Exception {
        ExecutorService ex = Executors.newFixedThreadPool(2);
        Callable<Integer> task = () -> 2 + 2;
        // INSERT CODE HERE
        ex.shutdown();
    }
}
```

- A. `System.out.println(ex.submit(task).get());`
- B. `System.out.println(ex.submit(task));`
- C. `System.out.println(ex.invokeAll(List.of(task)).get(0).get());`
- D. `System.out.println(ex.invokeAny(List.of(task)));`
- E. `ex.execute(task);`

Câu 08-16 · Vừa · objective 8.2, 8.3

Chương trình sau in ra gì?

```
import java.util.*;
import java.util.concurrent.*;

public class Cow {
    public static void main(String[] args) {
        List<Integer> list = new CopyOnWriteArrayList<>(List.of(1, 2, 3));
        int count = 0;
        for (Integer i : list) {
            list.add(i * 10);
            count++;
        }
        System.out.println(count + " " + list.size());
    }
}
```

- A. 6 6
- B. Ném `ConcurrentModificationException`
- C. 3 6
- D. Vòng lặp chạy mãi

Câu 08-17 · Khó · objective 8.3, 6.2

Điền thao tác nào vào cả hai chỗ ____ thì kết quả tuần tự và song song giống nhau (in ra same)? (Chọn 3 đáp án.)

```
import java.util.stream.*;

public class Assoc {
    public static void main(String[] args) {
        int seq = IntStream.rangeClosed(1, 5).____;
        int par = IntStream.rangeClosed(1, 5).parallel().____;
        System.out.println(seq == par ? "same" : "different");
    }
}
```

- A. `reduce(0, Integer::sum)`
- B. `reduce(1, (a, b) -> a * b)`
- C. `reduce(0, (a, b) -> a - b)`
- D. `reduce(5, Integer::sum)`
- E. `reduce(Integer.MIN_VALUE, Integer::max)`

Câu 08-18 · Vừa · objective 8.1

Chương trình sau in ra gì?

```
public class Builders {
    public static void main(String[] args) {
        Thread v = Thread.ofVirtual().name("v-", 1).unstarted(() -> { });
        Thread p = Thread.ofPlatform().name("p").unstarted(() -> { });
        System.out.println(v.getName() + " " + v.isVirtual() + " " + p.isVirtual() + " " + v.getState());
    }
}
```

- A. v- true false NEW
- B. v-1 true false NEW
- C. v-1 true false RUNNABLE
- D. v-1 false false NEW

Câu 08-19 · Dễ · objective 8.1

Chương trình sau in ra gì?

```
public class JoinOrder {
    public static void main(String[] args) throws InterruptedException {
        StringBuilder sb = new StringBuilder();
        Thread t = new Thread(() -> sb.append("T"));
        sb.append("A");
        t.start();
        t.join();
        sb.append("B");
        System.out.println(sb);
    }
}
```

- A. ATB
- B. TAB
- C. AB
- D. Thứ tự không xác định

Câu 08-20 · Vừa · objective 8.2

Chương trình sau in ra gì?

```
import java.util.concurrent.*;

public class Queue {
    public static void main(String[] args) {
        BlockingQueue<String> q = new ArrayBlockingQueue<>(2);
        System.out.print(q.offer("a") + " " + q.offer("b") + " " + q.offer("c") + " ");
        System.out.print(q.poll() + " " + q.peek() + " " + q.size());
    }
}
```

- A. true true true a b 2
- B. true true false a a 1
- C. true true false b b 1

- **D.** `true true false a b 1`

Lời giải

Câu 08-01 — Đáp án: A

- **Vì sao đúng:** `run()` chỉ là một lời gọi method bình thường, chạy trên thread hiện tại (`main`). `start()` mới tạo thread mới và thread đó gọi `run()` → tên `worker`. `join()` đợi thread `worker` in xong.
- **B sai:** `t.run()` không tạo thread mới; nó chạy trên `main`.
- **C sai:** `t.start()` chạy lambda trên thread mới tên `worker`.
- **D sai:** `t.run()` được gọi trước và chạy đồng bộ trên `main`.
- *Kiểm chứng:* `examples/questions/ch08/Q08_01/` — output confirmed (`python3 tools/book.py questions ch08`).

Câu 08-02 — Đáp án: D

- **Vì sao đúng:** Exception xảy ra **bên trong task** không bay thẳng ra thread gọi. `Future.get()` gói nó vào `ExecutionException`; exception gốc lấy bằng `getCause()`.
- **A sai:** Exception trong task được gói lại, `main` nhận `ExecutionException`.
- **B sai:** `getCause()` trả về exception gốc (`NumberFormatException`), không phải chính `ExecutionException`.
- **C sai:** Task thất bại nên `get()` ném exception, không trả về `null`.
- *Kiểm chứng:* `examples/questions/ch08/Q08_02/` — output confirmed (`python3 tools/book.py questions ch08`).

Câu 08-03 — Đáp án: B

- **Vì sao đúng:** `Runnable.run()` không khai báo `throws`, nên lambda `Runnable` không được ném checked exception như `InterruptedException` của `Thread.sleep` (L4). Lambda `() -> { return; }` không trả giá trị nên chỉ khớp `Runnable`; `submit(Runnable)` trả về `Future<?>`, không gán được cho `Future<Integer>` (L5).
- **A sai:** L5 cũng lỗi: lambda không trả giá trị → `submit(Runnable)` → `Future<?>`.
- **C sai:** `Callable.call()` khai báo `throws Exception`, nên L3 được phép gọi `Thread.sleep`.
- **D sai:** L3 hợp lệ (xem C).
- **E sai:** L4 cũng lỗi vì checked exception trong `Runnable`.
- *Kiểm chứng:* `examples/questions/ch08/Q08_03/` — compile error confirmed at `['L4', 'L5']` (`python3 tools/book.py questions ch08`).

Câu 08-04 — Đáp án: C

- **Vì sao đúng:** Thread vừa tạo (chưa `start()`) ở trạng thái `NEW`. Sau `join()`, thread chắc chắn đã chạy xong → `TERMINATED` và `isAlive()` là `false`.
- **A sai:** `join()` đợi thread kết thúc, nên không còn `RUNNABLE`.
- **B sai:** Không có trạng thái `CREATED` trong `Thread.State`; trạng thái đầu là `NEW`.
- **D sai:** Không có trạng thái `DEAD`; trạng thái cuối là `TERMINATED`.
- *Kiểm chứng:* `examples/questions/ch08/Q08_04/` — output confirmed (`python3 tools/book.py questions ch08`).

Câu 08-05 — Đáp án: A, B

- **Vì sao đúng:** A: virtual thread luôn là daemon (JVM không đợi chúng khi thoát). B: `start(...)` của `Thread.Builder` tạo và start thread luôn (khác `unstarted(...)`).
- **C sai:** Constructor `Thread` tạo **platform thread**; virtual thread tạo qua `Thread.ofVirtual()`, `Thread.startVirtualThread` hoặc `executor`.
- **D sai:** Đổi virtual thread thành non-daemon → `IllegalArgumentException`.
- **E sai:** Executor này tạo **một virtual thread mới cho mỗi task**, không giới hạn theo số CPU (ví dụ: 200 task cùng chờ nhau vẫn chạy xong).
- *Kiểm chứng:* `examples/questions/ch08/Q08_05/` — each option proven true/false by a program (`python3 tools/book.py questions ch08`).

Câu 08-06 — Đáp án: C

- **Vì sao đúng:** `invokeAll` đợi **tất cả** task xong và trả về list `Future` theo **đúng thứ tự của list task**, không theo thứ tự hoàn thành.
- **A sai:** Đây là thứ tự hoàn thành; `invokeAll` giữ thứ tự của danh sách đầu vào.
- **B sai:** Như A — thứ tự kết quả là thứ tự task.
- **D sai:** Thứ tự list kết quả được bảo đảm khớp với list task.
- *Kiểm chứng:* `examples/questions/ch08/Q08_06/` — output confirmed (`python3 tools/book.py questions ch08`).

Câu 08-07 — Đáp án: D

- **Vì sao đúng:** `getAndAdd(5)` trả về giá trị **cũ** (10), `a = 15`. `incrementAndGet()` trả về giá trị **mới** (16). `compareAndSet(15, 0)` thất bại vì giá trị hiện tại là 16 → `false`, không đổi. `updateAndGet(v * 2)` → 32.
- **A sai:** `getAndXxx` trả về giá trị trước khi đổi; và `compareAndSet` thất bại vì `a` đang là 16.

- **B sai:** `compareAndSet` chỉ thành công khi giá trị hiện tại **bằng đúng** giá trị mong đợi (15), mà a là 16.
- **C sai:** `getAndAdd` trả về 10 (giá trị cũ), và `incrementAndGet` chỉ tăng một lần.
- *Kiểm chứng:* `examples/questions/ch08/Q08_07/` — output confirmed (python3 `tools/book.py questions ch08`).

Câu 08-08 — Đáp án: B

- **Vì sao đúng:** Khối `synchronized (x)` cần x là **tham chiếu object**; `count` là `int` → L3 lỗi. Constructor không được khai báo `synchronized` → L5 lỗi. Method instance và static đều có thể `synchronized` (khóa trên `this` hoặc trên `Sync.class`).
- **A sai:** L5 cũng lỗi: constructor không dùng được `synchronized`.
- **C sai:** static `synchronized` hợp lệ (khóa trên object `Class`).
- **D sai:** Method `synchronized (L1)` hợp lệ.
- **E sai:** L3 cũng lỗi vì primitive không làm khóa được.
- *Kiểm chứng:* `examples/questions/ch08/Q08_08/` — compile error confirmed at ['L3', 'L5'] (python3 `tools/book.py questions ch08`).

Câu 08-09 — Đáp án: A

- **Vì sao đúng:** `ReentrantLock` cho phép cùng một thread `lock()` nhiều lần; mỗi lần tăng `hold count`. Phải `unlock()` đúng số lần mới thật sự mở khóa. `unlock()` khi không giữ khóa → `IllegalMonitorStateException`.
- **B sai:** Sau lần `unlock()` đầu, `hold count` còn 1 nên khóa vẫn đang bị giữ.
- **C sai:** Khóa được lấy hai lần nên `hold count` là 2.
- **D sai:** `unlock()` thêm lần nữa khi không giữ khóa sẽ ném exception.
- *Kiểm chứng:* `examples/questions/ch08/Q08_09/` — output confirmed (python3 `tools/book.py questions ch08`).

Câu 08-10 — Đáp án: A, B

- **Vì sao đúng:** A: `ConcurrentHashMap` cấm cả key lẫn value `null`. B: `CopyOnWriteArrayList` tạo bản sao mảng mỗi lần ghi; iterator duyệt ảnh chụp cũ nên không ném exception.
- **C sai:** `unlock()` khi không giữ khóa ném `IllegalMonitorStateException`.
- **D sai:** `getAndIncrement()` trả về giá trị **cũ** (5); `incrementAndGet()` mới trả về giá trị mới.
- **E sai:** offer trả về `false` ngay khi đầy; put mới là method chặn.
- *Kiểm chứng:* `examples/questions/ch08/Q08_10/` — each option proven true/false by a program (python3 `tools/book.py questions ch08`).

Câu 08-11 — Đáp án: A

- **Vì sao đúng:** `forEachOrdered` xử lý phần tử theo **thứ tự gặp (encounter order)** của nguồn có thứ tự (`List`), kể cả khi stream song song, và không gọi action đồng thời. Vì vậy output luôn giống nhau.
- **B sai:** Đó là hành vi của `forEach` trên parallel stream.
- **C sai:** Không có collection nào bị sửa khi đang duyệt.
- **D sai:** `sb` là effectively final (không bị gán lại); gọi method trên object là được phép.
- *Kiểm chứng:* `examples/questions/ch08/Q08_11/` — output confirmed (python3 `tools/book.py questions ch08`).

Câu 08-12 — Đáp án: D

- **Vì sao đúng:** `countDown()` khi count đã về 0 thì không làm gì (không âm). `await()` trả về ngay khi count là 0.
- **A sai:** Count không bao giờ âm.
- **B sai:** Gọi `countDown()` thừa không ném exception.
- **C sai:** Count đã về 0 nên `await()` không chờ.
- *Kiểm chứng:* `examples/questions/ch08/Q08_12/` — output confirmed (python3 `tools/book.py questions ch08`).

Câu 08-13 — Đáp án: C

- **Vì sao đúng:** `execute(Runnable)` (của `Executor`) trả về `void` → không gán cho `Future` được (L2). `execute` chỉ nhận `Runnable`; `lambda () -> "w"` trả về giá trị và "w" không phải một câu lệnh hợp lệ cho `void` → không khớp `Runnable` (L4). `submit` có bản nhận `Callable` nên L3 hợp lệ.
- **A sai:** L4 cũng lỗi: `execute` không nhận `Callable`.
- **B sai:** L2 cũng lỗi: `execute` trả về `void`.
- **D sai:** `submit(Callable<String>)` trả về `Future<String>` → L3 hợp lệ.
- **E sai:** L1 hợp lệ: `execute` nhận `Runnable`.
- *Kiểm chứng:* `examples/questions/ch08/Q08_13/` — compile error confirmed at ['L2', 'L4'] (python3 `tools/book.py questions ch08`).

Câu 08-14 — Đáp án: B

- **Vì sao đúng:** Task đầu chạy xong (`get()` đợi). Sau `shutdown()`, executor **từ chối** task mới → `RejectedExecutionException`. Không còn task nào nên `awaitTermination` kết thúc và `isTerminated()` là `true`.
- **A sai:** Sau `shutdown()`, không nhận task mới.
- **C sai:** Không còn task đang chạy, nên executor kết thúc (`terminated`) ngay.

- **D sai:** `get()` đợi task đầu in A trước khi main tiếp tục.
- *Kiểm chứng:* `examples/questions/ch08/Q08_14/` — output confirmed (python3 tools/book.py questions ch08).

Câu 08-15 — Đáp án: A, C, D

- **Vì sao đúng:** A: `submit` trả về `Future`, `get()` đợi và lấy kết quả. C: `invokeAll` trả về list `Future`. D: `invokeAny` trả về **trực tiếp** kết quả của một task hoàn thành thành công.
- **B sai:** In ra `toString()` của `Future` (dạng `java.util.concurrent.FutureTask@...`), không phải 4.
- **E sai:** `execute` chỉ nhận `Runnable`, không nhận `Callable` → lỗi biên dịch.
- *Kiểm chứng:* `examples/questions/ch08/Q08_15/` — variants: ACD satisfy output (python3 tools/book.py questions ch08).

Câu 08-16 — Đáp án: C

- **Vì sao đúng:** Iterator của `CopyOnWriteArrayList` duyệt **ảnh chụp** tại lúc tạo iterator (3 phần tử). Các phần tử thêm sau không được duyệt, nhưng vẫn nằm trong list → size 6.
- **A sai:** Vòng lặp chỉ thấy 3 phần tử của ảnh chụp ban đầu.
- **B sai:** `CopyOnWriteArrayList` không ném exception này.
- **D sai:** Phần tử mới không được đưa vào vòng lặp đang chạy, nên vòng lặp kết thúc.
- *Kiểm chứng:* `examples/questions/ch08/Q08_16/` — output confirmed (python3 tools/book.py questions ch08).

Câu 08-17 — Đáp án: A, B, E

- **Vì sao đúng:** `reduce` song song đúng khi identity là phần tử trung hoà thật và hàm có tính **kết hợp (associative)**. Cộng với 0, nhân với 1, max với `Integer.MIN_VALUE` đều thoả.
- **C sai:** Phép trừ không kết hợp: $(a - b) - c \neq a - (b - c)$. Kết quả song song phụ thuộc cách chia (trên máy kiểm tra: khác tuần tự).
- **D sai:** 5 không phải identity của phép cộng; mỗi phần nhỏ đều cộng thêm 5 nên kết quả song song lớn hơn.
- *Kiểm chứng:* `examples/questions/ch08/Q08_17/` — variants: ABE satisfy output (python3 tools/book.py questions ch08).

Câu 08-18 — Đáp án: B

- **Vì sao đúng:** `name(prefix, start)` đặt tên = prefix + số đếm (bắt đầu từ start) → v-1. `unstarted` tạo thread nhưng chưa chạy → `NEW`. `ofVirtual` tạo virtual thread, `ofPlatform` tạo platform thread.
- **A sai:** `name("v-", 1)` nối thêm số đếm vào tên.
- **C sai:** `unstarted` không start thread.
- **D sai:** `Thread.ofVirtual()` tạo virtual thread → `isVirtual()` là true.
- *Kiểm chứng:* `examples/questions/ch08/Q08_18/` — output confirmed (python3 tools/book.py questions ch08).

Câu 08-19 — Đáp án: A

- **Vì sao đúng:** A được thêm trước khi start thread. `join()` làm main đợi thread t kết thúc (và bảo đảm thấy được thay đổi của nó), rồi mới thêm B. Vì vậy luôn là ATB.
- **B sai:** Thread t chỉ bắt đầu sau khi A đã được thêm.
- **C sai:** `join()` đợi thread t chạy xong, nên T chắc chắn đã được thêm.
- **D sai:** `start()` sau A và `join()` trước B làm thứ tự được xác định.
- *Kiểm chứng:* `examples/questions/ch08/Q08_19/` — output confirmed (python3 tools/book.py questions ch08).

Câu 08-20 — Đáp án: D

- **Vì sao đúng:** Sức chứa là 2: `offer("c")` trả về false (không chặn). `poll()` lấy và xoá phần tử đầu (a). `peek()` chỉ xem phần tử đầu mới (b). Còn lại 1 phần tử.
- **A sai:** Hàng đợi đầy sau 2 phần tử nên `offer("c")` thất bại.
- **B sai:** `poll()` đã xoá a, nên `peek()` thấy b.
- **C sai:** Queue là FIFO: `poll()` lấy a (vào trước).
- *Kiểm chứng:* `examples/questions/ch08/Q08_20/` — output confirmed (python3 tools/book.py questions ch08).

Đọc thêm (link chính thức — bị chặn trong sandbox nên mình chưa mở được)

- JEP 444 Virtual Threads: <https://openjdk.org/jeps/444>
- Javadoc gói `java.util.concurrent`: <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/concurrent/package-summary.html>
- JLS Chương 17 Threads and Locks: <https://docs.oracle.com/javase/specs/jls/se21/html/jls-17.html>

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-28:

- `Thread.java` (virtual thread luôn daemon, `Thread.Builder`):
<https://raw.githubusercontent.com/openjdk/jdk21u/master/src/java.base/share/classes/java/lang/Thread.java>
- `ExecutorService.java` (`close()`, `shutdownNow`, `invokeAny`):
<https://raw.githubusercontent.com/openjdk/jdk21u/master/src/java.base/share/classes/java/util/concurrent/ExecutorService.java>
- `ReentrantLock.java`:
<https://raw.githubusercontent.com/openjdk/jdk21u/master/src/java.base/share/classes/java/util/concurrent/locks/ReentrantLock.java>
- `CountDownLatch.java`:
<https://raw.githubusercontent.com/openjdk/jdk21u/master/src/java.base/share/classes/java/util/concurrent/CountDownLatch.java>

Chương 9 — I/O và NIO.2 (Using Java I/O API)

Mục tiêu

- 9.1 Đọc và ghi dữ liệu console và file bằng I/O stream.
- 9.2 Serialize (tuần tự hoá) và deserialize object.
- 9.3 Tạo, duyệt, tạo mới, đọc, ghi `Path` và thuộc tính của nó bằng API `java.nio.file` (NIO.2).

Giải thích đơn giản

Java có hai “thể hệ” API cho file:

1. **java.io** (cũ): các **stream** — dòng dữ liệu một chiều. Có hai họ:
 - **Byte stream** (`InputStream/OutputStream`) cho dữ liệu nhị phân: ảnh, file .ser...
 - **Character stream** (`Reader/Writer`) cho văn bản, có **encoding** (UTF-8...).
2. **java.nio.file** (NIO.2, Java 7+): `Path` (một đường dẫn) và `Files` (các method static để thao tác file: đọc, ghi, copy, move, delete, duyệt thư mục). Hiện đại và dễ dùng hơn.

Chú ý: “stream” của I/O **không liên quan** tới Stream API của chương 6 (dù `Files.lines` trả về một `Stream<String>`).

Serialization là biến một object thành chuỗi byte (để lưu file hoặc gửi mạng) và ngược lại.

Ví dụ

Code trong `examples/ch09/`. Chạy lại: `python3 tools/book.py examples ch09`. Mỗi ví dụ chạy trong một thư mục tạm (file được tạo bằng đường dẫn tương đối). Output thật, JDK 21.0.10.

1. Byte stream

`examples/ch09/Ex01_ByteStreams.java`

```
// objective: 9.1
// Byte stream: FileOutputStream / FileInputStream (+ Buffered...). read() trả về -1 khi hết dữ liệu.
import java.io.*;

public class Ex01_ByteStreams {
    public static void main(String[] args) throws IOException {
        File f = new File("data.bin");
        try (OutputStream out = new BufferedOutputStream(new FileOutputStream(f))) {
            out.write(65); // ghi một byte (chỉ 8 bit thấp)
            out.write(new byte[]{66, 67, 68});
            out.write(321); // 321 = 256 + 65 → ghi byte 65
        }
        System.out.println("size = " + f.length() + " bytes");

        try (InputStream in = new BufferedInputStream(new FileInputStream(f))) {
            int b;
            StringBuilder sb = new StringBuilder();
            while ((b = in.read()) != -1) sb.append((char) b).append(' ');
            System.out.println("read: " + sb.toString().trim());
        }

        try (InputStream in = new FileInputStream(f)) {
            byte[] buf = new byte[3];
            int n = in.read(buf); // đọc tối đa 3 byte, trả về số byte đọc được
            System.out.println("read(buf) = " + n + " → " + new String(buf, 0, n));
            System.out.println("skip(1) = " + in.skip(1) + ", next = " + (char) in.read() + ", then = " + in.read());
        }

        try (FileOutputStream append = new FileOutputStream(f, true)) { // true = ghi nối (append)
            append.write('Z');
        }
        try (InputStream in = new FileInputStream(f)) {
            System.out.println("after append: " + new String(in.readAllBytes()));
        }
    }
}
```

Output thật (JDK 21.0.10):

```
size = 5 bytes
read: A B C D A
read(buf) = 3 → ABC
```

```
skip(1) = 1, next = A, then = -1  
after append: ABCDAZ
```

2. Character stream

examples/ch09/Ex02_CharStreams.java

```
// objective: 9.1  
// Character stream: Writer/Reader cho văn bản (có encoding). BufferedReader.readLine() trả về null khi hết.  
import java.io.*;  
import java.nio.charset.StandardCharsets;  
  
public class Ex02_CharStreams {  
    public static void main(String[] args) throws IOException {  
        try (BufferedWriter w = new BufferedWriter(new FileWriter("notes.txt", StandardCharsets.UTF_8))) {  
            w.write("Xin chào");  
            w.newLine();  
            w.write("Java I/O");  
        }  
        try (PrintWriter pw = new PrintWriter(new FileWriter("notes.txt", StandardCharsets.UTF_8, true))) {  
            pw.println();  
            pw.printf("%s=%d\n", "total", 3);          // PrintWriter không ném IOException; dùng checkError()  
        }  
        try (BufferedReader r = new BufferedReader(new FileReader("notes.txt", StandardCharsets.UTF_8))) {  
            String line;  
            int n = 0;  
            while ((line = r.readLine()) != null) System.out.println(++n + ": " + line);  
        }  
        try (Reader r = new FileReader("notes.txt", StandardCharsets.UTF_8)) {  
            char[] buf = new char[8];  
            int n = r.read(buf);  
            System.out.println("first " + n + " chars: " + new String(buf, 0, n));  
        }  
        System.out.println("bytes on disk: " + new File("notes.txt").length() + " (tiếng Việt có dấu tốn nhiều byte hơn trong UTF-8)");  
        StringWriter sw = new StringWriter();  
        try (PrintWriter pw = new PrintWriter(sw)) { pw.print("in memory"); }  
        System.out.println(sw);  
    }  
}
```

Output thật (JDK 21.0.10):

```
1: Xin chào  
2: Java I/O  
3: total=3  
first 8 chars: Xin chào  
bytes on disk: 27 (tiếng Việt có dấu tốn nhiều byte hơn trong UTF-8)  
in memory
```

3. Console

examples/ch09/Ex03_Console.java

```
// objective: 9.1  
// Console: System.console() có thể là null (không có terminal); System.in/out/err là các stream chuẩn.  
import java.io.*;  
  
public class Ex03_Console {  
    public static void main(String[] args) throws IOException {  
        Console c = System.console();  
        if (c == null) {  
            System.out.println("System.console() == null: chương trình không chạy trong terminal tương tác");  
        } else {  
            c.printf("Hello %s\n", "console");  
        }  
        PrintStream out = System.out;  
        out.printf("%-6s|%5.2f|%03d\n", "ab", 3.14159, 7);  
        out.format("%s %b %c\n", "format", true, 'x');  
  
        // Đọc dữ liệu "console" từ một nguồn giả lập thay cho System.in  
        InputStream fakeIn = new ByteArrayInputStream("Nobin\n30\n".getBytes());  
        try (BufferedReader in = new BufferedReader(new InputStreamReader(fakeIn))) {  
            String name = in.readLine();  
            int age = Integer.parseInt(in.readLine());  
            System.out.println("name=" + name + ", age+1=" + (age + 1) + ", next=" + in.readLine());  
        }  
        System.err.flush();  
    }  
}
```

Output thật (JDK 21.0.10):

```
System.console() == null: chương trình không chạy trong terminal tương tác
ab | 3.14|007
format true x
name=Nobin, age+1=31, next=null
```

4. Serialization

examples/ch09/Ex04_Serialization.java

```
// objective: 9.2
// Serialization: Serializable, transient, static, serialVersionUID; constructor nào chạy khi deserialize?
import java.io.*;

public class Ex04_Serialization {
    static class Base { // KHÔNG Serializable
        String baseName = "unset";
        Base() { System.out.println(" Base() constructor"); baseName = "from Base()"; }
    }

    static class User extends Base implements Serializable {
        private static final long serialVersionUID = 1L;
        static int instances = 0; // static: không được serialize
        String name;
        transient String password; // transient: bỏ qua
        int age = 18;
        User(String name, String password) {
            System.out.println(" User(...) constructor");
            this.name = name; this.password = password; this.baseName = "set by User";
            instances++;
        }
        { System.out.println(" instance initializer của User"); }
    }

    record Point(int x, int y) implements Serializable {
        Point { System.out.println(" Point canonical constructor"); }
    }

    public static void main(String[] args) throws Exception {
        System.out.println("serialize:");
        User u = new User("nobin", "secret");
        u.age = 30;
        try (ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream("user.ser"))) {
            out.writeObject(u);
            out.writeObject(new Point(1, 2));
        }
        User.instances = 99;
        System.out.println("deserialize:");
        try (ObjectInputStream in = new ObjectInputStream(new FileInputStream("user.ser"))) {
            User back = (User) in.readObject();
            Point p = (Point) in.readObject();
            System.out.println("name=" + back.name + " password=" + back.password + " age=" + back.age
                + " baseName=" + back.baseName + " instances=" + User.instances + " | " + p);
        }
        try (ObjectOutputStream out = new ObjectOutputStream(new ByteArrayOutputStream())) {
            out.writeObject(new Base());
        } catch (NotSerializableException e) {
            System.out.println("NotSerializableException: " + e.getMessage());
        }
    }
}
```

Output thật (JDK 21.0.10):

```
serialize:
Base() constructor
instance initializer của User
User(...) constructor
Point canonical constructor
deserialize:
Base() constructor
Point canonical constructor
name=nobin password=null age=30 baseName=from Base() instances=99 | Point[x=1, y=2]
Base() constructor
NotSerializableException: Ex04_Serialization$Base
```

5. Path

examples/ch09/Ex05_PathBasics.java

```
// objective: 9.3
// Path: tạo, lấy thành phần, resolve, relativize, normalize (chỉ thao tác chuỗi, không cần file tồn tại).
import java.nio.file.*;
```

```

public class Ex05_PathBasics {
    public static void main(String[] args) {
        Path p = Path.of("/home/nobin/projects/app/src/Main.java");
        System.out.println(p.getFileName() + " | " + p.getParent() + " | " + p.getRoot() + " | " + p.getNameCount());
        System.out.println(p.getName(0) + " | " + p.subpath(1, 3) + " | " + p.isAbsolute() + " | " + Path.of("a/b").isAbsolute());
        System.out.println(p.startsWith("/home") + " | " + p.startsWith("home") + " | " + p.endsWith("Main.java") + " | " +
p.endsWith(".java"));

        Path base = Path.of("/data/reports");
        System.out.println(base.resolve("2024/jan.csv") + " | " + base.resolve("/tmp/x") + " | " + base.resolveSibling("logs"));
        Path a = Path.of("/data/reports/2024");
        Path b = Path.of("/data/images/logo.png");
        System.out.println(a.relativeize(b) + " | " + b.relativeize(a));
        System.out.println(Path.of("a/./b/./c/./d").normalize() + " | " + Path.of(".././x").normalize() + " | " +
Path.of("a/b/../../").normalize());
        System.out.println(Path.of("docs", "java", "ocp.md") + " | " + Paths.get("x", "y") + " | " + Path.of("") .getNameCount());
        try {
            Path.of("rel/path").relativeize(Path.of("/abs/path"));
        } catch (IllegalArgumentException e) {
            System.out.println("IllegalArgumentException: không relativize giữa path tuyệt đối và tương đối");
        }
        for (Path part : Path.of("x/y/z")) System.out.print "[" + part + " ]";
        System.out.println();
    }
}

```

Output thật (JDK 21.0.10):

```

Main.java | /home/nobin/projects/app/src | / | 6
home | nobin/projects | true | false
true false true false
/data/reports/2024/jan.csv | /tmp/x | /data/logs
.././images/logo.png | ../../reports/2024
a/c/d | .././x | ..
docs/java/ocp.md | x/y | 1
IllegalArgumentException: không relativize giữa path tuyệt đối và tương đối
[x][y][z]

```

6. Files: tạo, đọc, ghi, copy, move, delete

examples/ch09/Ex06_FilesOps.java

```

// objective: 9.3
// Files: tạo thư mục, đọc/ghi, copy, move, delete và các exception thường gặp.
import java.io.IOException;
import java.nio.file.*;
import java.util.List;

public class Ex06_FilesOps {
    public static void main(String[] args) throws IOException {
        Path dir = Path.of("work/in");
        Files.createDirectories(dir); // tạo cả thư mục cha, không lỗi nếu đã có
        Path file = dir.resolve("todo.txt");
        Files.writeString(file, "buy milk\nlearn java\n");
        Files.write(file, List.of("sleep"), StandardOpenOption.APPEND);
        System.out.println(Files.readAllLines(file) + " size=" + Files.size(file));
        System.out.println("exists? " + Files.exists(file) + ", notExists(x)? " + Files.notExists(dir.resolve("x")));

        Path copy = Path.of("work/copy.txt");
        Files.copy(file, copy);
        try {
            Files.copy(file, copy); // đích đã tồn tại
        } catch (FileAlreadyExistsException e) {
            System.out.println("FileAlreadyExistsException: " + e.getMessage());
        }
        Files.copy(file, copy, StandardCopyOption.REPLACE_EXISTING);

        Path moved = Files.move(copy, Path.of("work/moved.txt"));
        System.out.println("moved: " + moved + ", copy exists? " + Files.exists(copy));
        try {
            Files.createDirectory(Path.of("work/a/b")); // cha "work/a" chưa có
        } catch (NoSuchFileException e) {
            System.out.println("NoSuchFileException: " + e.getMessage());
        }
        try {
            Files.delete(Path.of("work")); // thư mục không rỗng
        } catch (DirectoryNotEmptyException e) {
            System.out.println("DirectoryNotEmptyException: " + e.getMessage());
        }
        System.out.println("deleteIfExists(nothing) = " + Files.deleteIfExists(Path.of("work/nothing")));
        Files.delete(moved);
        System.out.println("readString: " + Files.readString(file).lines().count() + " lines");
    }
}

```

Output thật (JDK 21.0.10):

```
[buy milk, learn java, sleep] size=26
exists? true, notExists(x)? true
FileAlreadyExistsException: work/copy.txt
moved: work/moved.txt, copy exists? false
NoSuchFileException: work/a/b
DirectoryNotEmptyException: work
deleteIfExists(nothing) = false
readString: 3 lines
```

7. Thuộc tính file

examples/ch09/Ex07_Attributes.java

```
// objective: 9.3
// Thuộc tính file: isDirectory, isRegularFile, size, BasicFileAttributes, isSameFile, symbolic link.
import java.io.IOException;
import java.nio.file.*;
import java.nio.file.attribute.BasicFileAttributes;
import java.nio.file.attribute.FileTime;

public class Ex07_Attributes {
    public static void main(String[] args) throws IOException {
        Path dir = Files.createDirectories(Path.of("attr"));
        Path f = Files.writeString(dir.resolve("a.txt"), "12345");
        System.out.println(Files.isDirectory(dir) + " " + Files.isRegularFile(f) + " " + Files.isRegularFile(dir)
            + " " + Files.size(f) + " " + Files.isReadable(f) + " " + Files.isHidden(Path.of(".hidden")));

        BasicFileAttributes attrs = Files.readAttributes(f, BasicFileAttributes.class);
        System.out.println("attrs: dir=" + attrs.isDirectory() + " file=" + attrs.isRegularFile() + " size=" + attrs.size()
            + " link=" + attrs.isSymbolicLink());

        FileTime t = FileTime.fromMillis(0);
        Files.setLastModifiedTime(f, t);
        System.out.println("lastModified = " + Files.getLastModifiedTime(f));

        Path link = dir.resolve("link.txt");
        Files.createSymbolicLink(link, f.getFileName()); // link tương đối trỏ tới a.txt
        System.out.println("isSymbolicLink=" + Files.isSymbolicLink(link) + " isRegularFile(link)=" + Files.isRegularFile(link)
            + " noFollow=" + Files.isRegularFile(link, LinkOption.NOFOLLOW_LINKS) + " sameFile=" + Files.isSameFile(link, f)
            + " target=" + Files.readSymbolicLink(link));
        System.out.println("Path.of(\"attr/../attr/a.txt\") same as f? " + Files.isSameFile(Path.of("attr/../attr/a.txt"), f)
            + ", equals? " + Path.of("attr/../attr/a.txt").equals(f));
    }
}
```

Output thật (JDK 21.0.10):

```
true true false 5 true true
attrs: dir=false file=true size=5 link=false
lastModified = 1970-01-01T00:00:00Z
isSymbolicLink=true isRegularFile(link)=true noFollow=false sameFile=true target=a.txt
Path.of("attr/../attr/a.txt") same as f? true, equals? false
```

8. Duyệt thư mục: list, walk, find, lines

examples/ch09/Ex08_WalkFind.java

```
// objective: 9.3
// Duyệt cây thư mục: Files.list (1 cấp), Files.walk (đệ quy, có maxDepth), Files.find (có điều kiện).
import java.io.IOException;
import java.nio.file.*;
import java.util.stream.Stream;

public class Ex08_WalkFind {
    public static void main(String[] args) throws IOException {
        Path root = Path.of("tree");
        Files.createDirectories(root.resolve("src/main"));
        Files.createDirectories(root.resolve("docs"));
        Files.writeString(root.resolve("README.md"), "# tree");
        Files.writeString(root.resolve("src/main/App.java"), "class App {}");
        Files.writeString(root.resolve("src/main/Util.java"), "class Util {}");
        Files.writeString(root.resolve("docs/guide.md"), "guide");

        try (Stream<Path> s = Files.list(root)) { // Stream phải được đóng!
            System.out.println("list : " + s.map(Path::toString).sorted().toList());
        }
        try (Stream<Path> s = Files.walk(root)) {
            System.out.println("walk : " + s.map(Path::toString).sorted().toList());
        }
        try (Stream<Path> s = Files.walk(root, 1)) {
            System.out.println("depth1: " + s.count() + " paths (gồm cả chính root)");
        }
    }
}
```

```

    }
    try (Stream<Path> s = Files.find(root, 10, (p, attr) -> attr.isRegularFile() && p.toString().endsWith(".java"))) {
        System.out.println("find : " + s.map(p -> p.getFileName().toString()).sorted().toList());
    }
    try (Stream<String> lines = Files.lines(root.resolve("README.md"))) {
        System.out.println("lines: " + lines.toList());
    }
}
}

```

Output thật (JDK 21.0.10):

```

list : [tree/README.md, tree/docs, tree/src]
walk : [tree, tree/README.md, tree/docs, tree/docs/guide.md, tree/src, tree/src/main, tree/src/main/App.java,
tree/src/main/Util.java]
depth1: 4 paths (gồm cả chính root)
find : [App.java, Util.java]
lines: [# tree]

```

9. StandardOpenOption

examples/ch09/Ex09_OpenOptions.java

```

// objective: 9.1, 9.3
// Files.newBufferedReader/Reader với StandardOpenOption: CREATE_NEW, APPEND, TRUNCATE_EXISTING.
import java.io.*;
import java.nio.charset.StandardCharsets;
import java.nio.file.*;

public class Ex09_OpenOptions {
    public static void main(String[] args) throws IOException {
        Path p = Path.of("log.txt");
        try (BufferedWriter w = Files.newBufferedWriter(p, StandardCharsets.UTF_8, StandardOpenOption.CREATE_NEW)) {
            w.write("line1");
            w.newLine();
        }
        try (BufferedWriter w = Files.newBufferedWriter(p, StandardOpenOption.APPEND)) {
            w.write("line2");
            w.newLine();
        }
        try (BufferedReader r = Files.newBufferedReader(p)) {
            System.out.println(r.lines().toList());
        }
        try {
            Files.newBufferedWriter(p, StandardOpenOption.CREATE_NEW).close();
        } catch (FileAlreadyExistsException e) {
            System.out.println("CREATE_NEW trên file đã có → FileAlreadyExistsException");
        }
        try (BufferedWriter w = Files.newBufferedWriter(p)) { // mặc định: CREATE + TRUNCATE_EXISTING + WRITE
            w.write("fresh");
        }
        System.out.println(Files.readString(p));
        try {
            Files.newBufferedWriter(Path.of("missing.txt"), StandardOpenOption.APPEND).close();
        } catch (NoSuchFileException e) {
            System.out.println("APPEND không kèm CREATE trên file chưa có → NoSuchFileException");
        }
    }
}

```

Output thật (JDK 21.0.10):

```

[line1, line2]
CREATE_NEW trên file đã có → FileAlreadyExistsException
fresh
APPEND không kèm CREATE trên file chưa có → NoSuchFileException

```

10. DataStream và ObjectOutputStream

examples/ch09/Ex10_DataStreams.java

```

// objective: 9.1, 9.2
// DataOutputStream/DataInputStream (ghi primitive nhị phân) và ObjectOutputStream với nhiều object.
import java.io.*;
import java.util.ArrayList;
import java.util.List;

public class Ex10_DataStreams {
    record Item(String name, int qty) implements Serializable { }

    public static void main(String[] args) throws Exception {
        try (DataOutputStream out = new DataOutputStream(new BufferedOutputStream(new FileOutputStream("nums.dat")))) {

```

```

        out.writeInt(42);
        out.writeDouble(2.5);
        out.writeUTF("xin chào");
        out.writeBoolean(true);
    }
    System.out.println("nums.dat size = " + new File("nums.dat").length());
    try (DataInputStream in = new DataInputStream(new BufferedInputStream(new FileInputStream("nums.dat")))) {
        System.out.println(in.readInt() + " " + in.readDouble() + " " + in.readUTF() + " " + in.readBoolean());
        try {
            in.readInt(); // hết dữ liệu
        } catch (EOFException e) {
            System.out.println("EOFException khi đọc quá cuối file");
        }
    }

    List<Item> cart = new ArrayList<>(List.of(new Item("pen", 2), new Item("book", 1)));
    try (ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream("cart.ser"))) {
        out.writeObject(cart); // ArrayList và Item đều Serializable
    }
    try (ObjectInputStream in = new ObjectInputStream(new FileInputStream("cart.ser"))) {
        @SuppressWarnings("unchecked")
        List<Item> back = (List<Item>) in.readObject();
        System.out.println(back + " equals? " + back.equals(cart) + " same? " + (back == cart));
    }
}

```

Output thật (JDK 21.0.10):

```

nums.dat size = 24
42 2.5 xin chào true
EOFException khi đọc quá cuối file
[Item[name=pen, qty=2], Item[name=book, qty=1]] equals? true same? false

```

11. mark/reset, InputStreamReader, transferTo

examples/ch09/Ex11_ReaderUtilities.java

```

// objective: 9.1
// mark/reset, skip, transferTo, InputStreamReader (cầu nối byte → char).
import java.io.*;
import java.nio.charset.StandardCharsets;

public class Ex11_ReaderUtilities {
    public static void main(String[] args) throws IOException {
        try (BufferedReader r = new BufferedReader(new StringReader("ABCDEFGF"))) {
            System.out.print((char) r.read()); // A
            r.mark(10); // đánh dấu vị trí hiện tại
            System.out.print((char) r.read()); // B
            System.out.print((char) r.read()); // C
            r.reset(); // quay lại chỗ mark
            System.out.print((char) r.read()); // B
            r.skip(2); // bỏ C, D
            System.out.println((char) r.read() + " markSupported=" + r.markSupported()); // E
        }
        byte[] utf8 = "đẹp".getBytes(StandardCharsets.UTF_8);
        System.out.println("bytes=" + utf8.length + " chars=" + new String(utf8, StandardCharsets.UTF_8).length());
        try (Reader r = new InputStreamReader(new ByteArrayInputStream(utf8), StandardCharsets.UTF_8)) {
            System.out.println("first char via InputStreamReader: " + (char) r.read());
        }
        ByteArrayOutputStream sink = new ByteArrayOutputStream();
        long n = new ByteArrayInputStream("copy me".getBytes()).transferTo(sink);
        System.out.println("transferTo copied " + n + " bytes: " + sink);
    }
}

```

Output thật (JDK 21.0.10):

```

ABCBE markSupported=true
bytes=6 chars=3
first char via InputStreamReader: đ
transferTo copied 7 bytes: copy me

```

12. Lỗi biên dịch với Path và Files.lines

examples/ch09/Ex12_IOCompileErrors.java

```

// objective: 9.1, 9.3
// expect: compile-error
// Lỗi hay gặp: Path là interface (không new được); Files.lines trả về Stream<String>, không phải List.
import java.nio.file.*;

```

```
public class Ex12_IOCompileErrors {
    public static void main(String[] args) {
        Path p = new Path("a.txt");
        java.util.List<String> lines = Files.lines(Path.of("a.txt"));
    }
}
```

Output thật của javac (JDK 21.0.10) — cố ý không biên dịch được:

```
Ex12_IOCompileErrors.java:8: error: Path is abstract; cannot be instantiated
    Path p = new Path("a.txt");
                ^
Ex12_IOCompileErrors.java:9: error: incompatible types: Stream<String> cannot be converted to List<String>
    java.util.List<String> lines = Files.lines(Path.of("a.txt"));
                                   ^
2 errors
```

13. IOException là checked exception

examples/ch09/Ex13_CheckedIO.java

```
// objective: 9.1, 9.3
// expect: compile-error
// Hầu hết method của Files và I/O stream ném IOException (checked) → phải catch hoặc khai báo throws.
import java.nio.file.*;

public class Ex13_CheckedIO {
    public static void main(String[] args) {
        String all = Files.readString(Path.of("a.txt"));
        System.out.println(all);
    }
}
```

Output thật của javac (JDK 21.0.10) — cố ý không biên dịch được:

```
Ex13_CheckedIO.java:8: error: unreported exception IOException; must be caught or declared to be thrown
    String all = Files.readString(Path.of("a.txt"));
                                   ^
1 error
```

Đi sâu

Các lớp stream hay gặp

Loại	Đọc	Ghi	Ghi chú
Byte, file	FileInputStream	FileOutputStream(name, append)	read() trả về int 0..255, hoặc -1 khi hết
Byte, đệm	BufferedInputStream	BufferedOutputStream	Bọc (wrap) stream khác
Byte, primitive	DataInputStream	DataOutputStream	readInt/writeInt, readUTF... hết dữ liệu → EOFException
Byte, object	ObjectInputStream	ObjectOutputStream	readObject() ném thêm ClassNotFoundException
Char, file	FileReader	FileWriter(name, append)	Có constructor nhận Charset (Java 11+)
Char, đệm	BufferedReader (readLine() → null khi hết)	BufferedWriter (newLine())	
Char, định dạng	—	PrintWriter (println, printf, format)	Không ném IOException; dùng checkError()
Cầu nối	InputStreamReader (byte → char)	OutputStreamWriter (char → byte)	Chỉ định charset
Bộ nhớ	ByteArrayInputStream, StringReader	ByteArrayOutputStream, StringWriter	Hữu ích để test

Method chung: close() (dùng try-with-resources), flush() (ghi phần đệm ra đích), skip(n), mark(limit) / reset() (kiểm tra markSupported()), readAllBytes(), transferTo(out).

Console: System.in (InputStream), System.out/System.err (PrintStream), System.console() có thể trả về null khi không có terminal; Console có readLine(), readPassword() (trả char[]), printf, writer(), reader().

Serialization (9.2)

1. Lớp phải implements `Serializable` (interface đánh dấu, không có method). Mọi field được lưu cũng phải serialize được (hoặc là `transient`), nếu không → `NotSerializableException`.
2. `transient` và `static` **không** được lưu. Sau deserialize: field `transient` có giá trị mặc định.
3. Khi deserialize: constructor của lớp `Serializable` **không** chạy, khởi tạo field và instance initializer của nó cũng không chạy. Constructor **không tham số** của lớp cha **đầu tiên không Serializable** thì **có** chạy (nó phải truy cập được).
4. `private static final long serialVersionUID` — “phiên bản” của lớp; khác nhau khi đọc → `InvalidClassException`.
5. **Record** được deserialize qua **canonical constructor** (nên validation trong constructor vẫn chạy).
6. Tùy biến: `writeObject/readObject` private, `readResolve`, `writeReplace` (ít gặp trong đề).

Path (9.3)

- Tạo: `Path.of("a", "b")`, `Paths.get(...)`, `file.toPath()`; ngược lại `pathToFile()`.
- Thành phần: `getFileName()`, `getParent()` (null nếu không có), `getRoot()` (null nếu tương đối), `getNameCount()`, `getName(i)` (gốc không tính), `subpath(begin, end)`, `isAbsolute()`, `toAbsolutePath()`, iterator (for-each).
- `resolve(other)`: nối; nếu `other` tuyệt đối → trả về `other`. `resolveSibling`.
- `relativize(other)`: đường đi từ `path` này tới `other`; cả hai phải cùng tuyệt đối hoặc cùng tương đối, nếu không → `IllegalArgumentException`.
- `normalize()`: bỏ `.` và tên `..` (chỉ thao tác chuỗi). `toRealPath()`: hỏi hệ thống file (file phải tồn tại, giải quyết symbolic link).
- `equals/startsWith/endsWith` so sánh theo **thành phần path** (không phải chuỗi con): `p.endsWith(".java")` là `false` với `Main.java`. `Files.isSameFile` mới hỏi hệ thống file.

Files (9.3)

Nhóm	Method
Kiểm tra	<code>exists</code> , <code>notExists</code> , <code>isDirectory</code> , <code>isRegularFile</code> , <code>isSymbolicLink</code> , <code>isReadable</code> , <code>isWritable</code> , <code>isExecutable</code> , <code>isHidden</code> , <code>isSameFile</code>
Tạo	<code>createFile</code> (đã có → <code>FileAlreadyExistsException</code>), <code>createDirectory</code> (cha thiếu → <code>NoSuchFileException</code>), <code>createDirectories</code> , <code>createTempFile</code>
Đọc	<code>readString</code> , <code>readAllLines</code> (→ <code>List</code>), <code>readAllBytes</code> , <code>lines</code> (→ <code>Stream</code> , phải đóng), <code>newBufferedReader</code> , <code>newInputStream</code>
Ghi	<code>writeString</code> , <code>write(path, bytes lines, options...)</code> , <code>newBufferedWriter</code> , <code>newOutputStream</code>
Copy / move / xóa	<code>copy</code> (đích đã có → <code>FileAlreadyExistsException</code> , trừ khi <code>REPLACE_EXISTING</code>), <code>move</code> (<code>ATOMIC_MOVE</code>), <code>delete</code> (không có → <code>NoSuchFileException</code> ; thư mục không rỗng → <code>DirectoryNotEmptyException</code>), <code>deleteIfExists</code> (→ <code>boolean</code>)
Thuộc tính	<code>size</code> , <code>getLastModifiedTime</code> , <code>setLastModifiedTime</code> , <code>readAttributes(path, BasicFileAttributes.class)</code> , <code>getAttribute</code> , <code>getFileAttributeView</code>
Duyệt	<code>list</code> (1 cấp), <code>walk(start[, maxDepth])</code> (gồm cả <code>start</code> , depth-first), <code>find(start, maxDepth, BiPredicate<Path, BasicFileAttributes>)</code>

`StandardOpenOption`: `CREATE`, `CREATE_NEW`, `APPEND`, `TRUNCATE_EXISTING`, `READ`, `WRITE`. Mặc định khi ghi: `CREATE + TRUNCATE_EXISTING + WRITE`. `LinkOption.NOFOLLOW_LINKS` để không đi theo symbolic link. Hầu hết method của `Files` ném `IOException` (checked).

Lỗi và bẫy thường gặp (Exam traps)

1. `read()` trả về `int`; hết dữ liệu là `-1` (byte/char stream) — nhưng `readLine()` trả về `null` và `DataInputStream` ném `EOFException`.
2. `new FileOutputStream(f)` ghi đè; cần `new FileOutputStream(f, true)` để nối.
3. `PrintWriter/PrintStream` không ném `IOException`.
4. `System.console()` có thể là `null`.
5. `transient` và `static` không được serialize; constructor lớp `Serializable` không chạy khi deserialize.
6. `readObject()` ném cả `ClassNotFoundException`.
7. `Path` là interface: không `new Path(...)`.
8. `getName(0)` là phần đầu **sau** gốc; `getNameCount()` không tính gốc.
9. `resolve` với `path` tuyệt đối trả về chính `path` đó; `relativize` trộn tuyệt đối/tương đối → exception.
10. `normalize()` không kiểm tra file tồn tại; `Path.equals` không `normalize`.
11. `Files.walk` gồm cả thư mục bắt đầu; `Files.list` thì không.
12. `Files.lines`, `list`, `walk`, `find` trả về `Stream` phải đóng (try-with-resources).
13. `Files.copy/createFile` trên đích đã có → `FileAlreadyExistsException`.
14. `Files.delete` thư mục không rỗng → `DirectoryNotEmptyException`.

Góc nhìn từ TypeScript

Node.js / TypeScript	Java	Ghi chú
fs.readFileSync(p, "utf8")	Files.readString(p)	Java mặc định UTF-8 (Java 18+)
fs.writeFileSync(p, s)	Files.writeString(p, s)	
fs.appendFileSync	Files.writeString(p, s, StandardOpenOption.APPEND)	
path.join, path.resolve, path.relative, path.normalize	Path.resolve, toAbsolutePath, relativize, normalize	path.resolve("/a", "b") ~ Path.of("/a").resolve("b")
fs.readdirSync(dir, { recursive: true })	Files.walk(dir)	
fs.createReadStream + pipe	InputStream + transferTo(out)	
JSON.stringify(obj)	Java serialization (nhị phân) — hoặc thư viện JSON như Jackson	Serialization của Java khó đọc bằng mắt và có rủi ro bảo mật
Lỗi file: EONENT, EEXIST	NoSuchFileException, FileAlreadyExistsException	Checked exception trong Java

Tóm tắt

- java.io: byte stream vs character stream; bọc bằng Buffered/Data/Object/Print; đóng bằng try-with-resources.
- Hết dữ liệu: -1 (read), null (readLine), EOFException (DataInputStream).
- Serialization: Serializable, transient, static không lưu, constructor lớp cha không Serializable chạy.
- NIO.2: Path (thao tác chuỗi) + Files (thao tác thật trên đĩa, ném IOException).
- Duyệt: list (1 cấp), walk (đệ quy, gồm start), find (có điều kiện); nhớ đóng Stream.

Bài tập (có lời giải)

Mọi đáp án đã được tools/book.py biên dịch và chạy để xác nhận (code: examples/questions/ch09/). Mỗi câu chạy trong thư mục tạm riêng.

Câu hỏi

Câu 09-01 · Dễ · objective 9.3

Chương trình sau in ra gì?

```
import java.nio.file.*;

public class Parts {
    public static void main(String[] args) {
        Path p = Path.of("/var/log/app/server.log");
        System.out.println(p.getFileName() + " " + p.getNameCount() + " " + p.getName(1) + " "
            + p.getParent().getFileName());
    }
}
```

- A. server.log 5 var app
- B. server.log 4 var log
- C. /server.log 4 log app
- D. server.log 4 log app

Câu 09-02 · Vừa · objective 9.3

Chương trình sau in ra gì?

```
import java.nio.file.*;

public class Rel {
    public static void main(String[] args) {
        Path a = Path.of("/a/b/c");
        Path b = Path.of("/a/x");
        System.out.println(a.relativize(b) + " " + a.resolve("../d").normalize() + " " + b.resolve("/y"));
    }
}
```

- A. ../x /a/b/d /a/x/y
- B. ../../x /a/b/c/d /y
- C. ../../x /a/b/d /y

- D. x /a/d /y

Câu 09-03 · Vừa · objective 9.1

Chương trình sau in ra gì?

```
import java.io.*;
import java.nio.file.*;

public class Lines {
    public static void main(String[] args) throws IOException {
        Files.writeString(Path.of("f.txt"), "one\nntwo\n");
        try (BufferedReader r = Files.newBufferedReader(Path.of("f.txt"))) {
            String line;
            int n = 0;
            while ((line = r.readLine()) != null) n++;
            System.out.println(n);
        }
    }
}
```

- A. 2
- B. 3
- C. 4
- D. Ném EOFException

Câu 09-04 · Khó · objective 9.2

Chương trình sau in ra gì?

```
import java.io.*;

public class Ser {
    static class Animal {
        String sound = "?";
        Animal() { sound = "generic"; }
    }

    static class Dog extends Animal implements Serializable {
        String name;
        transient int age;
        Dog(String n, int a) { name = n; age = a; sound = "woof"; }
    }

    public static void main(String[] args) throws Exception {
        try (var out = new ObjectOutputStream(new FileOutputStream("dog.ser"))) {
            out.writeObject(new Dog("Rex", 5));
        }
        try (var in = new ObjectInputStream(new FileInputStream("dog.ser"))) {
            Dog d = (Dog) in.readObject();
            System.out.println(d.name + " " + d.age + " " + d.sound);
        }
    }
}
```

- A. Rex 5 woof
- B. Rex 0 woof
- C. Rex 0 generic
- D. null 0 generic

Câu 09-05 · Vừa · objective 9.1, 9.3

Những dòng nào gây lỗi biên dịch?

```
import java.io.*;
import java.nio.file.*;

public class Io {
    static void a() throws IOException { Files.readString(Path.of("x")); } // L1
    static void b() { Files.exists(Path.of("x")); } // L2
    static void c() { new FileReader("x"); } // L3
    static void d() { Path.of("x").resolve("y"); } // L4
    static void e() { Files.delete(Path.of("x")); } // L5

    public static void main(String[] args) { }
}
```

- A. Chỉ L3
- B. L3 và L5

- C. L2 và L3
- D. L2, L3 và L5
- E. Chỉ L5

Câu 09-06 · Vừa · objective 9.3

Chèn dòng nào vào chỗ `// INSERT CODE HERE` thì chương trình biên dịch được? (Chọn 3 đáp án.)

```
import java.io.File;
import java.nio.file.*;

public class Make {
    public static void main(String[] args) {
        // INSERT CODE HERE
    }
}
```

- A. `Path p = Path.of("a", "b");`
- B. `Path p = Paths.get("a/b");`
- C. `Path p = new Path("a/b");`
- D. `Path p = new File("a/b").toPath();`
- E. `Path p = Path.get("a/b");`

Câu 09-07 · Vừa · objective 9.3

Chương trình sau in ra gì?

```
import java.io.IOException;
import java.nio.file.*;

public class Copy {
    public static void main(String[] args) throws IOException {
        Path src = Files.writeString(Path.of("s.txt"), "S");
        Path dst = Files.writeString(Path.of("d.txt"), "D");
        try {
            Files.copy(src, dst);
        } catch (FileAlreadyExistsException e) {
            System.out.print("exists ");
        }
        Files.copy(src, dst, StandardCopyOption.REPLACE_EXISTING);
        System.out.print(Files.readString(dst));
    }
}
```

- A. exists S
- B. S
- C. exists D
- D. D

Câu 09-08 · Khó · objective 9.3

Chương trình sau in ra gì?

```
import java.io.IOException;
import java.nio.file.*;
import java.util.stream.Stream;

public class Walk {
    public static void main(String[] args) throws IOException {
        Path root = Path.of("root");
        Files.createDirectories(root.resolve("sub/deep"));
        Files.writeString(root.resolve("a.txt"), "a");
        Files.writeString(root.resolve("sub/b.txt"), "b");
        Files.writeString(root.resolve("sub/deep/c.txt"), "c");
        try (Stream<Path> w1 = Files.walk(root, 1);
             Stream<Path> w2 = Files.walk(root);
             Stream<Path> l = Files.list(root)) {
            System.out.println(w1.count() + " " + w2.filter(Files::isRegularFile).count() + " " + l.count());
        }
    }
}
```

- A. 2 3 2
- B. 3 3 3
- C. 2 2 2
- D. 3 3 2

Câu 09-09 · Vừa · objective 9.2

Hai phát biểu nào đúng về serialization? (Chọn 2 đáp án.)

- A. Field `static` được lưu và khôi phục khi serialize/deserialize.
- B. Field `transient` nhận giá trị mặc định sau khi deserialize.
- C. Serialize object của lớp không implements `Serializable` ném `NotSerializableException`.
- D. Constructor của lớp `Serializable` được gọi khi deserialize.
- E. Mọi lớp cha của một lớp `Serializable` cũng phải là `Serializable`.

Câu 09-10 · Dễ · objective 9.1

Chương trình sau in ra gì?

```
import java.io.*;
import java.nio.file.*;

public class Append {
    public static void main(String[] args) throws IOException {
        try (var out = new FileOutputStream("a.txt")) { out.write("AB".getBytes()); }
        try (var out = new FileOutputStream("a.txt")) { out.write("C".getBytes()); }
        try (var out = new FileOutputStream("a.txt", true)) { out.write("D".getBytes()); }
        System.out.println(Files.readString(Path.of("a.txt")));
    }
}
```

- A. ABCD
- B. CD
- C. D
- D. ABD

Câu 09-11 · Vừa · objective 9.3

Chương trình sau in ra gì?

```
import java.nio.file.*;

public class Norm {
    public static void main(String[] args) {
        System.out.println(Path.of("../a/../../b").normalize() + " " + Path.of("../x").normalize() + " "
            + Path.of("a/b/./c/..").normalize().getNameCount());
    }
}
```

- A. b ../x 2
- B. ../b /x 3
- C. ../b /x 2
- D. b /x 2

Câu 09-12 · Khó · objective 9.3

Chèn dòng nào vào chỗ `// INSERT CODE HERE` thì chương trình in ra `true`? (Chọn 2 đáp án.)

```
import java.io.IOException;
import java.nio.file.*;

public class Same {
    public static void main(String[] args) throws IOException {
        Files.createDirectories(Path.of("data"));
        Files.writeString(Path.of("data/f.txt"), "x");
        Path a = Path.of("data/f.txt");
        Path b = Path.of("data/../data/f.txt");
        // INSERT CODE HERE
    }
}
```

- A. `System.out.println(a.equals(b));`
- B. `System.out.println(Files.isSameFile(a, b));`
- C. `System.out.println(a.equals(b.normalize()));`
- D. `System.out.println(a.toString().equals(b.toString()));`
- E. `System.out.println(a.compareTo(b) == 0);`

Câu 09-13 · Vừa · objective 9.2

Dòng nào gây lỗi biên dịch?

```
import java.io.*;

public class Read {
    static Object a(ObjectInputStream in) throws IOException { return in.readObject(); } // L1
    static Object b(ObjectInputStream in) throws IOException, ClassNotFoundException { return in.readObject(); } // L2
    static Object c(ObjectInputStream in) throws Exception { return in.readObject(); } // L3
    static void d(ObjectOutputStream out) throws IOException { out.writeObject("x"); } // L4

    public static void main(String[] args) { }
}
```

- A. L1
- B. L2
- C. L3
- D. L1 và L3
- E. Không dòng nào

Câu 09-14 · Vừa · objective 9.1, 9.3

Chương trình sau in ra gì?

```
import java.io.IOException;
import java.nio.file.*;
import java.util.List;
import java.util.stream.Stream;

public class Sum {
    public static void main(String[] args) throws IOException {
        Path p = Path.of("n.txt");
        Files.write(p, List.of("3", "1", "2"));
        try (Stream<String> s = Files.lines(p)) {
            System.out.println(s.mapToInt(Integer::parseInt).sum() + " " + Files.readAllLines(p).get(0)
                               + " " + Files.size(p));
        }
    }
}
```

- A. 6 3 3
- B. 321 3 6
- C. 6 1 6
- D. 6 3 6

Câu 09-15 · Khó · objective 9.1

File mã nguồn được lưu bằng UTF-8. Chương trình sau in ra gì?

```
import java.io.IOException;
import java.nio.file.*;

public class Utf {
    public static void main(String[] args) throws IOException {
        String s = "Việt";
        Files.writeString(Path.of("v.txt"), s);
        System.out.println(s.length() + " " + Files.size(Path.of("v.txt")) + " "
                           + Files.readString(Path.of("v.txt")).length());
    }
}
```

- A. 4 4 4
- B. 4 6 4
- C. 6 6 6
- D. 4 6 6

Câu 09-16 · Vừa · objective 9.3

Hai phát biểu nào đúng về lớp Files? (Chọn 2 đáp án.)

- A. Files.createDirectory ném exception nếu thư mục cha chưa tồn tại.
- B. Files.createDirectories ném exception nếu thư mục đã tồn tại.
- C. Files.delete trên file không tồn tại ném NoSuchFileException.
- D. Files.deleteIfExists ném exception nếu file không tồn tại.
- E. Files.move giữ lại file nguồn.

Câu 09-17 · Dễ · objective 9.1

Chương trình được chạy với output chuyển hướng vào file (java Cons > out.txt). Nội dung out.txt là gì?

```
import java.io.Console;

public class Cons {
    public static void main(String[] args) {
        Console c = System.console();
        System.out.println(c == null ? "no console" : "console");
    }
}
```

- A. no console
- B. console
- C. Ném NullPointerException
- D. Không biên dịch được vì Console không có constructor public

Câu 09-18 · Khó · objective 9.1

Chương trình sau in ra gì?

```
import java.io.*;

public class Mark {
    public static void main(String[] args) throws IOException {
        try (Reader r = new BufferedReader(new StringReader("12345"))) {
            r.read();
            r.mark(5);
            r.read();
            r.read();
            r.reset();
            r.skip(1);
            System.out.println((char) r.read());
        }
    }
}
```

- A. 2
- B. 4
- C. 3
- D. 1

Câu 09-19 · Vừa · objective 9.1

Chương trình sau in ra gì?

```
import java.io.*;
import java.nio.file.*;

public class DataSize {
    public static void main(String[] args) throws IOException {
        try (var out = new DataOutputStream(new FileOutputStream("d.bin"))) {
            out.writeInt(1);
            out.writeLong(2);
            out.writeByte(3);
        }
        System.out.println(Files.size(Path.of("d.bin")));
    }
}
```

- A. 3
- B. 12
- C. 24
- D. 13

Câu 09-20 · Vừa · objective 9.1, 4.1

Chèn dòng nào vào chỗ // INSERT CODE HERE thì chương trình biên dịch và in ra x? (Chọn 3 đáp án.)

```
import java.io.*;
import java.nio.file.*;

public class Twr {
    public static void main(String[] args) throws IOException {
        Files.writeString(Path.of("t.txt"), "x");
        // INSERT CODE HERE
    }
}
```

```
}  
}
```

- **A.** `try (var r = new BufferedReader(new FileReader("t.txt"))) { System.out.println(r.readLine()); }`
- **B.** `try (Path p = Path.of("t.txt")) { System.out.println(Files.readString(p)); }`
- **C.** `try (var in = Files.newInputStream(Path.of("t.txt"))) { System.out.println((char) in.read()); }`
- **D.** `try (String s = Files.readString(Path.of("t.txt"))) { System.out.println(s); }`
- **E.** `try (var lines = Files.lines(Path.of("t.txt"))) { lines.forEach(System.out::println); }`

Lời giải

Câu 09-01 — Đáp án: D

- **Vì sao đúng:** Các thành phần tên (name elements) là `var`, `log`, `app`, `server.log` — gốc / **không** tính. Vậy có 4 phần, `getName(1)` là `log` (chỉ số từ 0). `getParent()` là `/var/log/app`, tên cuối của nó là `app`.
- **A sai:** Gốc / không phải một name element nên chỉ có 4 phần.
- **B sai:** `getName(0)` là `var`, nên `getName(1)` là `log`; `getParent().getFileName()` là `app`.
- **C sai:** `getFileName()` chỉ trả về phần cuối, không có /.
- **Kiểm chứng:** `examples/questions/ch09/Q09_01/` — output confirmed (`python3 tools/book.py questions ch09`).

Câu 09-02 — Đáp án: C

- **Vì sao đúng:** `a.relativeTo(b)`: từ `/a/b/c` đi lên 2 cấp tới `/a`, rồi vào `x` → `../../x.resolve("../d")` cho `/a/b/c/../d`, `normalize()` bỏ `c/..` → `/a/b/d.resolve` với path **tuyệt đối** trả về chính path đó → `/y`.
- **A sai:** Phải đi lên 2 cấp (`c` và `b`); và `resolve("/")` trả về `/y`, không nối.
- **B sai:** `normalize()` xoá cặp `c/..`, nên không còn `c`.
- **D sai:** `relativeTo` cần `../..` để đi lên từ `/a/b/c`.
- **Kiểm chứng:** `examples/questions/ch09/Q09_02/` — output confirmed (`python3 tools/book.py questions ch09`).

Câu 09-03 — Đáp án: B

- **Vì sao đúng:** `readLine()` trả về `"one"`, `""` (dòng trống vẫn là một dòng), `"two"`, rồi `null`. Ký tự xuống dòng cuối cùng không tạo thêm dòng.
- **A sai:** Dòng trống được `readLine()` trả về là chuỗi rỗng `""`, không phải `null`, nên vẫn được đếm.
- **C sai:** `\n` cuối file chỉ kết thúc dòng `two`, không tạo dòng thứ tư.
- **D sai:** `readLine()` báo hết dữ liệu bằng `null`, không ném exception.
- **Kiểm chứng:** `examples/questions/ch09/Q09_03/` — output confirmed (`python3 tools/book.py questions ch09`).

Câu 09-04 — Đáp án: C

- **Vì sao đúng:** Khi `deserialize`, constructor của lớp `Serializable (Dog)` **không** chạy; field của nó lấy từ dữ liệu đã lưu (`name`), trừ `transient (age = 0)`. Lớp cha **không** `Serializable (Animal)` thì constructor không tham số của nó **có** chạy, nên field `sound` của `Animal` được khởi tạo lại thành `"generic"`.
- **A sai:** `transient` không được lưu → `age = 0`; và `sound` thuộc lớp cha không `Serializable` nên không được lưu.
- **B sai:** Field `sound` khai báo ở `Animal` (không `Serializable`) nên được tạo lại bởi `Animal()` → `generic`.
- **D sai:** `name` là field của `Dog (Serializable)` nên được khôi phục.
- **Kiểm chứng:** `examples/questions/ch09/Q09_04/` — output confirmed (`python3 tools/book.py questions ch09`).

Câu 09-05 — Đáp án: B

- **Vì sao đúng:** Constructor `FileReader(String)` ném `FileNotFoundException` và `Files.delete` ném `IOException` — cả hai là checked nên phải catch hoặc khai báo (L3, L5). `Files.exists` và các method thao tác chuỗi của `Path` không ném checked exception.
- **A sai:** `Files.delete` cũng ném `IOException` (checked).
- **C sai:** `Files.exists` trả về boolean, không ném checked exception.
- **D sai:** L2 hợp lệ (xem C).
- **E sai:** `new FileReader("x")` ném `FileNotFoundException` (checked).
- **Kiểm chứng:** `examples/questions/ch09/Q09_05/` — compile error confirmed at ['L3', 'L5'] (`python3 tools/book.py questions ch09`).

Câu 09-06 — Đáp án: A, B, D

- **Vì sao đúng:** `Path` là interface, tạo bằng factory: `Path.of(...)` (Java 11+), `Paths.get(...)`, hoặc chuyển từ `File` bằng `toPath()`.
- **C sai:** `Path` là interface, không new được.
- **E sai:** `Path` không có method `get`; đó là `Paths.get` hoặc `Path.of`.
- **Kiểm chứng:** `examples/questions/ch09/Q09_06/` — variants: ABD satisfy compiles (`python3 tools/book.py questions ch09`).

Câu 09-07 — Đáp án: A

- **Vì sao đúng:** `Files.copy` mặc định **không ghi đè**: đích đã tồn tại → `FileAlreadyExistsException`. Lần copy thứ hai có `REPLACE_EXISTING` nên ghi đè → nội dung S.
- **B sai:** Lần copy đầu ném exception vì `d.txt` đã có.
- **C sai:** Lần copy thứ hai với `REPLACE_EXISTING` đã ghi đè nội dung.
- **D sai:** Như C.
- *Kiểm chứng:* `examples/questions/ch09/Q09_07/` — output confirmed (`python3 tools/book.py questions ch09`).

Câu 09-08 — Đáp án: D

- **Vì sao đúng:** `walk(root, 1)` gồm **chính root** và các mục con trực tiếp: `root, a.txt, sub` → 3. `walk(root)` đi hết độ sâu; có 3 file thường. `list(root)` chỉ liệt kê con trực tiếp, **không** gồm `root`: `a.txt, sub` → 2.
- **A sai:** `Files.walk` luôn trả về cả path bắt đầu (`root`).
- **B sai:** `Files.list` không bao gồm chính thư mục `root`.
- **C sai:** `walk` đệ quy toàn bộ nên thấy cả `c.txt` trong `sub/deep`; và `walk(root, 1)` có 3 phần tử.
- *Kiểm chứng:* `examples/questions/ch09/Q09_08/` — output confirmed (`python3 tools/book.py questions ch09`).

Câu 09-09 — Đáp án: B, C

- **Vì sao đúng:** B: `transient` không được ghi, nên khi đọc lại nhận giá trị mặc định (`null` cho object). C: chỉ lớp implements `Serializable` mới serialize được.
- **A sai:** `static` thuộc về lớp, không thuộc object → không được lưu (giá trị 99 được giữ nguyên).
- **D sai:** Constructor của lớp `Serializable` không chạy khi deserialize (đếm chỉ tăng 1 lần lúc `new`).
- **E sai:** Lớp cha không `Serializable` vẫn được, miễn là có constructor không tham số truy cập được.
- *Kiểm chứng:* `examples/questions/ch09/Q09_09/` — each option proven true/false by a program (`python3 tools/book.py questions ch09`).

Câu 09-10 — Đáp án: B

- **Vì sao đúng:** `new FileOutputStream(name)` mặc định **ghi đè** (xóa nội dung cũ): còn C. Tham số thứ hai `true` là chế độ **append**: thêm D vào cuối → CD.
- **A sai:** Lần mở thứ hai (không append) đã xóa AB.
- **C sai:** Lần cuối mở ở chế độ append nên giữ C.
- **D sai:** Lần mở thứ hai ghi đè AB bằng C.
- *Kiểm chứng:* `examples/questions/ch09/Q09_10/` — output confirmed (`python3 tools/book.py questions ch09`).

Câu 09-11 — Đáp án: C

- **Vì sao đúng:** `normalize()` bỏ `.` và triệt tiêu tên/... `./a/././b` → `a/..` mất, còn `../b` (không có gì để triệt tiêu `..` còn lại trong path tương đối). Với path tuyệt đối, `..` ngay sau gốc bị bỏ → `/x. a/b/./c/..` → `a/b` (2 phần).
- **A sai:** Path tương đối giữ `..` không triệt tiêu được; path tuyệt đối thì bỏ `..` sau gốc.
- **B sai:** `c/..` triệt tiêu nhau nên còn `a/b` — 2 phần.
- **D sai:** `..` thừa trong path tương đối được giữ lại: `../b`.
- *Kiểm chứng:* `examples/questions/ch09/Q09_11/` — output confirmed (`python3 tools/book.py questions ch09`).

Câu 09-12 — Đáp án: B, C

- **Vì sao đúng:** `Path.equals/compareTo` so sánh **chuỗi đường dẫn**, không nhìn hệ thống file. `normalize()` biến `b` thành `data/f.txt` nên C đúng. `Files.isSameFile` hỏi hệ thống file xem hai path có trỏ cùng một file không → B đúng.
- **A sai:** Hai chuỗi path khác nhau nên `equals` là `false`.
- **D sai:** Chuỗi `data/f.txt` khác `data/../data/f.txt`.
- **E sai:** `compareTo` so sánh chuỗi, khác 0.
- *Kiểm chứng:* `examples/questions/ch09/Q09_12/` — variants: BC satisfy output (`python3 tools/book.py questions ch09`).

Câu 09-13 — Đáp án: A

- **Vì sao đúng:** `readObject()` khai báo throws `IOException`, `ClassNotFoundException`. `ClassNotFoundException` **không** phải lớp con của `IOException`, nên L1 thiếu nó → lỗi. L2 khai báo đủ, L3 khai báo `Exception` (bao cả hai).
- **B sai:** L2 khai báo đủ cả hai checked exception.
- **C sai:** throws `Exception` bao gồm cả `IOException` và `ClassNotFoundException`.
- **D sai:** L3 hợp lệ (xem C).
- **E sai:** L1 thiếu `ClassNotFoundException`.
- *Kiểm chứng:* `examples/questions/ch09/Q09_13/` — compile error confirmed at ['L1'] (`python3 tools/book.py questions ch09`).

Câu 09-14 — Đáp án: D

- **Vì sao đúng:** `Files.write(path, lines)` ghi mỗi phần tử kèm ký tự xuống dòng: `3\n1\n2\n` = 6 byte. `Files.lines` trả về `Stream<String>` (lười, cần đóng); tổng = 6. `readAllLines` trả về `List` theo thứ tự trong file → phần tử đầu 3.
- **A sai:** Mỗi dòng có thêm ký tự `\n`, nên file có 6 byte.
- **B sai:** `mapToInt(Integer::parseInt).sum()` cộng số, không nối chuỗi.
- **C sai:** `readAllLines` giữ thứ tự của file; dòng đầu là 3.
- **Kiểm chứng:** `examples/questions/ch09/Q09_14/` — output confirmed (python3 tools/book.py questions ch09).

Câu 09-15 — Đáp án: B

- **Vì sao đúng:** `String.length()` đếm ký tự (char): V, i, ệ, t → 4. `Files.writeString` mặc định dùng UTF-8; ký tự ệ cần 3 byte nên file có $1 + 1 + 3 + 1 = 6$ byte. Đọc lại bằng UTF-8 được đúng 4 ký tự.
- **A sai:** Kích thước file tính bằng byte; ệ chiếm 3 byte trong UTF-8.
- **C sai:** `length()` của `String` đếm ký tự, không đếm byte.
- **D sai:** `readString` giải mã UTF-8 lại thành 4 ký tự.
- **Kiểm chứng:** `examples/questions/ch09/Q09_15/` — output confirmed (python3 tools/book.py questions ch09).

Câu 09-16 — Đáp án: A, C

- **Vì sao đúng:** A: `createDirectory` chỉ tạo một cấp; cha thiếu → `NoSuchFileException`. C: `delete` bắt buộc file phải tồn tại.
- **B sai:** `createDirectories` tạo mọi cấp còn thiếu và không lỗi khi thư mục đã có.
- **D sai:** `deleteIfExists` trả về `false` khi không có gì để xóa.
- **E sai:** `move` di chuyển (đổi tên) nên file nguồn không còn.
- **Kiểm chứng:** `examples/questions/ch09/Q09_16/` — each option proven true/false by a program (python3 tools/book.py questions ch09).

Câu 09-17 — Đáp án: A

- **Vì sao đúng:** `System.console()` trả về `null` khi JVM không gắn với một terminal tương tác (ví dụ output bị chuyển hướng hoặc chạy trong script). Vì vậy code dùng `Console` luôn phải kiểm tra `null`.
- **B sai:** Khi output bị chuyển hướng, không có console tương tác.
- **C sai:** Code chỉ so sánh `c == null`, không gọi method trên `c`.
- **D sai:** Code không `new Console()`; nó lấy qua `System.console()`.
- **Kiểm chứng:** `examples/questions/ch09/Q09_17/` — output confirmed (python3 tools/book.py questions ch09).

Câu 09-18 — Đáp án: C

- **Vì sao đúng:** Đọc 1 → mark tại vị trí của 2. Đọc 2, 3. `reset()` quay về chỗ mark (trước 2). `skip(1)` bỏ 2. Lần đọc tiếp theo là 3.
- **A sai:** `skip(1)` bỏ qua 2 sau khi reset.
- **B sai:** `reset()` quay lại vị trí mark, không tiếp tục từ 4.
- **D sai:** Mark đặt sau khi đã đọc 1, nên reset không quay về 1.
- **Kiểm chứng:** `examples/questions/ch09/Q09_18/` — output confirmed (python3 tools/book.py questions ch09).

Câu 09-19 — Đáp án: D

- **Vì sao đúng:** `DataOutputStream` ghi dạng nhị phân với kích thước cố định: `int` 4 byte, `long` 8 byte, `byte` 1 byte → 13 byte.
- **A sai:** Không ghi dạng chữ số; mỗi kiểu có kích thước nhị phân cố định.
- **B sai:** `long` chiếm 8 byte, `int` 4 byte và `byte` 1 byte: tổng 13.
- **C sai:** Không có đệm (padding); tổng là $4 + 8 + 1$.
- **Kiểm chứng:** `examples/questions/ch09/Q09_19/` — output confirmed (python3 tools/book.py questions ch09).

Câu 09-20 — Đáp án: A, C, E

- **Vì sao đúng:** Tài nguyên trong try-with-resources phải là `AutoCloseable`: `BufferedReader` (A), `InputStream` (C) và `Stream` (E, `Stream` implements `AutoCloseable` — `Files.lines` nên luôn được đóng).
- **B sai:** `Path` không implements `AutoCloseable`.
- **D sai:** `String` không implements `AutoCloseable`.
- **Kiểm chứng:** `examples/questions/ch09/Q09_20/` — variants: ACE satisfy output (python3 tools/book.py questions ch09).

Đọc thêm (link chính thức — bị chặn trong sandbox nên mình chưa mở được)

- Javadoc `java.nio.file.Files`: <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/nio/file/Files.html>
- Java Object Serialization Specification: <https://docs.oracle.com/en/java/javase/21/docs/specs/serialization/index.html>

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-28:

- `Files.java`: <https://raw.githubusercontent.com/openjdk/jdk21u/master/src/java.base/share/classes/java/nio/file/Files.java>
- `Path.java` (`resolve`, `relativize`, `normalize`, `endsWith`):
<https://raw.githubusercontent.com/openjdk/jdk21u/master/src/java.base/share/classes/java/nio/file/Path.java>
- `ObjectInputStream.java` (deserialize record qua canonical constructor, constructor lớp cha không `Serializable`):
<https://raw.githubusercontent.com/openjdk/jdk21u/master/src/java.base/share/classes/java/io/ObjectInputStream.java>
- `Serializable.java`: <https://raw.githubusercontent.com/openjdk/jdk21u/master/src/java.base/share/classes/java/io/Serializable.java>

Chương 10 — Bản địa hoá (Implementing Localization)

Mục tiêu

- 10.1 Bản địa hoá (localization) bằng `Locale` và `ResourceBundle`.
- 10.2 Parse (đọc) và format (định dạng) thông điệp, ngày, giờ, số — kể cả tiền tệ và phần trăm.

Giải thích đơn giản

Internationalization (i18n) là thiết kế chương trình để hỗ trợ nhiều ngôn ngữ/vùng. **Localization (l10n)** là việc làm cho chương trình hiển thị đúng với một vùng cụ thể: ngôn ngữ, cách viết số (1,234.5 hay 1.234,5), tiền tệ (\$ hay đ), ngày tháng (12/25/24 hay 25/12/24).

- **Locale** mô tả “vùng”: ngôn ngữ (vi) + quốc gia (VN) → vi_VN.
- **ResourceBundle** chứa các chuỗi dịch theo từng locale (`Messages_vi.properties...`). Code chỉ gọi `bundle.getString("greeting")`.
- **NumberFormat**, **DateTimeFormatter**, **MessageFormat** định dạng số, ngày giờ, thông điệp theo locale.

Ví dụ

Code trong `examples/ch10/`. Chạy lại: `python3 tools/book.py examples ch10`. Output thật, JDK 21.0.10 (dữ liệu locale CLDR đi kèm JDK). Nhiều locale dùng **khoảng trắng không ngắt** (U+00A0, U+202F); trong ví dụ, chúng được in thành `_` cho dễ nhìn.

1. Locale

`examples/ch10/Ex01_Locales.java`

```
// objective: 10.1
// Locale: ngôn ngữ (language) + quốc gia (country). Tạo bằng Locale.of (Java 19+), hằng số, hoặc language tag.
import java.util.Locale;

public class Ex01_Locales {
    public static void main(String[] args) {
        Locale vi = Locale.of("vi", "VN"); // constructor new Locale(...) đã deprecated từ Java 19
        Locale fr = Locale.FRANCE;
        Locale en = Locale.ENGLISH; // chỉ có ngôn ngữ, không có quốc gia
        Locale tag = Locale.forLanguageTag("ja-JP");
        Locale built = new Locale.Builder().setLanguage("de").setRegion("CH").build();
        System.out.println(vi + " " + fr + " " + en + " " + tag + " " + built);
        System.out.println(vi.getLanguage() + " " + vi.getCountry() + " [" + en.getCountry() + "] " + vi.toLanguageTag());
        System.out.println(vi.getDisplayName(Locale.US) + " | " + vi.getDisplayName(vi) + " | " + fr.getDisplayCountry(Locale.US));
        System.out.println("default = " + Locale.getDefault());
        Locale.setDefault(vi); // chỉ ảnh hưởng JVM này
        System.out.println("default = " + Locale.getDefault() + ", display: " + Locale.GERMANY.getDisplayName());
        System.out.println(Locale.of("VI", "vn") + " (tự chuẩn hoá chữ hoa/thường)");
    }
}
```

Output thật (JDK 21.0.10):

```
vi_VN fr_FR en_JP de_CH
vi_VN [] vi_VN
Vietnamese (Vietnam) | Tiếng Việt (Việt Nam) | France
default = en_US
default = vi_VN, display: Tiếng Đức (Đức)
vi_VN (tự chuẩn hoá chữ hoa/thường)
```

2. NumberFormat: số, tiền tệ, phần trăm, compact

Để ý dòng 2 4 -2 và 0.2 0.3 7.0: `NumberFormat` làm tròn kiểu **HALF_EVEN** (nửa về số chẵn), và 0.35 thật ra là 0.34999... trong hệ nhị phân nên thành 0.3.

`examples/ch10/Ex02_NumberFormat.java`

```
// objective: 10.2
// Định dạng số, tiền tệ, phần trăm theo locale. Lưu ý: nhiều locale dùng khoảng trắng KHÔNG NGẮT (U+00A0 / U+202F);
// ở đây in chúng thành '_' để dễ nhìn.
import java.text.NumberFormat;
import java.util.Locale;

public class Ex02_NumberFormat {
    static String show(String s) { return s.replace(' ', '_').replace('.', '_'); }

    public static void main(String[] args) {
```

```

double value = 1234567.891;
Locale[] locales = {Locale.US, Locale.of("vi", "VN"), Locale.GERMANY, Locale.FRANCE, Locale.JAPAN};
for (Locale l : locales) {
    System.out.printf("%-6s number=%-14s currency=%-18s percent=%s\n", l,
        show(NumberFormat.getInstance(l).format(value)),
        show(NumberFormat.getCurrencyInstance(l).format(value)),
        show(NumberFormat.getPercentInstance(l).format(0.256)));
}
NumberFormat integer = NumberFormat.getIntegerInstance(Locale.US);
System.out.println(integer.format(2.5) + " " + integer.format(3.5) + " " + integer.format(-2.5)); // HALF_EVEN
NumberFormat nf = NumberFormat.getInstance(Locale.US);
nf.setMaximumFractionDigits(1);
nf.setMinimumFractionDigits(1);
System.out.println(nf.format(0.25) + " " + nf.format(0.35) + " " + nf.format(7));
NumberFormat compact = NumberFormat.getCompactNumberInstance(Locale.US, NumberFormat.Style.SHORT);
NumberFormat compactLong = NumberFormat.getCompactNumberInstance(Locale.US, NumberFormat.Style.LONG);
System.out.println(compact.format(1_500) + " " + compact.format(2_345_678) + " " + compactLong.format(2_345_678));
}
}

```

Output thật (JDK 21.0.10):

en_US	number=1,234,567.891	currency=\$1,234,567.89	percent=26%
vi_VN	number=1.234.567,891	currency=1.234.568 đ	percent=26%
de_DE	number=1.234.567,891	currency=1.234.567,89_€	percent=26_%
fr_FR	number=1_234_567,891	currency=1_234_567,89_€	percent=26_%
ja_JP	number=1,234,567.891	currency=¥1,234,568	percent=26%
2 4 -2			
0.2 0.3 7.0			
2K 2M 2 million			

3. Parse số và DecimalFormat

examples/ch10/Ex03_ParseAndPatterns.java

```

// objective: 10.2
// parse số theo locale (ParseException, dùng ở ký tự lạ) và DecimalFormat với pattern.
import java.text.*;
import java.util.Locale;

public class Ex03_ParseAndPatterns {
    public static void main(String[] args) throws ParseException {
        NumberFormat us = NumberFormat.getInstance(Locale.US);
        NumberFormat de = NumberFormat.getInstance(Locale.GERMANY);
        System.out.println(us.parse("1,234.5") + " " + de.parse("1.234,5") + " " + us.parse("1.234,5"));
        System.out.println(us.parse("12abc") + " " + us.parse("42").getClass().getSimpleName()
            + " " + us.parse("4.2").getClass().getSimpleName());

        try {
            us.parse("abc");
        } catch (ParseException e) {
            System.out.println("ParseException: " + e.getMessage() + " at " + e.getErrorOffset());
        }

        NumberFormat money = NumberFormat.getCurrencyInstance(Locale.US);
        System.out.println(money.parse("$9.99") + " " + money.format(-3.456));

        DecimalFormat df1 = new DecimalFormat("#,##0.00", DecimalFormatSymbols.getInstance(Locale.US));
        DecimalFormat df2 = new DecimalFormat("000.#", DecimalFormatSymbols.getInstance(Locale.US));
        DecimalFormat df3 = new DecimalFormat("$#,###.## 'total'", DecimalFormatSymbols.getInstance(Locale.US));
        System.out.println(df1.format(1234.5) + " | " + df1.format(0.126) + " | " + df2.format(5.25) + " | "
            + df2.format(1234.56) + " | " + df3.format(98765.4321));
        System.out.println(new DecimalFormat("0.0%", DecimalFormatSymbols.getInstance(Locale.US)).format(0.4567));
    }
}

```

Output thật (JDK 21.0.10):

```

1234.5 1234.5 1.234
12 Long Double
ParseException: Unparseable number: "abc" at 0
9.99 -$3.46
1,234.50 | 0.13 | 005.2 | 1234.6 | $98,765.43 total
45.7%

```

4. DateTimeFormatter

examples/ch10/Ex04_DateTimeFormatter.java

```

// objective: 10.2
// DateTimeFormatter: pattern (chữ hoa/thường khác nghĩa!), locale, FormatStyle, parse.
import java.time.*;
import java.time.format.*;
import java.time.temporal.UnsupportedTemporalTypeException;

```

```
import java.util.Locale;

public class Ex04_DateTimeFormatter {
    public static void main(String[] args) {
        LocalDateTime dt = LocalDateTime.of(2024, 3, 5, 14, 7, 9);
        System.out.println(dt.format(DateTimeFormatter.ofPattern("dd/MM/yyyy HH:mm:ss")));
        System.out.println(dt.format(DateTimeFormatter.ofPattern("d/M/yy h:mm a", Locale.US)));
        System.out.println(dt.format(DateTimeFormatter.ofPattern("EEEE, MMMM d", Locale.US)) + " | "
            + dt.format(DateTimeFormatter.ofPattern("EEEE, d MMMM", Locale.of("vi", "VN"))) + " | "
            + dt.format(DateTimeFormatter.ofPattern("EEE d MMM", Locale.FRANCE)));
        System.out.println(dt.format(DateTimeFormatter.ofPattern("yyyy-MM-dd'T'HH:mm 'o'clock'")));
        System.out.println(dt.format(DateTimeFormatter.ofPattern("MM")) + " vs " + dt.format(DateTimeFormatter.ofPattern("MM"))
            + " | " + DateTimeFormatter.ISO_LOCAL_DATE.format(dt) + " | " + DateTimeFormatter.BASIC_ISO_DATE.format(dt));

        for (FormatStyle st : new FormatStyle[]{FormatStyle.SHORT, FormatStyle.MEDIUM, FormatStyle.LONG, FormatStyle.FULL}) {
            System.out.println(st + ": " + dt.toLocalDate().format(DateTimeFormatter.ofLocalizedDate(st).withLocale(Locale.US))
                + " | " + dt.toLocalDate().format(DateTimeFormatter.ofLocalizedDate(st).withLocale(Locale.of("vi", "VN"))));
        }
        LocalDate parsed = LocalDate.parse("05.03.2024", DateTimeFormatter.ofPattern("dd.MM.yyyy"));
        System.out.println("parsed: " + parsed);
        try {
            LocalDate.parse("2024/03/05");
        } catch (DateTimeParseException e) {
            System.out.println("DateTimeParseException: " + e.getMessage());
        }
        try {
            LocalDate.of(2024, 3, 5).format(DateTimeFormatter.ofPattern("HH:mm"));
        } catch (UnsupportedTemporalTypeException e) {
            System.out.println("UnsupportedTemporalTypeException: LocalDate không có giờ");
        }
    }
}
```

Output thật (JDK 21.0.10):

```
05/03/2024 14:07:09
5/3/24 2:07 PM
Tuesday, March 5 | Thứ Ba, 5 tháng 3 | mar. 5 mars
2024-03-05T14:07 o'clock
07 vs 03 | 2024-03-05 | 20240305
SHORT: 3/5/24 | 05/03/2024
MEDIUM: Mar 5, 2024 | 5 thg 3, 2024
LONG: March 5, 2024 | 5 tháng 3, 2024
FULL: Tuesday, March 5, 2024 | Thứ Ba, 5 tháng 3, 2024
parsed: 2024-03-05
DateTimeParseException: Text '2024/03/05' could not be parsed at index 4
UnsupportedTemporalTypeException: LocalDate không có giờ
```

5. MessageFormat và String.format

examples/ch10/Ex05_MessageFormat.java

```
// objective: 10.2
// MessageFormat: thông điệp có tham số {0}, {1}, kèm định dạng number/date; dấu ' là ký tự escape.
import java.text.MessageFormat;
import java.util.Locale;

public class Ex05_MessageFormat {
    public static void main(String[] args) {
        System.out.println(MessageFormat.format("{0} có {1} tin nhắn mới", "Nobin", 3));
        System.out.println(MessageFormat.format("{1} trước {0}, lặp lại {1}", "A", "B"));
        System.out.println(MessageFormat.format("It's {0}; '{1}' is literal", "ok", "ignored"));
        MessageFormat mf = new MessageFormat("Total: {0,number,#,##0.00} ({1,number,percent})", Locale.US);
        System.out.println(mf.format(new Object[]{1234.5, 0.07}));
        MessageFormat de = new MessageFormat("Summe: {0,number}", Locale.GERMANY);
        System.out.println(de.format(new Object[]{1234.5}));
        System.out.println(MessageFormat.format("{0} {1} {2}", "only-one")); // thiếu tham số - in nguyên {1}
        System.out.println(String.format(Locale.GERMANY, "%.2f", 3.14159) + " vs " + String.format(Locale.US, "%.2f", 3.14159)
            + " vs " + String.format(Locale.US, "%d", 1234567));
    }
}
```

Output thật (JDK 21.0.10):

```
Nobin có 3 tin nhắn mới
B trước A, lặp lại B
It's ok; {1} is literal
Total: 1,234.50 (7%)
Summe: 1.234,5
only-one {1} {2}
3,14 vs 3.14 vs 1,234,567
```

6. ResourceBundle (.properties)

Ex06_ResourceBundles/Main.java

```
// objective: 10.1
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Locale.setDefault(Locale.US); // không có Messages_en_US / Messages_en
        for (Locale l : List.of(Locale.of("vi", "VN"), Locale.of("vi"), Locale.FRANCE, Locale.JAPAN)) {
            ResourceBundle rb = ResourceBundle.getBundle("Messages", l);
            System.out.printf("%-6s -> greeting=%s | farewell=%s | only.root=%s | bundle locale=[%s]\n",
                l, rb.getString("greeting"), rb.getString("farewell"), rb.getString("only.root"), rb.getLocale());
        }
        ResourceBundle vi = ResourceBundle.getBundle("Messages", Locale.of("vi", "VN"));
        System.out.println("keys (vi_VN, gồm cả key kế thừa từ bundle cha): " + new TreeSet<>(vi.keySet()));
        try {
            vi.getString("missing.key");
        } catch (MissingResourceException e) {
            System.out.println("MissingResourceException: " + e.getMessage());
        }
        try {
            ResourceBundle.getBundle("NoSuchBundle", Locale.US);
        } catch (MissingResourceException e) {
            System.out.println("MissingResourceException khi không có bundle nào");
        }
    }
}
```

Ex06_ResourceBundles/Messages.properties

```
# bundle gốc (default / root)
greeting=Hello
farewell=Goodbye
only.root=from root
```

Ex06_ResourceBundles/Messages_fr.properties

```
greeting=Bonjour
```

Ex06_ResourceBundles/Messages_vi.properties

```
greeting=Xin chào
farewell=Tạm biệt
```

Ex06_ResourceBundles/Messages_vi_VN.properties

```
greeting=Chào bạn (vi_VN)
```

Ex06_ResourceBundles/common.sh

```
# Helper sourced by every run.sh in ch07: prints each command, then its output with the temp path hidden.
unset JAVA_TOOL_OPTIONS
run() {
    echo "\$ $*"
    "$@" 2>&1 | sed "s|$PWD|.|g"
    return 0
}
```

Ex06_ResourceBundles/run.sh

```
#!/usr/bin/env bash
# objective: 10.1
# ResourceBundle dạng .properties: tìm theo thứ tự vi_VN -> vi -> (locale mặc định) -> gốc.
source ./common.sh
run javac -d out Main.java
cp *.properties out/
run java -cp out Main
```

Output thật (chạy bash run.sh, JDK 21.0.10):

```
$ javac -d out Main.java
$ java -cp out Main
vi_VN -> greeting=Chào bạn (vi_VN) | farewell=Tạm biệt | only.root=from root | bundle locale=[vi_VN]
vi -> greeting=Xin chào | farewell=Tạm biệt | only.root=from root | bundle locale=[vi]
fr_FR -> greeting=Bonjour | farewell=Goodbye | only.root=from root | bundle locale=[fr]
ja_JP -> greeting=Hello | farewell=Goodbye | only.root=from root | bundle locale=[]
keys (vi_VN, gồm cả key kế thừa từ bundle cha): [farewell, greeting, only.root]
```

```
MissingResourceException: Can't find resource for bundle java.util.PropertyResourceBundle, key missing.key
MissingResourceException khi không có bundle nào
```

7. Locale mặc định trong việc tìm bundle

Ex07_DefaultLocaleFallback/Labels.properties

```
title=Root title
```

Ex07_DefaultLocaleFallback/Labels_de.properties

```
title=Deutscher Titel
```

Ex07_DefaultLocaleFallback/Labels_en.properties

```
title=English title
```

Ex07_DefaultLocaleFallback/Main.java

```
// objective: 10.1
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Locale.setDefault(Locale.GERMANY);
        // Tìm cho ja_JP: Labels_ja_JP → Labels_ja → (mặc định) Labels_de_DE → Labels_de → Labels (gốc)
        System.out.println("ja_JP, default de_DE -> " + ResourceBundle.getBundle("Labels", Locale.JAPAN).getString("title"));
        System.out.println("en_US, default de_DE -> " + ResourceBundle.getBundle("Labels", Locale.US).getString("title"));
        Locale.setDefault(Locale.JAPAN);
        System.out.println("fr_FR, default ja_JP -> " + ResourceBundle.getBundle("Labels", Locale.FRANCE).getString("title"));
        ResourceBundle.Control noFallback = ResourceBundle.Control.getNoFallbackControl(ResourceBundle.Control.FORMAT_DEFAULT);
        Locale.setDefault(Locale.GERMANY);
        System.out.println("ja_JP, no fallback -> " + ResourceBundle.getBundle("Labels", Locale.JAPAN,
noFallback).getString("title"));
    }
}
```

Ex07_DefaultLocaleFallback/common.sh

```
# Helper sourced by every run.sh in ch07: prints each command, then its output with the temp path hidden.
unset JAVA_TOOL_OPTIONS
run() {
    echo "\$ $*"
    "$@" 2>&1 | sed "s|$PWD|.|g"
    return 0
}
```

Ex07_DefaultLocaleFallback/run.sh

```
#!/usr/bin/env bash
# objective: 10.1
# Vai trò của locale mặc định khi không tìm thấy bundle cho locale được yêu cầu.
source ./common.sh
run javac -d out Main.java
cp *.properties out/
run java -cp out Main
```

Output thật (chạy bash run.sh, JDK 21.0.10):

```
$ javac -d out Main.java
$ java -cp out Main
ja_JP, default de_DE -> Deutscher Titel
en_US, default de_DE -> English title
fr_FR, default ja_JP -> Root title
ja_JP, no fallback -> Root title
```

8. ListResourceBundle (bundle là lớp Java)

Ex08_ListResourceBundle/Main.java

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Locale.setDefault(Locale.US);
        ResourceBundle rb = ResourceBundle.getBundle("Prices", Locale.of("vi"));
        double vat = (Double) rb.getObject("vat");
    }
}
```

```

    int[] tiers = (int[]) rb.getObject("tiers"); // kế thừa từ bundle cha Prices
    System.out.println("vat=" + vat + " currency=" + rb.getString("currency") + " tiers=" + Arrays.toString(tiers));
    try {
        rb.getString("vat"); // giá trị không phải String
    } catch (ClassCastException e) {
        System.out.println("ClassCastException: getString trên giá trị Double");
    }
}
}

```

Ex08_ListResourceBundle/Prices.java

```

// objective: 10.1
import java.util.ListResourceBundle;

// Bundle dạng lớp Java: giá trị có thể là object bất kỳ, không chỉ String.
public class Prices extends ListResourceBundle {
    @Override protected Object[][] getContents() {
        return new Object[][]{{"vat", 0.10}, {"currency", "USD"}, {"tiers", new int[]{10, 20}}};
    }
}

```

Ex08_ListResourceBundle/Prices_vi.java

```

import java.util.ListResourceBundle;

public class Prices_vi extends ListResourceBundle {
    @Override protected Object[][] getContents() {
        return new Object[][]{{"vat", 0.08}, {"currency", "VND"}};
    }
}

```

Ex08_ListResourceBundle/Prices_vi.properties

```

# Cùng tên với lớp Prices_vi: lớp Java được ưu tiên, file này bị bỏ qua
currency=FROM PROPERTIES

```

Ex08_ListResourceBundle/common.sh

```

# Helper sourced by every run.sh in ch07: prints each command, then its output with the temp path hidden.
unset JAVA_TOOL_OPTIONS
run() {
    echo "\$ *"
    "$@" 2>&1 | sed "s|${PWD}|.|g"
    return 0
}

```

Ex08_ListResourceBundle/run.sh

```

#!/usr/bin/env bash
# objective: 10.1
# ListResourceBundle (lớp Java) và thứ tự ưu tiên lớp > .properties.
source ./common.sh
run javac -d out Prices.java Prices_vi.java Main.java
cp *.properties out/
run java -cp out Main

```

Output thật (chạy bash run.sh, JDK 21.0.10):

```

$ javac -d out Prices.java Prices_vi.java Main.java
$ java -cp out Main
vat=0.08 currency=VND tiers=[10, 20]
ClassCastException: getString trên giá trị Double

```

9. Currency

examples/ch10/Ex09_Currency.java

```

// objective: 10.2
// Currency và định dạng tiền tệ: mã ISO 4217, ký hiệu theo locale, số chữ số thập phân.
import java.text.NumberFormat;
import java.util.Currency;
import java.util.Locale;

public class Ex09_Currency {
    static String show(String s) { return s.replace('.', '_').replace('_', '.'); }

    public static void main(String[] args) {
        Currency vnd = Currency.getInstance(Locale.of("vi", "VN"));
    }
}

```

```

        Currency usd = Currency.getInstance("USD");
        System.out.println(vnd.getCurrencyCode() + " " + vnd.getDefaultFractionDigits() + " " + usd.getDefaultFractionDigits()
            + " " + show(vnd.getSymbol(Locale.of("vi", "VN"))) + " " + usd.getSymbol(Locale.US) + " " +
            usd.getSymbol(Locale.CANADA));
        NumberFormat f = NumberFormat.getCurrencyInstance(Locale.US);
        f.setCurrency(Currency.getInstance("EUR"));
        System.out.println(f.format(12.5) + " | " + show(NumberFormat.getCurrencyInstance(Locale.of("vi", "VN")).format(12345.678)));
        System.out.println(show(NumberFormat.getCurrencyInstance(Locale.JAPAN).format(1234.5)) + " | "
            + show(NumberFormat.getCurrencyInstance(Locale.of("en", "IN")).format(1234567.8)));
    }
}

```

Output thật (JDK 21.0.10):

```

VND 0 2 đ $ US$
€12.50 | 12.346 đ
¥1,234 | ₹1,234,567.80

```

10. Locale.Category

examples/ch10/Ex10_LocaleCategory.java

```

// objective: 10.1, 10.2
// Locale.Category: FORMAT (định dạng số/ngày) và DISPLAY (tên hiển thị) có thể đặt riêng.
import java.text.NumberFormat;
import java.util.Locale;

public class Ex10_LocaleCategory {
    static String show(String s) { return s.replace(' ', '_').replace('.', '_'); }

    public static void main(String[] args) {
        Locale.setDefault(Locale.US);
        Locale.setDefault(Locale.Category.FORMAT, Locale.GERMANY);
        Locale.setDefault(Locale.Category.DISPLAY, Locale.FRANCE);
        System.out.println("default=" + Locale.getDefault() + " FORMAT=" + Locale.getDefault(Locale.Category.FORMAT)
            + " DISPLAY=" + Locale.getDefault(Locale.Category.DISPLAY));
        System.out.println("NumberFormat.getInstance() dùng FORMAT: " + show(NumberFormat.getInstance().format(1234.5)));
        System.out.println("getDisplayCountry() dùng DISPLAY: " + Locale.JAPAN.getDisplayCountry());
        System.out.println("String.format dùng FORMAT: " + String.format("%.1f", 2.5));
        Locale.setDefault(Locale.US);
        System.out.println("sau setDefault(US): " + NumberFormat.getInstance().format(1234.5) + " " +
            Locale.JAPAN.getDisplayCountry());
    }
}

```

Output thật (JDK 21.0.10):

```

default=en_US FORMAT=de_DE DISPLAY=fr_FR
NumberFormat.getInstance() dùng FORMAT: 1.234,5
getDisplayCountry() dùng DISPLAY: Japon
String.format dùng FORMAT: 2,5
sau setDefault(US): 1,234.5 Japan

```

Đi sâu

Locale

- Tạo: `Locale.of("vi", "VN")` (Java 19+; constructor `new Locale(...)` đã deprecated), hằng số (`Locale.US`, `Locale.FRANCE`, `Locale.GERMAN` — chỉ ngôn ngữ), `Locale.forLanguageTag("vi-VN")`, `new Locale.Builder()`.
- `toString()` → `vi_VN`; `toLanguageTag()` → `vi-VN`. Ngôn ngữ viết thường, quốc gia viết hoa (tự chuẩn hoá).
- `getLanguage()`, `getCountry()` (có thể ""), `getDisplayName([inLocale])`, `getDisplayLanguage`, `getDisplayCountry`.
- `Locale.getDefault()` / `setDefault(locale)`; `Locale.Category.FORMAT` (số, ngày) và `DISPLAY` (tên hiển thị) đặt riêng được: `setDefault(Category, locale)`.

ResourceBundle

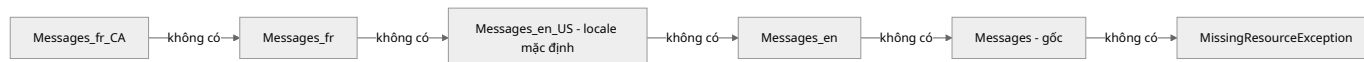
- Tên file: `BaseName_ngônngữ_QUỐCGIA.properties` (hoặc lớp `BaseName_ngônngữ_QUỐCGIA` kế thừa `ListResourceBundle`). File `.properties` được đọc bằng UTF-8 (Java 9+).
- `ResourceBundle.getBundle("Messages", locale)` tìm theo thứ tự (với locale yêu cầu `fr_CA`, mặc định `en_US`):

```

Messages_fr_CA → Messages_fr → Messages_en_US → Messages_en → Messages (gốc)
(yêu cầu)                (locale mặc định)

```

Bundle **đầu tiên tìm thấy** được chọn. Trong mỗi bước, lớp Java được thử trước file `.properties`.



- Sau khi chọn, key được tìm trong bundle đó rồi **lên các bundle cha** của nó (chỉ theo locale đã chọn, ví dụ `fr_CA` → `fr` → gốc) — **không** tìm trong bundle của locale mặc định.
- Không tìm thấy bundle nào → `MissingResourceException`; không tìm thấy key → `MissingResourceException`.
- `getString(key)` (giá trị phải là `String`, nếu không → `ClassCastException`), `getObject(key)`, `keySet()`, `containsKey`, `getLocale()`.
- `Properties` (không phải bundle): `getProperty(key)` trả về `null` nếu thiếu, `getProperty(key, default)`.

NumberFormat và DecimalFormat

Factory	Ví dụ (US)
<code>NumberFormat.getInstance()</code> / <code>getNumberInstance</code>	1,234.567 (tối đa 3 chữ số thập phân)
<code>getIntegerInstance()</code>	1,235 (HALF_EVEN)
<code>getCurrencyInstance()</code>	\$1,234.57
<code>getPercentInstance()</code>	26% (nhân 100)
<code>getCompactNumberInstance(1, Style.SHORT/LONG)</code>	2M, 2 million

- Làm tròn mặc định: **HALF_EVEN**. Chính: `setMaximumFractionDigits`, `setMinimumFractionDigits`, `setRoundingMode`.
- `parse(String)` trả về `Number` (`Long` nếu không có phần lẻ, `Double` nếu có), đọc từ đầu và **dừng** ở ký tự lạ; không đọc được gì → `ParseException` (checked). Dấu `,` / `.` hiểu theo locale.
- `DecimalFormat("#,##0.00")`: `0` bắt buộc, `#` tùy chọn, `,` phân cách nhóm, `.` thập phân, `%` nhân 100, `'text'` là chữ nguyên văn.

DateTimeFormatter

Chữ	Ý nghĩa	Ví dụ
<code>y / yy / yyyy</code>	năm	2024, 24
<code>M / MM / MMM / MMMM</code>	tháng	3, 03, Mar, March
<code>d / dd / D</code>	ngày trong tháng / ngày trong năm	5, 05, 65
<code>E / EEEE</code>	thứ	Tue, Tuesday
<code>H / HH</code>	giờ 0–23	14
<code>h / hh + a</code>	giờ 1–12 + AM/PM	02 PM
<code>m / mm</code>	phút	07
<code>s, S</code>	giây, phần giây	
<code>'...'</code> , <code>''</code>	chữ nguyên văn, dấu nháy	'T'

- `DateTimeFormatter.ofPattern(pattern[, locale])`, `ofLocalizedDate/Time/DateTime(FormatStyle) + withLocale`, hằng ISO (`ISO_LOCAL_DATE`, `BASIC_ISO_DATE`...).
- `date.format(f) = f.format(date)`; `LocalDate.parse(text, f)`. Sai định dạng → `DateTimeParseException` (unchecked). Pattern có giờ mà format `LocalDate` → `UnsupportedTemporalTypeException`.
- Parse văn bản mặc định phân biệt hoa thường.

MessageFormat

- `MessageFormat.format("{0} có {1} tin", a, b)`; kiểu con: `{0,number}`, `{0,number,###}`, `{0,number,percent}`, `{0,date,short}`...
- `'` là ký tự escape: `'{0}'` in nguyên văn `{0}`; `''` in một dấu `'`. Thiếu tham số → in nguyên `{n}`.
- `new MessageFormat(pattern, locale)` để định dạng số theo locale.

Lỗi và bẫy thường gặp (Exam traps)

- `Locale.GERMAN` (chỉ ngôn ngữ) ≠ `Locale.GERMANY` (`de_DE`).
- `toString()` dùng `_`, `toLanguageTag()` dùng `-`.
- Locale mặc định được thử **trước** bundle gốc.
- Key được thừa kế từ bundle cha của bundle đã chọn, không từ bundle của locale mặc định.
- Lớp Java bundle thẳng file `.properties` cùng tên.
- Thiếu key → `MissingResourceException`, không trả `null` (khác `Properties.getProperty`).
- `NumberFormat` là abstract; `parse` trả `Number` và ném `ParseException` (checked).
- `parse("12abc")` → 12, không lỗi; `parse("abc")` → `ParseException`.
- HALF_EVEN: 12.5 → 12, 13.5 → 14; percent 0.125 → 12%.
- MM tháng vs mm phút; HH 24h vs hh 12h.
- `'` trong `MessageFormat` và `DateTimeFormatter`.

12. Dấu phân cách theo locale: 1,5 với US là 15.

Góc nhìn từ TypeScript

TypeScript / JavaScript	Java	Ghi chú
<code>new Intl.NumberFormat("vi-VN").format(n)</code>	<code>NumberFormat.getInstance(Locale.of("vi", "VN")).format(n)</code>	Cùng dùng dữ liệu CLDR
<code>Intl.NumberFormat(1, { style: "currency", currency: "VND" })</code>	<code>NumberFormat.getCurrencyInstance(1) + setCurrency</code>	
<code>Intl.NumberFormat(1, { notation: "compact" })</code>	<code>NumberFormat.getCompactNumberInstance(1, SHORT)</code>	
<code>new Intl.DateTimeFormat(1, { dateStyle: "short" })</code>	<code>DateTimeFormatter.ofLocalizedDate(FormatStyle.SHORT).withLocale(1)</code>	
<code>Angular i18n (\$localize, file .xlf) / ngx-translate (vi.json)</code>	<code>ResourceBundle + Messages_vi.properties</code>	Cùng ý tưởng: key → chuỗi dịch
<code>ICU message format {count, plural, ...}</code>	<code>MessageFormat (ChoiceFormat cho số nhiều)</code>	
<code>navigator.language</code>	<code>Locale.getDefault()</code>	

Tóm tắt

- `Locale.of(lang, country)`, `forLanguageTag`, hằng số; `toString()` vs `toLanguageTag()`.
- `ResourceBundle.getBundle`: locale yêu cầu → locale mặc định → gốc; key thừa kế từ bundle cha; lớp Java trước `.properties`.
- `NumberFormat`: `number/integer/currency/percent/compact`; `HALF_EVEN`; `parse` trả `Number`, `ParseException`.
- `DateTimeFormatter`: `pattern (M/m, H/h, ')`, `FormatStyle` + `withLocale`, `DateTimeParseException`.
- `MessageFormat`: `{0}`, `{0,number,...}`, `'` escape.

Bài tập (có lời giải)

Mọi đáp án đã được `tools/book.py` biên dịch và chạy để xác nhận (code: `examples/questions/ch10/`); các câu về bundle dùng script tạo đúng file `.properties` rồi chạy chương trình.

Câu hỏi

Câu 10-01 · Dễ · objective 10.1

Chương trình sau in ra gì?

```
import java.util.Locale;

public class Loc {
    public static void main(String[] args) {
        Locale a = Locale.of("fr", "CA");
        Locale b = Locale.forLanguageTag("vi-VN");
        Locale c = Locale.GERMAN;
        System.out.println(a + " " + b + " " + c + " [" + c.getCountry() + "]");
    }
}
```

- A. `fr_CA vi_VN de []`
- B. `fr-CA vi-VN de_DE [DE]`
- C. `fr_CA vi-VN de []`
- D. `CA_fr VN_vi de []`

Câu 10-02 · Vừa · objective 10.2

Chương trình sau in ra gì?

```
import java.text.NumberFormat;
import java.util.Locale;

public class Money {
    public static void main(String[] args) {
        NumberFormat c = NumberFormat.getCurrencyInstance(Locale.US);
        NumberFormat p = NumberFormat.getPercentInstance(Locale.US);
        System.out.println(c.format(1234.567) + " " + p.format(0.125) + " " + c.format(-5));
    }
}
```

- A. \$1,234.57 13% -\$5.00
- B. \$1,234.57 12% -\$5.00
- C. \$1,234.56 12% (\$5.00)
- D. \$1234.57 12.5% -\$5.00

Câu 10-03 · Vừa · objective 10.2

Kết quả của chương trình là gì?

```
import java.text.*;
import java.util.Locale;

public class Parse {
    public static void main(String[] args) throws ParseException {
        NumberFormat nf = NumberFormat.getInstance(Locale.GERMANY);
        Number n1 = nf.parse("1.500,75");
        Number n2 = nf.parse("3,5kg");
        System.out.println(n1 + " " + n2 + " " + n2.getClass().getSimpleName());
    }
}
```

- A. In ra 1.50075 3.5 Double
- B. Ném ParseException vì có chữ kg
- C. In ra 1500.75 3 Long
- D. In ra 1500.75 3.5 Double

Câu 10-04 · Khó · objective 10.1

Classpath có các file bundle dưới đây. Chương trình Main in ra gì?

Menu.properties

title=Root

Menu_en.properties

title=English

Menu_vi_VN.properties

title=VN

Main.java

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Locale.setDefault(Locale.of("en", "US"));
        String a = ResourceBundle.getBundle("Menu", Locale.of("vi")).getString("title");
        String b = ResourceBundle.getBundle("Menu", Locale.of("vi", "VN")).getString("title");
        System.out.println(a + " " + b);
    }
}
```

- A. Root VN
- B. VN VN
- C. Ném MissingResourceException
- D. English VN

Câu 10-05 · Vừa · objective 10.1

Với các bundle dưới đây, chương trình in ra gì?

Messages.properties

a=root-a
b=root-b

Messages_fr.properties

a=fr-a

Messages_fr_CA.properties

b=ca-b

Main.java

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        ResourceBundle rb = ResourceBundle.getBundle("Messages", Locale.CANADA_FRENCH);
        System.out.print(rb.getString("a") + " " + rb.getString("b") + " ");
        try {
            System.out.println(rb.getString("c"));
        } catch (MissingResourceException e) {
            System.out.println("missing");
        }
    }
}
```

- A. root-a ca-b missing
- B. fr-a root-b missing
- C. fr-a ca-b missing
- D. fr-a ca-b null

Câu 10-06 · Vừa · objective 10.2

Chèn dòng nào vào chỗ `// INSERT CODE HERE` thì chương trình in ra `05/03/2024 02:07 PM`? (Chọn 2 đáp án.)

```
import java.time.*;
import java.time.format.*;
import java.util.Locale;

public class Pattern {
    public static void main(String[] args) {
        LocalDateTime dt = LocalDateTime.of(2024, 3, 5, 14, 7);
        // INSERT CODE HERE
    }
}
```

- A. `System.out.println(dt.format(DateTimeFormatter.ofPattern("dd/MM/yyyy hh:mm a", Locale.US)));`
- B. `System.out.println(dt.format(DateTimeFormatter.ofPattern("dd/mm/yyyy hh:mm a", Locale.US)));`
- C. `System.out.println(dt.format(DateTimeFormatter.ofPattern("dd/MM/yyyy HH:mm a", Locale.US)));`
- D. `System.out.println(DateTimeFormatter.ofPattern("dd/MM/yyyy hh:mm a", Locale.US).format(dt));`
- E. `System.out.println(dt.format(DateTimeFormatter.ofPattern("d/M/yyyy h:mm a", Locale.US)));`

Câu 10-07 · Khó · objective 10.2

Chương trình sau in ra gì?

```
import java.text.MessageFormat;

public class Quotes {
    public static void main(String[] args) {
        System.out.println(MessageFormat.format("'{0}' = {0}, it's {1}", "x"));
    }
}
```

- A. `x = x, it's {1}`
- B. `'x' = x, it's null`
- C. `{0} = x, it's {1}`
- D. `{0} = x, it's null`

Câu 10-08 · Vừa · objective 10.2

Chương trình sau in ra gì?

```
import java.time.LocalDate;
import java.time.format.*;
import java.util.Locale;

public class Styles {
    public static void main(String[] args) {
        LocalDate d = LocalDate.of(2024, 12, 25);
        System.out.println(d.format(DateTimeFormatter.ofLocalizedDate(FormatStyle.SHORT).withLocale(Locale.US))
            + " | " + d.format(DateTimeFormatter.ofLocalizedDate(FormatStyle.MEDIUM).withLocale(Locale.US)));
    }
}
```

```
}  
}
```

- A. 25/12/24 | 25 Dec 2024
- B. 12/25/2024 | December 25, 2024
- C. 2024-12-25 | Dec 25, 2024
- D. 12/25/24 | Dec 25, 2024

Câu 10-09 · Vừa · objective 10.1, 10.2

Hai phát biểu nào đúng? (Chọn 2 đáp án.)

- A. `Locale.of("vi").getCountry()` trả về chuỗi rỗng.
- B. `ResourceBundle.getBundle` ném exception khi không có bundle cho đúng locale yêu cầu, kể cả khi có bundle gốc.
- C. Locale mặc định của JVM có thể đổi lúc chạy bằng `Locale.setDefault`.
- D. Key trong `ResourceBundle` không phân biệt hoa thường.
- E. `NumberFormat.getInstance(Locale.US).parse("1,5")` trả về 1.5.

Câu 10-10 · Khó · objective 10.1

Có cả lớp `Prices_vi` (một `ListResourceBundle`) và file `Prices_vi.properties` trên classpath. Chương trình in ra gì?

`Prices.java`

```
import java.util.ListResourceBundle;  
public class Prices extends ListResourceBundle {  
    protected Object[][] getContents() { return new Object[][]{{"currency", "USD"}}; }  
}
```

`Prices_vi.java`

```
import java.util.ListResourceBundle;  
public class Prices_vi extends ListResourceBundle {  
    protected Object[][] getContents() { return new Object[][]{{"currency", "VND"}}; }  
}
```

`Prices_vi.properties`

```
currency=FROM PROPERTIES
```

`Main.java`

```
import java.util.*;  
public class Main {  
    public static void main(String[] args) {  
        System.out.println(ResourceBundle.getBundle("Prices", Locale.of("vi")).getString("currency"));  
    }  
}
```

- A. FROM PROPERTIES
- B. VND
- C. USD
- D. Ném `MissingResourceException`

Câu 10-11 · Vừa · objective 10.2

Chương trình sau in ra gì?

```
import java.text.*;  
import java.util.Locale;  
  
public class Patterns {  
    public static void main(String[] args) {  
        var sym = DecimalFormatSymbols.getInstance(Locale.US);  
        System.out.println(new DecimalFormat("#,##0.0#", sym).format(1234.5) + " "  
            + new DecimalFormat("00.00", sym).format(3.14159) + " "  
            + new DecimalFormat("#.#", sym).format(0.05));  
    }  
}
```

- A. 1,234.50 3.14 .1
- B. 1,234.5 03.14 .05
- C. 1,234.5 03.14 0.1

- D. 1234.5 03.14 0.1

Câu 10-12 · Vừa · objective 10.2

Chương trình sau in ra gì?

```
import java.text.NumberFormat;
import java.util.Locale;

public class Compact {
    public static void main(String[] args) {
        NumberFormat s = NumberFormat.getCompactNumberInstance(Locale.US, NumberFormat.Style.SHORT);
        NumberFormat l = NumberFormat.getCompactNumberInstance(Locale.US, NumberFormat.Style.LONG);
        System.out.println(s.format(1_250_000) + " " + l.format(3_000));
    }
}
```

- A. 1.25M 3 thousand
- B. 1M 3 thousand
- C. 1M 3K
- D. 1,250K 3 thousand

Câu 10-13 · Vừa · objective 10.2

Chương trình sau in ra gì?

```
import java.text.NumberFormat;
import java.util.Locale;

public class HalfEven {
    public static void main(String[] args) {
        NumberFormat nf = NumberFormat.getIntegerInstance(Locale.US);
        System.out.println(nf.format(12.5) + " " + nf.format(13.5));
    }
}
```

- A. 13 14
- B. 12 13
- C. 12 14
- D. 13 13

Câu 10-14 · Dễ · objective 10.2

Chương trình sau in ra gì?

```
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.Locale;

public class Midnight {
    public static void main(String[] args) {
        LocalDateTime t = LocalDateTime.of(0, 5);
        System.out.println(t.format(DateTimeFormatter.ofPattern("hh:mm a", Locale.US)) + " "
            + t.format(DateTimeFormatter.ofPattern("HH:mm")));
    }
}
```

- A. 12:05 AM 00:05
- B. 00:05 AM 00:05
- C. 12:05 AM 12:05
- D. 0:05 AM 00:05

Câu 10-15 · Vừa · objective 10.1, 10.2

Những dòng nào gây lỗi biên dịch?

```
import java.text.*;
import java.util.*;

public class L10n {
    public static void main(String[] args) throws ParseException {
        Locale a = Locale.of("en", "US"); // L1
        Locale b = new Locale.Builder().setLanguage("en").build(); // L2
        Locale c = Locale.US.of("fr"); // L3
        NumberFormat d = new NumberFormat(); // L4
        NumberFormat e = NumberFormat.getInstance(Locale.US); // L5
        double f = e.parse("1.5"); // L6
    }
}
```

```
}  
}
```

- A. Chỉ L4
- B. L4 và L6
- C. L3 và L4
- D. L3, L4 và L6
- E. Chỉ L6

Câu 10-16 · Khó · objective 10.1

Có các bundle `Msg.properties` (`bye=Goodbye`), `Msg_vi.properties` (`bye=Tạm biệt`), `Msg_vi_VN.properties` (`hello=Chào`). Chèn biểu thức nào vào chỗ `// INSERT CODE HERE` thì chương trình in ra `Tạm biệt`? **(Chọn 3 đáp án.)**

Main.java

```
import java.util.*;  
  
public class Main {  
    public static void main(String[] args) {  
        Locale.setDefault(Locale.US);  
        System.out.println(// INSERT CODE HERE);  
    }  
}
```

- A. `ResourceBundle.getBundle("Msg", Locale.of("vi", "VN")).getString("bye")`
- B. `ResourceBundle.getBundle("Msg", Locale.of("vi")).getString("bye")`
- C. `ResourceBundle.getBundle("Msg", Locale.US).getString("bye")`
- D. `ResourceBundle.getBundle("Msg").getString("bye")`
- E. `ResourceBundle.getBundle("Msg", Locale.of("vi", "US")).getString("bye")`

Câu 10-17 · Vừa · objective 10.2

Chương trình sau in ra gì?

```
import java.util.Locale;  
  
public class Fmt {  
    public static void main(String[] args) {  
        System.out.println(String.format(Locale.GERMANY, "%.2f", 1234.5) + " "  
            + String.format(Locale.US, "%.2f", 1234.5));  
    }  
}
```

- A. `1,234.50 1.234,50`
- B. `1234,50 1234.50`
- C. `1.234,5 1,234.5`
- D. `1.234,50 1,234.50`

Câu 10-18 · Dễ · objective 10.1

Chương trình sau in ra gì?

```
import java.util.Locale;  
  
public class Display {  
    public static void main(String[] args) {  
        Locale vi = Locale.of("vi", "VN");  
        System.out.println(vi.getDisplayLanguage(Locale.US) + " " + vi.getDisplayCountry(Locale.US) + " "  
            + vi.toLanguageTag());  
    }  
}
```

- A. Vietnamese Vietnam vi-VN
- B. Vietnamese Vietnam vi_VN
- C. vi VN vi-VN
- D. Tiếng Việt Việt Nam vi-VN

Câu 10-19 · Khó · objective 10.2, 1.4

Chương trình sau in ra gì?

```
import java.time.LocalDate;  
import java.time.format.*;
```

```
import java.util.Locale;

public class ParseDate {
    public static void main(String[] args) {
        DateTimeFormatter f = DateTimeFormatter.ofPattern("dd MMM yyyy", Locale.US);
        LocalDate d = LocalDate.parse("07 Jan 2024", f);
        System.out.print(d + " " + d.format(DateTimeFormatter.ofPattern("EEE D", Locale.US)) + " ");
        try {
            LocalDate.parse("07 jan 2024", f);
            System.out.println("ok");
        } catch (DateTimeParseException e) {
            System.out.println("fail");
        }
    }
}
```

- A. 2024-01-07 Sun 7 fail
- B. 2024-01-07 Sun 7 ok
- C. 2024-07-01 Mon 183 fail
- D. 2024-01-07 Sun 07 ok

Câu 10-20 · Vừa · objective 10.1

Có các bundle `Labels.properties` (`title=Root title`), `Labels_de.properties` (`title=Deutscher Titel`), `Labels_en.properties` (`title=English title`). Locale mặc định là `ja_JP`. `getBundle("Labels", Locale.FRANCE)` trả về giá trị `title` nào?

`Labels.properties`

`title=Root title`

`Labels_de.properties`

`title=Deutscher Titel`

`Labels_en.properties`

`title=English title`

`Main.java`

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        Locale.setDefault(Locale.JAPAN);
        System.out.println(ResourceBundle.getBundle("Labels", Locale.FRANCE).getString("title"));
    }
}
```

- A. Root title
- B. Deutscher Titel
- C. English title
- D. Ném `MissingResourceException`

Lời giải

Câu 10-01 — Đáp án: A

- **Vì sao đúng:** `Locale.toString()` có dạng ngôn ngữ_QUỐC GIA (dấu gạch dưới). `forLanguageTag("vi-VN")` đọc tag có gạch ngang nhưng `toString()` vẫn in `vi_VN`. `Locale.GERMAN` chỉ có ngôn ngữ, quốc gia là chuỗi rỗng.
- **B sai:** `toString()` dùng `_`; `Locale.GERMAN` (khác `Locale.GERMANY`) không có quốc gia.
- **C sai:** Dạng có gạch ngang là của `toLanguageTag()`, không phải `toString()`.
- **D sai:** Ngôn ngữ đứng trước quốc gia.
- *Kiểm chứng:* `examples/questions/ch10/Q10_01/` — output confirmed (`python3 tools/book.py questions ch10`).

Câu 10-02 — Đáp án: B

- **Vì sao đúng:** Tiền tệ US: 2 chữ số thập phân, có dấu phân cách hàng nghìn → `$1,234.57`. Percent nhân 100 và mặc định không có chữ số thập phân; `NumberFormat` làm tròn kiểu **HALF_EVEN** (nửa về số chẵn) → 12.5 thành 12%. Số âm in `-$5.00`.
- **A sai:** Chế độ làm tròn mặc định là **HALF_EVEN**: 12.5 → 12 (số chẵn gần nhất).
- **C sai:** 1234.567 làm tròn thành `.57`; và số âm hiển thị bằng dấu `-` (không phải ngoặc) với CLDR hiện tại.
- **D sai:** Có dấu phân cách hàng nghìn, và percent mặc định không có phần thập phân.
- *Kiểm chứng:* `examples/questions/ch10/Q10_02/` — output confirmed (`python3 tools/book.py questions ch10`).

Câu 10-03 — Đáp án: D

- **Vì sao đúng:** Ở Đức, `.` là dấu phân cách hàng nghìn và `,` là dấu thập phân → 1500.75. `parse(String)` đọc từ đầu chuỗi và **dừng** ở ký tự không hợp lệ đầu tiên (`k`), không ném exception → 3.5 (kiểu `Double` vì có phần thập phân).
- **A sai:** Với locale Đức, dấu `.` là phân cách hàng nghìn, không phải dấu thập phân.
- **B sai:** `parse` chỉ ném `ParseException` khi **không đọc được gì** ở đầu chuỗi.
- **C sai:** `,5` là phần thập phân theo locale Đức; kết quả có phần lẻ nên là `Double`.
- *Kiểm chứng:* `examples/questions/ch10/Q10_03/` — output confirmed (python3 tools/book.py questions ch10).

Câu 10-04 — Đáp án: D

- **Vì sao đúng:** Với ví: tìm `Menu_vi` (không có) → chuyển sang **locale mặc định** `en_US`: `Menu_en_US` (không có), `Menu_en` (có) → English. Locale mặc định được thử **trước** bundle gốc. Với `vi_VN`: có `Menu_vi_VN` → VN.
- **A sai:** Bundle gốc chỉ dùng khi cả locale yêu cầu lẫn locale mặc định đều không có bundle.
- **B sai:** `Menu_vi_VN` là bundle **con** của `vi`; tìm cho `vi` không đi xuống bundle con.
- **C sai:** Luôn có ít nhất bundle gốc `Menu`, nên không lỗi.
- *Kiểm chứng:* `examples/questions/ch10/Q10_04/` — script output confirmed (python3 tools/book.py questions ch10).

Câu 10-05 — Đáp án: C

- **Vì sao đúng:** Bundle được chọn là `Messages_fr_CA`, với chuỗi cha (parent chain) `Messages_fr` → `Messages`. Một key không có ở bundle con thì được tìm lên bundle cha: `a` lấy từ `fr`, `b` từ `fr_CA`. Không có ở đâu → `MissingResourceException`.
- **A sai:** `Messages_fr` định nghĩa `a`, và nó gần hơn bundle gốc.
- **B sai:** `Messages_fr_CA` định nghĩa `b`, được ưu tiên hơn bundle gốc.
- **D sai:** `getString` ném exception khi không tìm thấy key, không trả về `null`.
- *Kiểm chứng:* `examples/questions/ch10/Q10_05/` — script output confirmed (python3 tools/book.py questions ch10).

Câu 10-06 — Đáp án: A, D

- **Vì sao đúng:** `MM` = tháng (2 chữ số), `mm` = phút, `hh` = giờ 1–12 (2 chữ số), `HH` = giờ 0–23, `a` = AM/PM. Có thể gọi `dt.format(formatter)` hoặc `formatter.format(dt)` — cùng kết quả.
- **B sai:** `mm` là phút: in `05/07/2024 ...`.
- **C sai:** `HH` là giờ 24h: in `14:07 PM`.
- **E sai:** Một chữ cái (`d`, `M`, `h`) không thêm số 0 ở đầu: `5/3/2024 2:07 PM`.
- *Kiểm chứng:* `examples/questions/ch10/Q10_06/` — variants: AD satisfy output (python3 tools/book.py questions ch10).

Câu 10-07 — Đáp án: C

- **Vì sao đúng:** Trong `MessageFormat`, dấu `'` bao quanh phần văn bản được in nguyên văn: `'{0}'` → `{0}`. Hai dấu `''` → một dấu `'`. Placeholder không có tham số tương ứng (`{1}`) được in nguyên văn `{1}`.
- **A sai:** `'{0}'` nằm trong dấu nháy đơn nên không được thay thế.
- **B sai:** Dấu `'` là ký tự escape, không được in ra; và thiếu tham số không in `null`.
- **D sai:** Placeholder thiếu tham số giữ nguyên `{1}`.
- *Kiểm chứng:* `examples/questions/ch10/Q10_07/` — output confirmed (python3 tools/book.py questions ch10).

Câu 10-08 — Đáp án: D

- **Vì sao đúng:** Với `Locale.US`: `SHORT` là `M/d/yy` → `12/25/24`; `MEDIUM` là `MMM d, y` → `Dec 25, 2024`. (`LONG` sẽ là `December 25, 2024`.)
- **A sai:** Đó là thứ tự ngày/tháng của locale như `en_GB`; US để tháng trước.
- **B sai:** `SHORT` dùng năm 2 chữ số; tháng đầy đủ là của `LONG`.
- **C sai:** `2024-12-25` là định dạng ISO (`ISO_LOCAL_DATE`), không phụ thuộc locale.
- *Kiểm chứng:* `examples/questions/ch10/Q10_08/` — output confirmed (python3 tools/book.py questions ch10).

Câu 10-09 — Đáp án: A, C

- **Vì sao đúng:** A: locale chỉ có ngôn ngữ thì quốc gia là `""`. C: `Locale.setDefault` đổi locale mặc định cho JVM hiện tại.
- **B sai:** Nếu không có bundle cho locale yêu cầu (và locale mặc định), bundle gốc được dùng.
- **D sai:** Key phân biệt hoa thường: `Greeting` ≠ `greeting` → `MissingResourceException`.
- **E sai:** Với US, `,` là dấu phân cách hàng nghìn nên `1,5` được đọc thành 15.
- *Kiểm chứng:* `examples/questions/ch10/Q10_09/` — each option proven true/false by a program (python3 tools/book.py questions ch10).

Câu 10-10 — Đáp án: B

- **Vì sao đúng:** Với cùng một tên bundle, `ResourceBundle` (định dạng mặc định) tìm **lớp Java** trước, rồi mới tới file `.properties`. Lớp `Prices_vi` tồn tại nên được dùng.
- **A sai:** File `.properties` chỉ được dùng khi không có lớp cùng tên.

- **C sai:** Bundle `vi` có key `currency`, nên không cần lên bundle cha `Prices`.
- **D sai:** Bundle và key đều tồn tại.
- **Kiểm chứng:** `examples/questions/ch10/Q10_10/` — script output confirmed (`python3 tools/book.py questions ch10`).

Câu 10-11 — Đáp án: C

- **Vì sao đúng:** Trong pattern, `0` = chữ số bắt buộc (thêm số 0 nếu thiếu), `#` = chữ số tùy chọn. `0.0#` → ít nhất 1, nhiều nhất 2 chữ số thập phân → `1,234.5`. `00.00` → `03.14`. `##` → một chữ số thập phân: `0.05` (giá trị double thật hơi lớn hơn `0.05`) làm tròn thành `0.1`.
- **A sai:** `#` ở vị trí thập phân thứ hai là tùy chọn nên không in `0` thừa; `00.00` bắt buộc 2 chữ số phần nguyên.
- **B sai:** `##` chỉ giữ một chữ số thập phân.
- **D sai:** `##,##0` có dấu phân cách hàng nghìn.
- **Kiểm chứng:** `examples/questions/ch10/Q10_11/` — output confirmed (`python3 tools/book.py questions ch10`).

Câu 10-12 — Đáp án: B

- **Vì sao đúng:** Compact number format (Java 12+) rút gọn số theo locale: `SHORT` → `K, M...`; `LONG` → `thousand, million...` Mặc định không có chữ số thập phân nên `1.25M` làm tròn thành `1M`.
- **A sai:** Mặc định số chữ số thập phân tối đa là 0 → không in `.25`.
- **C sai:** Style `LONG` dùng chữ đầy đủ: `3 thousand`.
- **D sai:** Đơn vị được chọn là lớn nhất phù hợp: triệu (`M`).
- **Kiểm chứng:** `examples/questions/ch10/Q10_12/` — output confirmed (`python3 tools/book.py questions ch10`).

Câu 10-13 — Đáp án: C

- **Vì sao đúng:** `NumberFormat` mặc định dùng `RoundingMode.HALF_EVEN`: khi đúng nửa, làm tròn về số **chẵn** gần nhất. `12.5` → `12`, `13.5` → `14`. (Khác `Math.round`, luôn làm tròn nửa lên.)
- **A sai:** Đó là cách làm tròn `HALF_UP` (như `Math.round`), không phải mặc định của `NumberFormat`.
- **B sai:** `13.5` làm tròn về `14` (số chẵn).
- **D sai:** `12.5` → `12` và `13.5` → `14`.
- **Kiểm chứng:** `examples/questions/ch10/Q10_13/` — output confirmed (`python3 tools/book.py questions ch10`).

Câu 10-14 — Đáp án: A

- **Vì sao đúng:** `hh` là giờ theo đồng hồ 12 giờ (1–12): 0 giờ là `12 AM`. `HH` là giờ 0–23: `00`.
- **B sai:** `hh` không bao giờ in `00`; nửa đêm là `12 AM`.
- **C sai:** `HH` là giờ 24h, nên 0 giờ là `00`.
- **D sai:** `hh` có 2 chữ số và dùng dải 1–12 (`K` mới là 0–11).
- **Kiểm chứng:** `examples/questions/ch10/Q10_14/` — output confirmed (`python3 tools/book.py questions ch10`).

Câu 10-15 — Đáp án: B

- **Vì sao đúng:** `NumberFormat` là lớp **abstract** → không `new` được (L4); dùng factory `getInstance...` `parse` trả về `Number` (có thể là `Long` hoặc `Double`), không gán thẳng cho `double` được (L6); cần `.doubleValue()`. L3 gọi method static qua một object — kiểu viết xấu nhưng hợp lệ.
- **A sai:** L6 cũng lỗi: `Number` không tự unboxing thành `double`.
- **C sai:** Gọi method static `Locale.of` qua biến `Locale.US` vẫn biên dịch được (chỉ là cảnh báo về style).
- **D sai:** L3 hợp lệ (xem C).
- **E sai:** L4 cũng lỗi: `NumberFormat` là abstract.
- **Kiểm chứng:** `examples/questions/ch10/Q10_15/` — compile error confirmed at `['L4', 'L6']` (`python3 tools/book.py questions ch10`).

Câu 10-16 — Đáp án: A, B, E

- **Vì sao đúng:** A: bundle `vi_VN` không có `bye` nên lấy từ bundle cha `vi`. B: dùng thẳng `Msg_vi`. E: không có `Msg_vi_US`, nên lùi về `Msg_vi` (cùng ngôn ngữ).
- **C sai:** `Locale.US`: không có `Msg_en_US/Msg_en` → dùng bundle gốc → `Goodbye`.
- **D sai:** Không truyền locale → dùng locale mặc định `en_US` → bundle gốc → `Goodbye`.
- **Kiểm chứng:** `examples/questions/ch10/Q10_16/` — script variants: ABE in ra 'Tạm biệt' (`python3 tools/book.py questions ch10`).

Câu 10-17 — Đáp án: D

- **Vì sao đúng:** `String.format(Locale, ...)` dùng ký hiệu của locale: Đức dùng `.` cho hàng nghìn và `,` cho thập phân; US ngược lại. Cờ `,` bật phân cách hàng nghìn, `.2f` in đúng 2 chữ số thập phân.
- **A sai:** Đảo ngược: locale Đức đứng trước trong chuỗi.
- **B sai:** Cờ `,` trong `%,.2f` bật dấu phân cách hàng nghìn.
- **C sai:** `.2f` luôn in đủ 2 chữ số thập phân.
- **Kiểm chứng:** `examples/questions/ch10/Q10_17/` — output confirmed (`python3 tools/book.py questions ch10`).

Câu 10-18 — Đáp án: A

- **Vì sao đúng:** `getDisplayXxx(Locale inLocale)` trả về tên hiển thị **bằng ngôn ngữ của inLocale** (ở đây tiếng Anh). `toLanguageTag()` dùng dạng BCP 47 với gạch ngang: `vi-VN`.
- **B sai:** `toLanguageTag()` dùng `-`; `_` là của `toString()`.
- **C sai:** `getDisplayLanguage` trả tên đầy đủ; mã ngắn là `getLanguage()`.
- **D sai:** Tên được hiển thị bằng ngôn ngữ của tham số (`Locale.US`), không phải tiếng Việt.
- *Kiểm chứng:* `examples/questions/ch10/Q10_18/` — output confirmed (`python3 tools/book.py questions ch10`).

Câu 10-19 — Đáp án: A

- **Vì sao đúng:** `MMM` với `Locale.US` đọc tên tháng viết tắt `Jan`. `EEE` = thứ viết tắt (`Sun`), `D` = ngày thứ mấy trong năm (7). Mặc định việc parse văn bản **phân biệt hoa thường**, nên `jan` không khớp → `DateTimeParseException`.
- **B sai:** Parse mặc định phân biệt hoa thường (muốn khác phải dùng `DateTimeFormatterBuilder.parseCaseInsensitive()`).
- **C sai:** `dd` là ngày, `MMM` là tháng: 7 tháng 1.
- **D sai:** `D` một chữ cái không thêm số 0; và `jan` không parse được.
- *Kiểm chứng:* `examples/questions/ch10/Q10_19/` — output confirmed (`python3 tools/book.py questions ch10`).

Câu 10-20 — Đáp án: A

- **Vì sao đúng:** Thứ tự thử: `Labels_fr_FR`, `Labels_fr` (không có) → locale mặc định `Labels_ja_JP`, `Labels_ja` (không có) → bundle gốc `Labels`. Bundle `de` và `en` không liên quan tới cả `fr` lẫn `ja`.
- **B sai:** Không locale nào trong chuỗi tìm kiếm là tiếng Đức.
- **C sai:** Tiếng Anh không nằm trong chuỗi tìm kiếm (locale mặc định là `ja_JP`).
- **D sai:** Bundle gốc tồn tại nên luôn có kết quả.
- *Kiểm chứng:* `examples/questions/ch10/Q10_20/` — script output confirmed (`python3 tools/book.py questions ch10`).

Đọc thêm (link chính thức — bị chặn trong sandbox nên mình chưa mở được)

- Javadoc `ResourceBundle` (thuật toán tìm bundle):
<https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/ResourceBundle.html>
- Javadoc `DateTimeFormatter` (bảng chữ cái pattern):
<https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/time/format/DateTimeFormatter.html>

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-28:

- `ResourceBundle.java` (candidate locales, fallback locale, lớp trước properties):
<https://raw.githubusercontent.com/openjdk/jdk21u/master/src/java.base/share/classes/java/util/ResourceBundle.java>
- `NumberFormat.java` (`HALF_EVEN`, `parse`, `compact number`):
<https://raw.githubusercontent.com/openjdk/jdk21u/master/src/java.base/share/classes/java/text/NumberFormat.java>
- `DateTimeFormatter.java` (bảng pattern letters):
<https://raw.githubusercontent.com/openjdk/jdk21u/master/src/java.base/share/classes/java/time/format/DateTimeFormatter.java>
- `Locale.java` (`Locale.of`, constructor deprecated từ Java 19):
<https://raw.githubusercontent.com/openjdk/jdk21u/master/src/java.base/share/classes/java/util/Locale.java>

Lịch học 8 tuần (có phương án rút gọn 6 tuần)

Dành cho người đi làm full-time: ~1 giờ mỗi ngày thường (thứ 2–6), ~3 giờ mỗi ngày cuối tuần (thứ 7, CN) → khoảng 11 giờ/tuần, ~88 giờ cho cả 8 tuần.

Nguyên tắc

- 1. **Đọc → chạy → đoán → kiểm tra.** Với mỗi ví dụ: đọc code, **tự đoán output trước**, rồi mới xem output thật. Chạy lại bằng `python3 tools/book.py examples chNN` hoặc tự gõ lại bằng tay với `javac/java` (không IDE — như lúc thi).
- 2. **Số lỗi (error log).** Mỗi câu sai hoặc đoán mò: ghi 1 dòng “bây giờ, quy tắc đúng là gì”. Ôn số lỗi mỗi Chủ nhật.
- 3. **Ôn giãn cách.** Mỗi Chủ nhật ôn cheat sheet của các chương đã học.
- 4. **Thi thử có bấm giờ** (120 phút, không tra cứu). Mục tiêu: **≥ 80%** ở cả 3 đề (dư ra so với ngưỡng đậu 68%) rồi mới đăng ký thi.

8 tuần

Tuần	Thứ 2 – thứ 6 (1 giờ/ngày)	Thứ 7 (3 giờ)	Chủ nhật (3 giờ)
1	T2: đọc Giới thiệu, DECISION.md, cài JDK 21, thử <code>javac/java</code> . T3–T6: Chương 1 (primitive, wrapper, toán tử, Math, String/StringBuilder)	Chương 1: text block, <code>java.time</code> , DST; chạy 13 ví dụ	Làm 20 câu Chương 1; đọc lời giải; bắt đầu số lỗi; cheat sheet 1
2	T2–T4: Chương 2 (switch, pattern matching, vòng lặp). T5–T6: Chương 3 phần 3.1–3.2 (object, nested class, khởi tạo, record)	Chương 3 phần 3.3–3.5 (overloading, scope/var, kế thừa, sealed, cast)	20 câu Chương 2; ôn cheat sheet 1–2; số lỗi
3	T2–T3: Chương 3 phần 3.6–3.7 (interface, enum). T4: 20 câu Chương 3. T5–T6: Chương 4 (exception)	20 câu Chương 4; tự viết 3 ví dụ try-with-resources + suppressed	Ôn tổng hợp 1: làm lại mọi câu sai Chương 1–4; cheat sheet 3–4
4	T2–T5: Chương 5 (mảng, List/Set/Map/Deque, Comparator, generics). T6: 20 câu Chương 5	Chương 6 phần 1: lambda, functional interface, method reference, stream cơ bản, Optional	Chương 6 phần 2: reduce, collectors (groupingBy, partitioningBy, toMap), parallel
5	T2: 20 câu Chương 6. T3–T6: Chương 7 (module) — gõ lại các lệnh <code>javac -module-source-path, jar, jlink</code>	20 câu Chương 7; tự dựng 1 project 3 module có service	Ôn tổng hợp 2: câu sai Chương 4–7; cheat sheet 5–7
6	T2–T4: Chương 8 (thread, virtual thread, executor, lock, concurrent collections). T5: 20 câu Chương 8. T6: Chương 9 phần I/O stream	Chương 9: serialization, Path, Files; 20 câu Chương 9	Đề thi thử số 1 (120 phút) + 1 giờ chữa bài; ghi chủ đề yếu
7	T2–T3: Chương 10 (localization). T4: 20 câu Chương 10. T5–T6: học lại 2 chủ đề yếu nhất từ đề 1	Đề thi thử số 2 + chữa bài	Ôn sâu số lỗi; tự viết lại cheat sheet bằng tay cho 3 chương yếu nhất
8	T2–T4: làm lại tất cả câu từng sai (chương + đề 1, 2). T5: đọc cheat sheet 1–5. T6: cheat sheet 6–10	Đề thi thử số 3 + chữa bài	Ôn nhẹ; nếu cả 3 đề ≥ 80% → đăng ký thi. Nếu chưa, lặp lại tuần 8 cho chủ đề yếu

Phương án rút gọn 6 tuần (khi đã vững Java cơ bản)

Tuần	Nội dung
1	Chương 1 + 2 (+ 40 câu)
2	Chương 3 + 4 (+ 40 câu)
3	Chương 5 + 6 (+ 40 câu)
4	Chương 7 + 8 (+ 40 câu); Chủ nhật: Đề 1
5	Chương 9 + 10 (+ 40 câu); Chủ nhật: Đề 2
6	Làm lại câu sai + cheat sheet; thứ 7: Đề 3 ; Chủ nhật: ôn nhẹ

Theo dõi tiến độ (tự điền)

Mục	Ngày làm	Điểm	Ghi chú chủ đề yếu
Câu hỏi Chương 1		/20	
Câu hỏi Chương 2		/20	

Mục	Ngày làm	Điểm	Ghi chú chủ đề yếu
Câu hỏi Chương 3		/20	
Câu hỏi Chương 4		/20	
Câu hỏi Chương 5		/20	
Câu hỏi Chương 6		/20	
Câu hỏi Chương 7		/20	
Câu hỏi Chương 8		/20	
Câu hỏi Chương 9		/20	
Câu hỏi Chương 10		/20	
Đề thi thử 1		/50	
Đề thi thử 2		/50	
Đề thi thử 3		/50	

Mẹo làm bài thi

- 120 phút / 50 câu $\approx$ **2,4 phút/câu**. Câu nào quá 4 phút → đánh dấu (mark for review), làm câu khác.
- Đọc **câu hỏi trước**, code sau: “chọn 2” hay “chọn 3”? Hỏi output, lỗi biên dịch, hay exception?
- Luôn kiểm tra trước: code có **biên dịch được** không? (import thiếu, kiểu không khớp, biến ngoài phạm vi, checked exception.)
- Với câu “dòng nào lỗi”: xét **từng dòng độc lập**.
- Không để trống câu nào (không bị trừ điểm khi sai — theo thông lệ các kỳ thi Oracle; **UNVERIFIED** với 1Z0-830).

GLOSSARY — Thuật ngữ Anh-Việt

Sắp xếp theo bảng chữ cái tiếng Anh. Cột “Chương” chỉ chương giải thích kỹ nhất.

Thuật ngữ (English)	Giải thích tiếng Việt	Chương
abstract class	Lớp trừu tượng: không new được, có thể có method abstract (không thân)	3
accessor (record)	Method đọc giá trị component của record, cùng tên component: <code>x()</code>	3
annotation <code>@FunctionalInterface</code>	Nhãn yêu cầu compiler kiểm tra interface có đúng một method abstract	3
autoboxing / unboxing	Tự động đổi primitive ↔ wrapper (<code>int</code> ↔ <code>Integer</code>)	1
automatic module	JAR không có <code>module-info</code> đặt trên module path; tên suy ra từ file hoặc MANIFEST	7
bundle (ResourceBundle)	Tập chuỗi dịch cho một locale	10
byte stream / character stream	Dòng dữ liệu theo byte (<code>InputStream</code>) / theo ký tự (<code>Reader</code>)	9
canonical constructor	Constructor của record nhận đủ các component theo đúng thứ tự	3
cast (ép kiểu)	Đổi kiểu tường minh: <code>(int) 3.9</code> , <code>(Dog) animal</code>	1, 3
checked exception	Exception bắt buộc catch hoặc khai báo <code>throws</code>	4
collector	“Công thức” gom kết quả stream: <code>toList</code> , <code>groupingBy</code> ...	6
compact constructor	Constructor của record không có danh sách tham số, dùng để kiểm tra/chuẩn hoá	3
compile-time constant	Hằng số compiler biết giá trị lúc biên dịch (<code>final int K = 10</code>)	1
concurrency	Đồng thời: nhiều việc chạy xen kẽ/song song	8
covariant return type	Kiểu trả về của method override là kiểu con của kiểu gốc	3
daemon thread	Thread “nền”; JVM không đợi nó khi kết thúc	8
deadlock	Hai thread giữ khoá và chờ khoá của nhau mãi mãi	8
default method	Method có thân trong interface	3
deserialization	Đọc chuỗi byte để tạo lại object	9
diamond operator <code><></code>	Để compiler tự suy ra tham số kiểu sau <code>new</code>	5
dominance (pattern)	Case rộng hơn đứng trước “che” case hẹp hơn phía sau → lỗi	2
DST (daylight saving time)	Giờ mùa hè: đồng hồ nhảy lên/lùi xuống 1 giờ	1
effectively final	Biến không bị gán lại sau khi khởi tạo (dù không ghi <code>final</code>)	6
encapsulation	Đóng gói: ẩn dữ liệu (<code>private</code>), truy cập qua method	3
encounter order	Thứ tự phần tử theo nguồn của stream	6
enum	Kiểu liệt kê: tập hằng số cố định, vẫn là class	3
exhaustive (switch)	Switch phủ hết mọi giá trị có thể	2
exports / opens	Cho module khác dùng package (biên dịch + chạy) / cho reflection sâu lúc chạy	7
fall-through	Switch dạng : chạy tiếp xuống case sau nếu không <code>break</code>	2
flow scoping	Biến pattern chỉ có mặt ở nơi compiler chắc chắn nó đã được gán	2
functional interface	Interface có đúng một method abstract — dùng được với lambda	3, 6
garbage collection (GC)	Thu gom rác: JVM tự giải phóng object không còn dùng tới	3
generics	Kiểu tham số hoá: <code>List<String></code>	5
guard (when)	Điều kiện thêm cho một case pattern	2

Thuật ngữ (English)	Giải thích tiếng Việt	Chương
HALF_EVEN	Làm tròn nửa về số chẵn gần nhất (mặc định của <code>NumberFormat</code>)	10
hiding (field/static method)	Lớp con khai báo field/static method trùng tên → “che”, không phải override	3
immutable	Bất biến: không đổi được trạng thái sau khi tạo	1, 3
inner class	Lớp lồng không static, gắn với một object của lớp ngoài	3
interrupt	Yêu cầu một thread dừng/thức dậy (đặt cờ hoặc gây <code>InterruptedException</code>)	8
invariant (generics)	<code>List<Integer></code> không phải <code>List<Number></code>	5
JPMS	Java Platform Module System — hệ thống module	7
lambda expression	Hàm không tên: <code>x -> x * 2</code>	6
lazy evaluation	Stream chỉ chạy khi có terminal operation	6
locale	Ngôn ngữ + vùng, ví dụ <code>vi_VN</code>	10
localization (l10n)	Hiển thị theo ngôn ngữ/vùng cụ thể	10
method reference	Cách viết gọn lambda: <code>String::length</code>	6
module path / classpath	Nơi JVM tìm module / tìm lớp theo kiểu cũ	7
multi-catch	Một khối catch cho nhiều kiểu: <code>catch (A B e)</code>	4
named / unnamed module	Module có <code>module-info</code> / code chạy từ classpath	7
NIO.2	API <code>java.nio.file</code> (<code>Path</code> , <code>Files</code>)	9
numeric promotion	Nâng kiểu số khi tính toán (<code>byte + byte → int</code>)	1
Optional	Hộp chứa 0 hoặc 1 giá trị, thay cho <code>null</code>	6
overloading	Nạp chồng: nhiều method cùng tên, khác tham số	3
overriding	Ghi đè: lớp con viết lại method instance của lớp cha	3
parallel stream	Stream chạy song song trên nhiều thread	6, 8
pattern matching	So khớp kiểu và “mở” giá trị: <code>o instanceof String s, case Point(int x, int y)</code>	2
platform thread / virtual thread	Thread gắn với thread hệ điều hành / thread nhẹ do JVM quản lý	8
polymorphism	Đa hình: cùng lời gọi, hành vi theo object thật	3
primitive type	Kiểu nguyên thủy: <code>int</code> , <code>double</code> , <code>boolean</code> ...	1
race condition	Kết quả sai do nhiều thread cùng sửa dữ liệu không đồng bộ	8
record	Lớp dữ liệu ngắn gọn, bất biến nồng	3
record pattern	Pattern “mở” record: <code>Point(int x, int y)</code>	2
reduction	Rút gọn stream thành một giá trị: <code>reduce</code> , <code>sum</code> , <code>count</code>	6
reentrant lock	Khoá mà thread đang giữ có thể lấy lại nhiều lần	8
requires transitive	Phụ thuộc được “truyền” cho module dùng mình (implied readability)	7
runtime image	Bộ JRE thu gọn tạo bằng <code>jlink</code>	7
sealed class/interface	Giới hạn lớp con được phép (<code>permits</code>)	3
serialization	Biến object thành chuỗi byte	9
service / provider / consumer	Interface dịch vụ / module cung cấp cài đặt (<code>provides</code>) / module sử dụng (<code>uses</code>)	7
short-circuit	Dừng sớm khi đã biết kết quả (<code>&&</code> , <code>& & </code> , <code>findFirst</code> , <code>limit</code>)	1, 6
stack trace	Danh sách lời gọi method tại lúc exception xảy ra	4
static nested class	Lớp lồng có <code>static</code> , không cần object lớp ngoài	3
stream (Stream API)	Dây chuyền xử lý dữ liệu: nguồn → trung gian → kết thúc	6
string pool	Vùng lưu chuỗi literal dùng chung	1
suppressed exception	Exception phụ (thường từ <code>close()</code>) đính kèm exception chính	4

Thuật ngữ (English)	Giải thích tiếng Việt	Chương
switch expression	Switch trả về giá trị	2
terminal / intermediate operation	Thao tác kết thúc / trung gian của stream	6
text block	Chuỗi nhiều dòng <code>""" ... """</code>	1
thread-safe	An toàn khi nhiều thread dùng cùng lúc	8
transient	Field không được serialize	9
try-with-resources	<code>try (R r = ...)</code> tự đóng tài nguyên	4
type erasure	Thông tin kiểu generic bị xóa lúc chạy	5
type inference (<code>var</code>)	Compiler tự suy ra kiểu biến cục bộ	3
unchecked exception	<code>RuntimeException</code> , <code>Error</code> và lớp con — không bắt buộc xử lý	4
unreachable statement	Lệnh không bao giờ chạy tới → lỗi biên dịch	2
varargs	Tham số biến số lượng: <code>int... nums</code>	3
wildcard (<code>?</code> , <code>? extends</code> , <code>? super</code>)	Kiểu “bất kỳ” có giới hạn trong generics (PECS)	5
wrapper class	Lớp bao cho primitive: <code>Integer</code> , <code>Double</code> ...	1

COVERAGE — Mục tiêu thi 1Z0-830 → chương → ví dụ → câu hỏi

File này được **sinh tự động** bởi python3 tools/book.py coverage từ: tools/objectives.yaml (danh sách mục tiêu), dòng // objective: x.y trong từng ví dụ, và trường topic của từng câu hỏi trong examples/questions/*/questions.yaml.

Danh sách mục tiêu lấy từ bảng mục tiêu trong repo eh3rrera/ocpj21-book (ghi là mục tiêu chính thức của Oracle); **UNVERIFIED** so với trang Oracle (bị chặn trong sandbox, kiểm tra 2026-09-28). Số thứ tự x.y là của sách này.

Độ phủ: 26/26 mục tiêu có đủ chương + ví dụ + câu hỏi chương.

ID	Mục tiêu (Oracle, tiếng Anh)	Chương	Ví dụ	Câu hỏi chương	Câu hỏi thi thử
1.1	Use primitives and wrapper classes.	ch01	ch01/Ex01_Primitives, ch01/Ex02_Wrappers, ch01/Ex13_NarrowingError	01-01, 01-10, 01-13, 01-19, 01-20	—
1.2	Evaluate arithmetic and boolean expressions, using the Math API and by applying precedence rules, type conversions, and casting.	ch01	ch01/Ex03_Promotion, ch01/Ex04_Casting, ch01/Ex05_Operators, ch01/Ex06_MathApi, ch01/Ex13_NarrowingError	01-02, 01-03, 01-11, 01-12, 01-17	—
1.3	Manipulate text, including text blocks, using String and StringBuilder classes.	ch01	ch01/Ex07_Strings, ch01/Ex08_StringBuilder, ch01/Ex09_TextBlocks	01-04, 01-05, 01-06, 01-14, 01-15	—
1.4	Manipulate date, time, duration, period, instant and time-zone objects including daylight saving time using Date-Time API.	ch01	ch01/Ex10_LocalDate, ch01/Ex11_PeriodDuration, ch01/Ex12_ZonesDst	01-07, 01-08, 01-09, 01-16, 01-18, 10-19	—
2.1	Create program flow control constructs including if/else, switch statements and expressions, loops, and break and continue statements.	ch02	ch02/Ex01_IfElse, ch02/Ex02_SwitchStatement, ch02/Ex03_SwitchExpression, ch02/Ex04_PatternSwitch, ch02/Ex05_Loops, ch02/Ex06_Labels, ch02/Ex07_FlowScoping, ch02/Ex08_RecordPatterns, ch02/Ex09_Unreachable, ch02/Ex10_NotExhaustive, ch02/Ex11_SwitchTypes, ch02/Ex12_DoWhileScope, ch02/Ex13_Dominance	02-01, 02-02, 02-03, 02-04, 02-05, 02-06, 02-07, 02-08, 02-09, 02-10, 02-11, 02-12, 02-13, 02-14, 02-15, 02-16, 02-17, 02-18, 02-19, 02-20	—
3.1	Declare and instantiate Java objects including nested class objects, and explain the object life-cycle including creation, reassigning references, and garbage collection.	ch03	ch03/Ex01_Lifecycle, ch03/Ex02_NestedClasses	03-01, 03-02	—
3.2	Create classes and records, and define and use instance and static fields and methods, constructors, and instance and static initializers.	ch03	ch03/Ex03_InitOrder, ch03/Ex04_Records	03-03, 03-04, 03-05	—
3.3	Implement overloading, including var-arg methods.	ch03	ch03/Ex05_Overloading	03-06, 03-07	—

ID	Mục tiêu (Oracle, tiếng Anh)	Chương	Ví dụ	Câu hỏi chương	Câu hỏi thi thử
3.4	Understand variable scopes, apply encapsulation, and create immutable objects. Use local variable type inference.	ch03	ch03/Ex06_ScopeAndVar, ch03/Ex07_Immutable	03-08, 03-09, 03-10	—
3.5	Implement inheritance, including abstract and sealed types as well as record classes. Override methods, including that of the Object class. Implement polymorphism and differentiate between object type and reference type. Perform reference type casting, identify object types using the instanceof operator, and pattern matching with the instanceof operator and the switch construct.	ch03	ch02/Ex04_PatternSwitch, ch02/Ex07_FlowScoping, ch02/Ex08_RecordPatterns, ch02/Ex13_Dominance, ch03/Ex04_Records, ch03/Ex08_Polymorphism, ch03/Ex09_AbstractSealed, ch03/Ex10_Casting, ch03/Ex11_ObjectMethods, ch03/Ex15_OverrideRules	02-06, 02-07, 02-12, 02-14, 02-15, 03-04, 03-05, 03-11, 03-12, 03-13, 03-14, 03-20, 04-09	—
3.6	Create and use interfaces, identify functional interfaces, and utilize private, static, and default interface methods.	ch03	ch03/Ex12_Interfaces, ch03/Ex13_FunctionalInterface, ch06/Ex01_LambdaSyntax, ch06/Ex02_FunctionalInterfaces, ch06/Ex12_LambdaErrors, ch06/Ex13_EffectivelyFinal	03-15, 03-16, 03-17, 06-06, 06-11, 06-17	—
3.7	Create and use enum types with fields, methods, and constructors.	ch03	ch03/Ex14_Enums	03-18, 03-19	—
4.1	Handle exceptions using try/catch/finally, try-with-resources, and multi-catch blocks, including custom exceptions.	ch04	ch04/Ex01_Hierarchy, ch04/Ex02_TryCatchFinally, ch04/Ex03_MultiCatch, ch04/Ex04_TryWithResources, ch04/Ex05_Suppressed, ch04/Ex06_CustomException, ch04/Ex07_FinallyReturn, ch04/Ex08_CatchOrder, ch04/Ex09_CommonExceptions, ch04/Ex10_InitializerError, ch04/Ex11_Rethrow, ch04/Ex12_StackTrace	04-01, 04-02, 04-03, 04-04, 04-05, 04-06, 04-07, 04-08, 04-09, 04-10, 04-11, 04-12, 04-13, 04-14, 04-15, 04-16, 04-17, 04-18, 04-19, 04-20, 09-20	—
5.1	Create arrays, List, Set, Map and Deque collections, and add, remove, update, retrieve and sort their elements.	ch05	ch05/Ex01_Arrays, ch05/Ex02_ArraysUtil, ch05/Ex03_Lists, ch05/Ex04_ModifyWhileIterating, ch05/Ex05_Sets, ch05/Ex06_Maps, ch05/Ex07_Deque, ch05/Ex08_Sorting, ch05/Ex09_Generics, ch05/Ex10_Sequenced, ch05/Ex11_Unmodifiable, ch05/Ex12_GenericsErrors	05-01, 05-02, 05-03, 05-04, 05-05, 05-06, 05-07, 05-08, 05-09, 05-10, 05-11, 05-12, 05-13, 05-14, 05-15, 05-16, 05-17, 05-18, 05-19, 05-20	—
6.1	Use Java object and primitive Streams, including lambda expressions implementing functional interfaces, to create, filter, transform, process, and sort data.	ch06	ch06/Ex01_LambdaSyntax, ch06/Ex02_FunctionalInterfaces, ch06/Ex03_Laziness, ch06/Ex04_FilterMapSort, ch06/Ex07_Optional, ch06/Ex08_PrimitiveStreams, ch06/Ex10_ShortCircuit, ch06/Ex12_LambdaErrors, ch06/Ex13_EffectivelyFinal	06-01, 06-02, 06-05, 06-06, 06-08, 06-10, 06-11, 06-13, 06-14, 06-16, 06-17, 06-18, 06-20	—

ID	Mục tiêu (Oracle, tiếng Anh)	Chương	Ví dụ	Câu hỏi chương	Câu hỏi thi thử
6.2	Perform decomposition, concatenation, and reduction, and grouping and partitioning on sequential and parallel streams.	ch06	ch06/Ex05_FlatMapConcat, ch06/Ex06_Reduction, ch06/Ex09_Collectors, ch06/Ex11_ParallelReduce	06-03, 06-04, 06-07, 06-09, 06-12, 06-15, 06-19, 08-17	—
7.1	Define modules and expose module content, including that by reflection, and declare module dependencies, define services, providers, and consumers.	ch07	ch07/Ex01_FirstModule, ch07/Ex02_RequiresExports, ch07/Ex03_Transitive, ch07/Ex04_OpensReflection, ch07/Ex05_Services, ch07/Ex10_JdkModules, ch07/Ex11_ModuleErrors	07-01, 07-02, 07-03, 07-04, 07-05, 07-06, 07-11, 07-12, 07-15, 07-17, 07-18, 07-19	—
7.2	Compile Java code, create modular and non-modular jars, runtime images, and implement migration to modules using unnamed and automatic modules.	ch07	ch07/Ex01_FirstModule, ch07/Ex06_Jars, ch07/Ex07_AutomaticModules, ch07/Ex08_Jlink, ch07/Ex09_Jdeps, ch07/Ex10_JdkModules	07-07, 07-08, 07-09, 07-10, 07-13, 07-14, 07-16, 07-20	—
8.1	Create both platform and virtual threads. Use both Runnable and Callable objects, manage the thread lifecycle, and use different Executor services and concurrent API to run tasks.	ch08	ch08/Ex01_CreateThreads, ch08/Ex02_Lifecycle, ch08/Ex03_ExecutorCallable, ch08/Ex04_VirtualExecutor, ch08/Ex05_Scheduled, ch08/Ex12_ShutdownNow	08-01, 08-02, 08-03, 08-04, 08-05, 08-06, 08-13, 08-14, 08-15, 08-18, 08-19	—
8.2	Develop thread-safe code, using locking mechanisms and concurrent API.	ch08	ch08/Ex06_RaceCondition, ch08/Ex07_Locks, ch08/Ex08_ConcurrentCollections, ch08/Ex09_Atomics, ch08/Ex10_Coordination	08-07, 08-08, 08-09, 08-10, 08-12, 08-16, 08-20	—
8.3	Process Java collections concurrently and utilize parallel streams.	ch08	ch06/Ex11_ParallelReduce, ch08/Ex08_ConcurrentCollections, ch08/Ex11_ParallelCollections	06-09, 08-11, 08-16, 08-17	—
9.1	Read and write console and file data using I/O streams.	ch09	ch09/Ex01_ByteStreams, ch09/Ex02_CharStreams, ch09/Ex03_Console, ch09/Ex09_OpenOptions, ch09/Ex10_DataStreams, ch09/Ex11_ReaderUtilities, ch09/Ex12_IOCompileErrors, ch09/Ex13_CheckedIO	09-03, 09-05, 09-10, 09-14, 09-15, 09-17, 09-18, 09-19, 09-20	—
9.2	Serialize and de-serialize Java objects.	ch09	ch09/Ex04_Serialization, ch09/Ex10_DataStreams	09-04, 09-09, 09-13	—
9.3	Construct, traverse, create, read, and write Path objects and their properties using the java.nio.file API.	ch09	ch09/Ex05_PathBasics, ch09/Ex06_FilesOps, ch09/Ex07_Attributes, ch09/Ex08_WalkFind, ch09/Ex09_OpenOptions, ch09/Ex12_IOCompileErrors, ch09/Ex13_CheckedIO	09-01, 09-02, 09-05, 09-06, 09-07, 09-08, 09-11, 09-12, 09-14, 09-16	—
10.1	Implement localization using locales and resource bundles.	ch10	ch10/Ex01_Locales, ch10/Ex06_ResourceBundles, ch10/Ex07_DefaultLocaleFallback, ch10/Ex08_ListResourceBundle, ch10/Ex10_LocaleCategory	10-01, 10-04, 10-05, 10-09, 10-10, 10-15, 10-16, 10-18, 10-20	—

ID	Mục tiêu (Oracle, tiếng Anh)	Chương	Ví dụ	Câu hỏi chương	Câu hỏi thi thử
10.2	Parse and format messages, dates, times, and numbers, including currency and percentage values.	ch10	ch10/Ex02_NumberFormat, ch10/Ex03_ParseAndPatterns, ch10/Ex04_DateTimeFormatter, ch10/Ex05_MessageFormat, ch10/Ex09_Currency, ch10/Ex10_LocaleCategory	10-02, 10-03, 10-06, 10-07, 10-08, 10-09, 10-11, 10-12, 10-13, 10-14, 10-15, 10-17, 10-19	—

SOURCES — Nguồn học và nguồn kiểm chứng

Ngày kiểm tra: 2026-09-28. Mình chỉ ghi “đã mở” với link mình thật sự mở được từ sandbox. Nhiều trang chính thức bị chặn trong sandbox — xem mục 5.

1. Nguồn chính thức (ưu tiên cao nhất)

Nguồn	Link	Trạng thái
Trang kỳ thi 1Z0-830 (Oracle University)	https://education.oracle.com/java-se-21-developer-professional/pexam_1Z0-830	Bị chặn trong sandbox — chưa mở
Java Language Specification, Java SE 21 (JLS)	https://docs.oracle.com/javase/specs/jls/se21/html/index.html	Bị chặn trong sandbox — chưa mở
Java SE 21 API docs (Javadoc)	https://docs.oracle.com/en/java/javase/21/docs/api/index.html	Bị chặn trong sandbox — chưa mở
Mã nguồn OpenJDK 21 (bản cập nhật)	https://github.com/openjdk/jdk21u	Đã mở (GPL-2.0). Javadoc nằm ngay trong mã nguồn
Javadoc trong <code>src.zip</code> của JDK đã cài	<code>/usr/lib/jvm/java-21-openjdk-amd64/lib/src.zip</code>	Đã dùng để đọc Javadoc gốc offline

Cách mình kiểm chứng API khi Javadoc online bị chặn: đọc Javadoc trong mã nguồn JDK 21 (`src.zip` hoặc `raw.githubusercontent.com/openjdk/jdk21u/...`) và chạy code thật với JDK 21.0.10.

2. So sánh repo GitHub dùng để ôn 1Z0-830

Tiêu chí “repo uy tín” (theo RULES): commit trong 12 tháng, license rõ ràng, có người dùng thật, còn bảo trì.

URL	Stars	Last push	License	Có gì hữu ích	Đánh giá	Ngày kiểm tra
https://github.com/eh3rrera/ocpj21-book	144	2026-09-03	CC BY-NC-SA 4.0 (ghi trong README; GitHub hiện “Other”)	Sách ôn 1Z0-830 đầy đủ 14 chương, bảng objective → chương	Tốt nhất để đọc thêm. License NC-SA: chỉ đọc tham khảo, không copy vào sách này	2026-09-28
https://github.com/ThiagoBfim/java-21-certification	42	2026-08-26	MIT	Ghi chú (Guide.md) + code ví dụ Java 21	Tốt, license rõ, còn cập nhật	2026-09-28
https://github.com/Anasss/java21docCards	10	2026-06-19	MIT	Flashcard + quiz Java 21	Dùng để ôn nhanh; ít người dùng	2026-09-28
https://github.com/jose-de-souza/1Z0-830	29	2026-08-28	Không có license	Code khóa học	Không đạt (không có license)	2026-09-28
https://github.com/boyarsky/sybex-1Z0-830-chapter-12	18	2024-07-30	Không có license	Code chương 12 sách Sybex	Không đạt (cũ hơn 12 tháng, không license)	2026-09-28

Số liệu stars / last push / license lấy qua GitHub search API ngày 2026-09-28; nội dung repo mở bằng trình đọc web. Không repo nào được sao chép vào sách này.

3. Sách và khóa học nổi tiếng (chỉ giới thiệu link — có phí)

Các trang này **bị chặn trong sandbox**, mình chỉ thấy chúng trong kết quả tìm kiếm (2026-09-28), **chưa mở được**:

- Jeanne Boyarsky & Scott Selikoff — *OCP Oracle Certified Professional Java SE 21 Developer Study Guide: Exam 1Z0-830* (Sybex/Wiley): <https://www.wiley.com/en-us/OCP+Oracle+Certified+Professional+Java+SE+21+Developer+Study+Guide-p-9781394286621> — trang của tác giả (errata): <https://www.selikoff.net/ocpj21/>
- Enthuware — bộ đề luyện 1Z0-830 (có phí): <https://enthuware.com/java-certification-mock-exams/oracle-certified-professional/ocpj-java-21-exam-1z0-830>
- Esteban Herrera — bản web miễn phí của sách ocpj21: <https://ocpj21.javastudyguide.com/>
- Pearson — *Java SE 21 Developer (1Z0-830)* video: <https://www.oreilly.com/videos/java-se-21/9780135461846/>
- Pluralsight — loạt khóa “Java SE 21 Developer (Exam 1Z0-830)”: <https://www.pluralsight.com/courses/java-se-21-developer-1z0-830-concurrent-programming>
- Diễn đàn Coderanch OCPJP (hỏi đáp về kỳ thi): <https://coderanch.com/f/24/java-programmer-OCPJP>

4. Chính sách bản quyền của sách này

- Mọi giải thích, ví dụ và câu hỏi đều **tự viết**. Không dùng exam dump, không dịch sách có bản quyền.
- Danh sách mục tiêu thi (tên objective) là thông tin mô tả kỳ thi, trích ngắn để đối chiếu.

5. Host bị chặn trong sandbox (không phải link hỏng)

education.oracle.com, docs.oracle.com, mylearn.oracle.com, dev.java, openjdk.org, enthuware.com, www.selikoff.net, www.wiley.com, ocpj21.javastudyguide.com, www.oreilly.com, www.pluralsight.com, coderanch.com. Kết quả link checker: `tools/check_links.py` (xem PROGRESS.md).